



ФАКУЛЬТЕТ
ВЫЧИСЛИТЕЛЬНОЙ
МАТЕМАТИКИ И
КИБЕРНЕТИКИ
МГУ ИМЕНИ
М.В. ЛОМОНОСОВА

teach-in
ЛЕКЦИИ УЧЕНЫХ МГУ

АРХИТЕКТУРА ЭВМ И ЯЗЫК АССЕМБЛЕРА

ПАДАРЯН
ВАРТАН АНДРОНИКОВИЧ

ВМК МГУ

КОНСПЕКТ ПОДГОТОВЛЕН
СТУДЕНТАМИ, НЕ ПРОХОДИЛ
ПРОФ. РЕДАКТУРУ И МОЖЕТ
СОДЕРЖАТЬ ОШИБКИ.
СЛЕДИТЕ ЗА ОБНОВЛЕНИЯМИ
НА [VK.COM/TEACHINMSU](https://vk.com/teachinmsu).

ЕСЛИ ВЫ ОБНАРУЖИЛИ
ОШИБКИ ИЛИ ОПЕЧАТКИ,
ТО СООБЩИТЕ ОБ ЭТОМ,
НАПИСАВ СООБЩЕСТВУ
[VK.COM/TEACHINMSU](https://vk.com/teachinmsu).



БЛАГОДАРИМ ЗА ПОДГОТОВКУ КОНСПЕКТА
СТУДЕНТКУ ФИЗИЧЕСКОГО ФАКУЛЬТЕТА МГУ
ГЕОРГИЕВСКУЮ ЕКАТЕРИНУ ПАВЛОВНУ

Оглавление

1	Лекция 1. Введение.	10
1.1	Цель курса	10
1.2	Несколько проблем программирования	10
1.3	Зачем нужен язык ассемблера	14
1.4	История первых компьютеров	14
1.5	Архитектура фон Неймана	15
1.6	Другие варианты архитектур	17
1.7	Модельный цикл работы ЭВМ	18
1.8	Ключевые понятия и термины	19
2	Лекция 2. Машина, на которой работает пользовательская программа	21
2.1	Машина, на которой работает пользовательская программа	21
2.2	Порядок размещения байт в памяти	23
2.3	Команда сложения	25
2.4	Интерпретация конструкции с командой add с незадаанным размером	25
2.5	Команда пересылки	26
2.6	Типы адресации	26
2.7	IA-32	27
2.8	Примеры	28
3	Лекция 3. Устройство ассемблерной программы	31
3.1	Пример	31
3.2	Обратный путь	35
3.3	Устройство ассемблерной программы	37
3.4	Пример заполнения секции .data	39
3.5	Секция .text	39
3.6	Основные арифметические команды	40
4	Лекция 4. Регистр EFLAGS.	43
4.1	Регистр EFLAGS	43
4.2	Арифметические FLAGи	44
4.3	Переполнение при сложении двух беззнаковых чисел	44
4.4	FLAG OF	44
4.5	Изменение естественного порядка выполнения программы	46

4.6	Коды	49
4.7	Варианты сравнения	50
4.8	Регистр EFLAGS и инструкции	51
4.9	Обратная задача	51
4.10	Просмотр содержимого исполняемого файла	54
5	Лекция 5. Вызов функции	55
5.1	Вызов функции	55
5.2	Аппаратный стек IA-32	56
5.3	Команда push	56
5.4	Команда pop	58
5.5	Языки программирования (ЯП), базирующиеся на стеке вызовов	59
5.6	Порядок вызова функции	60
5.7	Стек фреймов	62
5.8	Организация фрейма в IA-32/Linux	64
6	Лекция 6. Система команд.	69
6.1	Группы команд	69
6.2	Команда LEA	70
6.3	Косвенная адресация	71
6.4	Реализация стрелки Пирса	72
6.5	Сдвиги	74
6.6	Форматы	76
6.7	Команды вращения	77
6.8	Обратная задача	79
6.9	Сложение 64-разрядных чисел на 32 разрядных регистрах	79
6.10	Умножение 64 разрядных чисел на 32 разрядных регистрах	80
7	Лекция 7. Как организована передача управления.	83
7.1	Сравнение беззнаковых чисел	83
7.2	Сравнение знаковых чисел	83
7.3	Сравнение: со знаком и без	84
7.4	Реализация операндов с помощью известных команд	86
7.5	Условная передача данных	89
7.6	Конвейер – совмещение разных действий в один момент времени	90
7.7	Оператор do-while	93
7.8	Оператор while	93
7.9	Оператор for	94
8	Лекция 8. Оператор управления switch	99
8.1	Обратная задача	99
8.2	Оператор управления switch	100
8.3	Duff's Device	104

8.4	Обратная задача	112
8.5	Указатели	114
8.6	Массивы	115
9	Лекция 9. Многомерные массивы.	116
9.1	Массивы	116
9.2	Две обратные задачи	117
9.3	Выделение памяти	119
9.4	Доступ к строкам	120
9.5	Оптимизация доступа к многомерным массивам.	122
9.6	Обращение к элементу	122
9.7	Оптимизация доступа к элементам массива	123
10	Лекция 10. Структуры	126
10.1	Повторение	126
10.2	Структуры	126
10.3	Выравнивание полей в структурах	127
10.4	Объединения	130
10.5	Порядок байт	131
10.6	Битовые поля	131
10.7	Немного про функции	132
10.8	Соглашения CDECL	132
10.9	Сохранение регистров	133
11	Лекция 11. Функция main	134
11.1	ABI - двоичный интерфейс приложения	134
11.2	Функция main	135
11.3	Начальное состояние стека	135
11.4	Сохранение требований по выравниванию	137
11.5	Конструкторы и деструкторы	138
11.6	Оболочка вокруг main	139
12	Лекция 12. Особенности архитектуры x86-64.	140
12.1	Выравнивание фрейма	140
12.2	Отказ от указателя фрейма	140
12.3	Компиляторы и соглашения вызова функции	142
12.4	Соглашение FASTCALL	144
12.5	Особенности 64-разрядной процессорной архитектуры	145
12.6	Архитектура x86_64	146
13	Лекция 13. Безопасность программного обеспечения	148
13.1	Пример 1 “Заглянуть за горизонт”	148
13.2	Пример 2. “Нескучная арифметика”	150
13.3	Пример 3. “return-to-libc”	151

13.4	Способы защиты	154
14	Лекция 14. Динамическая память	156
14.1	Управление динамической памятью	156
14.2	Выделение динамической памяти	156
14.3	Ограничения	158
14.4	Производительность. Пропускная способность	158
14.5	Внутренняя фрагментация. Внешняя фрагментация	158
14.6	Проблемы реализации менеджера памяти	159
15	Лекция 15. Работа с числами с плавающей точкой. Часть 1	165
15.1	Дробные двоичные числа	165
15.2	Представимые рациональные числа	165
15.3	Представление чисел с плавающей точкой	165
15.4	Диапазоны значений	167
15.5	Округление	167
15.6	Арифметические операции	168
15.7	Математические свойства сложения и умножения	170
15.8	Числа с плавающей точкой в языке Си	171
15.9	Упрощённая схема x87	171
16	Лекция 16. Работа с числами с плавающей точкой. Часть 2. Сопроцессор x87	173
16.1	Слово (регистр) состояния	173
16.2	Управляющий регистр	175
16.3	Регистр признаков	175
16.4	NASM и числа с плавающей точкой	175
16.5	Пример 1	176
16.6	Пример 2	178
16.7	Порядок действий	178
16.8	Дополнительные команды	179
16.9	Сравнение чисел	180
16.10	Дополнительные возможности работы сопроцессора	181
17	Лекция 17. Элементы системы программирования	182
17.1	Система программирования	182
17.2	Система программирования языка Си	182
17.3	Компиляция	184
17.4	Схема работы ассемблера	184
17.5	Пример Си-программы	185
17.6	Статическая компоновка	186
18	Лекция 18. Сборка Си-программы	191
18.1	Предварительное определение переменных	191
18.2	Сильные и слабые символы. Пример.	191

18.3	Правила работы с символами	192
18.4	Задача. Ошибка компоновки	193
18.5	Пример (Правила перебазирования)	193
18.6	Работа с общими функциями	196
18.7	Статические библиотеки	196
18.8	Загрузка исполняемого объектного файла	197
19	Лекция 19. Динамические библиотеки. Динамическое связывание	199
19.1	Динамические библиотеки	199
19.2	Пример	200
19.3	Позиционно независимый код в IA-32	201
19.4	Код при вызове функций	204
19.5	Загрузка динамически скомпонованного исполняемого файла	205
20	Лекция 20. Аппаратное обеспечение	207
20.1	Логические вентили	207
20.2	Сумматор. Полусумматор	209
20.3	Мультиплексор	209
20.4	Арифметико-логическое устройство	210
20.5	Регистр: сохранение 1 бита	210
20.6	Статическая память	211
20.7	Отличие статической памяти от динамической памяти	211
20.8	Разработка интегральных схем	211
20.9	Этапы разработки и изготовления компонент ЭВМ	212
20.10	Закон Мура	212
20.11	Закон Гроша	213
20.12	Закон Белла	213
20.13	Различные классы компьютеров	213
20.14	Оперативная память	214
21	Лекция 21. Организация шин	217
21.1	Шины и адресные пространства.	217
21.2	Организация ввода/вывода через пространство портов и через память.	220
21.3	Шина. Точка зрения системного программиста	222
21.4	Пример устройства на шине: WatchDog.	224
21.5	Физические принципы устройства шин. Основные характеристики шин.	225
21.6	Развитие системы связанных шин персональных компьютеров	226
21.7	Примеры шин: фронтальная шина	229
21.8	Синхронизация обращений к памяти	229
21.9	Примеры шин: PCI.	232

22	Лекция 22. Использование шин в современных компьютерах	234
22.1	Примеры шин: AGP, USB, SATA	234
22.2	Жесткий диск	235
22.3		237
22.4	Ввод/вывод (блочного) SATA-устройства.	238
22.5	Твердотельные диски (SSD)	239
22.6	Локальность в программах	242
22.7	Иерархическая организация памяти, кэширование.	244
22.8	Кэш-память: способы организации.	246
22.9	Кэш прямого отображения.	247
23	Лекция 23. Работа кэша. Количественные характеристики работы компьютера	248
23.1	Запись в кэш	248
23.2	Метрики производительности кэша	249
23.3	Способы оценки производительности компьютеров и программ	250
23.4	Аппаратные средства измерения времени.	251
23.5	Оценка производительности памяти: синтетический бенчмарк (контрольная задача)	252
23.6	Микроархитектура процессора, ее связь с другими архитектурными уровнями.	254
23.7	Пропускная способность vs. Латентность	256
23.8	Оценка латентности памяти на синтетическом тесте.	257
23.9	Микроархитектурные решения	260
23.10	Конвейерная обработка команд, проблемы конвейерной обработки	260
23.11	CISC и RISC архитектуры.	266
23.12	RISC-V – архитектура RISC-микропроцессоров с открытым исходным кодом.	267
23.13	Конвейер RISC-V	269
23.14	Развитие системы команд в архитектуре x86	270
24	Лекция 24. Системное управление работой современного компьютера	272
24.1	Режимы работы современного процессора архитектуры Intel64/AMD64, загрузка.	272
24.2	Встраиваемое ПО, обеспечивающее начальную загрузку компьютера: BIOS, ACPI, UEFI	273
24.3	Многозадачная работа компьютера: требования к аппаратуре.	275
24.4	Модели памяти.	276
24.5	Аппарат защиты памяти.	278
24.6	Страничная виртуальная память.	280

24.7	Упрощенная схема извлечения данных из виртуальной памяти . .	284
24.8	Уровни (кольца) защиты	285
24.9	Прерывания.	285
24.10	Hello, world!	289

Лекция 1. Введение.

Цель курса

1. Формирование связного представления об организации современных вычислительных систем.
 - Аппаратура, системные программы, прикладные программы
2. Понимание взаимосвязей между архитектурными решениями на уровнях аппаратуры ЭВМ и ПО.
3. Понимание факторов, влияющих на качественные и количественные характеристики ЭВМ, производительность и безопасность всей вычислительной системы в целом.
 - Язык Си как средство демонстрации
 - того, как работает аппаратура
 - того, как следует использовать аппаратуру для реализации языка программирования
 - Результат
 - Понимание причин хорошей/плохой производительности программы
 - Понимание причин ошибок
 - Основа для других курсов: ОС, сети, компиляторы, параллельная обработка данных ...

Несколько проблем программирования

В качестве введения рассмотрим несколько проблем современного программирования. Первая проблема – **свойства чисел**. В математических курсах мы зачастую имеем дело с некоторыми абстракциями, которые могут принимать бесконечные значения. В реальности вычислительные машины не могут представить бесконечность, так как количество представимых чисел ограничено. Также возникают вопросы, касающиеся чисел с плавающей точкой (округление) и некоторые другие. Примеры данных проблем можно представить в виде следующего списка:

- $X^2 \geq 0$?
 - float – всегда выполняется
 - int?

$$* 40000 \cdot 40000 = 1600000000$$

$$* 50000 \cdot 50000 = ?$$

- $(x + y) + z = x + (y + z)$?
 - signed/unsigned int – всегда выполняется
 - float
 - * $(1e20 + -1e20) + 3.14 \rightarrow 3.14$
 - * $1e20 + (-1e20 + 3.14) \rightarrow 3.14$

- Свойства типов данных могут быть использованы для причинения вреда

Вторая проблема – **язык ассемблера**. Существует довольно известный индекс популярности языков программирования который оценивает, как часто программисты обсуждают на разных тематических ресурсах те или иные языки программирования. Отметим, что язык ассемблера не входит в число популярных языков. Этот язык зачастую находится во второй десятке обсуждаемых языков, причём непонятно, о каком конкретно языке идёт речь, так как существует много архитектур.

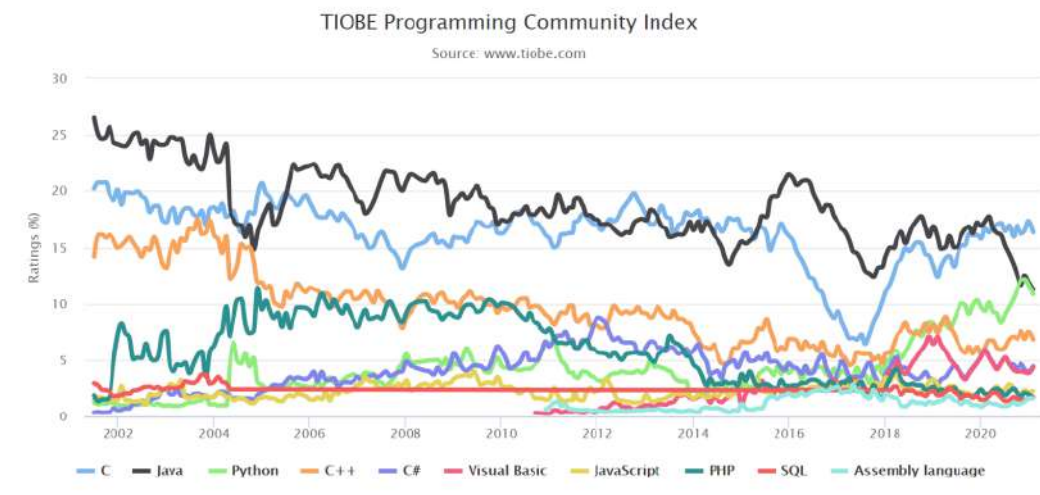


Рис. 1.1: TIOBE Programming Community Index

Третья проблема – **отладка**. Зачастую поиск ошибки в программе занимает значительное время. Более того, чем больше программа, тем сложнее найти ошибки. Последствия ошибки могут проявиться в любом месте программы. Также все усложняется тем, что в текущий момент времени может выполняться несколько параллельных операций на различных ядрах.

Отметим, что (в случае языка Си) возможны ситуации, которые описаны как неопределённое поведение. То есть, на разной аппаратуре компилятор может построить код разными способами.

- Ошибки работы с памятью
 - Выход на пределы массива
 - Некорректные указатели
 - malloc/free
- Неопределённое поведение

Пример. Обратимся к массиву из двух элементов.

```
double fun(int i){  
    volatile double d[1]={3.14};  
    volatile long int a[2];  
    a[i] = 1073741824;  
    return d[0];  
}
```

Результат:

```
fun(0)→ 3.14  
fun(1)→ 3.14  
fun(2)→ 3.139998664856  
fun(3)→ 2.0000061035156  
fun(4)→ 3.14, а затем segmentation fault
```

Четвертая проблема – **производительность**.

- Память имеет многоуровневую, иерархическую структуру
- Скорость работы программы зависит от порядка выполняемых действий

Пример. Рассмотрим две программы для умножения матриц. В сущности они делают одно и то же, однако вследствие того, что циклы в них переставлены местами, время работы данных программ будет разным.

Первая программа имеет вид:

```
void copyij(int src[2048][2048], int dst[2048][2048]) {  
    int i, j;  
    for (i = 0; i < 2048; i++)  
        for (j = 0; j < 2048; j++)
```

```
        dst[i][j] = src[i][j];  
    }
```

Вторая программа:

```
void copyji(int src[2048][2048], int dst[2048][2048]) {  
    int i, j;  
    for (j = 0; j < 2048; j++)  
        for (i = 0; i < 2048; i++)  
            dst[i][j] = src[i][j];  
}
```

будет работать в 20 раз медленнее.

Более того, известен случай, когда время работы подобной программы было улучшено в 161 раз за счёт грамотной реорганизации вычисления. В этом случае матрицы были приведены к блочной структуре, размеры блоков были подобраны таким образом, чтобы они целиком находились в кэше, число блоков определенным образом распределялось между вычислительными ядрами.

Пятая проблема – **безопасность**. В программе могут обрабатываться так называемые чувствительные данные (персональные данные, банковская информация и т.д.). Например, мы вводим некоторый пароль. Под размещение этого пароля выделяется некоторая память. После работы с этим паролем его содержимое удаляется. Современный компилятор, просматривая потоки данных внутри программы и то, как управление передается от одного оператора к другому, может решить, что операция `memset` является лишней (если мы освобождаем кусок памяти, далее мы с ним ничего не собираемся делать). Получается, что пароль продолжает жить в памяти. Далее этот участок памяти может появиться в какой-то другой программе.

- Оптимизации компилятора способны нарушать безопасность кода

```
memset(password, '\0', len); -03  
free(password);                ⇒ free(password);
```

- Язык высокого уровня не специфицирует существенные для безопасности аспекты

```
int a[2], b;  
memset(a, 0, 12);
```

- Соответствие исходного и исполняемого кода

– Необходим доступ к системе сборки и конфигурационным файлам

- Исходный код может быть просто недоступен

Зачем нужен язык ассемблера

- Язык ассемблера позволяет понимать поведение машины
- Системное программное обеспечение создается людьми, в том числе выпускниками ВМК
 - Компиляторы и многие другие инструменты разработки программ
 - Операционные системы
 - Средства защиты информации
- Отладка ошибок разработчиком программы
- Настройка производительности программы
 - Почему оптимизация программы компилятором не дает ожидаемого результата
- Безопасность программного обеспечения (software security, cyber security)
 - Сертификация программ
 - Средства защиты
 - Вредоносный код: ботнеты, вирусы/черви, руткиты, и т.д.

История первых компьютеров

- Чарлз Бэббидж, Ада Лавлейс 1820-1833
- Конрад фон Цузе
 - Z1 1938г., Z2 1939 г., Z3 1941г., Z4 1945г.
- Экерт и Моучли
 - ENIAC 1946 г.
 - фон Нейман
 - * First Draft of a Report on the EDVAC, 30 июня 1945
- Атанасов и Берри
 - ABC 1942 г.
- Mark I
 - 1949 г.

- Исаак Брук
 - М-1 1951 г.
- Сергей Лебедев
 - МЭСМ 1950 г.
 - БЭСМ-1 1952 г.

Применение первых компьютеров:

- задачи математической физики
- шифрование/дешифрование сообщений
- управление (военной) техникой

Современное применение компьютеров:

- хранение информации и предоставление удобного доступа для работы с ней
- игры
- передача информации
- управление технологическими процессами
- работа со всем, что нас окружает

Архитектура фон Неймана

Рассмотрим принципы фон Неймана. Первый принцип – информация хранится в двоичном виде. Пусть есть некоторое физическое механическое устройство, которое хранит в себе один бит информации. Например, пусть есть некоторое основание, на котором расположен шарнир. Укрепленная на этом механизме штанга может находиться только в одном конкретном положении (принимать только одно определенное значение). Если взять достаточное количество подобных устройств, мы сможем хранить нужный объем информации в виде вектора подобных устройств. Однако, такие устройства большие, работа с ними неудобна. Более того, нужно уметь быстро перезаписывать, считывать информацию и др., поэтому подобная система оказывается неудобной. На смену механической модели приходит электрическая. Данная модель компактна и с нее удобно считывать.

Все данные и команды, с которыми мы работаем, записываются в оперативной памяти. То есть, есть некоторое запоминающее устройство, где все хранится. Данное устройство можно условно представить в виде некоторого массива элементов. Однако данные

элементы – не отдельные биты, а, например, октеты (то есть байты). Соответственно запись в виде двоичной информации удобно делать не в виде последовательности нулей и единиц, а в шестнадцатиричном виде. Тогда вектор принимает более компактный вид, с которым удобно работать.

Отметим, что физическая память физически разделена с тем, где происходят вычисления. Сообщения между процессором и памятью осуществляется через шину. В процессоре можно выделить две основных компоненты – управляющее устройство (руководит ходом работы процессора) и арифметико-логическое устройство (выполняет преобразование информации). Для локального хранения используют регистры.

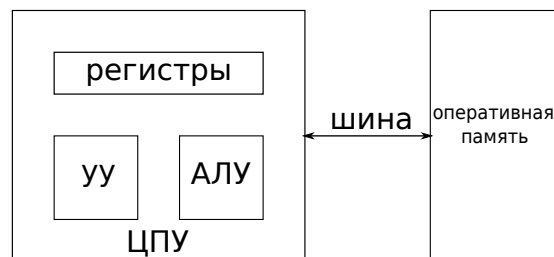


Рис. 1.2: Иллюстрация к объяснению

Чтобы забрать или вызвать что-то из оперативной памяти, нужно через шину указать адрес – номер ячейки. При этом потенциальная адресуемость всегда соответствует некоторой степени двойки.

Команды выполняются (по умолчанию) последовательно. Пусть где-то в памяти лежат числа: 89 C8 8b 15 .. Внутри управляющего устройства есть служебный регистр – счетчик команд – в котором хранится адрес текущей команды, которую он хочет выполнить. Согласно значению счетчика команд, процессор получает числа, разбирается, какой команде соответствуют данные числа, после чего счетчик команд передвигается на следующую адресуемую величину и продолжает выполнение.

Обращение к регистру выполняется с помощью имен. На самом деле машина никаких имен не знает. В коде команды есть определённая группа битов, которой соответствует определённый регистр. Однако для человека такая запись неудобна, поэтому существует также текстовая запись. Ниже представлены две формы записи одних и тех же команд:

```
89 c8                |mov eax, ecx
8b 15 20 20 00 00    |mov edx, dword [0x2021]
c7 03 41 00 00 00    |mov dword [ebx], 'A'
```

Поэтому любая ассемблерная команда имеет следующую структуру: КОП (код операции), ОП1, ОП2 ... (последовательность операндов).

Также в команде в виде операнда может встретиться некоторая константная величина или то, что кодируется непосредственно в теле самой команды (см. третью строку приведенного выше примера).

Итак, операндами могут быть:

- регистры
- память
- константы

Таким образом, **принципы фон Неймана** можно записать в виде

1. Двоичное кодирование информации
2. Неразличимость команд и данных
3. Адресуемость памяти
4. Последовательное выполнение команд

Другие варианты архитектур

Существует также так называемая гарвардская архитектура, в которой код программы и данные, с которыми он работает, физически разнесены. В результате появляется две шины, вследствие чего можно одновременно закидывать в процессор команду и данные. Если появляется ошибка и мы ошибочно пишем не в то место памяти, то мы испортим только данные (в отличие от фон Неймановской архитектуры).

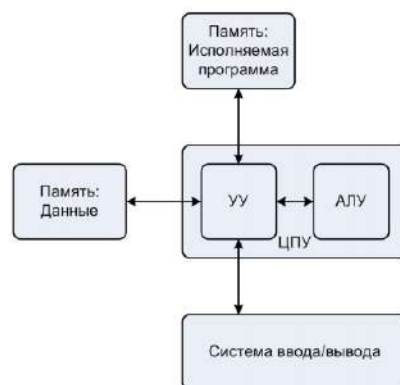


Рис. 1.3: Гарвардская архитектура

Отметим также, что зачастую, говоря об архитектуре фон Неймана, указывают систему ввода-вывода, что указано на рис. 1.4.

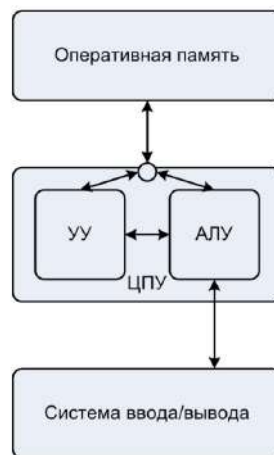


Рис. 1.4: Вариант изображения архитектуры фон Неймана

Модельный цикл работы ЭВМ

Процессор машины (в случае фон Неймановской архитектуры) выполняет цепочку действий. Эти действия условно группируются на некоторое количество шагов. Рассмотрим их:

1. Извлечение инструкции из памяти

Используя текущее значение счетчика команд, процессор извлекает некоторое количество байт из памяти и помещает их в буфер команд

2. Декодирование команды

Процессор просматривает содержимое буфера команд и определяет код операции и ее операнды. Длина декодированной команды прибавляется к текущему значению счетчика команд

3. Загрузка операндов

Извлекаются значения операндов. Если операнд размещен в ячейках памяти — вычисляется исполнительный адрес.

4. Выполнение операции над данными

5. Запись результата

Результат может быть записан в том числе и в счетчик команд для изменения естественного порядка выполнения

Рассмотрим подробнее третий пункт. Пусть есть некоторое запоминающее устройство из восьми разрядов. В случае работы с данными значения должны быть преобразованы в напряжения той или иной величины (см. рис. 1.5).

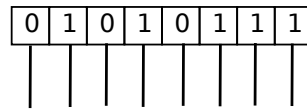


Рис. 1.5: Иллюстрация к объяснению

При этом, если мы обращаемся в память, нам нужно сформировать так называемый исполнительный адрес, то есть тот адрес, с которого у нас будет тянуться исполнение. Этот адрес закодирован внутри команды.

Ключевые понятия и термины

- **Регистр** – запоминающее устройство, которое физически присутствует непосредственно внутри процессора, в виде ассемблерной инструкции он идентифицируется по своему имени. Число элементов внутри регистра определяет его разрядность. То, что записано в регистре естественно интерпретировать в виде некоторого числа. В большинстве машин данный битовый вектор может интерпретировать разными способами.
- **Память** – линейный адресуемый массив. Адресация идет кусками – байтами. В случае архитектуры фон Неймана вне зависимости от того, по какому адресу мы обращаемся, время доступа (физическое время, необходимое для того, чтобы считать содержимое ячейки и через шину процессору) будет одинаковым. Однако в реальных машинах это не так (время доступа будет разным).
- **Цикл** – это то, что делает машина, обрабатывая каждую команду. Тактовую частоту можно условно соотнести с тем, сколько таких циклов за секунду компьютер успевает выполнить. **Счетчик команд** отвечает за то, какая команда выполняется следующей.
- **Адресность** – это то, сколько операндов записано внутри команды.
- **Исполнительный (действительный) адрес** (effective (executive) address) – адрес, на котором проявляется некий эффект работы команды.

Замечание. Битовый вектор, о котором говорится в первом пункте приведенного списка, можно интерпретировать как беззнаковое число:

$$U(\vec{x}) = \sum_{i=0}^{N-1} 2^i x_i \quad (1.1)$$

или можно использовать знаковую интерпретацию:

$$T(\vec{x}) = -2^{N-1}x_{N-1} + \sum_{i=0}^{N-2} 2^i x_i \quad (1.2)$$

Лекция 2. Машина, на которой работает пользовательская программа

Машина, на которой работает пользовательская программа

Рассмотрим 32-разрядную машину, то есть машину, для адресации памяти которой используется 32 разряда. Таким образом, доступное пространство оперативной памяти, куда можно писать, имеет размер 4Гб. Условное изображение памяти можно представить в виде:

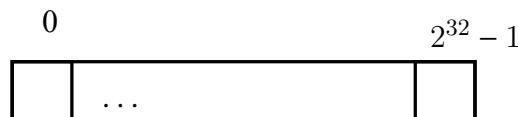


Рис. 2.1: Иллюстрация к объяснению

Адресация в данной машине байтовая. В указанной памяти, в непрерывном массиве программа держит все, что к ней относится (код, данные, статические и динамические переменные и др.). Операционная система отвечает за разделение ресурсов, при этом она делает так, что для каждой отдельно взятой пользовательской программы аппаратура представляется в следующем виде (как будто она одна работает на процессоре и использует оперативную память):

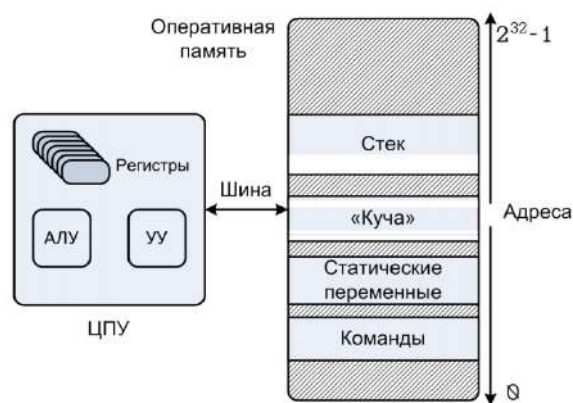


Рис. 2.2: Архитектура IA-32

Отметим, что несмотря на то, что все байты в указанных четырех гигабайтах потенциально адресуемы (можно обращаться к любой точке памяти), на самом деле операционная система будет позволять обращаться только к определённым диапазонам адресов.

Регистры, которыми мы можем распоряжаться, 32-разрядные. Регистров общего назначения всего восемь (см. рис. 2.3).

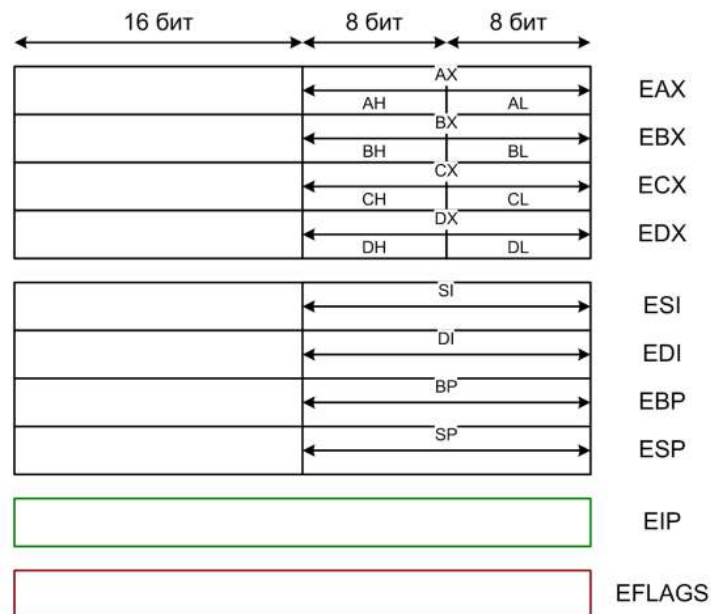


Рис. 2.3: Регистры

Рассмотрим один регистр – EAX (см. рис. 2.4). Содержимое регистра – разряды, то есть в данном случае некоторые запоминающие устройства, которые запоминают по одному биту.

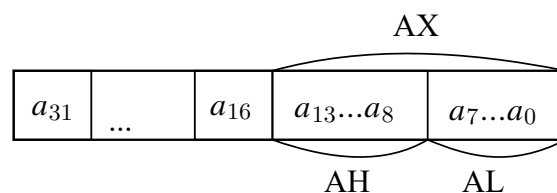


Рис. 2.4: Иллюстрация к примеру

Ранее предполагалось, что у каждого регистра есть свое предназначение. Поэтому вторая буква в названии регистра обозначала это предназначение. Например, буква А в EAX, означает accumulator, В в EBX – base и так далее. Сейчас нет ограничений на использование регистров, поэтому нет необходимости запоминать данную информацию.

В современных процессорах регистров зачастую гораздо больше. Это удобно для компилятора, для того, чтобы держать данные на регистрах рядом с арифметико-логическим устройством. Чем больше регистров, тем меньше обращений в память. При этом лучше делать регистры унифицированными в плане использования. Тогда их можно комбинировать для составления сложных выражений.

Есть пара служебных регистров, которые можно отнести к управляющему устройству – счетчик команд (EIP) и EFLAGS (в этом регистре можно сохранять некоторые побочные эффекты от того, как та или иная программа отработает).

Порядок размещения байт в памяти

Пример. Рассмотрим конкретный пример из четырех команд. Рассмотрим часть кода:

```
mov eax, ecx  
mov eax, dword [0x2019]  
add eax, 0xff  
mov dword [ebp-4], eax
```

Закодируем текстовую запись четырех команд (см. рис. 2.5).

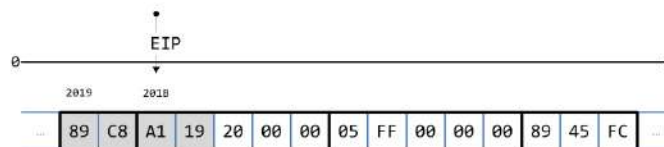


Рис. 2.5: Иллюстрация к примеру

В какой-то момент времени EIP указывает на первую команду. Этой команде будет соответствовать исполняемый адрес 2019. Далее должен запускаться некоторый такт работы: извлечение команды, ее декодирование, получение операндов. Как только произошло декодирование, нужно, определив размер команды, нужно переставить счетчик команд на следующую позицию. Таким образом, после этапа декодирования счетчик принимает значение 201B.

В первой команде `mov` происходит пересылка. Отметим, что в
`MOV EAX, ECX`

`MOV` – код операции, `EAX`, `ECX` – первый и второй операнды. Возникает вопрос, в каком порядке рассматриваются операнды. Несмотря на то, что в машине язык ассемблера один и тот же, есть два разных синтаксиса, которые в текстовом виде описывают, что делает команда. Первый синтаксис – Intel. В этом случае, если что-то происходит, то результат попадает в первый операнд (в данном примере в `EAX`). Тогда (в данном примере) команда `MOV` делает пересылку: она берет значения второго операнда (`ECX`) и отправляет в первый операнд. При этом содержимое первого операнда безвозвратно утрачивается. После копирования `EAX` содержит в себе тот же битовый вектор, что и `ECX`.

Опишем в формульном виде то, что может быть в качестве операндов:

$$MOV\ r/m\ \frac{8}{16},\ r/m/i\ \frac{8}{32},\ m, m \quad (2.1)$$

Всего существует три типа операндов:

- Регистр r
- Память m

- Константа i

Размеры операндов:

- byte 8
- word 16
- dword 32
- qword 64

Если речь идет о пересылке, то данные нужно отправлять куда-то: данные могут быть запомнены и сохранены, поэтому это или регистр или память. Отправлять можно из регистра или из памяти или же использовать прямо записанную в команде константу. После букв, которые специфицируют типы, специфицируются размеры. То есть, выше указано, что размеров может быть три. При этом, каждый раз, когда мы берем некоторый размер, далее (то есть после $r/m/i$) размер должен быть соответствующим (если мы пересылаем битовые вектора, информация не может потеряться или расширяться).

Рассмотрим вторую команду пересылки: `mov eax, dword [0x2019]`. Здесь операнд назначения – EAX, а второй операнд – память. При работе с памятью нужно задавать адрес и число извлекаемых байтов. Здесь спецификатор `dword` означает, что извлекается 4 байта.

Возникает вопрос: в каком порядке в памяти расположены байты и в каком порядке они будут составлены после пересылки? Два наиболее распространенных подхода

- Порядок от младшего к старшему (англ. *little-endian*) **используется в IA-32**
- Порядок от старшего к младшему (англ. *big-endian*) как правило используется в процессорах, предназначенных для обработки сетевых данных

В данной машине используется порядок от младшего к старшему, то есть в начале идет самый младший байт, а далее по старшинству. Таким образом, если идет пересылка, мы просто разворачиваем список. То есть, после первой пересылки получаем значение EAX:

$0x\ 19\ a_1\ c_8\ 89$

Если бы пересылка выполнялась бы с соседнего адреса, то есть с 201B, мы бы получили:

$00\ 20\ 19\ a_1$

В других процессорных архитектурах можно встретить обратный порядок следования. Однако такое встречается очень редко, так как такой прием неудобен. Пусть есть некоторое 32-разрядное число, которое лежит где-то в памяти. Есть некоторый адрес

[x], по которому мы к нему обращаемся. Если сначала идет младший байт, то, чтобы от 32-разрядного перейти к 16-разрядному, нужно вместо `dword[x]` написать `word[x]`. То есть получается, что тип данных сузился.

Команда сложения

Третья команда в рассматриваемом примере – команда сложения. Запишем формат этой команды:

$$ADD\ r/m\ \frac{8}{16},\ r/m/i\ \frac{8}{16},\ \underline{m},\ \underline{m}$$

В данном случае есть всего два регистра, причем сложение осуществляется следующим образом:

$$OP1 \leftarrow OP1 + OP2$$

То есть то, что было в первом операнде, безвозвратно теряется.

Таким образом, мы рассмотрели второй способ задания операнда. При этом заметим, что сложение регистра аккумулятора кодируется кодом операции 05, а второй операнд, который является непосредственно закодированной константой, находится дальше в теле кода (то есть в данном примере `ff 00 00 00`). В тот момент, когда команда выполняется и извлекается через шину памяти внутрь процессора, вместе с пересылкой самого тела команды также копируется распоряжение и значение того операнда, с которым мы будем работать.

Отметим такое, что указанная константа будет расширена до нужной длины, прежде чем произойдет сложение (так как складываются битовые вектора одинакового размера). При этом битовые вектора, которые мы складываем, могут интерпретироваться как знаковые числа или как беззнаковые числа (из-за особенностей кодирования разницы нет).

Интерпретация конструкции с командой `add` с незадаанным размером

Рассмотрим команду

```
add eax, [x]
```

где `x` – некоторый адрес. Однако в данной конструкции не задан размер. В этом случае программный ассемблер может посмотреть на размер первого операнда и подставить в указанную строку спецификатор размера так, чтобы запись была корректна.

Рассмотрим запись вида

```
add [x], 1
```

то есть мы берем некоторую память и прибавляем к ней константу. Однако в этом случае программный ассемблер выдаст ошибку, так как в этом случае неясно, о каком

размере идет речь. Чтобы исправить это, нужно у одного из двух (или у обоих) операндов приписать спецификатор размера.

Команда пересылки

Ранее мы рассматривали ситуации обращения к памяти, прямо указав тот адрес, с которого мы хотим что-то считать. Однако в последней команде рассмотренного примера в квадратных скобках указано имя еще одного регистра (`mov dword [ebp-4], eax`). Оказывается, что этот адресный код (код, который описывает операнд, с которым мы хотим работать) может быть самым разнообразным. В общем виде конструкция внутри квадратных скобок может быть следующей:

$$[\text{рег_база} + \text{масштаб} * \text{рег_индекс} + \text{смещение}]$$

где `рег_база` – некоторый регистра, к которому можно прибавить `рег_индекс`. перед указанными квадратными скобками должен быть спецификатор размера.

Таким образом, при после первой пересылки осталось только смещение. В последней же рассматриваемой команде появился регистр и смещение -4. В некоторых случаях мы не знаем заранее, где нужно держать переменные. В этом случае нужно на ходу определять значение некоторого регистра.

Типы адресации

Адресация операндов:

1. прямой
2. непосредственный
3. косвенная

Прямой способ адресации был рассмотрен в примере в первой команде (напрямую указывали имена регистров), во второй команде (напрямую указывали по имени первый регистр и адрес той ячейки памяти, куда будем обращаться). Термин адресация определяет то, как записывается (в двоичном коде) то, откуда мы будем брать данные.

Непосредственный способ встретился нам в третьей команде. В теле этой команды мы непосредственно записали значение одного из операндов.

Косвенная адресация заключается в следующем. Сразу же после того, как команда была декодирована, мы не знаем, с чем мы будем работать, так как необходим отдельный этап выполнения внутри машины, когда мы сможем сформировать исполнительный адрес, по которому будем обращаться в память. Для его формирования нужно обратиться к

регистру, считать его значение, произвести вычисление согласно указанной конструкции, получить некоторое число (с обрезанием результатов внутри соответствующей разрядной сетки). Таким образом, все вычисления в итоге выдают 32-разрядное число. После получения исполнительного адреса производится считывания регистров, который позволяет в рамках косвенной адресации задать нужное. Работа с такой адресацией предполагает дополнительное обращение к запоминающим устройствам.

Сопоставим способы адресации с типами операндов. В случае прямой адресации – регистр/память, в случае непосредственной адресации – константа, в случае косвенной адресации – память.

IA-32

Условно развитие архитектур можно записать так: 8086 → IA – 3 → Intel64. Изначально архитектура имела вид, представленный на рис. 2.6. Далее архитектура расширилась (рис. 2.7). Современный процессор в сильном упрощении можно представить в виде, указанном на рис. 2.8.

«Положительные» особенности IA-32:

- Особенности аппаратуры и операционная система (Windows/Linux) позволяют использовать в программах модель плоской памяти
- Значительно ослаблены ограничения на использование регистров в командах по сравнению с 8086
- «Количественно проще» чем Intel64

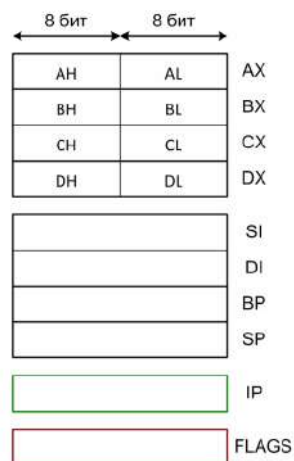


Рис. 2.6: Иллюстрация к объяснению

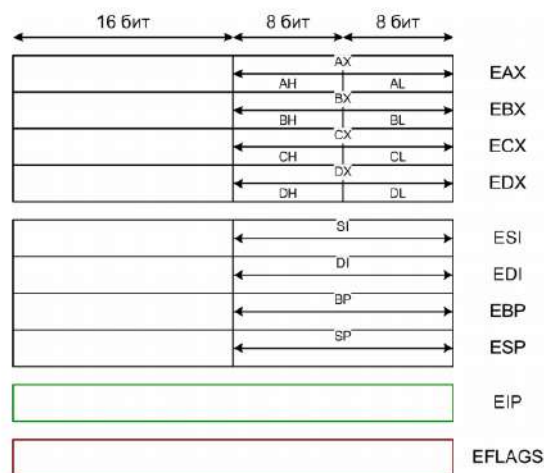


Рис. 2.7: Архитектура IA-32

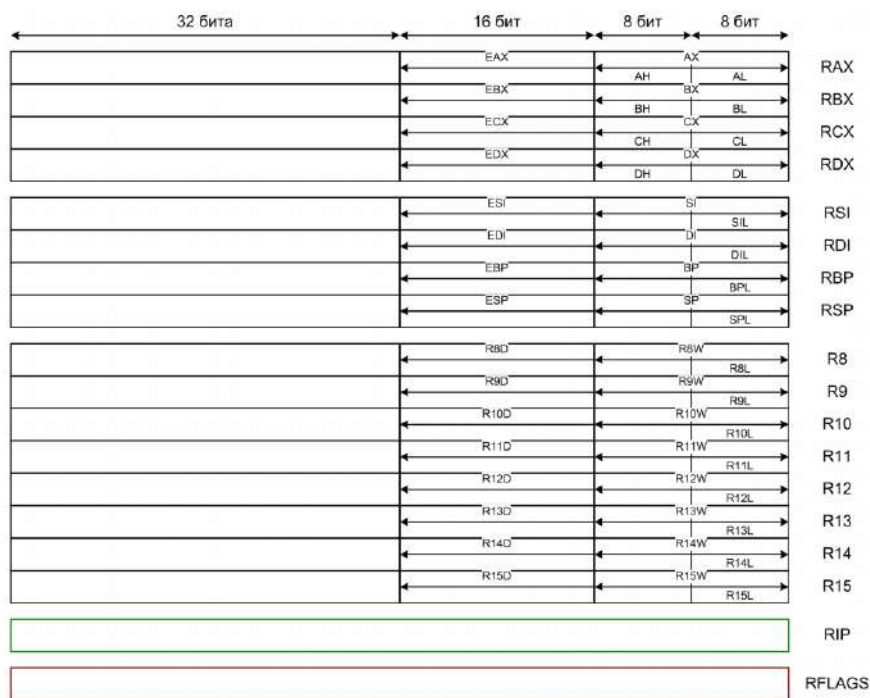


Рис. 2.8: Архитектура Intel64

Примеры

Рассмотрим несколько конкретных примеров. Рассмотрим следующий код на языке Си и сопоставим ему ассемблерные команды, которые будут построены компилятором при выполнении написанного.

```
void f() {
    static int cntr = 0;           // 1
```

```
int x = 2, y = 1, z = 0;    // 2
unsigned short w = 282;    // 3
signed char q = 13;        // 4
++cntr;                    // 5
z = -x + q * w * y - w;    // 6
}
```

Полный листинг, выданный ассемблером для данной функции, будет иметь вид:

```
section .bss
; Резервирование 4 байт памяти
cntr resd 1
section .text
global f
; Точка входа в программу
```

```
f:
push ebp
mov ebp, esp
sub esp, 16
mov dword [ebp-16], 2    ; (1)
mov dword [ebp-12], 1    ; (2)
mov dword [ebp-8], 0     ; (3)
mov word [ebp-4], 282    ; (4)
mov byte [ebp-1], 13     ; (5)
add dword [cntr], 1      ; (6)
movsx eax, byte [ebp-1]  ; (7)
movzx edx, word [ebp-4]  ; (8)
imul eax, edx             ; (9)
imul eax, dword [ebp-12]; (10)
sub eax, dword [ebp-16]  ; (11)
sub eax, edx             ; (12)
mov dword [ebp-8], eax   ; (13)
leave
ret
```

Рассмотрим строки (1)-(13), так как они и будут соответствовать строкам кода 1-6.

Каждый раз, когда появляется точка с запятой (;), все результаты в программе должны быть зафиксированы. В отсутствие оптимизации приведенные выше операторов можно соотнести с какими-то фиксированными командами.

В рассматриваемом коде пять переменных, из которых четыре – автоматические и одна статическая. Изобразим участок памяти (рис. 2.9). В некотором участке памяти должны

быть размещены статические переменные, которые создаются в одном экземпляре на весь срок работы программы. По ходу преобразования языка Си в язык ассемблера будет несколько этапов обработки. В итоге внутри адресного кода (внутри квадратных скобок) достаточно написать некоторое имя ([Cntr]). Позже, когда программа ассемблер будет строить код, она преобразует символьное имя, соответствующее некоторому адресу, в некоторое число. При этом не будет никаких пересечений с выделенными местами для хранения переменных. Для автоматических переменных на каждый вызов где-то в

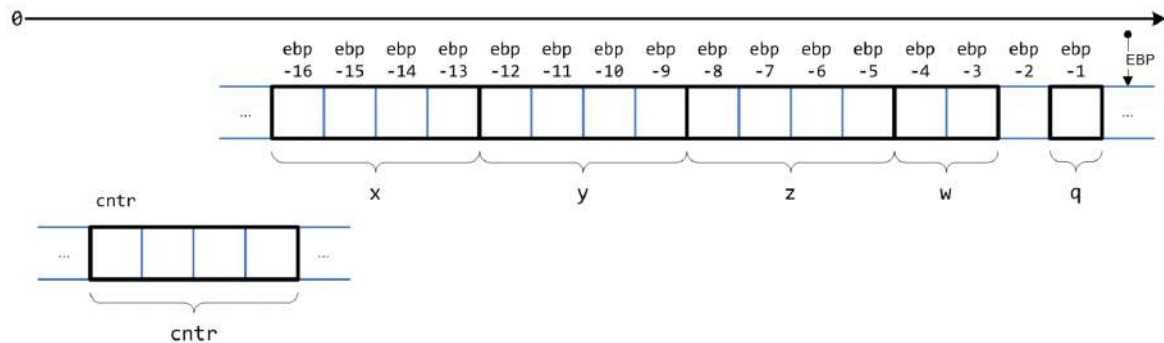


Рис. 2.9: Иллюстрация к примеру

памяти должно быть выделено место. При этом пространство должно быть распределено таким образом, чтобы эти переменные не налезали друг на друга. Это распределение за нас делает компилятор. При этом переменные содержатся в памяти следующим образом:

X	ebp-16
y	ebp-12
z	ebp-8
w	ebp-4
q	ebp-1

Команда 1-5 производят инициализацию автоматических локальных переменных. Выполнение второго оператора в коде разбивается на команды (1)-(3). В результате все автоматические локальные переменные получают нужные значения. Рассмотрим пятую строку кода. Оператор ++ в данном случае выражается через add. На самом деле при выполнении данного сложения выполняется два обращения в память. Первое обращение – когда после декодирования получаем значения операндов. Обращаемся в память на чтение, считаем значение переменной, помещаем это значение в процессор, прибавляем единицу, после чего новое значение отправляем по шине обратно в память.

Выполнение шестой строки – выполнение большого выражения. Все вычисляемые величины можно держать на регистрах как временные вспомогательные величины. После того, как вычисления завершены, результат отправляется обратно в память.

Лекция 3. Устройство ассемблерной программы

Пример

На прошлой лекции мы рассмотрели конкретный пример работы части кода на языке Си. Продолжим рассмотрение этого примера. В указанном участке кода были определены пять переменных, которые должны создаваться каждый раз при вызове функции. Поэтому адреса этих переменных нужно соотносить с некоторым динамически определяемым адресом, в вычислении которого участвует один из регистров:

x	ebp-16
y	ebp-12
z	ebp-8
w	ebp-4
q	ebp-1

Сборка программы происходит по схеме, указанной на рис. 3.1.

При вызове программы через командную строку или графическую среду разработки, эта программа через себя вызывает целую цепочку других программ в определённой последовательности. Эти программы связаны между собой через вход и выход программы. Препроцессор обрабатывает полученный код (на языке Си), находит директивы по включению других заголовочных файлов в которых определены типы и собирает весь Си-код, который должен быть скомпилирован. После этого включается компилятор, который смотрит на описание функций и типов и строит ассемблерный листинг.

Рассмотрим объявление и инициализацию переменной:

```
\int x=z;
```

результат выполнения си-компилятора:

```
mov dword[ebp-16],2
```

Далее программа ассемблер транслирует такую текстовую форму в двоичное представление:

```
c7 45 f0 02000000
```

Такую форму записи уже принимает машина, однако прежде чем эти байты сохранятся в процессор, пройдет еще несколько этапов (программа собирается не мгновенно, а частями, за что отвечает компоновщик).

На прошлой лекции мы остановились на рассмотрении шестой строчки кода:

```
void f() {  
    static int cnt = 0;           // 1
```

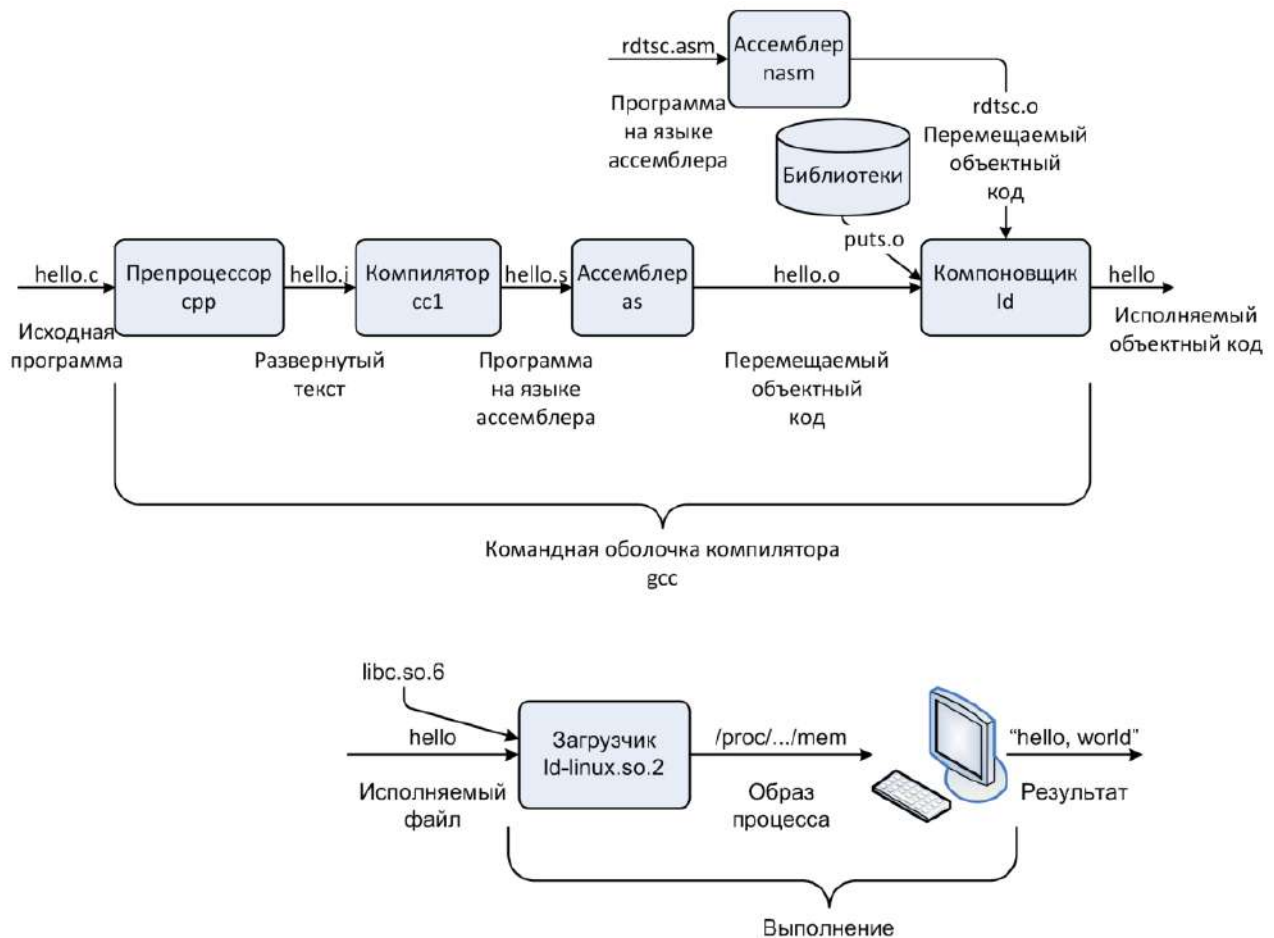


Рис. 3.1: Иллюстрация к объяснению

```
int x = 2, y = 1, z = 0;    // 2
unsigned short w = 282;    // 3
signed char q = 13;        // 4
++cntr;                    // 5
z = -x + q * w * y - w;    // 6
}
```

Листинг, выданный ассемблером для данной функции, будет иметь вид:

```
...
mov dword [ebp-16], 2      ; (1)
mov dword [ebp-12], 1      ; (2)
mov dword [ebp-8], 0       ; (3)
mov word [ebp-4], 282      ; (4)
mov byte [ebp-1], 13       ; (5)
add dword [cntr], 1        ; (6)
movsx eax, byte [ebp-1]    ; (7)
```



```
movzx edx, word [ebp-4]    ; (8)
imul eax, edx              ; (9)
imul eax, dword [ebp-12]   ; (10)
sub eax, dword [ebp-16]    ; (11)
sub eax, edx               ; (12)
mov dword [ebp-8], eax     ; (13)
...
```

Современные компиляторы заточены на то, чтобы перестраивать код так, чтобы он работал максимально быстро. Форма представления преобразования данных каких-то выражений удобна для восприятия человеком, но не для восприятия машиной. Поэтому в компиляторе есть свои специализированные представления, с которыми он работает. Эти представления условно можно разделить на две группы: то, что строится в начале и то, что строится потом для анализа. Построим дерево, которое будет соответствовать вычислению рассматриваемого выражения (рис. 3.1).

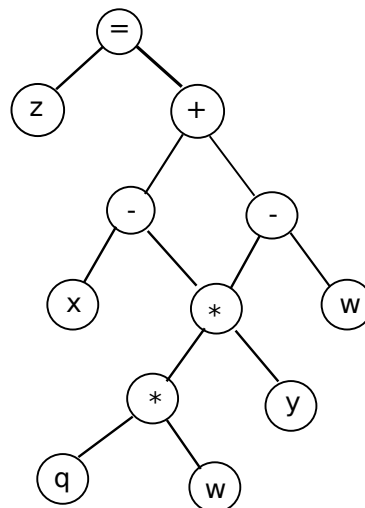


Рис. 3.2: Иллюстрация к вычислению выражения

Язык Си предполагает, что если прописана некоторая операция, то в подавляющем большинстве случаев она принимает на вход величины одного типа и выдает на выходе величину такого же типа. Однако в указанном дереве вычислений мы имеем листья разных типов. Поэтому компилятор будет делать неявное приведение типов. Изменим дерево следующим образом (рис. 3.3). Учтем, что w – беззнаковая переменная и приводить ее к типу, в котором она сможет гарантированно храниться, нужно одним способом, который мы назовем U . Переменная q – знаковая. Ее старший разряд будет интерпретироваться как степень двойки с минусом. Преобразование над этой переменной обозначим S .

После того, как данная конструкция попадет к компилятору, она будет перестроена в соответствии с внутренними критериями компилятора. В результате получим дерево,

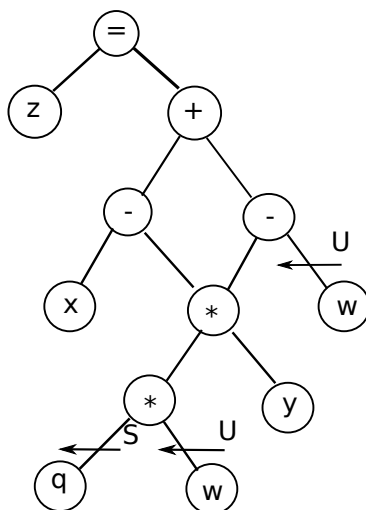


Рис. 3.3: Иллюстрация к объяснению

изображенное на рис. 3.4. Унарный минус будет отброшен. Отметим, что в данном дереве имеются два одинаковых поддерева (на рис. 3.4 они обведены).

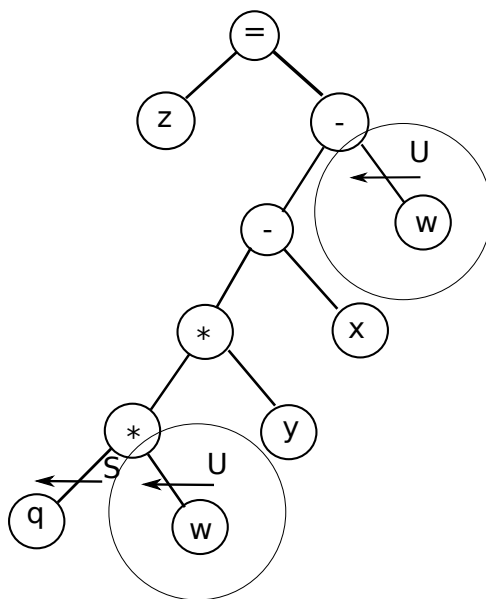


Рис. 3.4: Иллюстрация к объяснению

В листинге ассемблера, в седьмой строчке, чтобы получить на выходе (после умножения) переменную типа `int`, нужно рассматривать переменные того же типа. Для этого нужно преобразовать один байт в четыре байта. На прошлой лекции мы рассмотрели команду пересылки данных `move`, которая умеет поразрядно пересылать данные. Однако здесь нам нужно добавить еще разрядов. Для этого используют команду `MOVSX`, которая расширяет один битовый вектор из второго операнда до вектора большего размера.

```
MOVSX R \frac{16}{32}, r/m 8  
      R 32, r/m 16
```

ОП1 $\xleftarrow{s}$ ОП2

Поэтому, если в переменной q старший (седьмой) разряд – 0, то при пересылке в регистр eax все остальные 24 разряда тоже будут 0. Если старший разряд – 1, то он копируется по всем остальным 24 старшим разрядам, так, чтобы в дополнительном коде мы снова получили то же отрицательное число.

Восьмая строка представляет собой аналогичную операцию с небольшими отличиями. Величину, забранную из переменной w , отправляет в регистр edx и расширяет беззнаковым образом.

Теперь можем перейти к умножению. Двухадресная команда умножения целых чисел представляется в виде:

```
imul R \frac{16}{32}, r/m \frac{16}{32}
```

ОП1 $\leftarrow$ ОП1 * ОП2

Аспекты, присущие команде сложения, справедливы и для вычитания.

Рассмотрим двенадцатую строку. Воспользуемся тем, что регистров достаточно количество, поэтому та часть поддерева, которую мы считаем и запоминаем в регистре eax (команда (9)-(12)) также находится и в edx . Поэтому, не проводя повторное расширение, можно произвести вычисление.

Таким образом, строчки, которые соответствуют вычислениям дерева, можно обозначить на рис. 3.5.

Тринадцатая команда (пересылка) фиксирует побочный эффект выполнения присвоения (все, что мы посчитали, должно обновить значение переменных).

Программа может собираться из нескольких модулей. Поэтому, пока мы не видим, весь объем программы, из чего она будет собрана, трудно решить, по каким адресам нужно располагать статические переменные. Поэтому ассемблер сгенерирует дополнительную служебную информацию, которая будет описывать, что нужно сделать в отношении доопределения адресов.

После того, как программа будет собрана целиком, она отправляется на исполнение. Отдельная программа загрузчик будет копировать содержимое файла в определённые адреса памяти, после чего управление передается в некоторый адрес и начинается исполнение.

Обратный путь

Рассмотрим обратный путь – от исполняемого кода к читаемому тексту. В Linux есть пакет Binutils, который позволяет работать с двоичными файлами. Рассмотрим

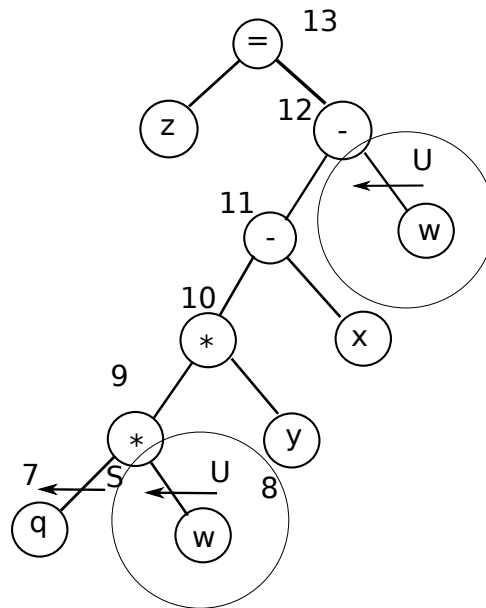


Рис. 3.5: Иллюстрация к объяснению

следующую ситуацию. Текстовый файл, в котором был листинг отдельной функции, транслировали и перевели в модуль с перемещаемым объектным кодом.

```
snoop@earth:~/samples$ nasm -f elf -o sample1.o sample1.asm
snoop@earth:~/samples$ objdump -M intel -d sample1.o
```

Здесь ключ -d означает дизассемблировать содержимое данного файла, ключ -M intel – хотим получить текстовую выдачу согласно синтаксису intel. Эта программа просматривает содержимое указанного файла и переводит двоичную запись в текстовую. В результате получим:

0: 55	push ebp
1: 89 e5	mov ebp,esp
3: 81 ec 10 00 00 00	sub esp,0x10
9: c7 45 f0 02 00 00 00	mov DWORD PTR [ebp-0x10],0x2
10: c7 45 f4 01 00 00 00	mov DWORD PTR [ebp-0xc],0x1
17: c7 45 f8 00 00 00 00	mov DWORD PTR [ebp-0x8],0x0
1e: 66 c7 45 fc 1a 01	mov WORD PTR [ebp-0x4],0x11a
24: c6 45 ff 0d	mov BYTE PTR [ebp-0x1],0xd
28: 81 05 00 00 00 00 01	add DWORD PTR ds:0x0,0x1
2f: 00 00 00	
32: 0f be 45 ff	movsx eax,BYTE PTR [ebp-0x1]
36: 0f b7 55 fc	movzx edx,WORD PTR [ebp-0x4]
3a: 0f af c2	imul eax,edx
3d: 0f af 45 f4	imul eax,DWORD PTR [ebp-0xc]

```
41: 2b 45 f0          sub    eax,DWORD PTR [ebp-0x10]
44: 29 d0             sub    eax,edx
46: 89 45 f8          mov    DWORD PTR [ebp-0x8],eax
49: c9               leave
4a: c3               ret
```

Самый левый столбец – смещение двоичной записи от начала функции (мы начали с какой-то команды, которая лежит по нулевому смещению от начала функции). Видно, что первая команда была однобайтовая, вторая команда лежит по смещению один, она двубайтового размера. Со смещения 9 начинается код, который мы разбирали выше. 28 смещение соответствует сложению.

Устройство ассемблерной программы

Рассмотрим верхнюю ветку сборки программы, то есть тот ассемблерный код, который мы пишем сами, транслируем его в то, что представляет перемещаемый объектный код.

```
%include 'io.inc'
```

```
section .data
a dw 1
addr dd $
var dd 0x1234F00D
```

```
section .bss
cntr resd 1
```

```
section .text
global CMAIN
CMAIN:
    add dword [cntr], 1
    mov eax, [addr]
    PRINT_HEX 4, eax
    NEWLINE
    PRINT_HEX 4, addr
    NEWLINE
    xor eax, eax
    ret
```

Пусть есть некоторый текстовый файл, который состоит из некоторого количества секций. Далее эти секции будут преобразованы в двоичное содержимое плюс некоторая

служебная информация. При этом перевод текстовой формы в двоичную запись выполняется ассемблером достаточно просто. Каждая строка – это либо текстовая запись какой-то команды, либо директива, которая диктует ассемблеру, что делать. Начало секции обозначается ключевым словом `section`, после него – имя секции. Основные секции, с которыми мы будем работать: `.data`, `.bss`, `.text`. Первые две секции – статические данные. В секции `.text` располагается программный код.

В секции `.text` записаны некоторые команды. Перед ними могут быть метки. Таким образом, можно записать

метка: КОП ОП1,...

Где КОП – код операции, ОП – операнд. Метки в коде требуются, когда нужно явно менять порядок выполнения команд.

Рассмотрим определение переменных:

имя-переменной [:]директива; комментарий

То, что указано, в квадратных скобках, может быть записано опционально (то есть необязательная запись).

В секции `.data` директивы могут быть следующими: `db val1 [,val2, ...]` – определяем один байт

`dw` – определяем 16-разрядное (слово)

`dd` – определяем 32-разрядную информацию

`dq` – определяем 8 байт

Здесь `val1 [,val2, ...]` – значения, которые должны появиться в указанном месте памяти.

Достаточно часто мы хотим что-то изначально инициализировать нулём. Возникает вопрос: нужно ли в этом случае этот ноль хранить? Ответ – нет. В случае секции `.data` при наличии одной из указанных директив действительно нужно генерировать некоторую байтовую последовательность, которую потом из текстовой формы переводим в двоичную запись и сохраняем внутри файла. Применительно же к секции `.bss`, в которой хранятся переменные, инициализируемые нулями, хранение такого содержимого не нужно. Нужно будет только в составе служебной информации записать, что суммарно под секцию `.bss` потребуется какое-то общее пространство (которое учитывает эти нули).

Отличие директив в данном случае будет в следующем: `resb` – определяем один байт

`resw` – определяем 16-разрядное (слово)

`resd` – определяем 32-разрядную информацию

`resq` – определяем 8 байт

Далее, в зависимости от директивы вводим какое-то число `N`, которое указывает, сколько раз мы хотим выделить памяти в указанном количестве.

Пример заполнения секции .data

Рассмотрим память работающей программы, куда после всех преобразований должна попасть секция data, будучи скопированной из общей программы (рис. 3.6). Первый байт соотносится с именем переменной a. Знак \$ означает, что в данное место нужно подставить текущий адрес. Однако, при трансляции текстового файла в двоичную запись мы ещё не знаем этот адрес. Поэтому про это место нужно сделать запись в служебный код. Поэтому пока что место под запись адреса (addr) остаётся пустым.

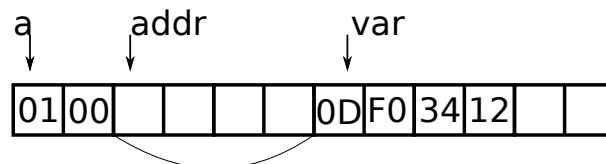


Рис. 3.6: Иллюстрация к примеру

Пусть известно, что после сборки программы секцию .data разместили на некоторый базовый адрес, например на 0x7000000. Теперь мы можем доопределить четыре байта, которые были выделены под адрес. Учтем, что в пустых клетках должен быть адрес места, которое относится к указанной директиве. В итоге получим (рис. 3.7):

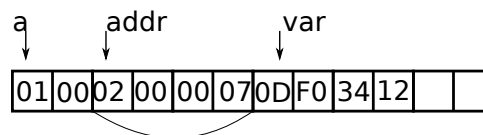


Рис. 3.7: Иллюстрация к примеру

Секция .text

Рассмотрим секцию .text:

```
section .text
global CMAIN
CMAIN:
    add dword [cntr], 1
    mov eax, [addr]
    PRINT_HEX 4, eax
    NEWLINE
    PRINT_HEX 4, addr
    NEWLINE
    xor eax, eax
    ret
```

Аналогично `section` – ключевое слово. После него – имя.text. Далее указывается конструкция `global CMAIN` (чтобы указать данную конструкцию, необходимо в начале включить некоторые дополнительные директивы), которая указывает точку входа. Далее идет метка `CMAIN`, команда сложения, пересылка и четыре команды, позволяющие выводить на печать значения реестров в памяти. Последние две команды соотносятся в языке Си с `return` (возвращает некоторый целочисленный код, по которому можно понять, хорошо отработала программа или нет). Строка `ret` позволяет выйти из функции.

Основные арифметические команды

- `MOV` – пересылка
- `MOVSX`, `MOVZX` – пересылка с расширением
- `ADD`, `SUB`
- `INC`, `DEC` – команды быстрой арифметики
- `NEG`
 - `r/m 8/16/32`
- `MUL`
 - `r/m 8/16/32`
- `IMUL`
 - `r/m 8/16/32`
 - `r 16/32`, `r/m 16/32`
 - `r 16/32`, `r/m 16/32`, `i 16/32`
- `DIV`, `IDIV`
 - `r/m 8/16/32`
- `CBW`, `CWD`, `CDQ`

`INC` – одноадресная команда (работает с одним операндом):

$$\text{inc } r/m \frac{8}{16}{32}$$

$$\text{ОП} \leftarrow \text{ОП} + 1$$

Команда DEC:

$$DEC\ r/m\ \frac{8}{16}{32}$$
$$ОП \leftarrow ОП - 1$$

Следующая одноадресная команда – обращение знака:

$$NEG\ r/m\ \frac{8}{16}{32}$$

$$-^t_w(x) = \begin{cases} -x, x > -2^{w-1} \\ -2^{w-1}, x = -2^{w-1} \end{cases}$$

Здесь t означает, что число представлено в двойном дополнительном коде некоторой разрядности w (то есть в числе w разрядов).

Рассмотрим новую форму команды умножения – MUL – умножение с расширением результата. Если операнд 8-разрядный, то результат будет записываться в регистр AX, причем мы берем от него младший байт, умножаем на операнд и получившееся значение записываем в 16 разрядов:

$$MUL\ r/m\ 8 : AX \leftarrow AL * ОП$$

В случае 16 разрядов:

$$MUL\ r/m\ 16 : DX : AX \leftarrow AX * ОП$$

В случае 32 разрядов:

$$MUL\ r/m\ 32 : EDX : EAX \leftarrow EAX * ОП$$

Отметим, что данное умножение трактует битовые вектора как беззнаковые числа (вследствие расширения).

Пример. Умножим два 8-разрядных числа.

$$0xFF * \overset{U}{8} 0xFF \rightarrow 0xFE01 \quad (3.1)$$

Также существует команда IMUL, у которой три формы допустимых форматов. Первая форма совпадает с тем, что было для беззнакового умножения. В этом случае при умножении тех же величин, что в примере выше, получим

$$0xFF * \overset{t}{8} 0xFF \rightarrow 0x0001 \quad (3.2)$$

Существует третий допустимый формат, который предполагает возможность явного указания константы в виде третьего операнда. В этом случае

$$\text{ОП1} \leftarrow \text{ОП2} * \text{ОП3}$$

Отметим, что в двухадресной форме не может быть константы в качестве второго оператора.

Рассмотрим команды деления `DIV` и `IDIV`. В случае заранее надо определить, знаковые величины или беззнаковые.

$$iDIV\ r/m \quad \frac{8}{16} \quad \frac{32}{32}$$
$$DIV\ r/m \quad \frac{8}{16} \quad \frac{32}{32}$$

При этом при работе с 8-байтовым операндом, мы изначально берем величину, которая по разрядности больше делителя в два раза. Далее мы в одной команде выполняем сразу две операции:

$$AX/\text{ОП} \rightarrow AL \text{ частное}, AH \text{ остаток}$$

Операция деления довольно сложная. Например, если на сложение приходится один такт, то на деление – порядка двадцати или даже больше в зависимости от реализации.

В случае 16-разрядного операнда:

$$DX : AX/\text{ОП} \rightarrow AX \text{ частное}, DX \text{ остаток}$$

В случае 32-разрядного операнда:

$$EDX : EAX/\text{ОП} \rightarrow EAX \text{ частное}, EDX \text{ остаток}$$

В случае знакового деления существует группа команд:

$$CBW\ AX \xleftarrow{s} AL$$
$$CWD\ DX : AX \xleftarrow{s} AX$$
$$CDQ\ EDX : EAX \xleftarrow{s} EAX$$

Лекция 4. Регистр EFLAGS.

Регистр EFLAGS

Основная компонента, которой мы будем пользоваться, чтобы влиять на ход выполнения – это служебный регистр флагов, который находится в управляющем устройстве и запоминает в себе некоторые состояния вычислений, которые происходили при выполнении арифметических и других операций. Например, мы выполнили сложение, получили некоторый результат. Тогда определённые свойства этого результата будут отражаться в указанном регистре. Регистры общего назначения мы рассматривали как битовые вектора, несущие в себе интерпретацию целых или положительных чисел. Регистры флагов следует рассматривать как некоторую совокупность отдельных устройств хранения информации и отдельные разряды или отдельные биты выражают то или иное свойство прошедших вычислений или результата. Сам регистр 32-разрядный (рис. 4.1). Далеко не все разряды несут в себе некоторую информацию (свойств модем быть меньше разрядов). Поэтому часть разрядов всегда равна нулю. Первый разряд всегда равен 1.

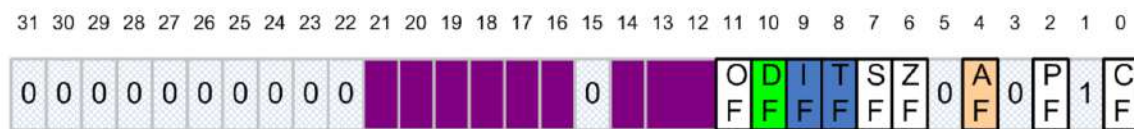


Рис. 4.1: Регистр EFLAGS

На рис. 4.1 фиолетовым цветом выделено то, что относится к функциям управления. Зеленый и синий флаги управляют ходом выполнения некоторых команд, которые позволяют работать с байтовыми строками. Флаг AF относится к так называемой двоично-десятичной арифметике. Помимо десятичного представления чисел, когда битовый вектор рассматривается как коэффициенты перед степенями двойки, существуют другие форматы представления чисел. Когда-то был довольно популярен формат двоично закодированных десятичных чисел (сокращенно BCD). пусть есть один байт. Этот байт делится на две части – каждые четыре бита могут закодировать число от 0 до 15, то есть их вполне достаточно, чтобы закодировать одну десятичную цифру.

Пример.

В привычном для машины виде число преобразуется в

$$16_{10} = 10_{16} = 10000_2$$

Однако, если закодировать указанное число по принципу, описанному выше, получим

совсем друга представление. Если закодировать 1, получим: 0001, а для 6: 0110.

Арифметические FLAGи

Рассмотрим FLAG чётности (PF) (второй разряд). В этом разряде взводится 1, если в младшем байте результата, который мы получили при вычислении предыдущей команды, четное количество единиц. Например, мы выполнили сложение, получили некоторое число 00010110. Здесь три единицы, то есть число нечетное, значит, флаг PF должен быть сброшен в ноль. Отметим, что само число четное, хотя флаг четности сбрасывается в 0.

Флаг ZF взводится в случае нулевого результата.

Флаг SF отвечает за знак результата. Например, рассмотрим некоторое положительное число. Его старший разряд – ноль, тогда и SF – 0. Если же число отрицательное, SF взводится.

Флаг CF – перенос за разрядную сетку. Например, мы осуществляем сложение. Если числа достаточно большие, то нам уже не хватит того числа разрядов, с которым мы работали. То есть, получаем ситуацию беззнакового переполнения. Эта ситуация и запоминается в указанном флаге. Таким образом, если мы наткнемся на переполнение, мы сможем узнать об этом по состоянию флага. Ситуацию переполнения мы будем получать достаточно часто при условии работы с числами, значения которых произвольны и равномерно распределены по всему диапазону представимых чисел.

Переполнение при сложении двух беззнаковых чисел

Рассмотрим числовую матрицу (рис. 4.2).

Числа, которые складываются по 4 разряда x и y пробегают значения от 0 до 15 каждый. Тогда, если цвет клетки белый, то результат представим и не возникает проблем с переполнением. Если же клетка оранжевого цвета, значит складываемые величины слишком большие и результат сложения не входит в четыре разряда (ситуация переполнения).

В трехмерном случае ситуацию переполнения можно представить в виде, представленном на рис. 4.3. Диагональ – граница представимых результатов. Отметим, что картина будет иметь один и тот же вид для разного числа разрядов. Так, на рис. 4.3 слева изображен график в случае 3-х разрядов, а справа – в случае четырех.

FLAG OF

Вернемся к рассмотрению флагов. Флаг OF – знаковое переполнение. Формально оно записывается следующим образом: перенос за разрядную сетку XOR перенос в старший разряд. Данная ситуация соответствует случаю сложения знаковых чисел при условии, что результат представить нельзя. Для машины нужен некоторый алгоритм, который

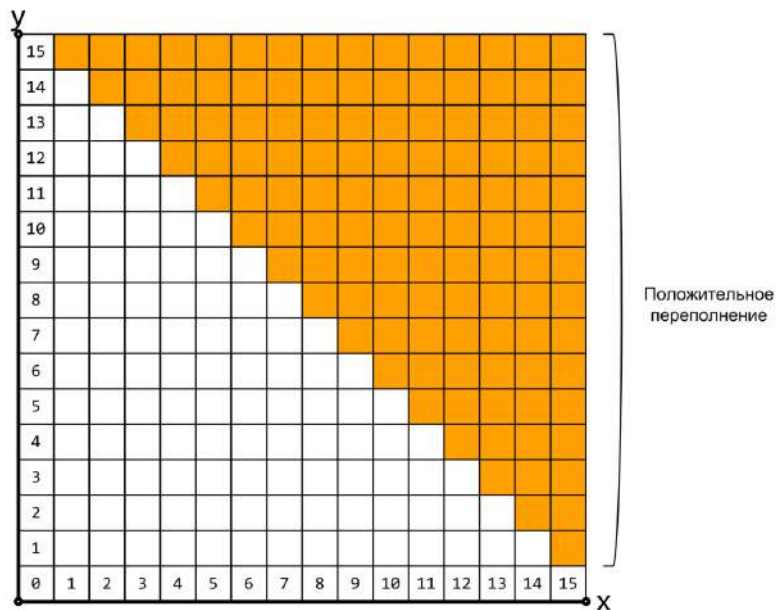


Рис. 4.2: Числовая матрица

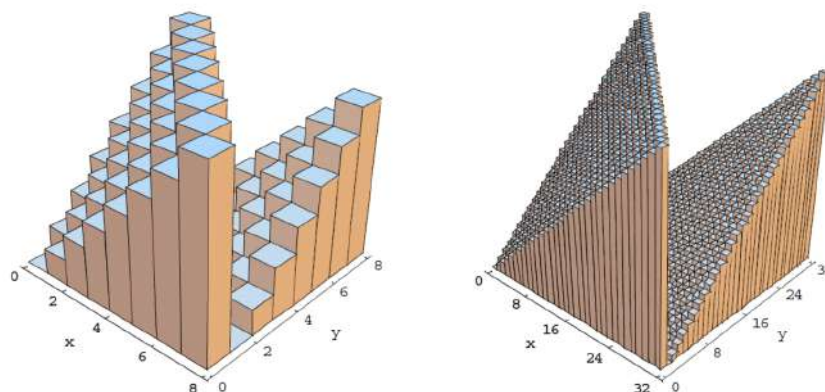


Рис. 4.3: Переполнение в 3D

описывает, что конкретно нужно делать с какими-то отдельными разрядами.

Пример.

Пусть есть некоторые битовые вектора, которые мы складываем. Пусть старшие разряды этих векторов были нулевые. Однако произошел перенос в старший разряд из предпоследнего (рис. 4.4). В результате получим ситуацию переполнения – сложили два положительных числа, а получили отрицательное.

Опишем знаковое переполнение с графической точки зрения. Рассмотрим числовую матрицу для случая сложения двух знаковых чисел (рис. 4.5). Учтем, что число знаковое, поэтому начало координат будет в центре. Правый верхний квадрант – положительные

числа, в нем ситуация аналогична рассмотренному выше случаю для беззнаковых чисел. В противоположной стороне получаем отрицательное переполнение. Отметим, что суммарная площадь зоны переполнения в два раза меньше, чем в рассмотренном выше случае с беззнаковым переполнением.

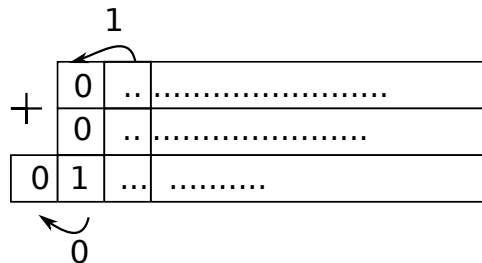


Рис. 4.4: Иллюстрация к примеру

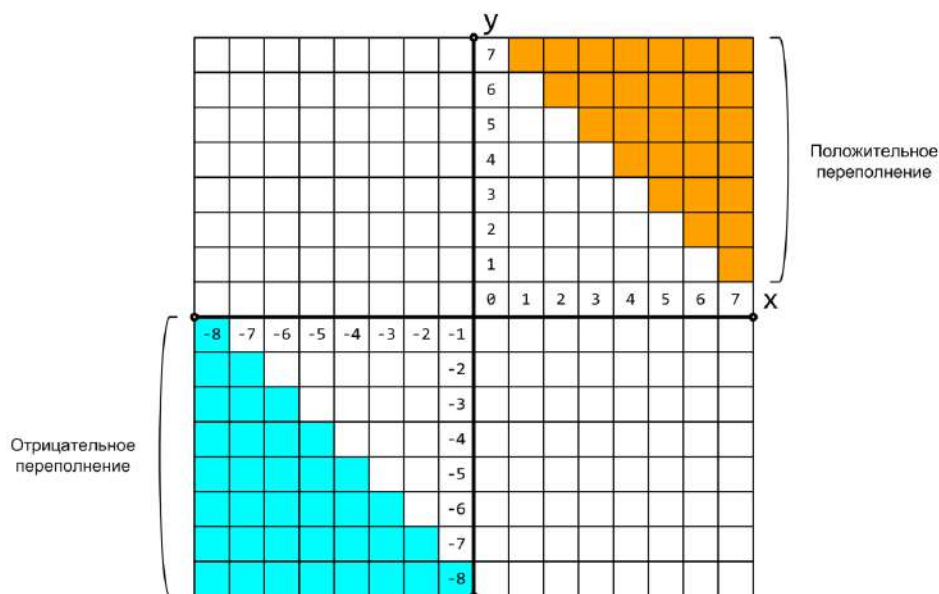


Рис. 4.5: Переполнение при сложении двух знаковых чисел

Отметим, что все флаги обновляются одновременно. То есть, машина не знает, как мы трактуем битовые вектора, живущие в регистрах или в памяти (рассматриваем мы их как знаковые величины или как беззнаковые). На основе значений этой группы флагов можно попросить машину сделать то или иное изменение порядка команд.

Изменение естественного порядка выполнения программы

На рис. 4.6 представлена таблица, первый столбец которой – адреса, следующий столбец – соответствующий машинный код, последний столбец – текстовая запись ассемблерных инструкций.

EIP	Машинный код	Длина	Ассемблерная инструкция
8048345	89 e5	2	mov ebp, esp
8048347	83 ec 10	3	sub esp, 0x10
804834a	c7 45 f0 02 00 00 00	7	mov dword [ebp-16],0x2

Рис. 4.6: Естественный порядок выполнения

В случае естественного порядка выполнения происходит выборка, декодирование, к текущему значению счетчика команд прибавляется длина команды. В результате получаем новое число, от которого снова идет выборка и далее как указано выше.

Влияние на поток выполнения осуществляется с помощью некоторых дополнительных команд. Если к команде мы добавляем некоторое условие, то изменение естественного порядка выполнения программы осуществляется по схеме, представленной на рис. 4.7.



Рис. 4.7: Изменение порядка выполнения программы

Вначале выполняется некоторое преобразование над величинами (сложение, вычитание и др.). После этого содержимое регистра EFLAGS обновляется. Специальная команда может посмотреть состояния разрядов в EFLAGS. В зависимости от того, что там выставлено, можно воздействовать на счетчик команд, тем самым перепрыгнуть куда-то в другое место, избежав естественного хода выполнения.

Рассмотрим некоторые другие преобразования (дополнительно к тем, что мы рассмотрели на прошлой лекции). Рассмотрим команды CMP и TEST. Отметим, что двухадресные команды могут быть неудобны, если операндов два (вычисленное знание разрушает первый операнд и переопределяет его). Если мы хотим поставить условие,

которое сравнивает значения операндов, то такие команды неприменимы. В подобных случаях можно воспользоваться командой CMP. Эта команда аналогична SUB, но без перезаписи результата. Таким образом, условно данную команду можно записать в виде

$$EFLAGS \leftarrow (ОП1 - ОП2)$$

Команда TEST аналогична команде AND. Допустимые типы операндов для указанных команд описаны ниже.

Рассмотрим команды переходов. Переходы бывают

- безусловные/условные
- абсолютные/относительные

Команда JMP – безусловный переход, который отрабатывает всегда. Если речь идет про регистр или память, то это безусловный абсолютный переход и фактически в EIP записывается значение операнда. В случае операнда-константы происходит безусловный относительный переход:

$$EIP \leftarrow (EIP + ОП)$$

В случае с условным переходом всегда может быть только некоторое 32-разрядное число, с помощью которого задается цель перехода.

Команда jcc – прыжок с условием.

Отметим, что для команд jmp и jcc метка в машинном коде будет представлена 32-разрядным числом, закодированным прямо в теле команды. Какое это будет конкретное число, посчитает ассемблер (при трансляции текстового представления в машинный код программа ассемблер знает, какой будет длина кодировок, поэтому он может определить, какое число должно быть записано в двоичном коде вместо метки). При этом, так как полученное число 32-разрядное, мы можем, находясь в любом месте памяти, перепрыгнуть относительным переходом на любое другое место потенциально адресуемой памяти, потому что тем самым задается 32-разрядное смещение.

Таким образом, осуществление естественного порядка выполнения программы может осуществляться с помощью команд

- Арифметические операции
- CMP
 - r/m 8/16/32, i 8/16/32
 - r/m 8/16/32, r 8/16/32
 - r 8/16/32, r/m 8/16/32

- TEST
 - r/m 8/16/32, i 8/16/32
 - r/m 8/16/32, r 8/16/32
- Переходы
 - Способ адресации: задается абсолютный или относительный адрес передачи управления
 - Условия выполнения: безусловная или условная передача управления
- Безусловная передача управления JMP
 - r/m/i 32
- Условная передача управления Jcc
 - i 32

Коды

Коды связаны с некоторой комбинацией взведенных или сброшенных разрядов в EFLAGS. Так, для команды jcc можно записать

$$[N] \frac{z}{E}$$

Здесь подразумевается, что z и E – взаимозаменяемы (z –результат равен нулю эквивалентно E – операнды равны).

Рассмотрим условную команду перехода на метку вида

$$jz \text{ label}$$

Она будет передать управление на помеченное вычисление в коде, тогда, когда раньше при вычислениях был взведен флаг ZF.

Аналогично со всеми остальными флагами. Например

$$[N] \{P, Z, S, C, P\}$$

Здесь в фигурных скобках указаны первые буквы названий флага, один из которых можно указать, чтобы прыгать к метке в том случае, когда указанных флаг взведён (при условии отсутствия буквы N перед указанием флага). Если же помимо флага указать N, то передвижение к метке будет осуществляться только когда указанный флаг сброшен.

Варианты сравнения

В случае беззнаковых величин кода будут следующие:

$$[N] \frac{A}{B} [E]$$

Здесь А и В – основные буквы (above и below), Е – опционально добавляется к основной букве. Например, если написать АЕ, то код условия означает, что при сравнении двух операндов ОП1vsОП2 первый будет больше или равен, чем второй. Если Е отсутствует, мы имеем дело со строгим сравнением.

В случае знакового сравнения:

$$[N] \frac{G}{L} [E]$$

Пример.

Реализуем функцию вида:

$$f(x) = \text{sign}(x)x^2 \quad (4.1)$$

Запишем соответствующий код, разместив итоговый результат в регистре ЕАХ. Для начала нужно получить величину, с которой мы будем работать:

```
GET_DEC 4, ECX
```

Чтобы не потерять первоначальное значение величины после умножения на себя, скопируем рассматриваемую величину в ЕАХ:

```
MOV EAX, ECX
```

Получаем квадрат:

```
IMUL EAX, EAX
```

Получим знак исходной величины:

```
CMP ECX, 0
```

Воспользуемся командой условного перехода. Если ECX больше или равен второму операнду, то есть 0, то мы перепрыгиваем на метку:

```
jge .end  
NEG EAX  
.end:
```

Система команд избыточна и позволяет добиваться результата, используя разные последовательности команд. За счет комбинирования этих последовательностей и выбора одного способа из многих можно достичь улучшения производительности. Поэтому предыдущий пример можно было осуществить с помощью команды TEST. Вместо команд CMP ECX, 0; jge .end можно записать

```
TEST ECX, ECX  
jNs .end
```

Регистр EFLAGS и инструкции

Подведем итоги. На рис. 4.8 представлены команды и основные флаги. Команды сложения, вычитания и обращения знака при своем выполнении меняют всю линейку арифметических флагов. Аналогично для команд CMP и TEST с единственным отличием – TEST всегда будет выставлять CF в ноль, так как при поразрядных операциях над битовыми векторами мы гарантированно не получаем перенос за разрядную сетку.

	OF	SF	ZF	PF	CF
ADD, SUB, NEG	M	M	M	M	M
INC, DEC	M	M	M	M	
IMUL, MUL	M	–	–	–	M
IDIV, DIV	–	–	–	–	–
CBW, CWD, CDQ					
MOV, MOVSX, MOVZX					
CMP	M	M	M	M	M
TEST	0	M	M	M	0

Рис. 4.8: Регистр EFLAGS и инструкции

Обозначения в таблице:

«M» инструкция обновляет флаг (сбрасывает или устанавливает)

«–» влияние инструкции на флаг не определено

« » инструкция на флаг не влияет

«0» инструкция сбрасывает флаг

Отметим, что инструкция может не оказывать никакого влияния на флаг. Это команды пересылки (как просто пересылки, так и с расширением). Также возможна ситуация неопределенного поведения – команда отработывает, после чего она может занулить выставить какой-то флаг.

Обратная задача

Рассмотрим так называемую обратную задачу. Пусть есть некоторый исполняемый код и пусть нужно представить его в виде языка более высокого уровня (то есть нужно пройти этапы компиляции в обратную сторону).

Пример.

Рассмотрим следующий код.

```
...  
SECTION .text  
GLOBAL CMAIN  
CMAIN:  
    MOV EAX, DWORD [a]      ; (1)  
    TEST EAX, EAX           ; (2)  
    JE .1                   ; (3)  
    MOV ECX, DWORD [b]      ; (4)  
    TEST ECX, ECX           ; (5)  
    JE .1                   ; (6)  
    CDQ                     ; (7)  
    IDIV ECX                 ; (8)  
    SUB DWORD [a], EDX       ; (9)  
.1:  
    XOR EAX, EAX             ; (10)  
    RET                      ; (11)
```

Напишем Си-код, который в сущности будет выполнять то же, что написано выше. Вариантов записи может быть несколько. Восстановим один из них.

Заметим, что в указанном коде обращение идет к двум абсолютным адресам памяти, скрытых от нас метками. Соответственно, где-то в Си-коде были объявлены

```
static      a;  
static      b;
```

Тип данных однозначно определить нельзя. Однако, нам известно, что во всех случаях обращения к адресам из памяти извлекалось 4 байта. Соответственно, это может быть тип `int` (аналогично можно записать `long`, это не будет ошибкой):

```
static int a;  
static int b;
```

Чтобы понять, знаковые величины или беззнаковые, нужно рассматривать коды операции, которые будут использоваться при работе с этими величинами.

Далее восстанавливается граф потока управления. В данном случае мы имеем дело с командами, у которых есть естественный ход выполнения и некоторые ситуации перепрыгивания из одного места в другое. Поэтому можно данную последовательность команд нарезать таким образом, чтобы получились блоки, связанные некоторыми дугами, передающими управление из одного блока в другой. Все, что находится внутри такого блока, гарантированно выполняется последовательно. Нарезка идет в зависимости от того, какую команду мы видим (передача управления или всё остальное) и от того, куда

мы можем перепрыгнуть. Например, может быть последовательность команд, которые всегда будут выполняться друг за другом, но они могут рваться меткой, вследствие чего можно извне попасть в центр данной последовательности. Обычно команды нарезаются в виде некоторых блоков, из которых составляется граф.

В данном случае первая, вторая и третья команды выполняются последовательно и образуют один блок. Третья команда может пойти дальше в четвертую или передать управление другой команде. Четвертая, пятая и шестая команды образуют еще один блок. Седьмая, восьмая и девятая команды образуют третий блок, а десятая и одиннадцатая – четвертый блок. Дополним места ветвления. В результате получим граф, изображенный на рис. 4.9.

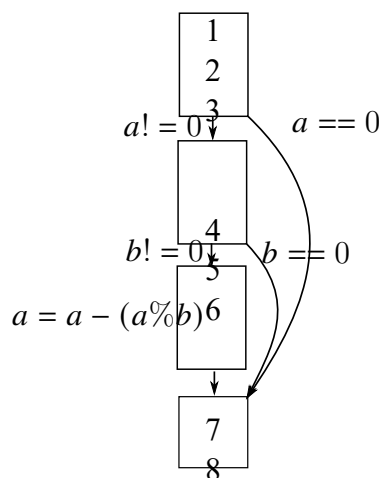


Рис. 4.9: Граф потока управления

Таким образом, в первой команде мы переслали в EAX a . Далее, если условие E выполняется, то мы идем на метку .1 (данное условие равносильно $a == 0$). Естественный ход выполнения продолжается в случае $a! = 0$ ¹⁰. Для команд 4-6 аналогично получаем, что в ECX переслали b . Далее, идем на метку, если $b == 0$ или продолжаем естественный ход, если $b! = 0$. Далее происходит деление. При этом, анализируя оставшийся код, можно сказать, что мы пользуемся результатом взятия остатка. Таким образом, в восьмой строке вычисляется выражение $a \% b$. В девятой строке полученное выражение вычитается из переменной a .

Запишем Си-код:

```
if (a != 0)
    if (b != 0)
        a -= (a % b);
```

Заметим, что возможна и более короткая запись:

```
if (a && b)
    a -= (a % b);
```

Просмотр содержимого исполняемого файла

Рассмотрим дизассемблированный листинг, который относится к полностью собранной исполняемой программе:

```
-bash-2.05b$ ./build_asm.sh backward.asm
-bash-2.05b$ objdump -d -M intel backward
```

`./build_asm.sh` – собираем код, `objdump -d -M intel backward` – дизассемблируем и переводим двоичную запись с использованием синтаксиса Intel. В результате получаем

```
080483e0 <main>:
80483e0: a1 fc 94 04 08      mov     eax,ds:0x80494fc
80483e5: 85 c0               test    eax,eax
80483e7: 74 13              je      80483fc <main.1>
80483e9: 8b 0d 00 95 04 08   mov     ecx,ds:0x8049500
80483ef: 85 c9               test    ecx,ecx
80483f1: 74 09              je      80483fc <main.1>
80483f3: 99                 cdq
80483f4: f7 f9              idiv    ecx
80483f6: 29 15 fc 94 04 08   sub     ds:0x80494fc,edx
080483fc <main.1>:
80483fc: 31 c0               xor     eax,eax
80483fe: c3                 ret
80483ff: 90                 nop
```

Вместо имен переменных у нас появились абсолютные значения адресов. Здесь можно четко проследить, в каком порядке и на каком расстоянии друг от друга размещены переменные. Вначале идет переменная `a`, сразу после нее без промежутка – переменная `b`. Третья и шестая строки имеют передачу управления.

Лекция 5. Вызов функции

Вызов функции

Определим, что необходимо для вызова функции и выхода из неё. Определим, что необходимо для вызова функции и выхода из неё.

- Вопросы
 - Передача управления и возвращение обратно
 - Вычисление значений фактических параметров и их размещение
 - Передача возвращаемого значения
 - Размещение автоматических локальных переменных
 - Порядок использования регистрового файла различными функциями
 - Какие именно машинные команды использовать для поддержки функций
- Ответы – Application Binary Interface (ABI)
 - Соглашение о вызовах (Calling Convention)

Рассмотрим каждый пункт отдельно. Сперва нужно передать управление на участок кода, где будут находиться команды, относящиеся к другой функции. После того, как функция отработана, необходимо вернуться обратно. Значит, нужно запоминать адрес возврата.

Следующий вопрос – параметры. Пусть над данными выполняются некоторые типовые действия. Эти данные нужно как-то передавать. В языке Си передача параметров происходит по значениям. Возникает вопрос, как именно нужно передавать. В случае нашей машины регистров немного и мы не будем ими пользоваться для решения данной задачи.

Аналогичный вопрос возникает и для возвращаемого значения. Функция может возвращать как примитивные типы данных, так и что-то производное. Если речь идет о целых числах или указателе, может хватить и регистра, но если возвращается структура, то возникает проблема, так как структура может занимать произвольное число байт.

Для автоматических локальных переменных вопрос решается примерно следующим образом. Где-то в памяти выделяется какое-то место. Туда располагаем один из регистров и далее смещаемся от этого места.

Следующий вопрос связан с регистрами. Всего регистров восемь, в свободном распоряжении ещё меньше. Если где-то вокруг вызова функции идет вычисление каких-то выражений, то снова нужно задать некоторые правила, чтобы не возникало коллизий (не должно быть так, что где-то снаружи функции используется регистр, а потом вызванная функция тоже захотела его использовать).

Сборник ответов на эти вопросы – один из разделов документа Application Binary Interface (ABI). Этот документ формируется для каждой аппаратной платформы, то есть для каждого процессора и для той операционной системы, которая на ней работает. При этом даже на одной и той же процессорной архитектуре но в разных операционной системах этот документ может отличаться. Помимо правил, описывающих работу соглашения вызова, в ABI есть различные требования по размещению данных в памяти в части выравнивания, то, в каких форматах следует хранить код и так далее.

Аппаратный стек AI-32

- Область памяти, работа с которой ведется согласно дисциплине стека
- Стек растет в направлении меньших адресов
- Регистр esp содержит адрес «верхушки» стека (наименьший адрес памяти)

В той архитектуре, с которой мы работаем, есть некоторая особенность – аппаратная поддержка стека. Есть специализированные команды и специализированное использование определённого регистра, что позволяет удобно работать со стеком. До этого память мы рисовали как одномерный массив. В случае со стеком эту область памяти лучше представлять другим образом. Естественное представление стека – стопка 4-байтных двойных слов, которые уложены друг на друга. Есть некая область памяти, которую операционная система обозначила как доступную для чтения и записи. Куда-то внутри этой памяти указывает регистр ESP. Этот регистр мы воспринимаем как указатель на верхушку стека. Помещать на стек можно размытые величины, причем укладываются туда и снимаются оттуда они порциями или в два, или в четыре байта. Мы будем рассматривать случай, когда эта порция – четыре байта и адреса выровнены и кратны четырём.

Команда push

Рассмотрим команду push, которая что-то кладет на верхушку стека. У нее может быть операндом или регистр или памяти. Или же можно использовать непосредственно закодированную величину в соответствии с написанным ниже:

$$push\ r/m\ \frac{16}{32}\ i/\frac{8}{16}$$

Стек растет вниз (рис. 24.10), а адреса увеличиваются снизу вверх. Это означает, что каждый раз, когда мы что-то помещаем командой push, помещаемое значение каждый раз будет находиться все ближе к нулевому адресу.



Рис. 5.1: Стек.

Рассмотрим, как конкретно происходит размещение в стек. Отметим, что размещение может быть в памяти, значит, может использоваться косвенная адресация, то есть для определения исполнительного адреса можно использовать какие-то регистры.

Рассмотрим следующий псевдокод. Берем операнд и запоминаем его значение:

$$tmp \leftarrow ОП$$

Уменьшаем ESP на размер того, что в не помещаем:

$$ESP \leftarrow ESP - 4$$

По этому уменьшенному адресу помещаем запомненное значение:

$$[ESP] \leftarrow tmp$$

Рассмотрим команду

`push dword [ESP]`

Перед запуском программы была выделена память, на верхний адрес операционная система установила ESP, чтобы было сформировано дно стека. Пусть команда начинает выполняться, ESP показывает на некоторый адрес. Копируем то, что было по этому адресу, и переносим по адресу, сдвинутому на 4. Путем использования последовательности таких команд можно расклонировать по стеку какую-то величину (рис. 5.2).

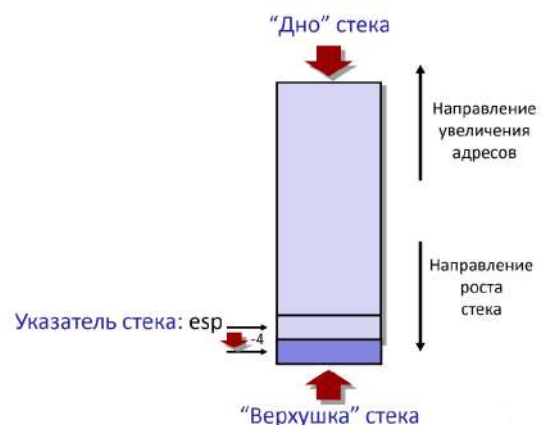


Рис. 5.2: Иллюстрация к объяснению

Отметим, что при работе с указанной командой получается, что можно работать с

памятью дважды – и на чтение, и на запись. Это происходит за счет того, что память не произвольно адресуемая, а в первом и втором случае для адресации того, с чем работаем, всегда используем регистр ESP, то есть не нужно дополнительно кодировать адреса.

Команда pop

Команда pop отвечает за выгрузку данных из стека. Эта команда работает или с регистром или с памятью соответствующих размеров:

$$r/m \frac{16}{32}$$

Рассмотрим работу команды. Запоминаем величину, на которую указывает ESP (указатель верхушки стека):

$$tmp \leftarrow ОП$$

После чего увеличиваем ESP на 4:

$$ESP \leftarrow ESP + 4$$

Отправляем в операнд запомненное выражение:

$$ОП \leftarrow tmp$$

Рассмотрим команду

`pop dword [ESP]`

Эта команда будет выполнять аналогичные действия, но в обратном направлении.

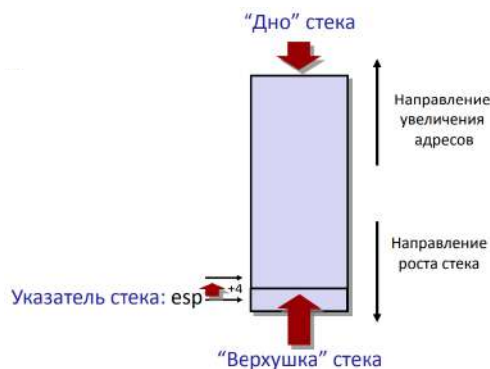


Рис. 5.3: Выполнение команды pop

При этом, данные, которые находились ниже верхушки стека, остаются там неизменными. При этом с логической точки зрения они перестают для нас существовать.

Все, что находится ниже верхушки стека, с логической точки зрения это некоторая свободная память.

Посмотрим, что будет делать команда

`pop esp`

`esp` – формально регистр общего назначения. Мы можем взять что-то со стека, переместить это значение в `esp`. Тогда указатель перескочит в другое место – мы продолжим работать с другой областью памяти. Которая будет рассматриваться машиной как стек. Мы сами должны озаботиться тем, чтобы с этой памятью можно было работать.

Языки программирования (ЯП), базирующиеся на стеке вызовов

Языки программирования (ЯП), базирующиеся на стеке вызовов:

- ЯП с поддержкой рекурсии
 - C, Pascal, Java, ...
 - Код функции можно вызывать повторно (“Reentrant”)
 - * Одновременно могут выполняться несколько вызовов функции
 - Необходимо выделять память под сохранение состояния каждого работающего вызова
 - * Аргументы
 - * Локальные переменные
 - * Адрес возврата
- Стек
 - Сохранять состояние вызова функции надо в ограниченный период времени: от момента вызова до момент выхода
 - Вызываемая функция всегда завершается до вызывающей
- Стек выделяется Фреймами
 - Состояние отдельного вызова функции

На стеке мы будем выделять области памяти, в которых будет размещаться вся необходимая служебная информация, для того, чтобы мог выполняться очередной вызов функции. Так или иначе стек требуется для любого языка программирования, поддерживающего рекурсию.

Под каждый вызов функции на стеке выделяется некоторый блок – **фрейм**. С этим фреймом мы работаем до тех пор, пока идет выполнение очередного вызова. Вызовы

функции по времени строго вложены друг в друга – до тех пор, пока очередной вызов не отработал, мы не возвращаемся в вызывающую функцию. Поэтому фреймы можно распределять на стеке.

Порядок вызова функции

Вернемся к проблеме возвращения из функции. Для решения данной проблемы есть команды `call` и `ret`. **Порядок вызова функции:**

- Аппаратный стек используется для вызова функций и возврата из них
- Вызов функции: `call label`
 - На стек помещается адрес возврата
 - Выполняется прыжок на метку `label`
- Адрес возврата:
 - Адрес инструкции непосредственно расположенной за инструкцией `call`

```
804854e:      e8 3d 06 00 00      call 8048b90 <main>
8048553:      50                push  eax
```
 - Адрес возврата = `0x8048553`
- Возврат из функции: `ret`
 - Выгрузка адреса из стека
 - Прыжок на этот адрес

Рассмотрим эти команды подробнее. Команда `call` передает управление и запоминает место, в которое нужно будет вернуться, чтобы продолжить работу (то есть она запоминает адрес следующей команды и кладет его на стек). Если записать, в каком виде она будет работать, получим:

```
call rel32
```

Здесь `rel32` указывает на то, что речь идет о константе, которая будет восприниматься как относительный адрес.

Другая форма (в качестве операнда можно использовать регистр или память):

```
call r/m 32
```

Здесь будут абсолютные адреса.

При этом происходит следующее. Определяется, куда мы прыгнем:

$$tmpDest \leftarrow Dest$$

Здесь *Dest* – исполнительный адрес, на который мы будем переходить. После этого берем текущее значение счетчика команд и кладем его на стек:

$$push(EIP)$$

После этого пересылаем запомненное значение в *EIP* (то есть осуществляем передачу управления).

$$EIP \leftarrow tmpDest$$

Рассмотрим команду *ret*. У нее нет операндов. Она берет то, что находится на текущей верхушке стека и отправляет в *EIP*:

$$Ret \quad EIP \leftarrow pop$$

Тем самым можно вернуться на ту же самую команду, которая разродилась после вызова *call*. При этом в каждом вызове адрес на верхушке стека может быть разный.

Рассмотрим следующую иллюстрацию с модельным стеком (рис. 5.4). Будем считать, что *esp* указывает на 108. После того, как выполнили *call*, адрес следующей команды появился внизу, под величиной 123, которая далее будет интерпретироваться как аргумент вызова. *esp* уменьшился на 4 и *eip* стал тем, на что мы передаем управление.

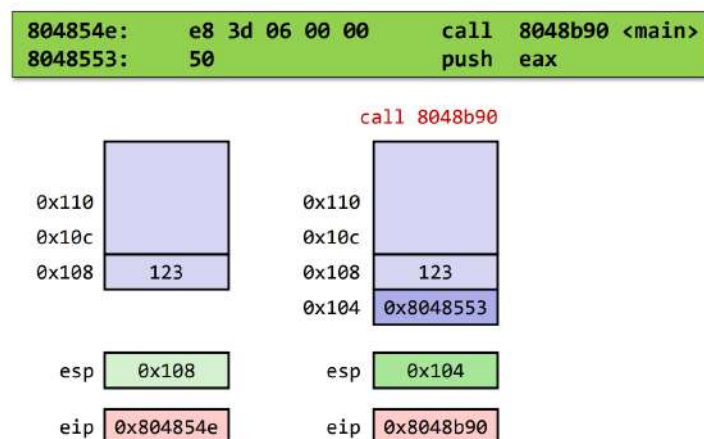


Рис. 5.4: Вызов функции

Внутри вызова функции может происходить все, что угодно. Мы можем сдвигать верхушку стека вниз, помещать туда что-то, работать со фреймом и прочее. Но, когда мы хотим вернуться из функции, нужно вернуть *esp* на то же самое место, где был

запомненный адрес возврата. В противном случае при вызове `ret` можно перепрыгнуть в непредсказуемое место.

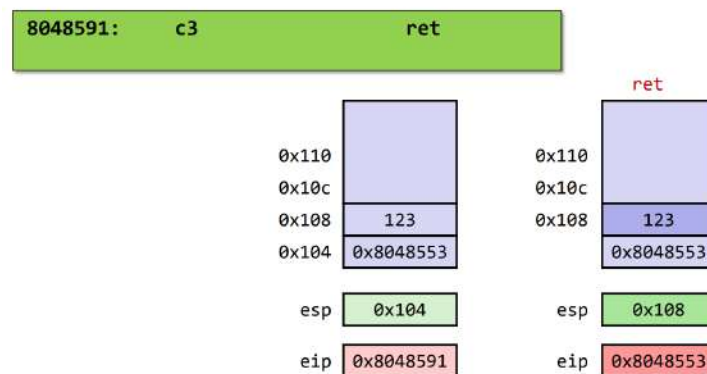


Рис. 5.5: Выход из функции

Указанный механизм позволяет обрабатывать сложные ситуации, в том числе рекурсии. Рассмотрим пример цепочки вызовов (рис. 5.6). Здесь есть три функции, которые вызывают друг друга. Граф вызовов указан на рис. 5.7. Переходя по этому графу, и правильно возвращаясь, мы будем описывать некоторое дерево вызовов. Например, дерево вызовов может быть таким, как на рис. 5.6 справа.

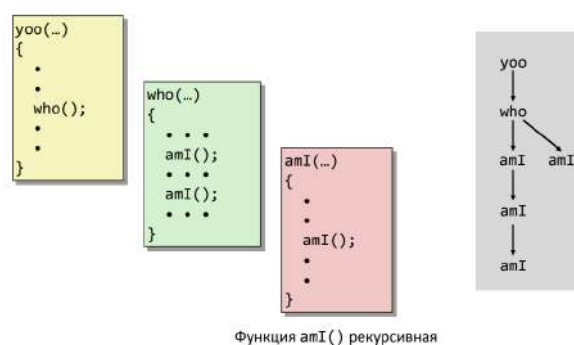


Рис. 5.6: Пример цепочки вызовов



Рис. 5.7: Граф вызовов

Стек фреймов

Во фрейме, помимо адреса возврата мы будем держать локальные переменные, какие-то временные переменные. Управлять такими фреймами будет некоторый

вспомогательный код, который не относится напрямую к логике программы. То есть, у нас есть тело функции, в котором мы реализуем нужные алгоритмы, но они обрамляются прологом и эпилогом. То есть, в тот момент, когда произошла передача управления и мы начали работать с функцией, отработывает пролог – кусок текста, который фрейм формирует – выделяет некоторое дополнительное место на стеке, с которым можно работать. Когда мы выходим, отработывает эпилог – некоторое количество функции, которое переставит указатель стека наверх и вернет указатель стека на то место, где находится адрес возврата и выполнит команду `ret`.

Итак,

- Во фрейме размещаются
 - Локальные переменные
 - Данные, необходимые для возврата из функции
 - Временные переменные
- Управление фреймами
 - Пространство выделяется во время входа в функцию
 - * «пролог» функции
 - Освобождается на выходе
 - * «эпилог» функции

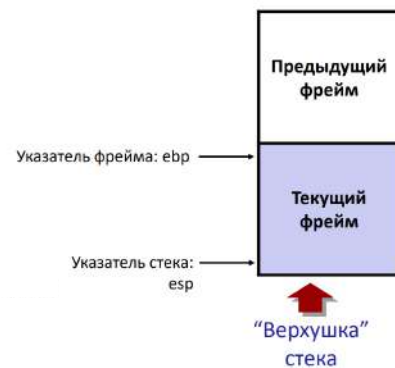


Рис. 5.8: Иллюстрация к объяснению

Таким образом, применительно к той последовательности вызовов, которые мы разбирали выше, последовательность фреймов на стеке будет формироваться следующим образом (рис. 5.9). Каждый фрейм оказывается зажатым между двумя указателями. `esp` указывает на нижнюю границу, а верхняя граница фрейма адресуется регистром `ebp`. Его использование –м вопрос соглашения (с ним не связаны никаких специализированные команды). Таким образом, отработала первая функция.

Вызывается вторая функция (рис. 5.10). В этот момент на стеке выделяется еще одна область (за счет того, что отработывает код пролога). Далее происходит третий вызов. Снова создается фрейм (рис. 5.11).

Далее происходит рекурсивный вызов (рис. 5.12). Таким образом, в данный момент времени существует два разных независимых фрейма, относящихся к одной и той же функции. Таких фреймов может быть несколько (рис. 5.13). Мы можем использовать столько места, сколько операционная система выделила нам под стек. Обычно эта область памяти не очень большая, так как она может переиспользоваться и внутри каждого фрейма не принято выделять много памяти. Для этого у нас есть динамическая память, в которую можно отправлять большие объёмы.

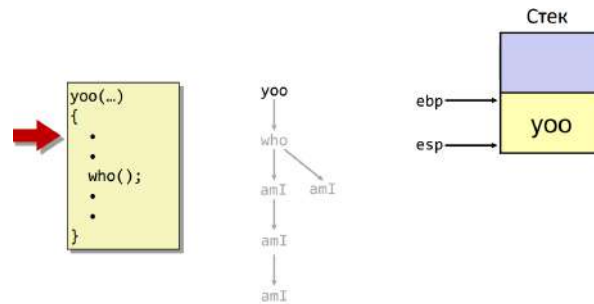


Рис. 5.9: Последовательность фреймов

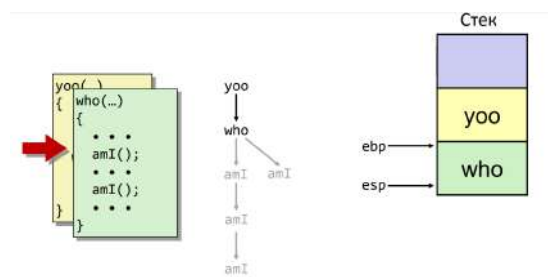


Рис. 5.10: Вызов функции

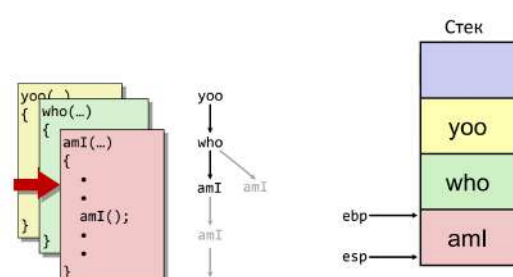


Рис. 5.11: Вызов функции

В случае выхода один фрейм исчезает (рис. 5.14), `ebp` и `esp` переставляются выше. Потом заканчивается еще один рекурсивный вызов за счет того, что эпилог переставил все наверх (рис. 5.15).

Возвращаемся в функцию `who` (рис. 5.16). Спускаемся вниз по операторам до следующего вызова и на том же самом месте в памяти возник еще один фрейм, относящийся к другому вызову с другого места (рис. 5.17).

Отрабатываем этот вызов (рис. 5.18 и рис. 5.19).

Организация фрейма в IA-32/Linux

- Текущий фрейм (от «верхушки» ко «дну»)
 - «Пространство параметров»: фактические параметры вызываемых функций
 - Локальные переменные
 - Сохраненные регистры
 - Прежнее значение указателя фрейма
- Фрейм вызывающей функции
 - Адрес возврата

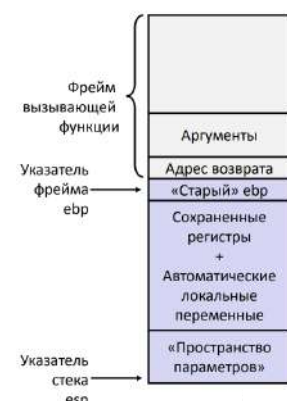


Рис. 5.20: Организация фрейма

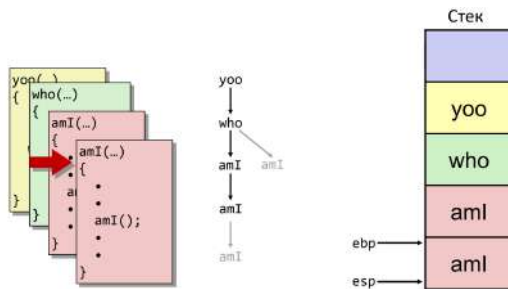


Рис. 5.12: Рекурсивный вызов

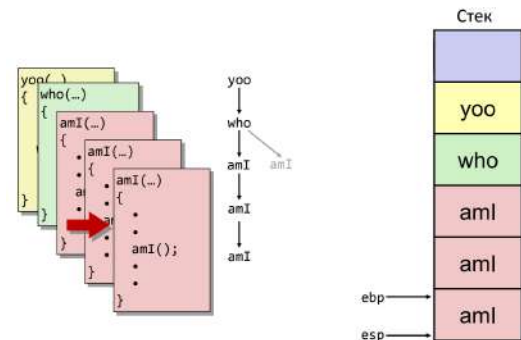


Рис. 5.13: Продолжение рекурсивного вызова

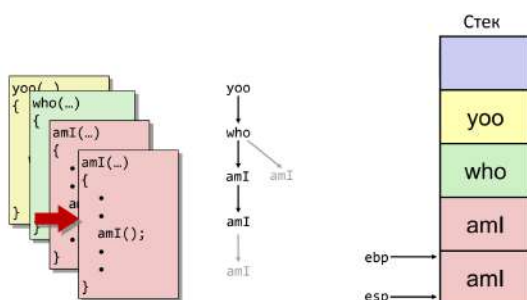


Рис. 5.14: Выход из функции

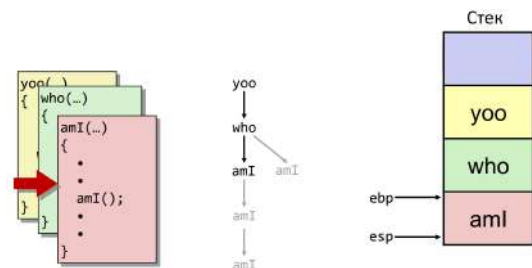


Рис. 5.15: Выход из рекурсивного вызова

* Помещается на стек инструкцией call

– Значения фактических аргументов для текущего вызова

Рассмотрим организацию фрейма подробнее. Есть 8 регистров общего назначения (РОН). Из них ESP, EBP выполняют некую служебную роль. Остальные шесть: EAX, ECX, EDX, EBX, ESI, EDI. В тот момент, когда вызывают некоторую функцию, по соглашению она считает, что тремя регистрами можно начать пользоваться сразу. Если функция разрушит их и перезапишет, ничего страшного не произойдет (это регистры EAX, ECX, EDX). Оставшиеся три регистра нужно предварительно сохранять. Соответственно, если есть некоторые величины, которые должны сохраниться, даже если в дальнейшем что-то будет вызываться, эти величины нужно сохранить на указанных трех регистрах. Если же есть какие-то текущие величины, которые потом можно стереть, то их следует помещать на первых трех регистрах.

Пример. Рассмотрим следующий код:

```
int main() {
    int a = 1, b = 2, c;
```

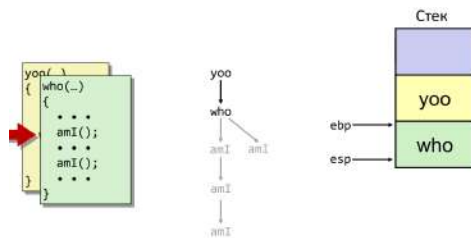


Рис. 5.16: Иллюстрация к объяснению

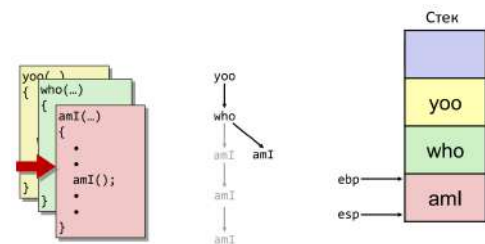


Рис. 5.17: Иллюстрация к объяснению

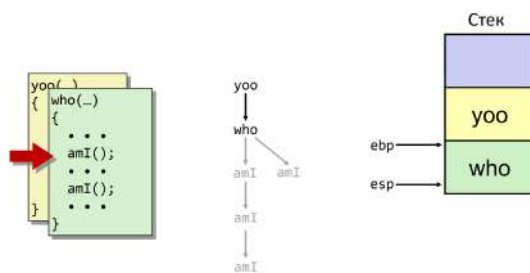


Рис. 5.18: Иллюстрация к объяснению

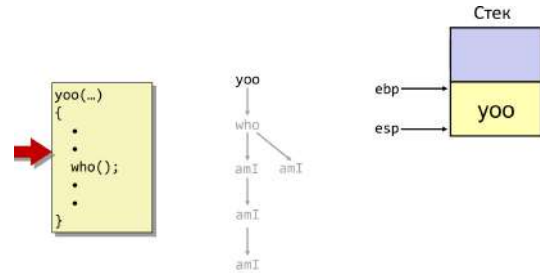


Рис. 5.19: Иллюстрация к объяснению

```
c = sum(a, b);
return 0;
}
```

```
int sum(int x, int y) {
    int t = x + y;
    return t;
}
```

Отображение:

```
include 'io.inc'
section .text
global CMAIN
CMAIN:
...
    mov dword [ebp-16], 0x1 ; (1)
    mov dword [ebp-12], 0x2 ; (2)
    mov eax, dword [ebp-12] ; (3)
    mov dword [esp+4], eax ; (4)
    mov eax, dword [ebp-16] ; (5)
    mov dword [esp], eax ; (6)
```

```

    call sum                ; (7)
    mov dword [ebp-8], eax  ; (8)
    ...
global sum
sum:
    push ebp                ; (9)
    mov ebp, esp            ; (10)
    sub esp, 0x10           ; (11)
    mov edx, dword [ebp+12] ; (12)
    mov eax, dword [ebp+8]  ; (13)
    add eax, edx             ; (14)
    mov dword [ebp-4], eax  ; (15)
    mov eax, dword [ebp-4]  ; (16)
    mov esp, ebp            ; (17)
    pop ebp                 ; (18)
    ret                     ; (19)

```

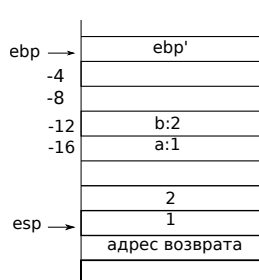


Рис. 5.21:
Иллюстрация
к примеру



Рис. 5.22:
Иллюстрация
к примеру

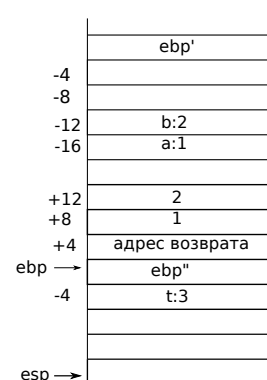


Рис. 5.23:
Иллюстрация
к примеру

Функция `main` начинается с того места, где идет инициализация локальных переменных. В тот момент, когда выполняется функция, есть регистр `ebp`, который указывает на значение `ebp` для верхнего фрейма (обозначим `ebp'`). Далее вниз идут смещения (рис. 5.21). Во второй ((2)) строке начинается вызов функции. Помещаем в стек аргументы (рис. 5.21). На седьмой строке вызывается функция `sum`. В этот момент осуществляется передача управления на метку, которая использовалась для обозначения начала тела функции и запоминается адрес возврата. Команда `ret` на девятнадцатой строке вернет нас на восьмую строку. Внутри функции `sum` начинает работать пролог функции. После того, как сработал `call`, `esp` переместился на адрес возврата. Команды `push ebp` и `mov ebp, esp` переставляют указатель верхней границы фрейма на новое место

согласно принятым соглашениям. Сразу под адресом возврата появляется значение `ebp'`. При этом, согласно десятой строке, `ebp` переставляется туда, на что указывает `esp` (рис. 5.22). В одиннадцатой строке выделяется автоматическая память (команда `sub`). Соответственно, в стеке появляется ещё четыре ячейки. Теперь `esp` должна указывать на последнюю ячейку (рис. 5.23). Далее пролог заканчивается пролог, возникает фрейм. Начинается чтение параметров. Они адресуются относительно `ebp`, который указывает на верхнюю границу (рис. 5.23). Команды (12)-(13): переносим 2 и 1 в `edx` и `eax`. Команда (14): складываем эти два регистра, сумма в `eax`. В строке (15) записываем значение в переменную `t`, которая лежит по смещению -4 (рис. 5.23).

В строке (16) переходим к оператору `return`. Возвращаемое значение помещаем в `eax` и переходим в эпилог функции. В эпилоге мы восстанавливаем указатель границы фрейма. Для этого возвращаем `esp` на прежнее место (рис. 5.24). После выполнения `pop` `ebp` значение ячейки восстановится, в нее попадет значение, которое было записано в `ebp` которое указывает на `ebp'`. Таким образом, в команде (18) мы возвращаем `ebp` на прежнее место (рис. 5.24). Команда (19) вызывает `ret`, после чего `esp` переходит на адрес возврата (рис. 5.25). `ret` возвращает нас на восьмую строку, в `eax` находится возвращенное значение. Соответственно, мы можем переслать это значение в переменную `s` (рис. 5.25).

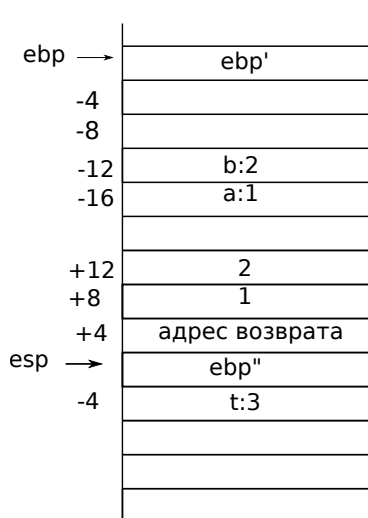


Рис. 5.24: Иллюстрация к примеру

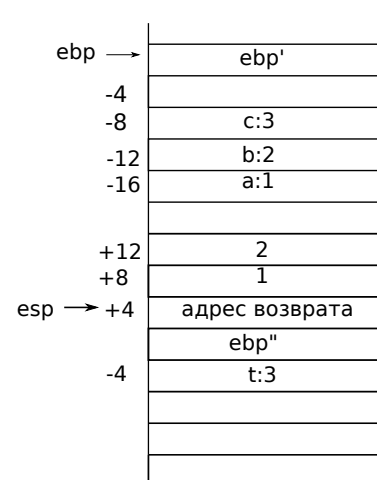


Рис. 5.25: Иллюстрация к примеру

Лекция 6. Система команд.

Группы команд

Набор команд архитектуры можно условно разделить на две группы – пользовательские и системные команды. Пользовательские команды используют для того, чтобы реализовать нужный функционал того или иного прикладного алгоритма, связанного с обработкой данных, с работой с информацией и пр. Системные команды позволяют управлять ходом выполнения пользовательских команд (настраивать работу с памятью, изолировать работающих программ друг от друга и др.). Система команд избыточна, нужный функционал можно реализовать не задействуя все то, чем располагает компилятор. Для разных процессоров набор команд, предлагаемый процессорной архитектурой, может несколько отличаться. Ниже зеленым указаны те группы команд, которые будут изучаться в курсе.

- **Общего назначения**
- **e87 FPU**
- MMX
- SSE
- IA-32e команды 64-разрядного режима работы
- **Системные команды**
- Аппаратная виртуализация
- ...

Где команды **общего назначения** можно разделить на

- **Пересылка данных**
- **Команды двоичной арифметики**
- Команды двоичнодесятичной арифметики
- **Логические команды**
- **Сдвиги и вращения**
- **Битовые и байтовые команды**
- **Передача управления**

- Строковые
- Ввод/Вывод
- Явное управление EFLAGS
- Вспомогательные

При этом **основные команды IA-32** можно разделить на следующие группы (рис. 6.1). Красным выделены не упоминавшиеся ранее на лекциях ассемблерные инструкции. Команды двоичной арифметики и логические команды представлены в полном составе.

Пересылка данных	Двоичная арифметика	Передача управления	Логические	Сдвиги и вращения	Битовые и байтовые	Прочее
• MOV • XCHG • BSWAP • MOVSX • MOVZX • CDQ • CWD • CBW • PUSH • POP • CMOVcc	• ADD • ADC • SUB • SBB • NEG • IMUL • MUL • IDIV • DIV • INC • DEC • CMP	• JMP • Jcc • CALL • RET	• AND • OR • XOR • NOT	• SAR • SHR • SAL, SHL • ROR • ROL • RCR • RCL	• SETcc • TEST	• LEA • NOP

Рис. 6.1: Группы команд

Команда LEA

Команда LEA позволяет задействовать все возможности по вычислению исполнительных адресов, которые присутствуют в процессоре и реализуются на этапе увеличения значения операндов. Адресный код, находящийся внутри квадратных скобок (см. рис. 6.2) способен закодировать достаточно сложное выражение. Запишем его в общем виде. Получим следующую конструкцию: есть базовый регистр, в качестве которого может выступать один из представленных на рис. 6.2 регистров, к нему прибавляется еще один регистр (возможен повтор). К этому индексному регистру можно приложить некоторый масштаб. В конце – смещение.

В документации команда LEA описывается несколько иначе. Ее допустимый формат:

LEA R32, m

Результат попадает в первый операнд – 32-разрядный регистр. Второй операнд – память без спецификатора размера.

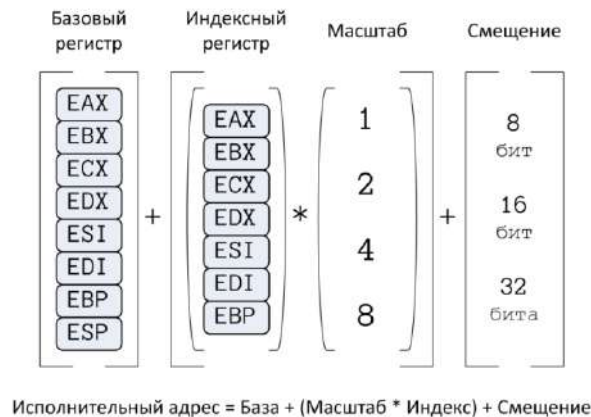


Рис. 6.2: Иллюстрация к объяснению

$$\text{ОП1} \leftarrow \text{АДРЕС (ОП2)}$$

В частности, описанное выше позволяет реализовывать посредством данной команды ту же арифметику, которую можно создавать с поморью команд `add`, `sub` и проч. Это делается за счёт того, что в дополнительном коде положительное смещение можно превратить в отрицательное число и после того, как все вычисления будут выполнены, результат будет обрезан по границам 32-разрядной сетки и никакие флаги не будут при этом обновляться. Видно, что при этом можно провести умножение на некоторые predetermined константы, комбинируя регистры. Например, можно осуществить:

$$\text{LEA EAX, [EAX + 8 * EAX]}$$

что эквивалентно умножению на число 9.

Отметим, что указанные выше действия выполняются в рамках одного из этапов выполнения команды, поэтому всё происходит очень быстро. По этой причине если у компилятора есть возможность, он использует команду `LEA` — использует все возможности косвенной адресации. Косвенность адресации связана с тем, что в вычислениях используется значения определенных регистров.

Косвенная адресация

Рассмотрим некоторые специализированные названия косвенной адресации. Первый вариант — **относительная адресация**. Её выражение:

$$\text{база} + \text{смещение}$$

Типичный пример: `[ebp-16]`. `ebp` — базовый регистр, который показывает верхнюю

границу текущего фрейма функции. Вычитаем из него 16 – получаем место внутри фрейма. В зависимости от того, как мы распределили содержимое фрейма, мы можем, например, обращаться к какой-то автоматической локальной переменной или области, которая используется для промежуточных результатов вычисления.

Базово-индексная. В составе адресного кода есть регистр база. К нему прибавляется значение регистра индекса:

база + индекс

Пример: `[EAX+ECX]`. При этом нельзя сразу сказать, где в указанной конструкции сразу указать, где база, а где – индекс. Для этого нужно располагать некоторым контекстом того, какой код мы пишем.

Если массив не однобайтный, нужно использовать коэффициент. Поэтому в этом случае используем несколько другой тип адресации. **Базово-индексная с масштабированием:**

база + масштаб*индекс

Пример: `[eax+4*ecx]`.

Рассмотрим соотношение типов для языка Си и ассемблера (таблица 1). Машина поддерживает основные целые типы данных.

Си	АСМ
char	byte *1
short	word *2
int	dword *4
long	dword *4
long long	qword *8

Таблица 1: Соотношение типов

Последняя форма – **Базово-индексная с масштабом и смещением**. Выражение:

База + Масштаб * Индекс + Смещение

Пример: `[eax+4*ecx+10]`.

Реализация стрелки Пирса

Пример. Реализация стрелки Пирса.

Известно, что число логических поразрядных операций не очень много. Машина дает нам некоторые простой базис. Машина умеет поразрядно выполнять некоторые действия (см.

таблицу table:2). В указанной таблице левый столбец – обозначение операции в языке Си, правый столбец – код мнемоники в машине. У всех трех команд одна и та же допустимая форма: $r/m \frac{8}{16}, r/m/i \frac{8}{16}$.

&	AND
	OR
^	XOR

Таблица 2: Стандартные операции

Описать реализацию можно следующим образом:

$$OP1 \leftarrow OP1 \text{ OP } OP2$$

Где ОП – операция над операндом ОП2. Отметим, что SF и ZF обновятся (при поразрядных действиях что-то будет в знаковом разряде, результат может быть как нулевым, так и ненулевым). Переносов нет, поэтому флаги OF и CF можно занулить.

Рассмотрим четвертую команду – NOT (~). У нее один операнд: $r/m \frac{8}{16}$. В этот операнд производится запись поразрядной инвертации для каждого отдельного разряда:

$$OP \leftarrow \sim OP$$

При этом флаги не меняются.

Комбинируя рассмотренные четыре команды, можно реализовать любую булеву функцию, выполняющуюся над отдельными разрядами.

В качестве примера рассмотрим реализацию стрелки Пирса:

```
unsigned pierce_arrow(unsigned a, unsigned b) {
    int t = ~(a | b);
    return t;
}
```

Справедливо выражение:

$$a \downarrow b = \neg(a \vee b)$$

Это действие оформлено в коде как самостоятельная функция:

```
section .text
global pierce_arrow
pierce_arrow:
    push ebp
    mov ebp, esp
    mov eax, dword [ebp+12] ; (1)
    or eax, dword [ebp+8] ; (2)
```

```
not eax ; (3)
pop ebp
ret
```

Пролог и эпилог здесь не занумерованы. После выполнения двух первых команд регистр `ebp` показывает на сохраненный `ebp'`. По смещению `+8` от `ebp` (относительная косвенная адресация) лежит параметр `a`, по смещению `+12` – параметр `b` (рис. 6.3). В первой команде мы перегружаем в `eax`. Далее перегружаем параметр `b`. Накапливая результат в `eax`, выполняем OR и поразрядный NOT. Последние две команды – эпилог и выход из функции.

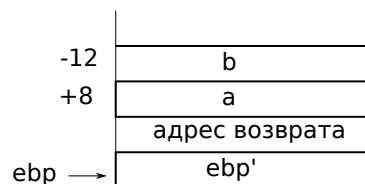


Рис. 6.3: Иллюстрация к объяснению

Сдвиги

Рассмотрим поразрядную пересылку одних разрядов в другие, то есть сдвиги. У сдвигов есть две независимые характеристики – куда двигаем и то, как мы интерпретируем то, что мы двигаем. Таким образом, имеем два параметра – направление и данные (знаковые и беззнаковые), которые создают матрицу, в которой будут коды операции (рис. 6.4). Первая буква для сдвигов – всегда буква `S`. Далее идет `H` – для беззнаковых или `A` – для знаковых данных. Третья буква: `L` или `R` в зависимости от направления.

Данные	Направление сдвига		
	Влево	Вправо	
Беззнаковые	SHL	SHR	<code>r/m 8/16/32, i8</code>
Знаковые	SAL	SAR	<code>r/m 8/16/32, cl</code>

Рис. 6.4: Сдвиги

В результате сдвигов влево неважно, как данные будут интерпретироваться, так как то, что не умещается в старших разрядах, теряется безвозвратно, а то, что освобождается справа, заполняется нулями (рис. 6.5). Интерпретация данных в данном случае будет одинаковой, поэтому кодировки `SHL` и `SAL` по сути эквивалентны.

В случае поразрядного сдвига вправо освободившиеся разряды будут заполняться нулями, последний разряд будет оказываться в разряде `CF` (рис. 6.6).



Рис. 6.5: Сдвиг влево



Рис. 6.6: Логический сдвиг вправо

В случае арифметического сдвига вправо старший разряд будет копироваться (рис. 6.7). Таким образом, арифметический сдвиг вправо имеет сходство с делением.



Рис. 6.7: Арифметический сдвиг вправо

Форматы

Для всех четырех команд первый операнд – регистр или память, второй операнд – непосредственно закодированная величина:

$$r/m \frac{8}{16}, i8 \text{ (masked 5 bit)}$$

Здесь masked 5 bit – маскированные пять разрядов. Это означает, что мы можем написать в i8 какое-то число, причем старшие три разряда выкинут, а оставшиеся пять будут интерпретировать как положительное число, на которое осуществляется сдвиг. Однако далеко не всегда заранее известно, на сколько зарядов нужно подвинуть. В этом случае первый операнд такой же, а второй операнд – регистр cl:

$$r/m \frac{8}{16}, cl$$

Флаги SF и ZF выставляются по результату.

Рассмотрим следующий пример:

```
char updown(char x) {  
    return (x << 8) >> 8;  
}
```

Логический результат работы данной программы – ноль. Полученный код:

```
section .text  
global updown  
updown:  
    push ebp  
    mov ebp, esp  
    movsx eax, byte [ebp+8]  
    sal eax, 8  
    sar eax, 8  
    pop ebp  
    ret
```

После того, как отработал пролог, первый параметр со смещения +8 интерпретируются как знаковое число, командой movsx расширяем его до полного eax, а далее сдвигаем число поочередно в обе стороны. Таким образом, если на вход поступило число 42, то и результат будет – 42.

Команды вращения

Рассмотрим так называемые **команды вращения**. У них есть два параметра – направление вращения битового вектора и участие флага CF (таблица 3). Допустимые форматы вращения аналогичны форматам сдвигов. При этом то, что в последний раз при вращении выходит за пределы разрядной сетки, копируется в разряд CF (рис. 6.8). RCL и RCF предполагают явное участие флага CF (он добавляется как дополнительный, 32-разряд).

		направление	
		лево	право
Участие CF	да	RCL	RCR
	нет	ROL	ROR

Таблица 3: Соотношение типов



Рис. 6.8: Иллюстрация к объяснению

В качестве примера Рассмотрим фрагмент кода криптографической хеш-функции:

```
unsigned sha256_f1(unsigned x) {
    unsigned t;
    t = ((x >> 2) | (x << ((sizeof(x) << 3) - 2))); // (1)
    t ^= ((x >> 13) | (x << ((sizeof(x) << 3) - 13))); // (2)
    t ^= ((x >> 22) | (x << ((sizeof(x) << 3) - 22))); // (3)
    return t;
}
```

В криптографических хешах стараются разработать алгоритм хеширования таким образом, чтобы коллизий было как можно меньше. В данном примере функция

sha256 вырабатывает так называемый дайджест длиной 256 разрядов. Эти разряды интерпретируются как число. В пространство таких чисел, ограниченных такой разрядной сеткой, много что отображается – коллизии будут всегда. Однако находить эти коллизии достаточно сложно, поэтому от функции требуется:

- Необратимость: $H(X) = m$, m – задано
- Стойкость к коллизиям первого рода: $H(M) = H(N)$, M – задано
- Стойкость к коллизиям второго рода: $H(M1) = H(M2)$

Код, который построил компилятор:

```
global sha256_f1
sha256_f1:
    push ebp
    mov ebp, esp
    mov edx, dword [ebp+8]    ; (1)
    pop ebp                  ; (2)
    mov eax, edx              ; (3)
    mov ecx, edx              ; (4)
    ror eax, 13                ; (5)
    ror ecx, 2                 ; (6)
    xor eax, ecx               ; (7)
    ror edx, 22                ; (8)
    xor eax, edx               ; (9)
    ret
```

Сначала отрабатывается пролог, после чего в `edx` загружается значение `unsigned x`. Потом `edx` копируется в `ecx` и `eax`. Таким образом, в трех регистрах находится одно и то же значение `x`. Далее поддерево команд, изображенное на рис. 6.9, может быть сведено к выполнению одной команды, что и происходит в строке (5).

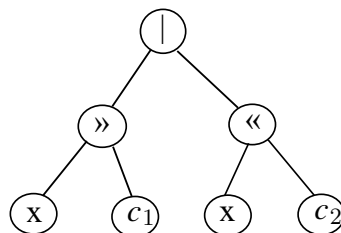


Рис. 6.9: Иллюстрация к объяснению

Во всех трех строчках Си-кода сдвиги влево и вправо скомбинированы так, чтобы получить вращения. Происходит вращение `eax` и `ecx`. Далее (в (7) строке) осуществляется хог с накоплением результата в `eax`. Далее – третье вращение, результат в `edx`. Потом вновь применяем хог и получаем результирующее значение.

Обратная задача

Восстановим некоторую конструкцию, реализованную в ассемблерном коде. В качестве подсказки имеем, что есть статические переменные:

```
static int a, b, c, d;
```

???

Ассемблерный код:

```
mov eax, dword [b]    ; (1)
mov edx, dword [c]    ; (2)
or al, -1              ; (3)
sal eax, 3             ; (4)
add edx, eax           ; (5)
mov dword [a], eax     ; (6)
mov eax, edx           ; (7)
sar edx, 31            ; (8)
idiv dword [d]         ; (9)
mov dword [c], eax     ; (10)
```

Получаем, что $EAX|_b$, $EDX|_c$. Далее от EAX используем только младший байт. Такую ситуацию можно изобразить следующим образом: $EAX|_{(b|0xFF)}$ (к переменной b применяется указанная константа). Далее происходит сдвиг вправо на три разряда: $EAX|_{(b|0xFF) \ll 3}$. Далее в EDX получаем сумму вида: $EDX|_{c+(b|0xFF) \ll 3}$. Далее происходит выгрузка EAX в переменную a: $a \leftarrow EAX$. Далее, если считать, что значение переменной a было зафиксировано, получим: $EDX|_{c+a}$. В (7) строке происходит копирование EDX в EAX. В (9) строке происходит знаковое деление, соответственно, делимое нужно подготовить, причём в данном случае подготовка к делению приходит с помощью арифметического сдвига. Таким образом, в (7) и (8) строчке приходила подготовка к делению. В (9) строке выполняется idiv, причем так как в (10) строке используется EAX, после команды деления берется частное, а не остаток. Это частное выгружается в переменную c. Запишем описанное выше в виде Си-кода:

```
a = (b | 0xFF) << 3;
c = (c + a) / d;
```

Сложение 64-разрядных чисел на 32 разрядных регистрах

Рассмотрим случай, когда для вычисления размера регистров не хватает. Рассмотрим пример с командой ADC. Формат у нее такой же, как у сложения с одним отличием:

$$ОП1 \leftarrow ОП1 + ОП2 + CF$$

В этом случае большой операнд, который не помещается в регистре, можно разбить на две логические части. Тогда далее можно будет выполнять определенное цепочечное действие над его кусками, начиная с младших разрядов.

```
long long f1(long long a, long long b) {  
    long long c;  
    c = a + b;  
    return c;  
}
```

Код ассемблера (начало и конец функции опущены):

```
mov eax, dword [ebp+16] ; (1)  
mov edx, dword [ebp+20] ; (2)  
add eax, dword [ebp+8] ; (3)  
adc edx, dword [ebp+12] ; (4)
```

В данном примере имеем следующее заполнение стека (рис. 6.10). Перегружаем b на EAX и EDX, далее:

$$a = a_h : a_l$$

$$b = b_h : b_l$$

После этого складываем младшие разряды. Если происходит переполнение, CF обновляется (0 или 1). Далее применяем команду ADC, которая скалывает старшие разряды. Если было переполнение, то переносится один разряд. Вычитание производится аналогично.

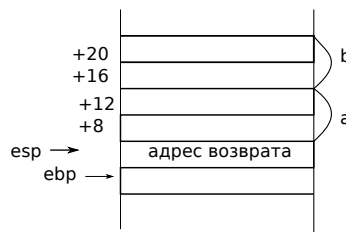


Рис. 6.10: Иллюстрация к объяснению

Умножение 64 разрядных чисел на 32 разрядных регистрах

Рассмотрим случай умножения 640разрядных чисел. В машине есть несколько видов умножения (знаковое и беззнаковое). При этом в машине нет специализированной поддержки умножения и сложения чисел двойной разрядности. Построим ручную код, который будет опираться на имеющиеся команды умножения, чтобы получить корректный результат.


```
long long f2(long long a,  
             long long b) {  
    long long c;  
    c = a * b;  
    return c;  
}
```

Отметим, что при сохранении результата в разрядной сетке мы будем иметь один и тот же код для знаковых и беззнаковых величин. Пусть есть некоторый битовый вектор. Его знаковая интерпретация:

$$C(\vec{x}_w) = x_{\text{зн}}$$

$$U(\vec{x}_w) = x'_{\text{беззн}}$$

Тогда при умножении беззнаковых величин:

$$(x' \cdot y') \bmod 2^w = (x \cdot y) \bmod 2^w = (x \cdot y') \bmod 2^w$$

Доказать это утверждение можно, если учесть, что справедливо

$$x' = x + 2^w x_{w-1}$$

Ассемблерный код:

```
globl f2  
f2:  
    push ebp  
    mov ebp, esp  
    push ebx  
    mov eax, dword [ebp+8]  
    mov edx, dword [ebp+16]  
    mov ecx, dword [ebp+20]  
    mov ebx, dword [ebp+12]  
    imul ecx, eax  
    imul ebx, edx  
    mul edx  
    add ecx, ebx  
    add edx, ecx  
    pop ebx  
    pop ebp  
    ret
```

Таким образом, имеем два вектора, которые мы разбиваем на старшие и младшие части и производим умножение:

$$\vec{Y}_w^h : \vec{Y}_w^l *_{2w} \vec{X}_w^h : \vec{X}_w^l = \{2^{2w} C(\vec{Y}_w^h) C(\vec{X}_w^h) + 2^w \cdot [C(\vec{Y}_w^h) U(\vec{X}_w^l) + C(\vec{X}_w^h) U(\vec{Y}_w^l)] + U(\vec{Y}_w^l) U(\vec{X}_w^l)\} \bmod 2^{2w}$$

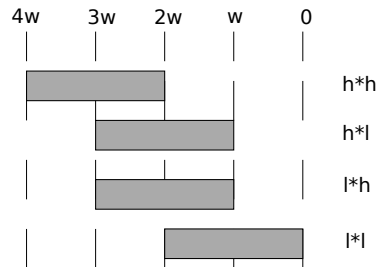


Рис. 6.11: Иллюстрация к объяснению

Распишем данное выражение для разрядной сетки (рис. 6.11). Результат умножения long long – 64-разрядный. Младшие части должны целиком попасть в сумму, при этом, так как эти части будут интерпретироваться как коэффициенты при положительных степенях двойки, эти части должны быть перемножены с помощью команды mul, работающей с беззнаковыми числами.

Лекция 7. Как организована передача управления.

Сравнение беззнаковых чисел

Рассмотрим следующие коды условий (рис. 7.1) и определим, как сложное условие, которые применяются для сравнения чисел, соотносятся с флагами, которые вырабатываются при выполнении арифметических действий.

Исс	Условие	Описание
JE	ZF	Равно / Ноль
JNE	~ZF	Не равно / Не ноль
JS	SF	Отрицательное число
JNS	~SF	Неотрицательное число
JG	~(SF^OF)&~ZF	Больше (знаковые числа)
JGE	~(SF^OF)	Больше либо равно (знаковые числа)
JL	(SF^OF)	Меньше (знаковые числа)
JLE	(SF^OF) ZF	Меньше либо равно (знаковые числа)
JA	~CF&~ZF	Больше (числа без знака)
JB	CF	Меньше (числа без знака)

Рис. 7.1: Коды условий

Самый простой пример – сравнение чисел по признаку равны или нет:

$$X \text{ vs. } Y \implies X - Y \text{ vs. } 0$$

Команда `str` вычитает свои операнды друг из друга и логически сравнивает результат с нулем. Получается комплект арифметических флагов, взведенных или сброшенных после выполнения данной операции. Тогда, если результат вычитания – ноль, то взведен флаг `ZF` и величины равны. Если флаг не взведен, величины не равны.

Рассмотрим числовую матрицу (рис. 7.2). Отсюда получаем, что коды условия сравнения для беззнаковых чисел соответствуют выражению:

$$jB : (x - y) <_u 0 : CF$$

$$jA : (x - y) >_u 0 : \sim CF \text{ и } \sim ZF$$

Сравнение знаковых чисел

Рассмотрим числовую матрицу небольшого размера для случая знаковых чисел (рис. 7.3). Флаги в этом случае учитывают также знаковое переполнение.

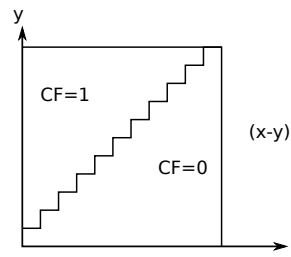


Рис. 7.2: Иллюстрация к объяснению



Рис. 7.3: Сравнение знаковых чисел

При вычитании $(x - y)$ на главной диагонали будет взведен ZF. Синей штриховкой на рис. 7.3 указана область знакового переполнения. Эта область соответствует случаю, когда при вычитании из отрицательного x вычли такой положительный y , что результат – отрицательная величина, которая не вмещается в размер типа. Дополнительно на эту матрицу можно наложить флаг SF (красная штриховка). Двойная штриховка соответствует случаю, когда из положительного числа вычитают отрицательное. При этом должно было получиться положительное число, но из-за переполнения оно стало отрицательным.

Машина комбинирует информацию о выставленных флажках в осмысленное условие на сравнение двух знаковых чисел. Например, для следующего кода условия:

$$jl : (x - y) <_c 0 : SF \wedge OF$$

$$jg : (x - y) >_c 0 : \sim (SF \wedge OF) \& \sim ZF$$

Сравнение: со знаком и без

Рассмотрим следующую ситуацию. Нам даны некоторые величины, которые должны восприниматься как знаковые, но коды условий были перепутаны. В этом случае результат скорее всего будет неправильным, так как если части матрицы изменить (рис. 7.4) мы

получим, срабатывание описанных выше выражений над флагами в областях, указанных на рис. 7.4. Беззнаковые числа будут соотноситься иначе с кодами для знаковых чисел (будут захватываться другие области).

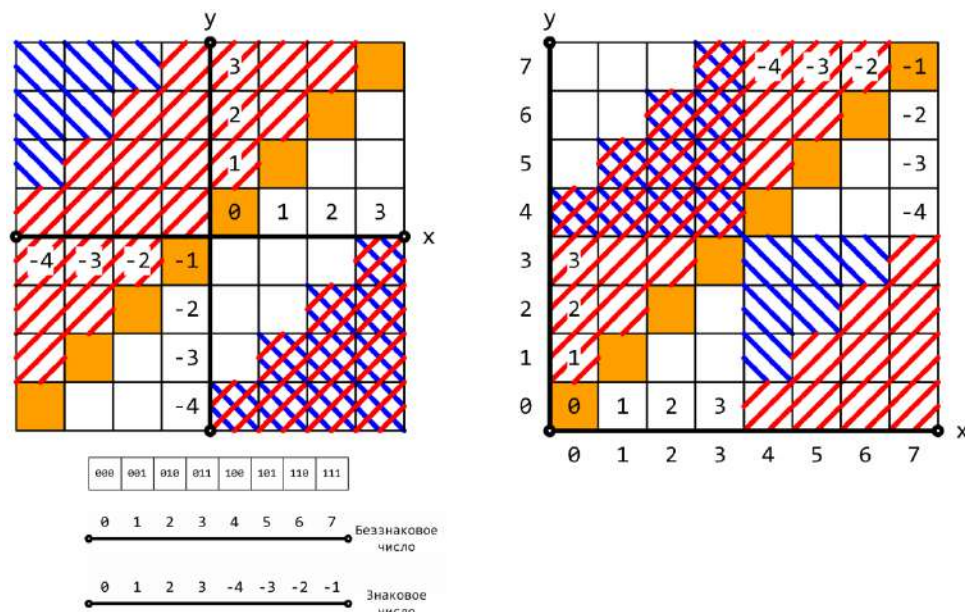


Рис. 7.4: Сравнение: со знаком и без

Рассмотрим пример работы с булевыми типами. В случае с типом `int` все ясно – под него выделяется нужное количество байт. Тип данных `_Bool` мы соотносим с байтом, который может принимать только два значения: `{0, 1}`. В машине есть специальная команда, которая позволяет работать с такими величинами: `SET`. У нее один операнд – регистр или память из 8 разрядов. В зависимости от условия она отправляет в операнд 1 или 0:

SEToc R/m8

```
if(cc){ОП ← 1}
else {ОП ← 0}
```

Задача. Рассмотрим следующую задачу, связанную с описанной командой. Пусть есть некоторая функция, описанная ассемблерным текстом:

CMP :

```
MOV EAX, [esp+8]    (1)
CMP [ESP+4], EAX    (2)
setl AL              (3)
ret                  (4)
```

Требуется правильным образом заполнить пробелы в заголовке функции и в том, что она будет возвращать:

```
_Bool cmp( x, y){  
    return x < y;  
}
```

Рассмотрим заполнение стека в этом случае (рис. 7.5). Типовой пролог в данном случае отсутствует, соответственно esp показывает прямо на адрес возврата. Таким образом, над ними располагаются параметры по смещению +4 и +8. Когда происходит выполнение первой команды с пересылкой в EAX, получаем $EAX|_y$. Следующая строка – сравнение, причем сначала идет x, а потом – y. Согласно коду условия, мы имеем знаковое сравнение. Соответственно, мы работаем со знаковым 32-разрядным числом, то есть в коде можно написать `int`.

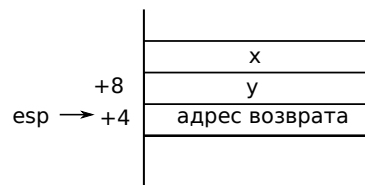


Рис. 7.5: Иллюстрация к объяснению

Из (2) и (3) строк следует, что между x и y нужно поставить знак <. Таким образом, имеем код:

```
_Bool cmp(int x, int y){  
    return x < y;  
}
```

Реализация операндов с помощью известных команд

Посмотрим, как конкретные операторы языка Си будут реализовываться с помощью изученных ранее команд (безусловный переход и переход с условием). В программах управляющие операторы могут образовывать различные конструкции; компилятор просматривает код и строит внутреннее представление, действуя по некоторым шаблонам. Рассмотрим выражение для оператора `if-else`:

```
if(expr){  
    tnen_stmt;  
}else{  
    else_stmt  
}
```

Это можно представить в виде:

```
_Bool t = !(expr)
```

Учтем, в машине есть возможность сравнения (cmp), перепрыгнуть по условию на какую-то ветку (jcc label). Этак конструкция соотносится с оператором:

```
if(cc) goto label;
```

В случае обычного прыжка (jmp label) имеем соотношение с

```
goto label;
```

Далее приводит код в так называемую goto-форму:

```
_Bool t = !(expr)
    if (!) goto Else;
    then_start;
    goto End;
else:
    else_start;
end:
```

Если изначально условие не выполняется, тогда, согласно коду, мы сразу переходим на вторую ветку (произошло формальное обращение условия). Далее, в случае естественного хода переходим к End. С помощью сравнений и условных переходов вида:

```
cmp ....
jcc      .else
[then ....]
jmp      .end
,else;
    [else]
,end:
```

можно реализовать if-else.

Пример:

```
int absdiff(int x, int y) {
    int result;
    if (x > y) {
        result = x-y;
    } else {
        result = y-x;
    }
    return result;
}
```

Ассемблерный код:

```
absdiff:
    push ebp
    mov ebp, esp
    mov edx, dword [8 + ebp]    ; (1)
    mov eax, dword [12 + ebp]   ; (2)
    cmp edx, eax                ; (3)
    jle .L6                     ; (4)
    sub edx, eax                ; (5)
    mov eax, edx                ; (6)
    jmp .L7                     ; (7)
.L6:                            ; (8)
    sub eax, edx                ; (9)
.L7:                            ; (10)
    pop ebp
    ret
```

Приводим программу в goto-вид:

```
int goto_ad(int x, int y) {
    int result;
    if (x <= y) goto Else;
    result = x-y;
    goto Exit;
Else:
    result = y-x;
Exit:
    return result;
}
```

Соотносим с ассемблерным кодом:

```
absdiff:
    push ebp
    mov ebp, esp
    mov edx, dword [8 + ebp]    ; (1)
    mov eax, dword [12 + ebp]   ; (2)
    cmp edx, eax                ; (3)
    jle .L6                     ; (4)
    sub edx, eax                ; (5)
    mov eax, edx                ; (6)
    jmp .L7                     ; (7)
```



```
.L6:                                ; (8)
    sub eax, edx                    ; (9)
.L7:                                ; (10)
    pop ebp
    ret
```

Условная передача данных

Помимо обычного if-else и ситуаций, когда нужно развести управление, есть также тернарный оператор, работа которого сводится к условному присвоению. В зависимости от условия идет присвоение либо одного, либо другого выражения и значения этих выражений присваиваются тому, что находится в левой части (см. код ниже).

```
val = Test ? Then_Expr : Else_Expr;
```

```
val = x > y ? x - y : y - x;
```

Такую конструкцию можно обработать следующим способом. Можно привести тернарный оператор к известным командам (сводим задачу к предыдущей):

```
    nt = !(Test);
    if (nt) goto Else;
    val = Then_Expr;
    goto Done;
Else:
    val = Else_Expr;
Done:
...
```

С другой стороны, возможна другая реализация:

```
tmp_val = Then_Expr;
val = Else_Expr;
t = Test;
if (t) val = tmp_val;
```

Здесь сначала вычисляется первое выражение. Его значение присваивается временной переменной. Потом вычисляется значение второго выражения. В зависимости от того, сработало ли условие, производится условное присвоение. В машине есть соответствующая команда, которая позволяет это осуществить – CMOV:

$$CMOVcc \ R \frac{16}{32}, \ r/m \frac{16}{32}$$

Она выполняет следующие действия:

```
if(cc){  
ОП1 ← ОП2  
}
```

Вторая рассмотренная стратегия дает выигрыш по времени выполнения работы.

Конвейер – совмещение разных действий в один момент времени

Пусть есть электрическая цепь, которая реализует выполнение команды. Можно разделить их на части, делая так, чтобы в один и тот же момент времени идет наложение последовательности выполняемых команд. Действие может разрезаться на три этапа, как описано ниже. **Конвейер** – совмещение разных действий в один момент времени.

- Общая для различных предметных областей методика
- Длительность обработки неизменна или несколько увеличивается
- Увеличение пропускной способности

На рис. 7.6 описан процесс наложения последовательности команд. В данном случае действие нарезается на три этапа. Как только первый этап кончился – начинается следующий этап следующего действия. Применительно к команде такое разделение приводит к понятию конвейера команды. Это позволяет независимо разрабатывать и развивать логику каждого отделочного этапа. Тогда на выходе конвейера каждая команда будет завершаться и выполнять свою функцию не за весь период ее выполнения, а за длительность работы отдельного этапа. Чем больше этапов и чем быстрее они выполняются, тем больше общая пропускная способность по командам. Такой подход сейчас используется во всех современных процессорах.

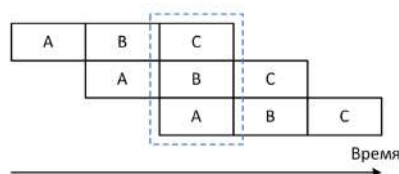


Рис. 7.6: Иллюстрация к объяснению

Таким образом, мы получаем увеличение пропускной способности, так как отдельный такт соотносится с работой отдельного этапа. Однако, следует учитывать, что для работы конвейера нужно обеспечивать его непрерывную загрузку. Каждый такт нужно отправлять на него команду, которая будет выполняться все этапы. Например, конвейер, состоящий из пяти этапов (извлечение инструкции из памяти, декодирование,

загрузка операндов, сама операция и запись результат) представлена на рис. 7.7. Здесь первая команда достигает этапа записи (рис. 7.7). За ней далее следуют другие команды. Рассмотрим выполнение последовательности команд с прыжком. Сначала происходит извлечение сравнения. В следующий момент времени эта команда начинает декодироваться, извлекается идущая за ней непосредственно в памяти. В следующий момент времени подтягиваются для сравнения значения регистров `edx` и `eax`. Далее команда второй строки декодируется, а команда третьей строки – извлекается. Далее (четвертый столбец на рис. 7.7) команда первой строки – сравнение, команда второй строки декодируется с меткой, команда третьей строки декодируется, команда четвертой строки – извлекается. Далее – этап записи. В этот момент времени мы переходим на следующий прыжок (так как уже получен некоторый результат), обновляем регистр флагов и перепрыгиваем на метку `.L6`. Здесь становится понятно, что запись результата – это перезапись регистра `EIP`, соответственно, нужно выкинуть всё из конвейера. Здесь пять этапов, соответственно, нужно выкинуть четыре команды, переставить счетчик команд на шестую строку и с него начать заново. После этого мы будем ждать пять тактов, пока конвейер полностью наполнится (пока первая загруженная команда дойдет до этапа записи результата). Чем больше неожиданных для загрузки конвейера, тем хуже выполняется работа.

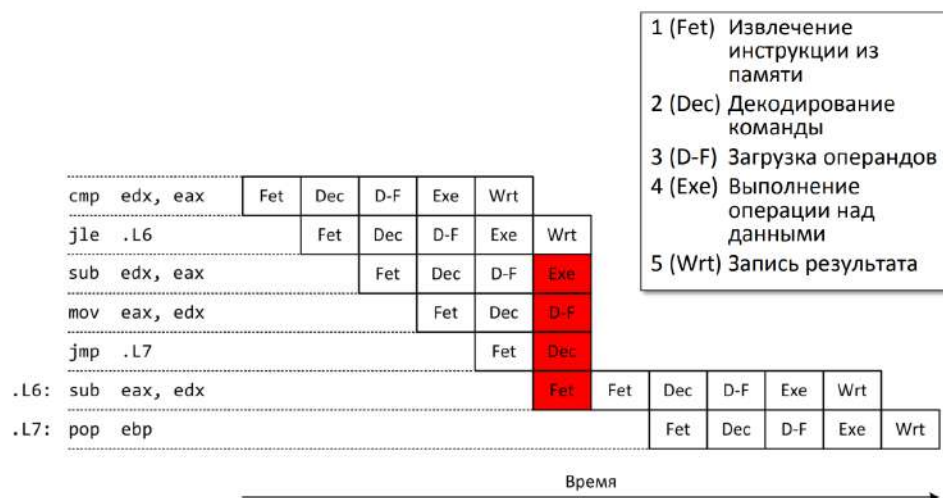


Рис. 7.7: Пример конвейера

Рассмотрим следующий пример вычисления модуля разности:

```
int absdiff(int x, int y) {
    int result;
    if (x > y) {
        result = x-y;
    } else {
        result = y-x;
    }
}
```

```
    }  
    return result;  
}
```

Более короткая запись (через тернарный оператор):

```
int absdiff(int x, int y) {  
    return (x > y)? x-y: y-x;  
}
```

Чтобы реализовать код с применением условной передачи данных, потребуется еще пара регистров. На тот момент, который рассматривается в ассемблерном коде ниже, параметры x и y загружены в регистры (рис. 7.8). x мы копируем в edx , после чего вычисляется разность $x - y$. Далее, в eax мы вычисляем обратную разность. Далее сравниваем edi и esi (то есть x) больше y , величина $x - y$ отправляется в eax . Отметим, что в данном случае все действия выполнили, ни разу не перезагрузив конвейер.

```
absdiff:  
    ...  
    mov edx, edi  
    sub edx, esi ; tmp_val:edx = x-y  
    mov eax, esi  
    sub eax, edi ; result:eax = y-x  
    cmp edi, esi ; Compare x:y  
    cmovg eax, edx ; If >, result:eax = tmp_val:edx  
    ...
```

Регистр	Значение
edi	x
esi	y

Рис. 7.8:
Расположение
параметров

Однако в некоторых ситуациях у нас нет возможности сводить процесс к условной передаче данных. Например,

```
int **p;  
xp? **p:0;
```

Попробуем реализовать указанное выше, отправив результат в eax и считая, что само значение переменной p лежит в edx . Казалось бы, это можно реализовать командами:

```
MOV EAX, 0  
TEST EDX, EDX  
CMOVNE EAX, [EDX]
```

Однако, вне зависимости от того, будет ли запись, команда начнет обращаться по адресу, указанному в третьей строке. Если этот адрес не ноль и указывает в правильное место, где что-то лежит, то все работает. Но если EDX показывает на ноль, команда не успеет до конца отработать – будет аварийное завершение. Такая ситуация называется **сбой при выполнении**. Возможна и другая ситуация – наличие **побочных эффектов** (например, присвоение каких-то значений в переменные и др.). Тогда, в результате использования

описанной выше стратегии, мы можем получить срабатывание побочного эффекта, которого не должно быть. Третий случай – **”тяжелые”вычисления**. В этом случае не достигается выигрыш по производительности.

Оператор do-while

В языке Си имеется три основных вида циклов. Рассмотрим реализацию цикла do-while. Тело цикла реализуется как есть, потом доходим до проверки и без обращения условий вычисляем expression. Если он верный, прыгаем обратно на метку. Таким образом, конструкция сводится к go-to-форме. Рассмотрим реализацию:

```
int pcount_do(unsigned x) {
    int result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if (x)
        goto loop;
    return result;
}
```

Ассемблерный код:

```
    mov ecx, 0 ; result = 0
.L2: ; loop:
    mov eax, edx
    and eax, 1 ; t = x & 1
    add ecx, eax ; result += t
    shr edx, 1 ; x >>= 1
    jne .L2 ; If !0, goto loop
```

Здесь ecx – общая сумма обнаруженных единиц в параметре функции pcount_do, в edx – текущее значение x. При реализации значение x будет разрушаться, поэтому мы копируем его в eax. Далее мы прибавляем его к ecx, далее текущее значение x сдвигается на 1. Сдвиг влияет на некоторые флаги, поэтому мы проверяем значение флага (если не ноль, прыгаем назад).

Оператор while

Другие циклы можно свести к форме, описанной выше. Например, для цикла while:

```
int pcount_while(unsigned x) {
    int result = 0;
```

```
while (x) {
    result += x & 0x1;
    x >>= 1;
}
return result;
}
```

Отсоединяем однократную проверку и переносим ее в начало:

```
int pcount_do(unsigned x) {
    int result = 0;
    if (!x) goto done;
loop:
    result += x & 0x1;
    x >>= 1;
    if (x)
        goto loop;
done:
    return result;
}
```

Если условия проверки не выполнены, мы переходим на done. То, что осталось, можно свести к do-while:

```
int pcount_do(unsigned x) {
    int result = 0;
loop:
    if (!x) goto done;
    result += x & 0x1;
    x >>= 1;
    goto loop;
done:
    return result;
}
```

Оператор for

Аналогично для оператора for. Сначала мы приводим его к оператору while, а потом – к do-while. Например:

```
#define WSIZE 8*sizeof(int)

int pcount_for(unsigned x) {
```

```
int i;  
int result = 0;  
for (i = 0; i < WSIZE; i++) {  
    unsigned mask = 1 << i;  
    result += (x & mask) != 0;  
}  
return result;  
}
```

Разворачиваем первую проверку:

```
int pcount_for_gt(unsigned x) {  
    int i;  
    int result = 0;  
    i = 0;  
    if (!(i < WSIZE))  
        goto done;  
loop:  
    {  
        unsigned mask = 1 << i;  
        result += (x & mask) != 0;  
    }  
    i++;  
    if (i < WSIZE)  
        goto loop;  
done:  
    return result;  
}
```

В машине есть специальная команда для работы с циклами – LLOP. Однако, на практике она не используется. Данная команда предполагает некоторое итерирование:

LOOP rel8

Существуют вариации:

LOOPE rel8

LOOPNE rel8

Во всех трех случаях операндом выступает метка. Рассмотрим фрагмент кода:

```
int fib(int x) { // x >= 1  
    int i;  
    int p_pred = 0;
```

```
    int pred = 1;
    int res = 1;
    x--;
    for (i = 0; i < x; i++) {
        res = p_pred + pred;
        p_pred = pred;
        pred = res;
    }
    return res;
}
```

Тогда:

```
fib:
    push ebp
    mov ebp, esp
    push ebx

    mov ecx, dword [ebp + 8] ; x
    xor edx, edx ; p_pred
    mov ebx, 1 ; pred
    mov eax, 1 ; res
    dec ecx

    jecxz .end

.loop:
    lea eax, [edx + ebx]
    mov edx, ebx
    mov ebx, eax
    loop .loop

.end:
    pop ebx
    pop ebp
    ret
```

В этом фрагменте кода есть loop, за ним идет метка, которая потом ассемблером будет заменена на некоторое числовое смещение. Во всех трех случаях команда уменьшает ECX на 1 и переходит по метке в случае, если ECX не ноль:

$$ECX \leftarrow ECX - 1$$

jmp rel8 if ECX ≠ 0

В случае LOOPE:

$\&ZF == 1$

В случае LOOPNE:

$\&ZF == 0$

Таким образом, есть некоторый специально выделенный регистр-счетчик, в который загружается число итераций, которые мы хотим выполнить. Далее мы проходим по телу цикла, доходим до рассмотренной команды, которая каждый раз уменьшает ECX, и итерируемся.

Если же мы пришли к циклу, а число итераций заранее неизвестно (это некоторая переменная величина) и оказывается, что число итераций равно нулю. В этом случае мы вначале ноль уменьшаем на 1, получаем 32 единицы. Проверка следующего условия говорит, что полученное число – не ноль, соответственно, начинается бесконечное число итераций. Для этого случая есть некоторое техническое решение – специальная команда `jesxz rel8`, то есть мы куда-то прыгаем, только если `ECX=0`. Рассмотрим следующий пример реализации чисел Фибоначчи через цикл `for`:

```
int fib(int x) { // x >= 1
    int i;
    int p_pred = 0;
    int pred = 1;
    int res = 1;
    x--;
    for (i = 0; i < x; i++) {
        res = p_pred + pred;
        p_pred = pred;
        pred = res;
    }
    return res;
}
```

Ассемблерный код:

```
fib:
    push ebp
    mov ebp, esp
    push ebx

    mov ecx, dword [ebp + 8] ; x
    xor edx, edx ; p_pred
    mov ebx, 1 ; pred
    mov eax, 1 ; res
```

```
    dec ecx

    jecxz .end
.loop:
    lea eax, [edx + ebx]
    mov edx, ebx
    mov ebx, eax
    loop .loop

.end:
    pop ebx
    pop ebp
    ret
```

Лекция 8. Оператор управления switch

Обратная задача

Рассмотрим следующую задачу. Дан некоторый ассемблерный код, который реализует оператор while.

f:

```
...  
mov edx, dword [ebp+8] ; (1)  
mov eax, 0 ; (2)  
test edx, edx ; (3)  
je .L7 ; (4)  
.L10: ;  
xor eax, edx ; (5)  
shr edx, 1 ; (6)  
jne .L10 ; (7)  
.L7: ;  
and eax, 1 ; (8)  
...
```

Требуется правильно заполнить пропуски:

```
int f(unsigned x) {  
    int val = 0;  
    while (_____) {  
        _____;  
        _____;  
    }  
    return _____;  
}
```

Отметим, что в данном случае только одно обращение в память. Под многоточием скрывается типовой пролог. Согласно первой строчке, $EDX|_x$. Далее идет присвоение: $EAX|_{val}$. В (3) и (4) строках происходит некоторое сравнение величины с нулём. Нарисуем граф потока управления (рис. 8.1). Реализация сравнение-переход осуществляется парой команд (3)-(4). Если EDX нулевой, то осуществляется переход, если нет – продолжается естественный ход, переходим на пятую строку. В (5) и (6) строках реализуются некоторые побитовые операции. В (7) строке реализуется еще одно ветвление (через использование флаг нуля). Таким образом, мы можем записать в виде ассемблерного листинга go-to-формы

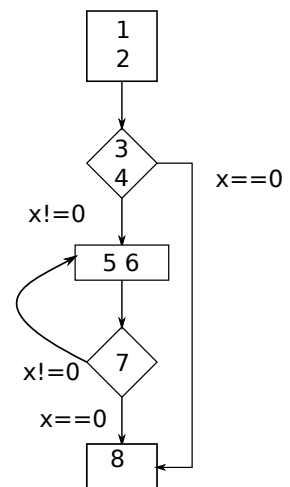


Рис. 8.1: Граф потока управления

```
if (x==0)
    goto end;

loop:
    val ^= x;
    x >>= 1;
    if (x)
        goto loop;
end:
    return val & 1;
```

Таким образом, получаем:

```
while (x) {
    val ^= x;
    x >>= 1;
}
return (val & 1);
```

Оператор управления switch

Рассмотрим следующий кусок в кода в качестве иллюстрации работы switch:

```
enum TargetPosition {
    TARGET_AT_BEGINNING,
    TARGET_AT_MIDDLE,
    TARGET_AT_END
};

switch (targetPosition) {

case TARGET_AT_BEGINNING:
    offsetInWindow = 0;
    break;
case TARGET_AT_MIDDLE:
    offsetInWindow = MIN (newSize - size, newSize / 2);
    break;
case TARGET_AT_END:
    offsetInWindow = newSize - size;
    break;
default:
    _error_code = IllegalData;
```

```
_error_msg = tr("requested target position"  
               " is not a TargetPosition enum member");  
}
```

В данном примере есть три условия, в зависимости от которых управление расходится по трем разным вариантам, а также ветка default, которая применяется в том случае, когда ничего не подошло. Если изобразить это графически в виде некоторого графа, получим следующее (рис. 8.2).

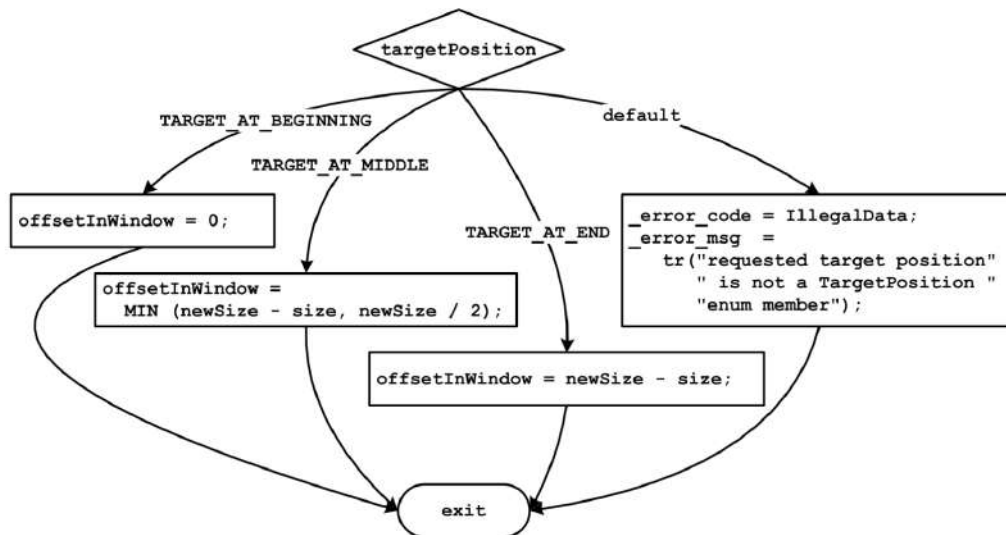


Рис. 8.2: Иллюстрация к объяснению

В блок-схеме наверху указано условие, в ней есть выражение-селектор, которое определяет выбор пути. Далее это выражение должно приходиться к какому-то целому числу. Если ничего не подходит, переходим в default. В конце все приходит в общую точку. Структура такого подграфа достаточно регулярна. Такой вариант может быть реализован следующим образом. Приведем описанное выше к цепочке if-else (см. код ниже). Отметим, что в среднем для того, чтобы пройти по такому коду, необходимо провести примерно $\frac{n-1}{2}$ сравнений, где n – число вариантов, приведенное в switch.

```
enum TargetPosition {  
    TARGET_AT_BEGINNING,  
    TARGET_AT_MIDDLE,  
    TARGET_AT_END  
};  
  
if (TARGET_AT_BEGINNING == targetPosition) {  
    offsetInWindow = 0;  
} else if (TARGET_AT_MIDDLE == targetPosition) {
```

```
    offsetInWindow = MIN (newSize - size, newSize / 2);
} else if (TARGET_AT_END == targetPosition) {
    offsetInWindow = newSize - size;
} else {
    _error_code = IllegalData;
    _error_msg = tr("requested target position"
                    " is not a TargetPosition "
                    " enum member");
}
```

На ассемблере это может быть реализовано в виде (в `edx` помещено значение управляющего выражения):

```
; .. targetPosition

    cmp edx, TARGET_AT_BEGINNING
    jne .comp2
; code for case TARGET_AT_BEGINNING:
    jmp .switch_exit

.comp2:
    cmp edx, TARGET_AT_MIDDLE
    jne .comp3
; code for case TARGET_AT_MIDDLE:
    jmp .switch_exit

.comp3:
    cmp edx, TARGET_AT_END
    jne .default
;     case TARGET_AT_END:
    jmp .switch_exit

.default:
;     default:
.switch_exit:
```

По этой цепочке мы двигаемся линейно. Сначала идет проверка на равенство с некоторой константой. Если условие не выполняется, то мы переходим на следующее сравнение. Если условие выполняется, то естественным ходом мы переходим к той части кода, которая реализует первую ветку. После ее выполнения мы переходим безусловным прыжком на выход. В результате такой последовательности действий последний блок должен быть `default`. Если все явные проверки не выполняются, то мы переходим к

последнему блоку и, соответственно, к выходу. Все это работало только потому, что в рассмотренном примере каждый кусок кода (каждая ветка) заканчивался оператором `break`, который явно передавал управление на выход из текущего оператора. Если не писать эти дополнительные операторы, мы получим несколько другую картину, которая соответствует чистому оператору `switch`, в котором есть его заголовок и тело, внутри которого находится некоторое количество `case`, среди которых, возможно, есть `default` (рис. 8.3).

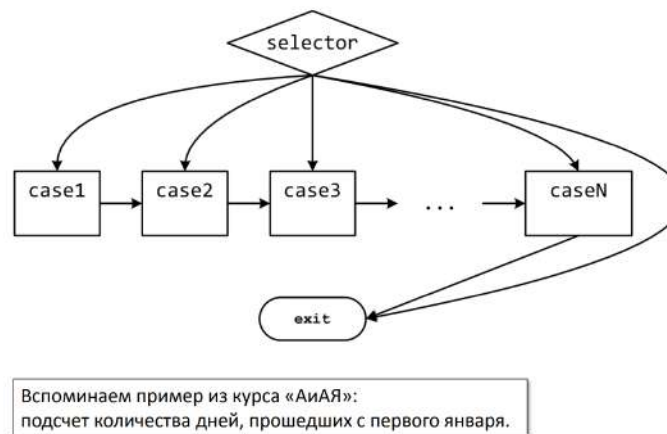


Рис. 8.3: Иллюстрация к объяснению

Если операторы `break` отсутствуют, мы находим какое-то ветвление в зависимости от текущего значения селектора, попадаем на соответствующий кусок кода и далее движемся до самого конца вправо (рис. 8.3). Таким образом, какие-то куски кода могут покрываться разными условиями и мы можем переходить туда по разным путям. Однако такая конструкция, в которой не получается создать один вход и один выход, неудобна для восприятия.

В случае появления `break` граф приобретает вид, указанный на рис. 8.4.

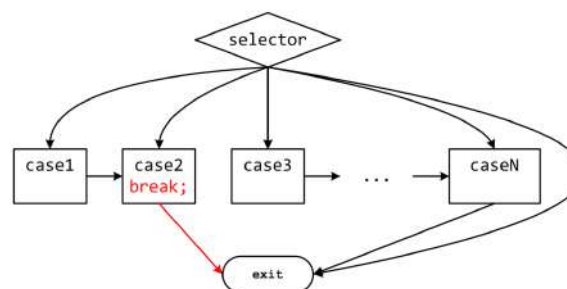


Рис. 8.4: Иллюстрация к объяснению

Duff's Device

Рассмотрим конструкцию под названием Duff's Device. Реализуем цикл, в котором происходит копирование данных с одного буфера на другой. Само копирование реализуется одной командой. Далее нужно проверять условия; в зависимости от того, выполнилось ли условие, мы перепрыгиваем обратно в начало цикла (а тело цикла – всего одна команда). Таким образом, цикл суммарно реализуется тремя командами, из которых полезную функциональность осуществляет только одна, остальные команды – накладные расходы на организацию цикла. Попробуем решить эту проблему путем развертывания цикла. Рассмотрим простой случай – заранее известно, что число итераций будет кратно некоторому числу, например, восьми. Тогда число итераций можно поделить на 8 и проверять на границу, уменьшенную в 8 раз. Тело цикла размножаем восьмикратно, раскопировав код:

```
void duffs_device(char *to, char *from, int count) {  
  
    register n = (count + 7) / 8; /* count > 0 assumed */  
  
    switch (count % 8) {  
        case 0: do { *to = *from++;  
        case 7: *to = *from++;  
        case 6: *to = *from++;  
        case 5: *to = *from++;  
        case 4: *to = *from++;  
        case 3: *to = *from++;  
        case 2: *to = *from++;  
        case 1: *to = *from++;  
            } while (--n > 0);  
    }  
}
```

Таким образом, код полезной нагрузки, который должен был выполняться, образует последовательно выполняющийся блок сильно увеличенного объема и соотношение полезного и служебного кодов меняется.

Рассмотрим стратегии реализации switch. Наша основная задача – сложность сравнений $\frac{n-1}{2}$ ($O(n)$), которая появилась при реализации цепочек if, была уменьшена до $O(1)$. Рассмотрим следующий пример:

```
long switch_eg  
    (long x, long y, long z)  
{
```



```
    long w = 1;
    switch (x) {
    case 1:
        w = y*z;
        break;
    case 2:
        w = y/z;
    case 3:
        w += z;
        break;
    case 5:
    case 6:
        w -= z;
        break;
    default:
        w = 2;
    }
    return w;
}
```

В данном случае для одного блока используется несколько различных меток. Есть ситуация, когда `break` отсутствует и управление “проваливается” – переходим на кусок кода, который соотносится с `case 3`. Некоторые значения могут отсутствовать (например, нет метки со значением 4). Отметим, что величины в кейсах распределены на достаточно компактом отрезке $[1, 6]$. В самом языке Си нет конструкций, которые позволили бы нам уменьшить сложность сравнений. Однако, компилятор `gcc` имеет некоторые расширения стандартного языка Си. В частности, имеется возможность завести массив, который ссылается на метки:

```
static void *JTab[]={&l_A, &l_B}
```

Таким образом, можно сформировать некоторую таблицу переходов. Тогда, если есть возможность посчитать некоторый индекс, в этом расширении языка Си появляется возможность сделать `goto` на одну из меток:

```
int index;
goto JTab[index];
```

Далее мы размещаем эти метки, что-то реализуем в помеченных областях:

```
l_A:
    ....
l_B
```

В ассемблере есть команда `jmp`, операндом которой может быть не только метка, но и регистр или память: `jmp R/m/i`. Отметим, что в рассматриваемом примере переменную `w` нужно инициализировать не во всех ветках, поэтому присвоение мы будем выполнять только в тех случаях, когда оно действительно требуется. Поэтому в коде ниже инициализация переменной `w` отложена на потом. Рассмотрим часть кода:

```
long switch_eg(long x, long y, long z) {
    long w = 1;
    switch (x) {
        . . .
    }
    return w;
}
```

Для него:

```
switch_eg:
    push ebp
    mov ebp, esp

    mov edx, dword [ebp + 8]    ; edx = x
    mov eax, dword [ebp + 12]   ; eax = y
    mov ecx, dword [ebp + 16]   ; ecx = z

    dec edx
    cmp edx, 5                  ;      (x-1)    5
    ja .L8                      ;      >u goto default
    jmp [.L4 + 4*edx]            ; goto *JTab[x]
```

Сначала идет типовой пролог. Далее появляются три переменные, которые берутся из параметров: $EDX|_x$, $EAX|_y$, $ECX|_z$. Все три регистра можно использовать сразу, предварительно сохранять их не надо (согласно принятому соглашению вызова). Далее переходим к `switch`. Чтобы куда-то перепрыгнуть, нужно где-то разместить метки. Впоследствии мы будем использовать некоторое количество меток; их адреса можно поместить в некоторый массив. Этот массив принято размещать в секции, которая доступна только для чтения. Секция `.rodata` – одна из типовых секций, куда принято помещать данные, доступные только на чтение. Начало этого массива будет отмечено меткой `L4`. Далее мы берем те значения кейсов, которые просматривали ранее, и соотносим их с некоторыми метками. Запишем массив значений `x` и их соотношения с локальными метками:

```
section .rodata align=4
.L4:
```

```
dd .L3 ; x = 1
dd .L5 ; x = 2
dd .L9 ; x = 3
dd .L8 ; x = 4
dd .L7 ; x = 5
dd .L7 ; x = 6
```

Учтем, что значения 5 и 6 были склеены и относятся к одному и тому же блоку, поэтому метка для них будет одинаковая. 1,2,3 относятся к разным частям кода, поэтому метки разные. Значения 4 не было, но его можно соотнести с выходом из цикла или с default. Далее возникает вопрос: можем ли мы вообще обращаться внутрь массива (массив имеет определенную длину, а x – произвольный). Рассмотрим границы

$$A \leq_c x \leq_c B$$

Это аналогично беззнаковому сравнению:

$$x - A \leq_U B - A$$

Рассмотрим числовой круг, на котором мы расположим значения битовых векторов, с которыми мы работаем. Для знаковых величин числовой круг изображен на рис. 8.5. Здесь w – разрядность. Тогда максимальное число $-2^{w-1} - 1$, переход на отрицательные значения начинается с $-2^{(w-1)}$. Далее, прибавляя по одному разряду, будем приходить к -1 . Вместе с тем все битовые вектора можно рассматривать как беззнаковые числа (внутренний круг на рис. 8.5). Рассмотрим некоторый сектор с границей $B - A$. Учтем, что интерпретация числовых кругов совпадает для беззнакового и знакового представления в правой части круга. Соответственно, если взять сектор так, как указано на рис. 8.5, получим, что проверка попадания в данный сектор для знаковых чисел равносильна двум проверкам – проверка на правую границу $B - A$ и проверка на то, что мы не попадаем в отрицательные числа. Однако, в случае беззнакового представления тот же набор битовых векторов начинается от 0, соответственно, одного сравнения будет достаточно.

Приведем сравниваемую величину к нулевой базе. Переходим от сравнения вида:

$$1 \leq_c x \leq_c 6$$

к сравнению вида

$$x - 1 \leq_U 5$$

Тогда, если x не попадает в указанный отрезок, то не существует первой ячейки ($x=1$) и мы не имеем права обращаться внутрь массива. Получаем:

```
switch_eg:
```

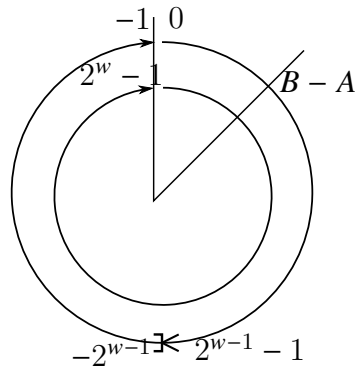


Рис. 8.5: Числовые круги

```

push ebp                                ;
mov ebp, esp                            ;

mov edx, dword [ebp + 8]                ; edx = x
mov eax, dword [ebp + 12]               ; eax = y
mov ecx, dword [ebp + 16]               ; ecx = z

dec edx

cmp edx, 5                             ; сравниваем (x - 1) и 5
ja .L8                                  ; если >_u goto default
jmp [.L4 + 4*edx]                       ; goto *JTab[x]

```

Чтобы описанное выше работало, метки должны гарантированно лежать в памяти без каких-либо промежутков. Таким образом, имеем

- Организация таблицы переходов:
 - Каждый элемент занимает 4 байта
 - Базовый адрес - .L4
- Переходы
 - **Прямые:** jmp .L2
 - Для обозначения цели перехода используется метка .L2
 - **Косвенные:** jmp [.L4 + 4*edx]
 - Начало таблицы переходов .L4
 - Коэффициент масштабирования должен быть 4 (в IA-32 метка содержит 32 бита = 4 байта)
 - Выбираем цель перехода через исполнительный адрес $.L4 + edx * 4$
 - * Только для $x : 0 \leq x - 1 \leq 5$

Перейдем к следующему вопросу: как размещать тело switch? Рассмотрим начало оператора switch:

```
switch(x) {  
case 1: // .L3  
    w = y*z;  
    break;  
  
    . . .  
  
case 5:  
case 6: // .L7  
    w -= z;  
    break;  
default: // .L8  
    w = 2;  
}
```

Первая метка L3 соответствует умножению. Здесь используем двухадресное умножение `imul`, результат – в EAX; после этого переходим на метку L2, которая соответствует выходу из цикла:

```
.L3: ; x == 1  
    imul eax, ecx ; w = y*z;  
    jmp .L2 ; goto switch_end
```

Продолжение оператора switch:

```
switch(x) {  
    . . .  
    case 2: // .L5  
        w = y/z;  
  
/* «проваливаемся» на точку слияния .L6 */  
  
    case 3: // .L9  
        w += z;  
        break;  
    . . .  
}
```

Здесь мы переходим к знаковому делению (L5). После этого сразу переходим на L6 – к точке слияния. L9 – начала case 3, где требуется инициализировать значение w. Далее доходим до точки слияния, где выполняется сложение и переходим на L2, то есть на выход.

```
.L5: ; x == 2
```

```
        cdq                ;
        idiv ecx           ; w = y/z;
        jmp .L6           ; goto merge (.L6)
.L9:                ; x == 3
        mov eax, 1        ; w = 1;

.L6:                ; точка слияния
        add eax, ecx      ; w += z;
        jmp .L2          ; goto switch_end
```

Окончание оператора switch:

```
switch(x) {
case 1: // .L3
    w = y*z;
    break;
    . . .
case 5:
case 6: // .L7
    w -= z;
    break;
default: // .L8
    w = 2;
}
```

L7 – ещё одна инициализация, далее вычитаем $w -= z$. Далее переходим к default одним из двух способов: первичная проверка показывает непопадание в границы или из метки L8.

```
.L7:                ; x == 5 or x == 6
        mov eax, 1    ; w = 1;
        sub eax, ecx  ; w -= z;
        jmp .L2       ; goto switch_end
.L8:                ; default
        mov eax, 2    ; w = 2

.L2:                ; выход из switch
```

Завершение функции:

```
return w;
```

В EAX находится требуемый результат, поэтому никаких дополнительных действий не требуется. Осталось выполнить типовой эпилог: `pop ebp`

`ret` ; возвращаемое значение уже размещено в `EAX` Отметим, что всё описанное выше со сложностью $O(1)$ только тогда, когда значения достаточно компактно сгруппированы. Тогда можно сделать таблицу переходов. Если при этом достаточное число `case`'в и между ними нет промежутков, тогда компилятор скорее всего выберет рассмотренную выше стратегию, реализуя `switch`. Если `case`'в мало, то скорее всего он выберет стратегию с цепочкой `if`'в. Подведем итог.

- Преимущества таблицы переходов:
 - Применение таблицы переходов позволяет избежать последовательного перебора значений меток
 - * Фиксированное время работы
 - Позволяет учитывать «дыры» и повторяющиеся метки
 - Код располагается упорядоченно, удобно обрабатывать «пропуски»
 - Инициализация `w = 1` не проводилась до тех пор пока не потребовалась
- В качестве меток используем значения типа `enum`

Иногда значения могут сильно расходиться. Например:

```
int f(int n, int *p) {
    int _res;
    switch (n) {
    default:
        _res = 0;
    case 1:
        *p = _res;
        break;
    case 64:
        _res = 1;
        break;
    case 63:
        _res = 2;
        *p = _res;
    case 256:
        _res = 3;
        break;
    case 65536:
        _res = 4;
    }
    return _res;
}
```

- В этом случае компилятор не будет строить таблицу переходов (таблица переходов получается неприемлемо большой). Для данного случая можно применить третью стратегию. Рассмотрим двоичное дерево поиска (рис. 8.6). Отметим, что в данном случае при выполнении условного перехода флаги не меняются. Поэтому, оказавшись в этом дереве в каком-то месте, мы можем разойтись по нему по трем направлениям. Во-первых, мы можем проверить на совпадение с конкретным ключом, который относится к текущей вершине:

```
cmp <selector>, val_i
```

Если совпало, можно сразу перепрыгнуть на ветку:

```
je label_j
```

Далее можно еще раз попытаться выполнить условный переход на какую-то другую метку:

```
jle label_{j+1}
```

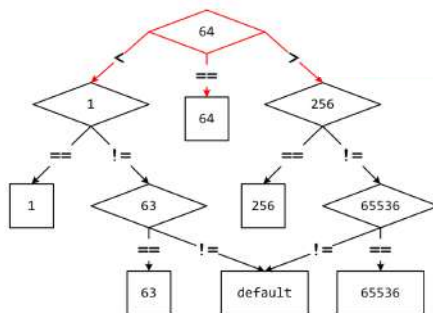


Рис. 8.6: Двоичное дерево поиска

Для данного примера:

```
...  
push ebp  
mov ebp, esp  
mov eax, 1  
mov edx, dword [ebp+8] ; n  
cmp edx, 64  
je .L9  
jle .L13  
...
```

Обратная задача

Рассмотрим следующую обратную задачу


```
int switchMeOnce(int x) {  
    int result = 0;  
  
    switch (x) {  
        . . .  
    }  
  
    return result;  
}
```

Ассемблерный код:

```
section .text  
    . . .  
    mov eax, dword [ebp+8]  
    add eax, 2  
    cmp eax, 6  
    ja .L2  
    jmp [.L8 + 4*eax]  
    . . .  
  
section .rodata  
    .L8 dd .L3, .L2, .L4, .L5, .L6, .L6, .L7
```

Нужно ответить на следующие вопросы:

1. Сколько раз было использовано ключевое слово case?
2. Какие константы использовались?
3. Какие ветки выполнения были объединены?
4. Что помечено .L2?

Для решения задачи рассмотрим таблицу соответствия индексов и меток (таблица 4).
Видно, что код выполняет следующие действия:

индексы	0	1	2	3	4	5	6
метки	13	12	14	15	16	16	17

Таблица 4: Таблица индексов

$$x + 2 \leq_U 6$$

В виде знаковых сравнений эти действия имеют вид:

$$-2 \leq_c x \leq_c 4$$

Таким образом, для x получаем следующие значения (см. таблицу. 5).

индексы	0	1	2	3	4	5	6
метки	L3	L2	L4	L5	L6	L6	L7
x	-2	-1	0	1	2	3	4

Таблица 5: Таблица значений

Если мы не попадаем внутрь рассматриваемого массива, то мы перепрыгиваем на метку L2. Это означает, что в этом месте не будет прыжка на кейс с числом. Таким образом, ответ на первый вопрос – 6. Всего элементов семь, в случае индексов 4 и 5 метка одна и та же. С кейсами соотносятся -2, 0, 1, 2, 3, 4. Ветки выполнения 2 и 3 были объединены. На четвертый вопрос нельзя ответить, пока мы не рассмотрим тело самого switch.

Указатели

Рассмотрим операцию взятия адреса со стороны ассемблера. Рассмотрим случай статической памяти:

```
static int *p_i;  
static char *p_c;
```

Адреса 32-разрядные, поэтому на первый указатель выделяется 4 байта. Этот указатель показывает куда-то еще (рис. 8.7). Следующие четыре байта ссылаются на char.

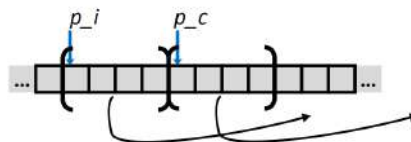


Рис. 8.7: Иллюстрация к объяснению

Если мы хотим обратиться с char (то есть выполнить `*p_c;`), нужно составить последовательность из двух пересылок:

```
mov eax, dword [p_c]  
mov al, byte [eax]
```

Чтобы взять адрес переменной (`&p_i;`), то должна быть выполнена пересылка величины в регистр:

```
mov eax, p_c
```

Рассмотрим случай автоматической локальной переменной. Пусть есть некоторая переменная `tmp`, которая объявлена внутри тела функции:

```
int f() {  
    int* tmp;...  
;  
}
```

Будем считать, что переменная `tmp` была сохранена сразу под сохраненным `ebp` (рис. 8.8). Тогда взятие адреса (`*tmp;`) соответствует

```
mov eax, [ebp-4]  
mov eax, [eax]
```

Операция `&tmp` соответствует

```
mov eax, ebp  
sub eax, 4
```

Есть способ определить указанный адрес одной командой:

```
lea eax, [ebp-4]
```



Рис. 8.8: Иллюстрация к объяснению

Массивы

Во время объявления массива одновременно происходит два процесса. Выделяется непрерывно идущая последовательность байт, каждый следующий элемент идет сразу за предыдущим. При этом происходит связывание имени массива с адресом первого элемента. Общий размер массива – произведение его длины на байтовый размер каждого элемента. Отметим, что в терминах языка Си есть совместимые типы. Например, рассмотрим массив элементов типа `T`, размер массива – `L`. Этот массив располагается в непрерывном блоке памяти размером `L * sizeof(T)` байт. Идентификатор `A` может использоваться как указатель на элемент массива с индексом 0. Тип указателя – `T*`.

Лекция 9. Многомерные массивы.

Массивы

Пусть есть некий массив A размера L с элементами типа T : $A[L]$. Это означает, что на уровне ассемблера была заведена метка, под нее зарезервирована память:

```
A res L*<sizeof T>
```

Мы можем работать с данным базовым адресом, после которого данные тянутся последовательно. Для обозначения адреса памяти будем использовать нотацию: χ_A – адрес A . Конструкция $M[\dots]$ – обращение к памяти. Рассмотрим следующие массивы (массивы переменных `int` и `short` и статический указатель):

```
int E[...];
short S[...];
static int *p;
```

Будем считать, что само значение расположено в `eax`, а базовый адрес массива – в `edx`, а в `ecx` – некоторая вспомогательная переменная. Заполним следующую таблицу (рис. 9.1):

Выражение	Тип	Значение	Ассемблерный код
E	<code>int*</code>	χ_E	<code>MOV EAX, EDX</code>
$E[0]$	<code>int</code>	$M[\chi_E]$	<code>MOV EAX, [EDX]</code>
$E[i]$	<code>int</code>	$M[\chi_E + 4 * i]$	<code>MOV EAX, [EDX + 4 * ECX]</code>
p	<code>int*</code>	$M[\chi_p]$	<code>MOV EAX, [p]</code>
$S+1$	<code>short*</code>	$\chi_S + 2$	<code>LEA EAX, [EDX + 2]</code>
$S[3]$	<code>short</code>	$M[\chi_S + 6]$	<code>MOV EAX, [EDX + 6]</code>
$\&S[i]$	<code>short*</code>	$\chi_S + 2 * i$	<code>LEA EAX, [EDX + 2 * ECX]</code>
$S[4 * i + 1]$	<code>short</code>	$M[\chi_S + 8 * i + 2]$	<code>MOV EAX, [EDX + 8 * ECX + 2]</code>
$S + i - 5$	<code>short*</code>	$\chi_S + 2 * i - 10$	<code>LEA EAX, [EDX + 2 * ECX - 10]</code>
$\&E[i] - E$	<code>ptrdiff_t</code>	i	<code>MOV EAX, ECX</code>

Рис. 9.1: Иллюстрация к объяснению

Первое выражение – переменная E , массив. По сути его можно рассматривать как указатель (с некоторыми допущениями). Согласно принятой нотации, в этом случае значение – χ_E . Результат должен оказаться в `EAX`, значение адреса лежит в `EDX`, поэтому необходимо сделать пересылку. Во втором случае ($E[0]$) в ассемблерном коде добавляются квадратные скобки (так как базовый адрес – адрес нулевого элемента). В случае $E[i]$ в качестве значения нужно указать $M[\chi_E + 4 * i]$, так как адресная арифметика на уровне ассемблера выражается в том, что мы сами должны определять размер типа и явно домножать, чтобы получать байтовое смещение. Аналогично для ассемблерного кода. В случае указателя p имеем тип `int*`, поэтому и значение – $M[\chi_p]$. Для ассемблерного кода обращаемся к участку памяти, где расположена статическая переменная p : `MOV EAX, [p]`. Отметим, что ранее мы ввели нотацию, которая явно

определяет то, как мы будем что-то воспринимать. Так, в языке Си под `r` будет пониматься имя переменной и ее значение, а в ассемблере под `r` мы будем понимать адрес. Контекстуально это одно и то же, но в зависимости от того, куда мы помещаем `r`, смысл может меняться. В данном случае точно имеется в виду, что для получения значения Си-выражения типа `r` мы обращаемся в память по адресу, связанным с этим именем. Рассмотрим случай `&S[i]`. В данном случае имеем тип `short*` (так как чтение выражения с учетом приоритета операций происходит следующим образом: сначала рассматриваем имя переменной, потом получаем `i`-й элемент, далее берем адрес от этого элемента). Далее берем адрес, который определяется базовым адресом переменной и прибавляем к нему `2*i`. Это можно выразить одной командой – LEA.

Две обратные задачи

Рассмотрим две задачи. Даны два фрагмента кода – функции с типовыми прологами и эпилогами. Нужно заполнить пропуски для Си-кода. Первый код:

f :

```
...
mov  edx , dword ebp +
movsx ax, byte [a + edx
mov  word [b + edx + edx ], ax
...
```

Си-код:

```
----- a[N];
----- b[N];

void f(____i ){
    _____;
}
```

Вторая задача:

g :

```
...
mov  edx , dword ebp + 8
movzx eax , word [c + edx + edx]
mov  byte [d + edx ], al
...
```

Си-код:

```
----- c[N];
```

```
----- d[N];  
  
void g(-----i){  
    -----;  
}
```

В обоих случаях есть некоторые статические массивы, так как в указанных трех командах используются имена, это означает, что иметь есть базовые адреса, откуда идут эти массивы, расположенные в статической памяти. Таким образом, определить тип. Пусть размерность известна. Также указано ключевое слово `dword`, соответственно, речь идет о 32-разрядном типе. Будем считать, что это тип `int` (это не будет ничему противоречить). Таким образом, в обоих случаях EDX_i . Далее берем базовый адрес, прибавляем к нему переменную `i`, знаковым образом расширяем до двух байт и отправляем в то место, которое уже связано с массивом `b`. Отметим, что так как в коде был указан байт, речь идет о типе `char`. Таким образом, имеем

```
signed    char a[N];  
          short b[N];  
unsigned short [N];  
          char d[N];  
b[i]=a[i]
```

Рассмотрим вторую часть кода. Здесь есть беззнаковое двухбайтное обращение к массиву (см. код выше). Далее расширяем до четырех байт, отсекаем последние четыре байта и записываем результат в `b`. Таким образом, продолжение кода будет следующим:

```
d[i]=(char) c[i]
```

Добавим к одномерному массиву еще один уровень. Рассмотрим переменную типа

```
zip_dig pgh[4]
```

При этом

- *zipdig pgh[4]* эквивалентно *int pgh[4][5]*
 - Переменная `pgh`: массив из 4 элементов, расположенных непрерывно в памяти
 - Каждый элемент – массив из 5 `int`'ов, расположенных непрерывно в памяти
- Всегда разворачивание по строкам (Row-Major) (сначала берем нулевой элемент внутренней размерности, непрерывно разворачиваем его, после чего переходим к следующему элементу).

Есть две особенности данного рассмотрения. Во-первых, промежутки отсутствуют, машина гарантированно будет работать, используя ту адресную арифметику для вычисления смещений, которую мы рассмотрели выше. Во-вторых, не во всех языках программирования разворачивание идет по строкам.

Выделение памяти

Рассмотрим принципы выделения памяти. Для примера рассмотрим тип

```
int **a;
```

Так как мы рассматриваем указатель, компилятор в данном случае выделит четыре байта. Этот указатель будет ссылаться на какую-то другую область памяти размером четыре байта, так как там тоже будет указатель. Далее идет ссылка на конечный тип `int`, который тоже где-то выделен (рис. 9.2).

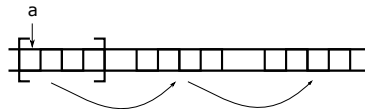


Рис. 9.2: Иллюстрация к объяснению

Далее рассмотрим следующий массив

```
int b0[20][20];
```

В этом случае компилятор гарантированно выделит непрерывный кусок памяти, базовый адрес будет связан с именем `b0`, а размер – произведение размерностей на размер типа `int` (рис. 9.2).

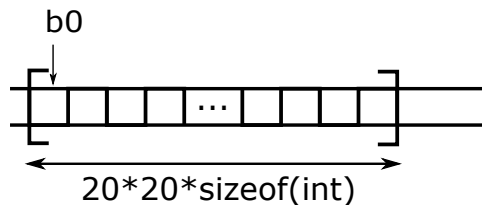


Рис. 9.3: Иллюстрация к объяснению

Если же обе размерности будут пропущены:

```
int b1[] [];
```

мы получим ошибку: `incomplete type`. Аналогично получим для случая

```
int b2[20] [];
```

Однако, в случае

```
int b3[] [20];
```

понятно, что каждый элемент представляет собой 20 `int`'в. В этом случае выделяется непрерывный буфер, массив по умолчанию будет создан единичного размера. Рассмотрим также пример

```
int* c[];
```

В этом случае предоставляется четыре байта, которые ссылаются на указатель.

Доступ к строкам

Если мы берем двумерный массив и пишем конструкцию вида $A[i]$, задав только одно индексное выражение, то на самом деле таким образом мы обращаемся к строке. В случае записи значения мы берем базовый адрес и прибавляем к нему величину i , умноженную на размер элемента и размер типа, то есть $\chi_A + i * c * sizeof(T)$. При этом обращения в память не происходит, мы вычисляем указатель. Например, в случае размерности пять:

```
int *get_pgh_zip(int index){
    return pgh[index];
}
```

```
#define PCOUNT 4
zip_dig pgh[PCOUNT] =
{{1, 5, 2, 0, 6},
 {1, 5, 2, 1, 3},
 {1, 5, 2, 1, 7},
 {1, 5, 2, 2, 1}};
```

В подобных случаях удобно использовать команду LEA:

```
; eax = index
    lea eax, [eax + 4 * eax] ; 5 * index
    lea eax, [pgh + 4 * eax] ; pgh + (20 * index)
```

Рассмотрим случай $A[i][j]$ – появляется второе индексное выражение, то есть появляется смещение до отдельного элемента внутри строки (рис. 9.4). После этого идёт обращение в память: $M[\chi_A + i * c * sizeof(T) + j * sizeof(T)]$.

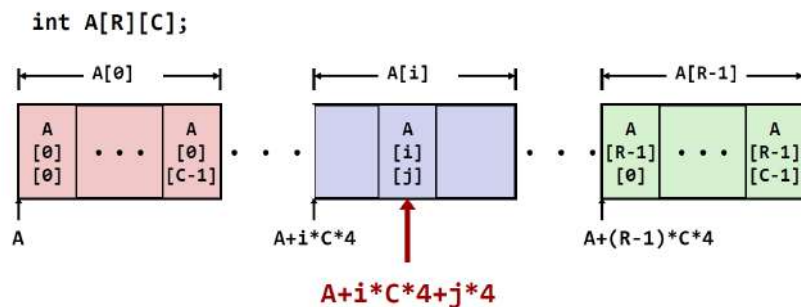


Рис. 9.4: Иллюстрация к объяснению

Рассмотрим пример:

```
int get_pgh_digit(int index, int dig)
    return pgh[index][
}
```


Ассемблерный код:

```
mov
eax , dword ebp + 8]                ; index
lea eax , eax + 4 * eax ]]           ; 5*index
add eax , dword ebp + 12]           ; index+dig
mov eax , dword pgh + 4 * eax ]]     ; 4*(5* index+dig)
```

В данном случае имеем два формальных параметра. После первой пересылки индекс отправляется в EAX. С помощью команды LEA мы умножаем его на 5 (число элементов внутри строки). Далее прибавляет второй индекс (переменная j). Таким образом, процесс состоит в следующем:

- $pgh[index][\text{тип int}]$
- Адрес : $pgh + 20 * index + 4 * dig = pgh + 4 * (5 * index + dig)$
- Вычисление адреса производится как $pgh + 4 * ((index + 4 * index) + dig)$

Рассмотрим массивы указателей. В этом случае данные не обязательно будут располагаться непрерывно друг за другом. Данная особенность имеет место для данных верхнего уровня (рис. 9.5). В этом случае указатели идут непрерывно друг за другом.

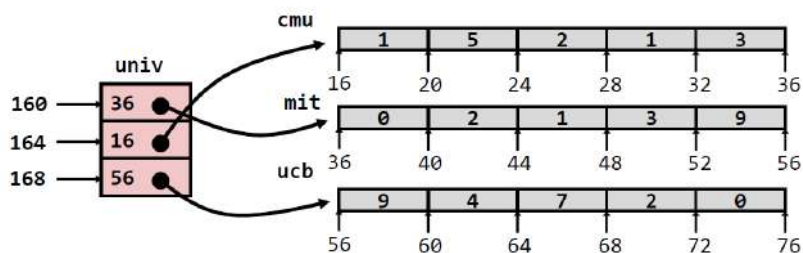


Рис. 9.5: Иллюстрация к объяснению

Заметим, что Си-код с двумя индексными выражениями имеет такой же вид, что и раньше:

```
int get_univ_digit (int index, int dig) {
    return univ[index][dig];
}
```

Но при этом ассемблерный код изменился:

```
mov eax, dword [ebp + 8] ; index
mov edx, dword [univ + 4 * eax] eax]; p = univ[
mov eax, dword [ebp + 12] 12]; dig
mov eax, dword [edx + 4 * eax] ; p[dig]
```

Здесь появляется два обращения в память. Кратко действия можно описать следующим образом:

- Доступ к элементу $Mem[Mem[univ + 4 * index] + 4 * dig]$
- Необходимо выполнить два чтения из памяти:
 - Первое чтение получает указатель на одномерный массив
 - Затем второе чтение выполняет выборку требуемого элемента этого одномерного массива

Оптимизация доступа к многомерным массивам.

Рассмотрим некоторые оптимизации, которые выполняются при работе с многомерными массивами. Как мы видели выше, взятие элемента массива соответствует довольно сложному вычислению. В случае циклов подобные вычисления могут занимать большую часть времени выполнения программы. Данный процесс можно оптимизировать. Для этого рассмотрим матрицу размера $N*N$. В случае динамически задаваемой размерности есть два возможных варианта. Поняв, какими будут размерности, выделяем одномерный массив и вручную реализуем адресную арифметику:

```
#define IDX(n, i, j) ((i)*(n)+(j))
/* Get element a[i][j] */
int vec_ele
    (int n, int *a , int i , int j)
{
    return a[IDX(n,i,j)];
}
```

Также существует некоторое расширение, которое поддерживается в компиляторе gcc. С его помощью мы можем прямо в объявлении функции указать, что матрица будет двумерная размера $n*n$, при этом значение переменной n определяется через другие параметры:

```
/* Get element a[i][j] */
int var_ele
    (int n, int a[n][n] n, int i, int j)
    return a[i][j];
}
```

Обращение к элементу

Рассмотрим доступ к элементу в случае матрицы $16*16$:

```
int fix_ele fix_matrix a , int i , int j) {  
    return a[i][j];  
}
```

Ассемблерный код:

```
mov edx , dword ebp + 12]      ; i  
sal edx , 6                    ; i *64  
mov eax , dword ebp + 16]      ; j  
sal eax , 2                    ; j*4  
add eax , dword ebp + 8]       ; a + j*4  
mov eax , dword eax + edx ]    ; *(a + j*4 + i *64)
```

Здесь передаётся три параметра, A, i, j. Они идут по смещению +8, +12, +16 соответственно. Если размеры являются степенями двойки, то правомерно использовать замену умножением в виде сдвигов. Базовый адрес поступает через параметр, эта величина неизвестная, поэтому в коде присутствуют два сложения – явное и неявное (последние две строки ассемблерного кода).

Рассмотрим матрицу неизвестной размерности. В этом случае появляется параметр n:

```
int var_ele(int n, int a[n][n], int i, int j){  
    return a[i][j];  
}
```

Из-за него появляется умножение в месте, где определяется смещение строки:

```
mov edx , dword ebp + 8]      ; n  
sal edx , 2                    ; n*4  
imul edx , dword ebp + 16]    ; i *n*4  
mov eax , dword ebp + 20]     ; j  
sal eax , 2                    ; j*4  
add eax , dword ebp + 12]     ; a + j*4  
mov eax , dword eax + edx ]]   ; *(a + j*4 + i *n*4)
```

Отметим, что само a будет восприниматься как указатель, то есть $\text{sizeof}(a) = 4$. В случае элемента массива $\text{sizeof}(a[i]) = 4*n$.

Оптимизация доступа к элементам массива

Рассмотрим пример, в котором осуществляется проход по столбцу. Разложим величины по регистрам (рис. 9.6). Мы заведём указатель, который будет указывать внутрь общего массива. А далее мы будем работать с этим указателем,

Регистр	Значение
ecx	ajp
ebx	dest
edx	i

Рис. 9.6: Иллюстрация к объяснению

переставляя его при переходе на строки. Нам известно, на сколько байт нужно прыгнуть, чтобы перейти к следующему массив, так как размер массива заранее известен. Си-код:

```
void fix_column
(
    fix_matrix a, int j, int dest
)
{
    int i;
    for (i = 0; i < N; i++)
        dest[i] = i ][j];
}
```

Ассемблерный код:

```
.L8:                                ; loop:
mov  eax , dword [ecx]              ; *ajp
mov  dword [edi + 4 * edx ], eax    ;
add  edx , 1                        ; i++
add  ecx , 64                       ; ajp += 4*N
cmp  edx , 16                       ; i vs. N
jne  .L8                            ; if !=, goto loop
```

Рассмотрим тело цикла. Указатель на элемент текущего столбца считывается и помещается в `eax`. Далее пересылаем его (вторая строка ассемблерного кода). Переход на следующее место в столбце соответствует прибавлению 64 (четвёртая строка).

В случае массива неизвестного размера (см. код ниже) нам потребуется дополнительный регистр, в котором необходимо держать шаг. Чтобы подготовить все вспомогательные величины, мы используем почти все доступные нам регистры (рис. 9.7).

```
void var_column
(
    int n, int a[n][n],
    int j, int *dest
)
{
    int i;
    for (i = 0; i < n; i++)
        dest[i] = i ][j];
}
```

Регистр	Значение
<code>ecx</code>	<code>ajp</code>
<code>edi</code>	<code>dest</code>
<code>edx</code>	<code>i</code>
<code>ebx</code>	<code>4*n</code>
<code>esi</code>	<code>n</code>

Рис. 9.7: Иллюстрация к объяснению

Тогда получаем реализацию:

```
.L18:                                ; loop:
mov  eax , dword ecx ]]              ;
mov  dword edi + 4 * edx ], eax      ;
```

```
add    edx , 1          ; i++
add    ecx , ebx        ; aip += 4*n
cmp     esi , edx        ; n vs. i
jg      .L18            ; if (>) goto loop
```

Считываем текущее значения указателя на столбец, пересылаем это в `dest[i]`, увеличиваем счётчик на единицу и прибавляем к `ebx` `ecx`.

Лекция 10. Структуры

Повторение

Рассмотрим следующий двумерный массив:

Если мы хотим обратиться к его элементу $A[i][j]$, это эквивалентно обращению в память:

$$M[\chi_A + c * i * \text{sizeof}(T) + j * \text{sizeof}(T)] \quad (10.1)$$

где χ_A - базовый адрес (начальное место в памяти).

Отметим, что один из аспектов эффективности заключается в том, что нет проверок попадания внутрь диапазона адресов памяти. Например, если взять индекс слишком большим, то мы вылетаем за одну границу. Можно выделить и за другую границу тоже.

В некоторых случаях по ассемблерному коду функции, которая работает с двумерным массивом можно определить его размеры.

Структуры

Структуры - это ещё один тип данных, который в некоторых вещах похож на массив. Для размещения в этом типе данных выделяется непрерывный блок памяти, как и в массиве. Отличие состоит в том, что в массиве все элементы однородные, а в структуре они могут быть произвольных типов. Элементы в структуре обладают именами, обращения потом будут происходить по этим именам. Для каждого поля внутри этого выделенного блока данных выделяется определённое место. Так как размеры типов данных известны во время компиляции, можно посчитать смещение до того или иного элемента в структуре от начала.

Пример объявления структуры в Си:

```
struct rec {  
    int i;  
    int j;  
    int a[3];  
    struct rec *p;  
}
```

Для этой структуры расположение в памяти будет выглядеть так, как представлено на рисунке 10.1.

Доступ к полям

Пусть имеется статическая структура типа `rec`:

```
static struct rec t;
```

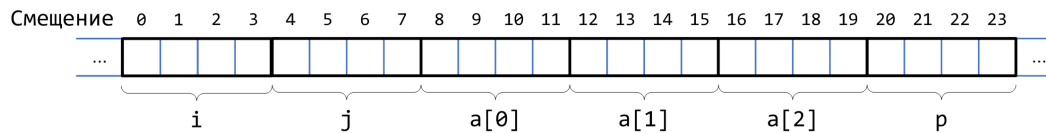


Рис. 10.1: Расположение в памяти структуры rec.

Пусть пользователь хочет обратиться к полю `t.p`. Так как структура статическая, то под неё выделяется блок в статической памяти, тогда для доступа к полю `p`, аналогично массиву, необходимо сместиться от базового адреса:

```
mov     eax, [t+20]
```

В случае указателя на структуру ситуация несильно отличается. Указатель на структуру хранит в себе базовый адрес, поэтому сначала он считывается. В случае взятия адреса ситуация снова похожа. Пусть есть код взятия адреса:

```
static struct rec *x;  
static int i;  
...  
&(x->a[i]);
```

Тогда на ассемблере это будет выглядеть следующим образом:

```
mov     edx, dword [i]  
mov     eax, dword [x]  
lea     eax, [eax + 4 * edx + 8]
```

То есть, сначала помещается значение индекса `i` в `edx`, затем базовый адрес структуры загружаем в `eax`. Последняя строка ассемблерного кода вычисляет общее смещение и определили искомый адрес не обращаясь в память.

Таким образом, появляется возможность писать более интересный код, работая со связными списками например.

Выравнивание полей в структурах

Выравнивание полей в структурах происходит немного сложнее, чем в массивах. В структурах поля идут не непрерывным потоком, и иногда между ними образуются промежутки - заполнители. Заполнители - это какое-то количество байт, которое не используется и присутствует для того, чтобы поля находились на выровненных адресах.

Правила выравнивания в общем случае следующим образом. Если имеется какой-то тип данных, для хранения которого требуется `k` байт, тогда адрес, где данный тип размещается должен быть кратен числу `k`. Для структуры:

```
struct S1 {  
    char c;
```

```
    int i[2];  
    double v;  
} *p;
```

Выровненные поля будут выглядеть, как на рисунке 10.2.

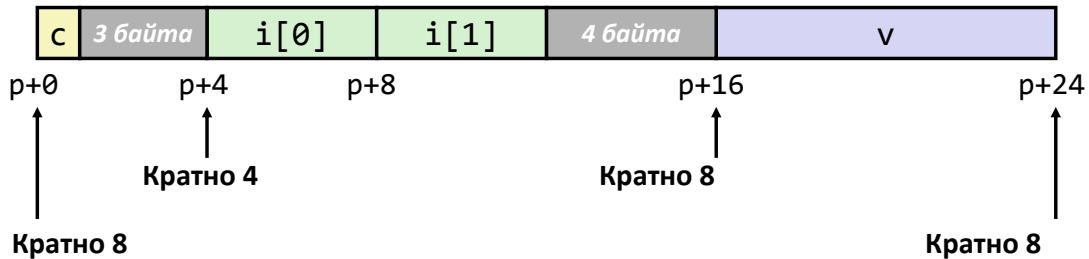


Рис. 10.2: Выровненные данные в структуре.

Общей размер структуры оказывается равным 8.

Причина выравнивания

Рассмотрим зачем нужно выравнивание. Вспомним машину фон Неймана, но добавим к процессору, памяти и шине контроллер памяти (mem ctrl). Когда очередная команда пытается обратиться к памяти, она получает исполнительный адрес, и с этого адреса нужно извлечь какое-то количество байт.

Рассмотрим кусок памяти в виде массива. Пусть нужно выгрузить из памяти что-то размером dword с адреса [0xFFF]. Четыре раза обращаться в память неэффективно. Шина пересылает данные порциями, пусть в данном примере она пересылает их порциями по 4 байта. Эти 4 байта будут считываться не с произвольных адресов, а с кратных четырём. То есть, 4 байта можно взять от нуля, от четырёх, от восьми и так далее.

Пусть мы пытаемся считать с невыровненного адреса. Тогда контроллеру памяти придётся запрашивать два соседних куска по 4 байта из памяти, затем вырезать из них нужные куски и склеивать. То есть, всё получится, но за два обращения. Память работает на меньших частотах, чем процессор. Однако, каналов, связывающих память и контроллер памяти может быть много, тогда данные можно посылать параллельно. Кроме того, во многих процессорах уже давно не одно ядро. Причём, процессор - это двумерная структура, с уменьшением числа транзисторов, число ядер растёт чуть ли не квадратично. В то же время, число каналов связи памяти и контроллера памяти с годами растёт линейно, из-за этого дисбаланс растёт и образуется так называемое bottleneck. В наше время можно считать, что в среднем на один канал памяти приходится 2-3 ядра, которые независимо друг от друга выполняют код. С каждого из этих ядер в любой момент может прийти запрос на получение данных из памяти, и контроллер может получить на один канал связи с памятью несколько запросов одновременно, получается задержка.

Таким образом, выравнивание, ценой потери нескольких байт, может существенно увеличить скорость работы всего компьютера в целом. Современный компилятор производит выравнивание очень активно.

Помимо общего требования кратности адресов к выравниванию, на разных операционных системах соглашения об устройстве исполняемого машинного кода имеют отличие.

Правила выравнивания для IA-32:

- 1 байт: char, Ограничений на биты нет.
- 2 байта: short, Младший бит адреса должен быть 0.
- 4 байта: int, long, float, char*, Два младших бита адреса должны быть нулями.
- 8 байт: double, В данном случае существует различие между Windows и Linux. В Windows выравнивание происходит таким образом, что младшие три бита адреса должны быть нулями. В linux выравнивание происходит по 4 байтной границе, как для int, long, ...
- 12 байт: long double. Два младших бита должны быть нулями.

Для современных 64 разрядных устройствах всё очень похоже. Отличие состоит в том, что адреса становятся 8-ми байтными, и что в Windows, что в Linux, выравнивание происходит таким образом, что младшие три бита адреса должны быть нулями. Также, long double выравнивается по 16-ти битной границе.

Ранее, на рисунке 10.2 рассматривалось выравнивание структуры для Windows или x86-64. Для других систем выравнивание той же структуры будет выглядеть по-другому. Примеры представлены на рисунках 10.3 и ??.

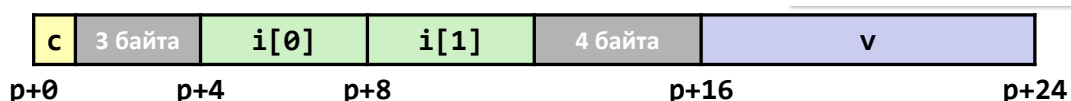


Рис. 10.3: Выровненные данные в структуре для x86-64 или IA-32 Windows.

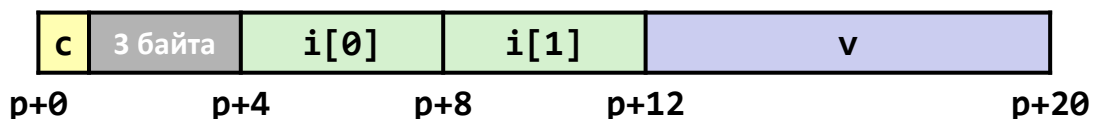


Рис. 10.4: Выровненные данные в структуре для IA-32 Linux.

Помимо выравнивания полей может быть необходимо произвести выравнивание всей структуры как целого, так как структура может быть вложена в какую-то другую структуру данных.

Массивы структур

Рассмотрим массив структур. Все элементы массива идут непрерывно, но каждый элемент имеет свои концовки в виде пустых байт. Для того, чтобы обращаться к элементам массива структур, сначала необходимо определить размеры всех типов с учётом требований выравниваний. Например, рассмотрим структуру S3:

```
struct S3 {  
    short i;  
    float v;  
    short j;  
} a[10];
```

Для неё определим функцию, которая возвращает поле j по заданному индексу в массиве структур:

```
short get_j(int idx) {  
    return a[idx].j  
}
```

Такая функция для данной структуры будет выглядеть в ассемблерном коде (с учётом всех выравниваний):

```
;      eax = idx  
lea     eax, [eax + 2 * eax] ; 3*idx  
movsx   eax, word [a + 4 * eax + 8]
```

В данном случае размер структуры - это 12, а поле j расположено со смещением 8 внутри структуры.

Существует просто правило, которое говорит о том, как заполнять место, чтобы было минимальное количество пустых байтов для выравнивания. Необходимо размещать поля в структуре не в произвольном порядке, а таким образом, чтобы наиболее крупные элементы структуры оказались в начале, а маленькие - в конце.

Объединения

Объединения - очень похожая структура данных. Отличие от структур состоит в том, что под каждое поле объединения выделяется то же самое место. Графически это можно представить следующим образом. На рисунке 10.5 представлено расположение в памяти структуры S1, а на рисунке 10.6 можно увидеть расположение в памяти объединения с теми же полями, что и структура S1.

```
struct S1 {  
    char c;  
    int i[2];
```

```
double v;  
} *sp;
```

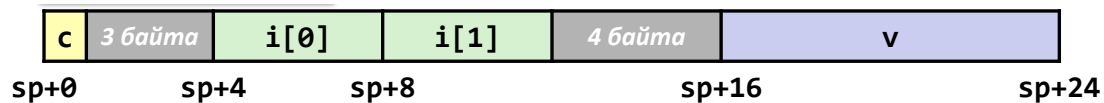


Рис. 10.5: Расположение в памяти структуры S1.

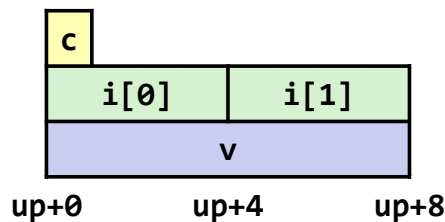


Рис. 10.6: Расположение в памяти объединения, имеющего поля как у структуры S1.

Порядок байт

Данные хранятся в памяти как последовательности байт. Возникает вопрос, где именно расположен старший (младший) байт. Существует два разных порядка:

- Big-endian (порядок от старшего к младшему). Старший байт имеет наименьший адрес.
- Little-endian (порядок от младшего к старшему). Младший байт имеет наименьший адрес. Такой порядок используется на процессорах Intel x86 (IA-32).

Различие в порядке байт может являться проблемой при пересылке двоичных данных между машинами с разной архитектурой. В наше время проблема не так актуальна, так как большинство машин имеют Little-endian архитектуру или переключаемый порядок байт. Переключаемый порядок байт имеют архитектуры: ARM, PowerPC, Alpha, SPARC V9, MIPS, PA-RISC и IA-64.

Битовые поля

В языке существует возможность задавать отдельные битовые поля. С ними почти всё зависит от конкретной реализации компилятора, потому что идти они могут как в одном порядке, так и в другом, могут пересекать границы машинных слов. В результате получается почти не переносимый код.

Пример использования - это применение тех или иных битовых масок, которые будут накладываться командами or, and и подобными.

Немного про функции

При вызове функций считаем, что сначала отработывает пролог, используется указатель фрейма `ebp`. От нас будет требоваться расписать карту памяти фрейма, в котором будут храниться свои и служебные данные. Внутри фрейма будет храниться некое текущее состояние вызова (сохранённые регистры, автоматические локальные переменные).

Схема фрейма, когда происходит вызов функции представлена на рисунке 10.7.

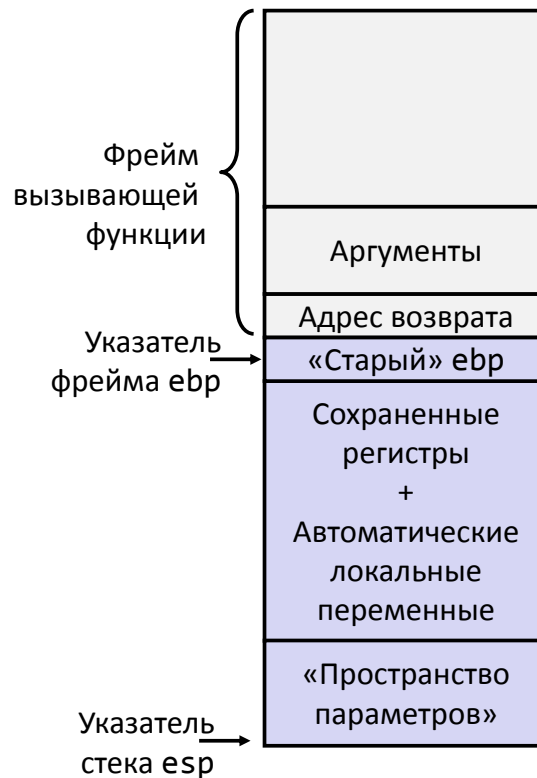


Рис. 10.7: Фрейм при вызове функции.

Соглашения CDECL

Функции поддерживаются на уровне соглашений. Одно из соглашений вызова - CDECL, оно является стандартом для операционной системы Linux.

- Параметры функции кладутся на стек с обязательным выравниванием по 4-ёх байтной границе (так как адрес возврата занимает 4 байта).
- Есть возможность посмотреть, сколько функций вызывается из текущей, какие у них аргументы вызова, сколько под них необходимо байт. Под эти нужды резервируется пространство аргументов, то есть, это пространство переиспользуется. Таким

образом, нет никаких гарантий, что то, что лежало в пространстве аргументов, при возврате туда после вызова функции, сохранит своё значение.

- Очистка стека от аргументов после вызова функции - это обязанность текущей вызывающей функции. Эта очистка проводится перед выходом из функции (в других соглашениях вызова это не так).
- Возвращаемое функцией значение - это `eax`. Если тип данных 8 байтный, то возврат через пару регистров `edx:eax`. Если структура большого размера, то возврат происходит через память.

Сохранение регистров

В 32-х разрядной машине разницы между Linux и Windows не будет. Регистров 8 штук. Два - служебные (`esp`, `ebp`), остальными можно пользоваться. Три можно пользоваться сразу (сохраняются вызванной функцией, регистры `ebx`, `esi`, `edi`). Ещё три необходимо сохранять вызывающей функцией (регистры `eax`, `edx`, `ecx`).

Лекция 11. Функция `main`

ABI - двоичный интерфейс приложения

Двоичный интерфейс приложения - это просто некоторый свод правил (договорённостей), которого должны придерживаться все программисты (преимущественно все, особенно те, которые пишут системный софт). Очень много кода уже давно написано, таким образом, несоблюдение сводов правил приведёт к нестыковкам с чужим кодом. Двоичный интерфейс задаёт правила взаимодействия между модулями программы на уровне исполняемого кода.

ABI - это один из нижних уровней соглашений, по которым необходимо пройти. Схема уровней интерфейсов представлена на рисунке 11.1. Самый нижний уровень - это ISA (instruction set architecture). Этот уровень описывает то, что умеет делать машина. Поверх ISA может быть “натянута” ABI, причём не одно. Над ABI естественным образом расположено API (application programming interface) - инструкции в терминах языка программирования. Поверх всего располагаются контракты функций. Это высокоуровневая логика, которую не выразить в терминах языка программирования (например для списка можно сказать, что он однонаправленный, имеет в себе `int` и другие).



Рис. 11.1: Уровни интерфейсов соглашений для работы с приложениями.

Существует небольшая сложность склейки описанных уровней интерфейсов между собой.

Функция main

В Си `main` - это точка входа в программу. На самом деле программа начинают свою работу с функции `start`, а только после запускается `main`. Утилита `nm` позволяет напечатать все метки, которые присутствуют в вызываемой программы.

Полное дерево вызовов, если аккуратно расписать все вспомогательные библиотечные функции (код поддержки времени выполнения), будет выглядеть так, как на картинке 11.2.

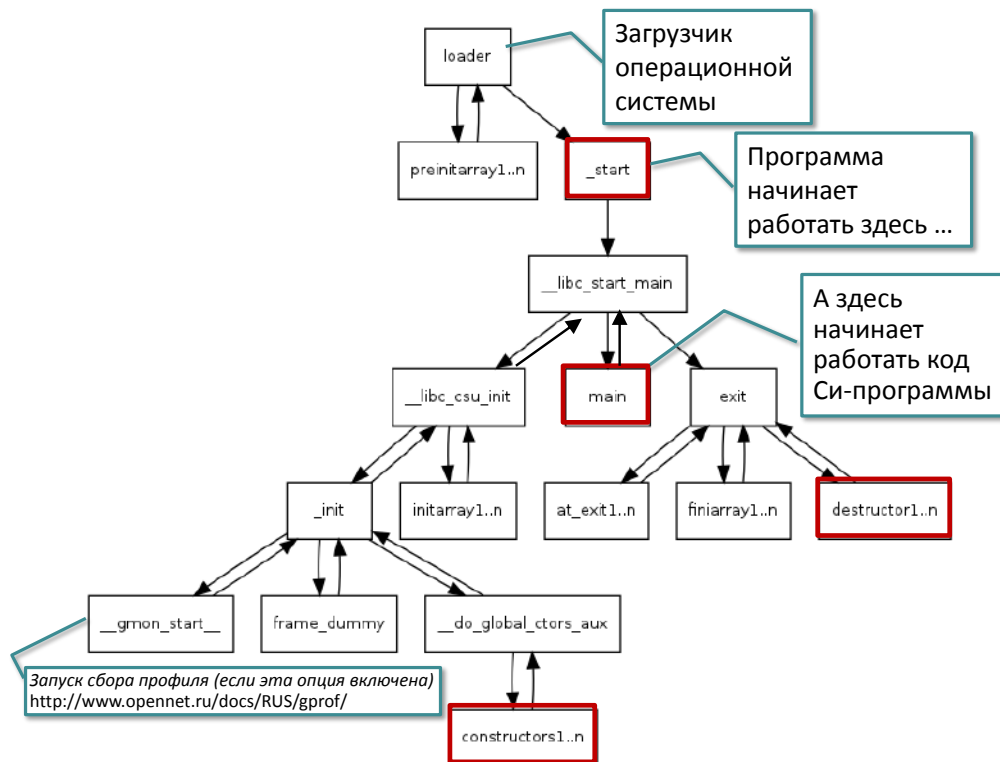


Рис. 11.2: Полное дерево вызовов.

Начальное состояние стека

Рассмотрим, что происходит внутри функции `_start`. Функция на самом деле достаточно короткая. Внутри файла `main` существует прописанное число (в структуре служебного заголовка), которое является адресом, куда операционная система отдаёт управление при запуске. По соглашению во время компоновки имя `_start` было превращено в адрес. В этот момент появляется стек - некое пространство памяти, подготовленное операционной системой, доступное для чтения и записи. Кроме того, операционная система положила на стек определённые данные.

В самом начале `esp` показывает на стек, где записано `argc` (число параметров командной строки, как минимум это будет 1 - имя запущенной программы всегда параметр). Далее идёт `argv` - массив указателей, которые ссылаются на строки.

Дальше идёт массив указателей, которые ссылаются на переменные окружения (envp). Переменное окружение представляет собой описание той среды, в которой запущена программа с помощью пар ключ-значение.

Например, при запуске чего-либо из командной строки записывается имя файла, его можно указать в коротком виде. А далее берётся переменная окружения PATH=dir1:dir2, где dir1, dir2 - имена директорий.

Выше по стеку располагаются все сточки, на которые идут ссылки ниже. Доаступная для работы память в каком-то месте кончается. По науке там начинается guard page - кусок памяти, который операционная система разметила, как недоступный для работы. Любая попытка что-то там сделать вызовет аварийную остановку. Ниже argc по стеку тоже находится guard page. Эти guard page контролируют работу со стеком.

Рассмотрим, что происходит с командами в `_start`. Сначала зануляется `ebp`, потому что в этот момент он “мусор”. Затем в регистр `esi` выталкивается величина `argc`. После устанавливается регистр `ecx` на начало `argv` на стеке. После этого к `esp` прикладывается маска, у которой последняя шестнадцатеричная цифра нуль. Это будет выравниванием по 16-ти байтной границе. При этом `esp` уедет куда-то вниз на стеке. Дальнейший код после выравнивания готовит параметры для вызова специальной функции `__libc_start_main`.

Функция `__libc_start_main` в своём заголовке имеет адрес функции `main` первым параметром. Кроме того, в параметрах есть `argc` и верхняя граница стека, с которой можно работать. Далее идут адреса функций инициализации, деинициализации, системной деинициализации. Последний параметр - конец стека. Перед концом на стек кладётся мусор, так как будет происходить выравнивание.

Для защиты от разного рода взлома, можно строить код, который не завязан на конкретные адреса, тогда нельзя будет ими манипулировать. Адреса определяются во время работы программы. Полный ассемблерный код функции `_start`:

```
080482e0 <_start>:
80482e0: xor    ebp, ebp
80482e2: pop    esi
80482e3: mov    ecx, esp
80482e5: and    esp, 0xfffffffff0
80482e8: push   eax
80482e9: push   esp
80482ea: push   edx
80482eb: call   8048313 <_start+0x33>
80482f0: add    ebx, 0x1d10
80482f6: lea    eax, [ebx-0x1b80]
80482fc: push   eax
80482fd: lea    eax, [ebx-0x1be0]
8048303: push   eax
```



```
8048304: push    ecx
8048305: push    esi
8048306: mov     eax, 0x80483fc
804830c: push    eax
804830d: call    80482c0 <__libc_start_main@plt>
8048312: hlt
8048313: mov     ebx, DWORD PTR [esp]
8048316: ret
```

Команда hlt

Команда `hlt` встречается в коде функции `_start`. В оригинале она означает, что машина должна остановиться и ждать каких-то событий (например связанных с вступлением каких-то данных извне, с переферией). В данном месте смысл этой команды другой. По дереву вызовов необходимо было дойти до функции `exit`, а затем операционная система останавливает работу программы. Таким образом, никак уже не вернуться на точку, где команда `hlt`. Например, если возникла какая-то ошибка, то управление возвращается и попытка выполнить команду `hlt` в пользовательском режиме приведёт к аварийной остановке. Дело в том, что команды, которые выполняет машина, делятся на две категории: пользовательские и системные. Сама операционная система может выполнять любые команды, а при передачи управления в пользовательскую программу в некоторых системных регистрах переключается режим, и попытка выполнить системную команду будет заканчиваться аварийной остановкой. Попытка переписать системные регистры тоже закончится аварийной остановкой.

Сохранение требований по выравниванию

При вызове функции `main` и формировании фреймов определённого размера сохраняются требования по выравниванию. После того, как начнёт выполняться `main`, будет наблюдаться следующая картина. Элемент стека, находящийся над адресом возврата гарантированно будет выровнен по 16-ти байтной границе. Вторая выровненная граница будет под `argc` (рисунок 11.3)

Действие по выравниванию стека было перенесено из `_start`.

Если рассмотреть ассемблерный код функции `main` (которая внутри себя вызывает `nullify` с аргументами `argc`, `argv`):

```
080483fc <main>:
80483fc:          55                push    ebp
80483fd:          89 e5             mov     ebp, esp
80483ff:          ff 75 0           cpush   dword [ebp+0xc]
8048402:          ff 75 08          push    dword [ebp+0x8]
```



Рис. 11.3: Фрейм функции main.

```
8048405:      e8 ec ff ff ff      call    nullify
804840a:      83 c4 08             add     esp, 0x8
804840d:      b8 00 00 00 00      mov     eax, 0x0
8048412:      c9                  leave
8048413:      c3                  ret
...
```

После отработывания `nullify`, `esp` сдвигается на 8, чтобы вернуть сохранённый `ebp`, `eax` зануляется, чтобы вернуть 0.

Далее идёт команда `leave`. Эта команда является полным функциональным аналогом двух команд. Она аналогично следующему коду:

```
mov     esp, ebp
pop     ebp
```

То есть, если стек куда-то передвигался при работе с ним, не используя `ebp`, с помощью команды `leave` можно восстановить `esp` на нужное место, где `ebp` уже сохранён, и восстановить его. Две команды записываются большим числом байт, чем `leave`, хотя чем пользоваться - это дело вкуса.

Конструкторы и деструкторы

Существуют ещё два места, куда можно добавить свой код. Стандарт языка Си ничего об этом не знает, но компилятору ничто не мешает взять и расширить стандарт. Каждый компилятор вынужденно вносит расширения. В `gcc` есть аппарат атрибутов, с помощью которых можно написать конструкцию с двойными скобками, в которых можно написать ключевое слово, которое `gcc` воспримет. В данном случае используются ключевые слова `constructor` и `destructor` (инструкции, которые нужно выполнить до и после работы программы). Пример того, как можно завести функции с такими ключевыми словами:

```
void __attribute__((constructor))
my_constructor() {
```

```
printf ("%s n", __FUNCTION__);  
}  
void __attribute__((destructor))  
my_destructor() {  
printf ("%s n", __FUNCTION__);  
}
```

Подчёркивание `__FUNCTION__` позволяет напечатать имя текущей функции. В этих функциях тоже будут сохраняться требования по выравниванию. Для примера разберём ассемблерный код функции `my_constructor`. Сначала происходит стандартный полог, сохраняется `ebp`. После выделяем на стеке 8 байт, дополнительно ещё 12, и в этот момент появляется некоторый параметр. Функция `printf` заменилась на более простую `puts`. Ассемблерный код:

```
08048426 <my_constructor>:  
8048426:      55                push    ebp  
8048427:      89 e5             mov     ebp, esp  
8048429:      83 ec 08          sub     esp, 0x8  
804842c:      83 ec 0c          sub     esp, 0xc  
804842f:      68 10 85 04 08    push    0x8048510  
8048434:      e8 a7 fe ff ff    call    80482e0 <puts@plt>  
8048439:      83 c4 10          add     esp, 0x10  
804843c:      90                nop  
804843d:      c9                leave  
804843e:      c3                ret
```

Конструкторов и деструкторов может быть произвольное число. Рассмотрим, как происходит их вызов. Компилятор собирает в дополнительную служебную секцию адреса всех функций со специальными атрибутами. Сейчас эти секции называются `init_array` и `fini_array`. В них будут некоторые последовательности байт. Если расписать последовательности, то можно будет увидеть адреса функций. Далее, для вызова функции происходит `call`.

Оболочка вокруг main

Вокруг `main` можно написать оболочку, которая будет производить выравнивание так, что внутри оболочки можно писать любой свой код. В результате откроется возможность свободно вызывать функции стандартной библиотеки.

Лекция 12. Особенности архитектуры x86-64.

Выравнивание фрейма

Ранее мы рассмотрели вопрос распределения данных при создании фрейма. Известно, что если используется стандартное соглашение `cdecl`, предполагающая сохранение регистра `ebp`, то мы сразу вычитаем восемь байт. Четыре байта идёт на адрес возврата, четыре – на сохранённые `ebp`. Остальное пространство (с учётом этих восьми байт) должно давать в сумме величину, кратную шестнадцати. Если выполнить указанное соглашение, то, разрабатывая ассемблерный программы, мы получаем возможность вызывать функции, которые были написаны на языке Си (при условии подключения нужных библиотек). Если рассматривать обычный персональный компьютер под управлением Windows/Linux, то нарушение данного соглашения по выравниваю общего размера фрейма зачастую не вызывает аварийных завершения, а только негативно влияет на производительность. Однако в некоторых случаях возможно аварийное завершение программы (например, в случае архитектуры x86 с операционной системой Apple). Это обосновано тем, что существует отдельно взятая команда `MOVDQA`, которая выполняет очень быстрое копирование данных в случае, если данные размещены на определённых кратных адресах (выровненные данные). Такое предположение позволяет с меньшими проверками на уровне аппаратуры производить копирование (из-за чего копирование производится быстрее).

Отказ от указателя фрейма

Можно предложить некоторые оптимизации, чтобы сократить время соглашения вызова. Это становится заметным, если тело функции не очень большое. Тогда создание и освобождение фрейма будут вносить некоторые дополнительные расходы, сравнимые с полезной нагрузкой. В некоторых ситуациях можно отказываться от использования указателя фрейма. На его особое применение рассчитывает отладчик (вопрос соглашения). Если мы собираем программу в так называемом отладочном режиме, без оптимизации, то нарушение данного соглашения по выравниваю общего размера фрейма зачастую не вызывает аварийных завершения. Если мы подготавливаем релизную сборку, чтобы можно было передать то, что мы разработали, потребителям, то включается оптимизация, которая в том числе меняет работу с `ebp`. Это означает, что адресация будет только относительно регистра `esp`. В этом случае у нас все всегда будет относительно верхушки стека и придётся больше пересчитывать, большим количеством смещение добраться до того, что лежит выше адреса возврата. Рассмотрим следующий пример. Есть некоторая рекурсивная функция, для которой с помощью приведённых ниже ключей был получен указанный далее ассемблерный листинг:

```
gcc -S -O0 -fomit-frame-pointer -fno-optimize-sibling-calls
```

`-masm=intel length.c`

Ключ - s нужен для того, чтобы вместо исполняемого кода был построен ассемблерный листинг. Ключ - OS – оптимизация по размеру, чтобы код получился покороче. Ключ - fomit-frame-pointer – смена использования регистра ebp. Ключ - fno-optimize-sibling-calling – разворачиваем рекурсивную функцию в плоский алгоритм, целиком закодированный в теле функции. Последний запрос: - masm=Intel – вывод в синтаксис языка Intel. Код выглядит следующим образом:

```
typedef struct link link;
struct link {
    int payload;
    link* next;
};

int length(link *p) {
    return p? length(p-->next) + 1 : 0;
}
```

Ассемблерный код:

```
length:
    sub    esp , 12
    xor    eax , eax
    mov    edx , dword [esp+16]
    test   edx , edx
    je     .L2
    mov    ecx , dword [edx+4]
    mov    [esp], ecx
    call   length
    inc    eax
.L2:
    add    esp , 12
    ret
```

Функция length работает со структурой. После попадания в тело функции формируется нужный размер фрейма. Первая строка ассемблерного кода создаёт фрейм размера 16 байт. Параметр p находится по смещению +16.

Далее считываем edx и проверяем, что он не нулевой. Если он нулевой, мы переходим к метке .L2, выполнение функции в этот момент сворачивается и мы выходим из функции. В противном случае, если указатель не нулевой, мы перешагиваем на следующее поле, вытаскиваем указатель на следующий элемент в однонаправленном списке. Далее ecx копируется в место, на которое указывает esp и происходит рекурсивный вызов. Когда

мы из него возвращаемся, нужно увеличить `eax` на единицу, далее – эпилог, увеличение `esp` и завершение функции.

На оптимизацию можно воздействовать отдельными ключами. Например

```
gcc -S -O3 -fno-omit-frame-pointer -fno
```

отвечает за требование сохранения `ebp`. Тогда для того же кода получим ассемблерный листинг вида

```
length:
    push    ebp
    xor     eax, eax
    mov     ebp, esp
    sub     esp, 8
    mov     edx, dword [esp+8]
    test    edx, edx
    je      .L2
    mov     ecx, dword [edx+4]
    mov     dword[esp], ecx
    call    length
    inc     eax
.L2:
    leave
    ret
```

Компиляторы и соглашения вызова функции

Оптимизация вызова функции раньше зависла от компилятора. Сейчас компиляторов немного:

- LLVM clang (open source)
 - AMD Optimizing C/C++ Compiler (AOCC)
- GNU gcc (open source)
- Microsoft Visual Studio vc
- Intel icc

В каждом подобном компиляторе были свои техники ускорения соглашения вызова. Можно выделить несколько основных соглашений – соглашение вызова, принятое в операционных системах Windows и соглашение `FASTCALL`. Рассмотрим соглашение `STDCALL`. Пусть дан код:

```
#include <stdio.h>

__attribute__((stdcall))
int sum( int x, int y);

int main() {
    int a = 1, b = 2, c;
    c = sum(a, b);
    printf ("%d n", c);
    return 0;
}

__attribute__((stdcall ))
int sum( int x, int y) {
    int t = x + y;
    return t;
}
```

Ассемблерный код:

```
sum:
    push    ebp
    mov     ebp, esp
    mov     eax, dword [ebp+12]
    add     eax, dword [ebp+8]
    pop     ebp
    ret     8
```

При этом

```
CMAIN:
    ; ...
    mov     dword [esp+4], 2
    mov     dword [esp], 1
    call    sum
    sub     esp, 8
    mov     dword [ebp+8], eax
    ; ...
```

STDCALL от CDECL отличается тем, кто будет освобождать стек от аргументов вызова. Ранее был предложен вариант команды `ret` с одним операндом (константа, которая указывает, на сколько байт нужно сдвинуть `esp`). Такой вариант удобен, если код пишется вручную либо не очень хорошо компилирующим компилятором. Сейчас

подобное улучшение не даёт существенного увеличения в скорости. В частности, соглашение `STDCALL` не позволяет реализовывать вызовы с переменным числом параметров, так как вызванная функция имеет чёткое предположение о том, сколько было параметров, и это число прибавляет `esp`. В свою очередь `CDECL` предполагает, что очищать будет тот, кто вызвал. Поэтому при переменной числе параметров только он может знать, сколько параметров расположено в стеке. Отметим, что функцию соглашения `STDCALL` можно вызывать изнутри функций, работающих в соглашении `CDECL`. Для этого надо указать, что соглашение вызова по умолчанию изменилось. В `gcc` используется известный механизм атрибутов: `__attribute__((stdcall))` – указанная далее функция будет использовать соглашение `STDCALL`. Далее она вызывается из функции `main`, в которой соглашение `CDECL`. Тогда нужно учесть, что при возвращении из функции `sum` уменьшается размер стека внутри неё. Это нужно компенсировать, что производится строкой `sub esp, 8`.

Соглашение `FASTCALL`

Соглашение `FASTCALL` использует для передачи параметров некоторое количество регистров. Однако, регистров общего назначения очень мало, а параметров может быть значительно количество. Поэтому принимают следующие соглашения:

- Первый и второй параметры размещаются в регистрах `ECX` и `EDX` (Если размер параметров позволяет).
- Остальные параметры на стеке, от них стек очищает вызванная функция, как и в `stdcall`.
- Такая форма соглашения принята в `gcc` и `MSVC`.

Рассмотрим случай, когда код был собран с указанием атрибута `fastcall`:

```
typedef struct chain chain
```

```
struct chain {  
    unsigned data;  
    chain *next;  
};
```

```
__attribute__((fastcall)) unsigned xorEmAll(chain*p,unsigned salt)  
{  
    if (p) {  
        return p-->data ^= xorEmAll (p -->next, salt);  
    } else {
```



```
        return salt;
    }
}
```

Здесь мы проходим по однонаправленному списку и каждый элемент полезной нагрузки подвергается xor со вторым параметром. Параметр *p* – указатель, поэтому он умещается в *ecx*, второй параметр помещается в *edx*, поэтому стек не используется для передачи параметров. Таким образом, если собрать функцию с включённой оптимизацией, мы получим:

```
xorEmAll:
    test    ecx , ecx
    mov     eax , edx
    je      .L6
    push    ebx
    mov     ebx , ecx
    sub     esp , 8
    mov     ecx , dword [ecx+4]
    call    xorEmAll
    xor     eax , dword [ebx]
    mov     dword [ebx], eax
    add     esp , 8
    pop     ebx
.L6:
    ret
```

Фактически пролог отсутствует, сразу переходим к работе с параметром *p*. В первой строке проверяем, что параметр *p* ненулевой. Если он нулевой, переходим к метке *.L6*. Та часть кода, которая находится ниже строки перехода к метке, описывает действия, происходящие внутри ветки *true* оператора *if*.

Особенности 64-разрядной процессорной архитектуры

Особенности полноценной 64-разрядной процессорной архитектуры:

- АЛУ оперирует 64-разрядными данными
- (Большой) набор 64-разрядных регистров общего назначения
- 64-разрядное (плоское) адресное пространство

Отсюда получаем преимущества 64-разрядной процессорной архитектуры:

- Эффективнее (быстрее) работаем с 64-разрядными данными

- Реже «проливаем» содержимое регистров
- Огромное пространство адресуемой памяти: $2^{64} = 16 \times 2^{30} \times 2^{30} = 16$ Эксбибайт

Примеры полноценной 64-разрядных архитектур: PowerPC, Sparc, Alpha, IA 64 (Itanium).

Архитектура x86_64

В момент появления x86_64 едва ли можно было отнести к полноценным 64 х разрядным архитектурам, так как

- Архитектура x86_64 была получена очередным эволюционным расширением ISA IA 32
- Двоичная кодировка команд IA 32 не изменилась. Работа с 64 х разрядными регистрами и данными реализована через специальные префиксы в коде операции, переопределяющие поведение процессора по умолчанию. Декодирование команд «оптимизировано» для работы с 32 х разрядными командами.
- Доступ к 64 х разрядному адресному пространству реализован через доработанный механизм сегментной памяти IA 32

AMD/Intel пошли таким путем, потому что

- Некоторые свойства 64 х разрядных процессоров достигаются, причем переход с 32 х разрядных процессоров получается проще
- Сохранена работоспособность ISA IA 32, а следовательно всех ранее написанных для данной архитектуры программ.

В данном случае нет честной адресации всех 16 Эксбибайт не существует. Аппаратура поддерживает чуть меньшее количество разрядов, с которыми позволяет работать. При этом регистров стало больше (16 вместо 8) и размер каждого регистра расширился до 64 разрядов. Привычные названия регистров сохранились, остальные восемь назвали по номерам. Регистр EIP превратился в регистр RIP, EFLAGS – в RFLAGS, тоже расширенный до 64 разрядов. Также была добавлена особенность – обращение к данным через смещение относительно счётчика команд. Однако возникает ограничение: в данной архитектуре мы почти никогда не можем задавать произвольные 64-разрядные константы. Такую величину можно закодировать только в команде MOV:

- Переслать непосредственно закодированный операнд можно только в регистр.
- В общем формате адресного кода операнда памяти смещение осталось 2^{32} . Можно задать абсолютный адрес 2^{64} , но только если это пересылка данных из RAX.

Следующие изменения произошли в соглашении вызова функции. В основное соглашение вызова было создано в некоторой степени аналогично рассмотренному выше. Идеологически оно было похоже на совмещение отказа от указателя фрейма и соглашения FASTCALL. При этом аргументы передаются не через пару регистров (ecx и edx или rcx и rdx), а через большее число регистров (рис. 12.1).

rax	Возвращаемое значение	r8	Аргумент #5
rbx	Сохраняется вызванной функцией	r9	Аргумент #6
rcx	Аргумент #4	r10	Сохраняется вызывающей функцией
rdx	Аргумент #3	r11	Сохраняется вызывающей функцией
rsi	Аргумент #2	r12	Сохраняется вызванной функцией
rdi	Аргумент #1	r13	Сохраняется вызванной функцией
rsp	Указатель стека	r14	Сохраняется вызванной функцией
rbp	Сохраняется вызванной функцией	r15	Сохраняется вызванной функцией

Рис. 12.1: Иллюстрация к объяснению

Таким образом, особенности регистров x86-64:

- Аргументы передаются в функцию через регистры
 - Если целочисленных параметров более 6, остальные передаются через стек
 - Регистры аргументы могут рассматриваться как сохраненные на стороне вызывающей функции
- Все обращения к фрейму организованы через указатель стека (Отпадает необходимость поддерживать значения EBP/RBP)
- Остальные регистры:
 - 6 регистров сохраняется вызванной функцией
 - 2 регистра сохраняется вызывающей функцией
 - 1 регистр для возвращаемого значения может рассматриваться как регистр, сохраненный на стороне вызывающей функции
 - 1 выделенный регистр указатель стека

Лекция 13. Безопасность программного обеспечения

В данной лекции рассмотрим как связан такой аспект как безопасность программного обеспечения с тем, что происходит на уровне языка ассемблера. Сначала рассмотрим три примера, на которых будут рассмотрены проявления ошибочных ситуаций, а в конце будет “разоблачение”.

Пример 1 “Заглянуть за горизонт”

Рассматривается фрагмент кода, печатающего элементы массива.

```
void f(int i) {  
    int a[3] = {1, 2, 3};  
    printf("%x\n", a[i]);  
}
```

Казалось бы, что ошибок в данной функции нет. Однако, контекст вызова данной функции может быть произвольным, таким образом функция может вызываться с любым значением аргумента. То есть, не только в интервале от 0 до 2, а и больше и меньше. Выйдя за границы, можно наткнуться на неопределённое поведение (Undefined behaviour). С точки зрения стандарта языка может происходить что угодно, а с точки зрения машины это будет вполне конкретная последовательность действий, которая будет выполняться согласно предписанию.

Если исполнить программу, в которой вызывается данная функция со значением параметра, например от 0 до 9. Тогда на выходе программа будет сначала печатать числа 1, 2, 3, а затем какие-то шестнадцатеричные числа. Казалось бы, что это случайные “мусорные” значения, но это не совсем так. Для этого необходимо рассмотреть сгенерированный ассемблерный код и нарисовать устройство фрейма.

Ассемблерный код для функции f выглядит так:

```
f :  
    push    ebp  
    mov     ebp, esp  
    sub     esp, 32  
    mov     eax, dword [ebp+8]  
    mov     dword [ebp-20], 1  
    mov     dword [ebp-16], 2  
    mov     dword [ebp-12], 3  
    push    dword [ebp-20+eax*4]  
    push    .LC1  
    call    printf  
    add     esp, 16
```

Фрейм вышеописанной функции представлен на картинке 13.1. В верхней части горизонтальной чертой отмечена граница выровненных адресов, над ней располагается параметр i , который являлся аргументом в момент вызова. Ниже находится адрес возврата, ниже находится ebp , так как отрабатывается стандартный пролог. Новое значение переставляется. Затем видно, что из esp вычитается 32, значит появляется ещё 8 элементов на стеке. Пространство стека разделим отсечками для удобства. Получилось два блока. Затем следует вызов функции $printf()$, она библиотечная и её нужно вызывать на выровненной границе. После её вызова добавляются два нижних элемента на стек. Нижний третий блок является пространством аргументов. В самом низу лежит ссылка на константу со строкой с форматирования ($LC1$). Над ней располагается ещё одна ссылка, которая идёт на определённый адрес до смещения -20 - отмечено на рисунке 13.1. Выделим массив a . Тогда видно, куда показывается вторая ссылка, она идёт по адресу $(ebp - 20 + eax \cdot 4)$ и отмечена синей стрелкой.

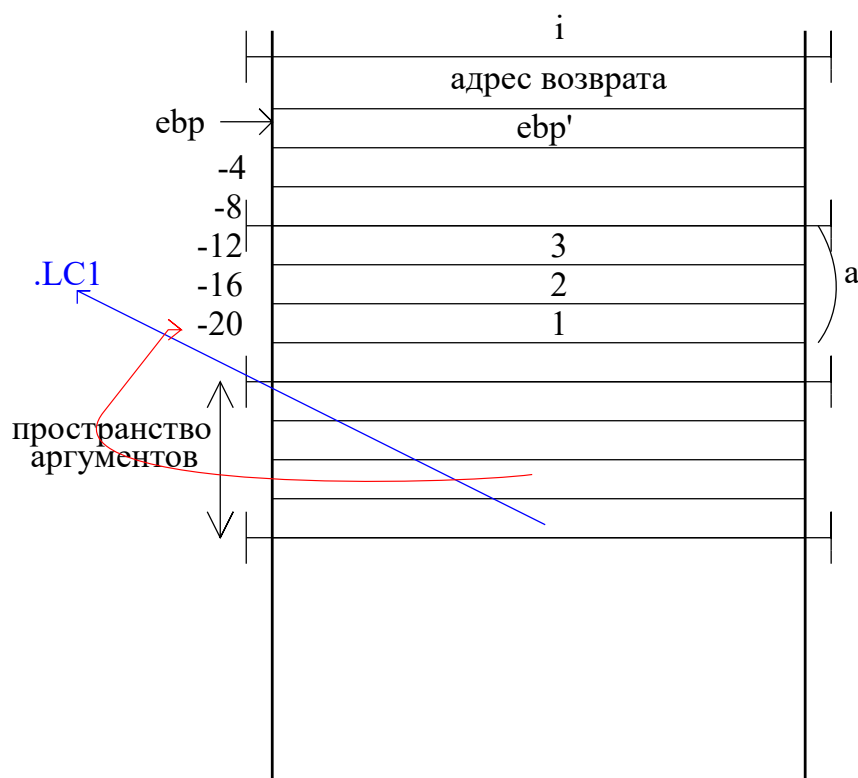


Рис. 13.1: Фрейм функции f при вызове с различными параметрами.

Таким образом, после того, как будут выведены три элемента, два значения будут напечатаны и будут являться “мусором” действительно, потому что они находятся в не инициализированной области стека (выше тройки две ячейки). Понятно, что тогда шестое выведенное значение будет являться *ebp*. Аналогично седьмое - адрес возврата, а потом

текущий аргумент. Затем стало печататься то, что находится во фрейме предыдущей функции.

Итого, при выходе за границ буфера программа продолжает работу, если память доступна для чтения. В этом случае нельзя рассчитывать на аварийное завершение.

Пример 2. “Нескучная арифметика”

Рассмотрим пример, в котором вызывается некоторая функция `f` из `main`, которая что-то записывает. Код фрагмента программы:

```
void f() {
    char buf[5];
    int *ptr;

    ptr = (int*)(buf + 1);
    (*ptr) += 2021
}

void main() {
    int x = 1;

    f();
    x += 1;
    printf("1 + 1 = %d\n", x);
}
```

Выполняя программу первый раз, она выведет ожидаемый результат 2. Кажется, что функция `f()` находится в отдельной области и другого быть не может. Однако, внутри неё происходит неопределённое поведение. Можно найти два способа, при которых при сложении $1 + 1$ получится 2. В этом примере придётся нарисовать два фрейма, относящихся к текущей функции `f` и к тому, что находится за её границей.

На рисунке 13.2 представлен фрейм функции `f`. Заштрихован массив `buf`. В предыдущем примере мы читали за границей фрейма. Переменная `X` живёт за границей фрейма функции `f`, внутри фрейма, относящегося к функции `main`. Смотря на ассемблерный код данной программы (который здесь не приводится из-за чрезмерного объёма), можно увидеть где будет на фрейме находиться переменная `X`. Расстояние от `buf` до `X` можно рассчитать, оно равно 29 байтам.

Таким образом, если в `X` будет помещена “-1”, то на выходе получится “нескучная арифметика”, а именно $1+1$ будет равняться 1.

Отметим, что в рассмотренных примерах при перемещении по фрейму можно было записать другое значение в адрес возврата. Например, в данном примере это является

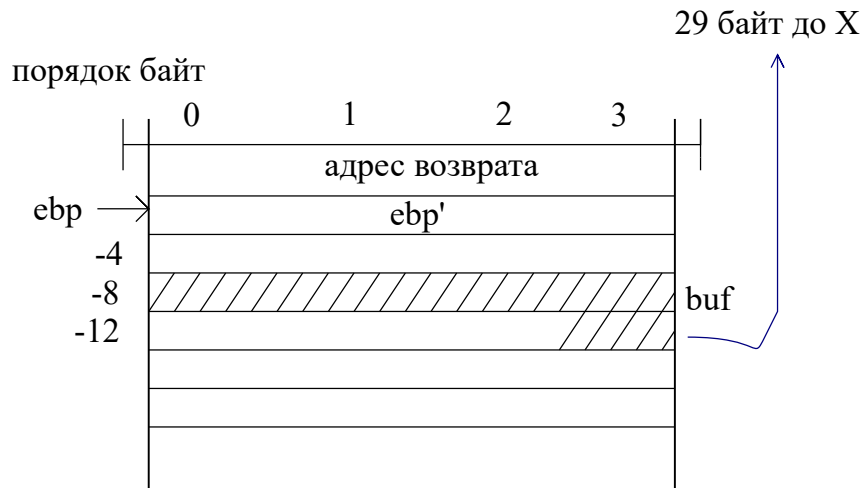


Рис. 13.2: Фрейм функции f.

вторым способом получить равенство $1 + 1 = 1$ как результат работы программы. Таким образом, мы повлияли на ход выполнения самой программы.

Пример 3. “return-to-libc”

В данном примере присутствуют две функции: **main** и **f**. Программа открывает некоторый файл, считывает оттуда данные, и эти данные затем печатает обратно на стандартный вывод. В конце программа спит одну секунду и завершает свою работу.

Код программы (фрагмент):

```
void f(FILE* fd) {
    char buf[16];
    fgets(buf, 256, fd);
    puts(buf);
}

int main(int argc, char* argv[]) {
    FILE *fd = fopen(argv[1], "rb");
    f(fd);
    fclose(fd);
    system("sleep 1");
    return 0;
}
```

Так как в программе содержится дефект, её работу можно поменять совершенно произвольным способом. Опасное место - функция **fgets**, которая никак не контролирует сколько будет считано байт по отношению к массиву **buf**. Использовать данное место можно таким образом, чтобы в итоге можно было добиться исполнения функции

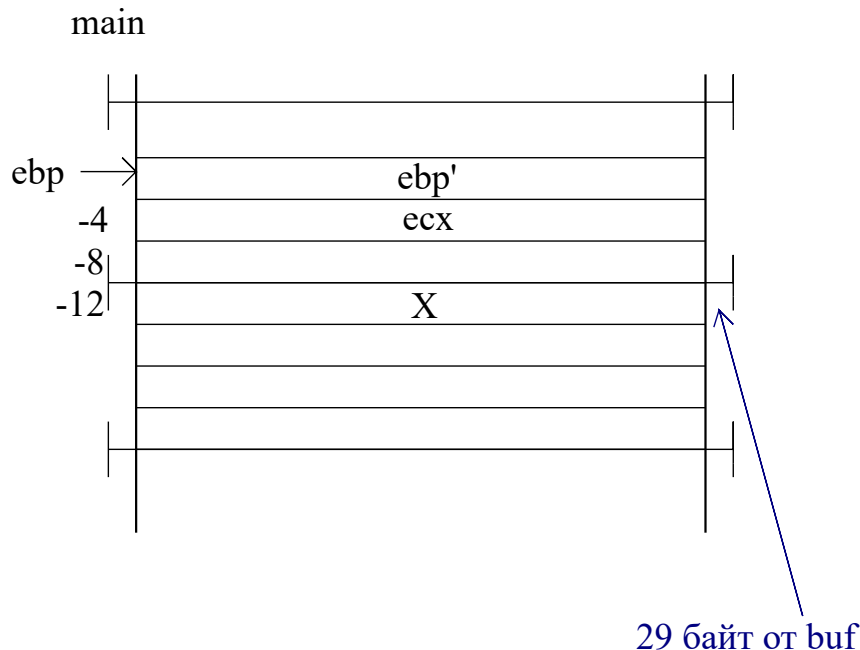


Рис. 13.3: Фрейм функции main.

system с совсем другим аргументом. Эта функция может превратиться в программный интерпретатор, если в качестве параметра ей подать **/bin/sh**.

Для того, чтобы понять каким образом это сделать, необходимо разобраться с детальным устройством программы (в каких адресах начался и закончился фрейм и его устройство). Можно разобраться в работе программы с помощью отладчика. В линуксе стандартный отладчик - это **GDB**. Однако, наблюдая за программой отладчиком, мы действуем инструментом на объект, который меняет поведение объекта. То есть, отладчик меняет переменное окружение.

Ассемблерный код функции f:

```
0x08048900    <+0>:    push ebx
0x08048901    <+1>:    sub esp,0x1c
0x08048904    <+4>:    push DWORD PTR [esp+0x24]
0x08048908    <+8>:    push 0x100
0x0804890d    <+13>:   lea ebx,[esp+0xc]
0x08048911    <+17>:   push ebx
0x08048912    <+18>:   call 0x80504f0 <fgets>
0x08048917    <+23>:   mov DWORD PTR [esp],ebx
0x0804891a    <+26>:   call 0x80509a0 <puts>
0x0804891f    <+31>:   add esp,0x28
0x08048922    <+34>:   pop ebx
0x08048923    <+35>:   ret
```

Рассмотрим фрейм при старте программы. На самом верху будут какие-то данные,

относящиеся к операционной системе, потом идёт переменное окружение, которое описывается ключевым словом **env**. Затем идут аргументы вызова и так далее. Так как отладчик добавит дополнительные элементы в **env**, то граница фрейма каждой вызываемой функции сдвинется на какую-то величину вниз, то есть картина будет отлична от той, что можно было бы наблюдать в реальной жизни без отладчика. Такой недостаток можно обойти, используя простой скрипт.

На рисунке 13.4 представлен фрейм наблюдаемой функции. В самом верху находится файловый дескриптор **fd**. Первый **push**, если посмотреть на ассемблерный код программы, записывает **fd** внизу, затем идёт 256, а затем будет записан адрес буфера **buf**, который занимает 16 байт.

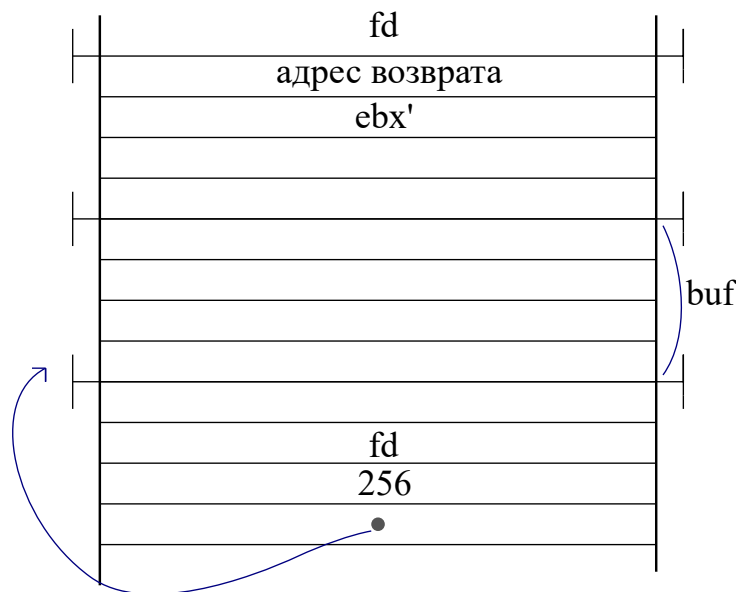


Рис. 13.4: Фрейм наблюдаемой функции для примера 3.

Когда начнётся запись информации, сначала полностью заполнится буфер **buf**, потом начнётся заполнение выше по фрейму. Таким образом, мы можем записать и в адрес возврата. Путём подмены адреса возврата, после завершения функции **f**, она прыгнет в начало функции **system**.

В тот момент, когда функция будет выполняться, она будет видеть на стеке следующую картину (рисунок 13.5). Она будет видеть некоторые элемент, куда показывает **esp**, и она будет искренне считать, что это адрес возврата той точки, откуда её вызвали (но был сделан прыжок, и **esp** будет показывать на **fd**). Выше будет лежать указатель, ссылающийся на строчку, лежащую выше, а над ним сама строчка. Теперь, чтобы всё заработало, нужно определить, какую величину надо записать, где лежит указатель на строку, чтобы этой величиной оказался адрес места памяти выше (это делается через отладчик). Адрес будет являться аргументом функции **system**, который пытались определить.

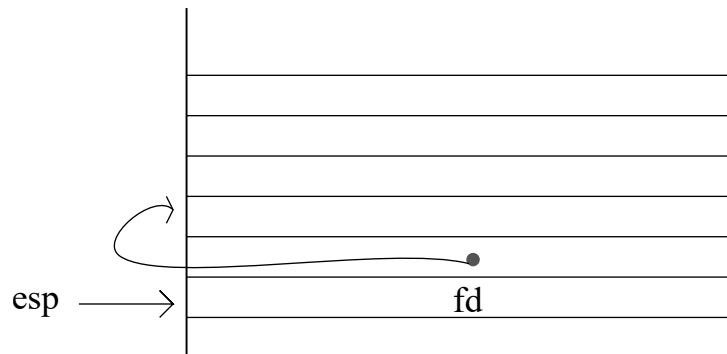


Рис. 13.5: Стек при исполнении функции в примере 3.

Построим то, что будем отправлять на вход рассматриваемой уязвимой функции для использования уязвимости. Данные на вход будут являться строкой. Первые 28 байт может быть любой информацией. После должен быть размещён адрес возврата, затем снова 4 байта, которые не принципиальны. Затем должен идти выше определённый адрес. В конце будет `/bin/sh` и нулевой байт.

Данный пример является упрощённой атакой **return-to-libc**. Такие специальные подготовленные входные данные называются **эксплойт**.

Способы защиты

Канарейка на стеке

Название позаимствовано из жизни шахтёров. Канарейка переставала петь при критической концентрации взрывоопасных газов. На стеке ситуация похожа. Можно разместить на фрейме некоторую величину, у которой будет фиксированное эталонное значение. Если функция отработала и в конце обнаружила убитую эталонную величину (переписанную), то это верный признак того, что в каких-то данных произошло переполнение при обращении на запись. В таком случае переполнение переписало к тому же ещё и адрес возврата с большой вероятностью. Из функции в таком случае выходить ни в коем случае нельзя. В коде такая вставка выглядит следующим образом:

```
...    ;  
mov    eax, dword [gs:20]  
mov    dword [ebp-12], eax  
xor    eax, eax  
...    ;  
mov    edx, dword [ebp-12]  
xor    edx, dword [gs:20]  
je     .L3  
call   __stack_chk_fail
```

.L3 :
... ;

После отработки пролога используется специальная структура данных для работы с эталонными значениями. Существуют сегментные регистры (например $[gs : 20]$), который можно определить так, что он будет показывать на некоторую служебную структуру данных, описывающую нужную для организации работы программы информацию. Называется это TLS - локальное хранилище для текущего потока выполнения. И из этого потока по смещению 20 (так как $[gs : 20]$) находится эталонное значение длиной в 4 байта - канарейка. Эту канарейку отправляют внутрь фрейма, которая находится сверху на стеке.

Другие способы

- Можно ограничить память так, чтобы она была доступна на чтение (исполнение), но не на запись. Существует специальный бит, который запрещает одновременную запись и исполнение (NX bit).
- Можно так размещать данные на фрейме, чтобы над массивами не оказывалось никаких указателей.
- Можно перемешать адреса на стеке (Address Space Layout Randomization (ASLR))
- Безопасные библиотеки

Кроме того, у некоторых стандартных функций появляются безопасные версии, которые компилятор может применять по своему усмотрению. Например, функцию `printf` он может заменить на функцию `printf_chk`, которая проводит все дополнительные проверки для форматной строки. Такая технология, которая применяет защищённые версии, называется *FORTIFY_SOURCE*. Механизмы безопасности могут работать одновременно. Комплекс защиты современных систем даёт хорошую защиту. Ядро `ubuntu` сейчас имеет различные технологии безопасности.

Лекция 14. Динамическая память

Управление динамической памятью

Динамическая память - это некоторое пространство, которое называется кучей, которой можно пользоваться для размещения по ходу выполнения программы каких-то данных. Причём, эти данные не будут ограничены областью своей жизни, как это происходит в автоматических локальных переменных. То есть, одно пространство можно запросить и заполнить из одной функции, а потом в совершенно другом месте программы этот блок освободить.

Вся работа происходит посредством интерфейса менеджера работы с динамической памятью. Ключевые функции в С, отвечающие за выделение (`calloc`, `malloc`), перевыделение (`realloc`) и освобождение памяти (`free`). Кучей пользуются, когда заранее неизвестен размер данных, время появления, область жизни.

```
void * calloc (size_t nmemb , size_t size);  
void * malloc (size_t size);  
void free(void *ptr)  
void *realloc (void ptr , size_t size);
```

Существует специальная интерфейсная функция **sbrk**, с помощью которой можно попросить у операционной системы добавить определённое дополнительное пространство, с которым хотим работать.

```
void *sbrk (intptr_t increment);
```

Пространство будет появляться в верхних адресах, относительно статических данных и расти в сторону стэка - рисунок 14.1. Текущую границу кучи можно узнать через **sbrk(0)**, так как **sbrk** как раз сдвигает верхнюю границу.

Границы стэка и кучи конечно могут сойтись, но их собственные колебания не влияют друг на друга.

Выделение динамической памяти

Менеджер работает не с отдельными байтами, а с блоками памяти. Менеджер рассматривает кучу, как последовательность блоков некоторого размера. Сами блоки состоят тоже не из байтов, а из машинных слов, так как будут определяться требования по выравниванию. Управление блоками бывает двух типов:

1. Явное управление: когда сам разработчик отвечает за то, в какой момент он запросит, а в какой память освободит. Яркий пример - язык С. Такой способ даёт выигрыш по производительности, но добавляет титанический объём ответственности.

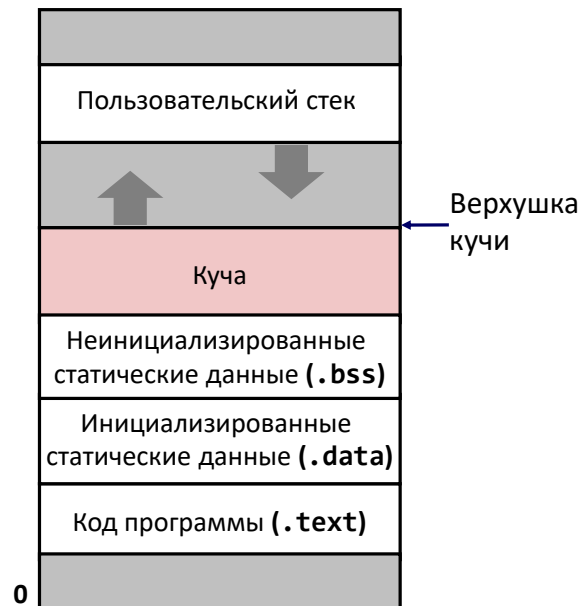


Рис. 14.1: Устройство кучи и стека в памяти, они растут навстречу друг другу.

2. Неявное управление. Разработчик выделяет память, но освобождает её за него сборщик мусора автоматически. Освобождаться в данном случае будут только те блоки, которые признаны, как недоступные для работы. Для них должно быть доказано, что обратиться из текущего потока программы к ним уже нельзя. Зачастую такое управление используется, когда код работает в виртуальном окружении. Даже в языке C++ есть библиотеки, позволяющие неявно управлять памятью в той или иной мере.

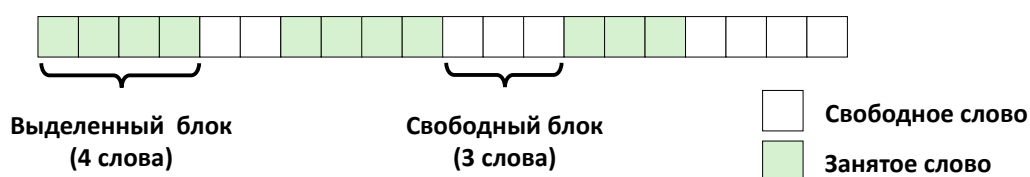


Рис. 14.2: Пример изображения кучи. Блоки могут иметь какой-то фиксированный размер, заполнение происходит слева направо. Если присутствует свободное пространство размером, вмещающим блок, который запрашивается, то он выделяется. Иначе ищется свободное пространство дальше, таким образом образуются “дыры”.

Отметим, что **malloc** может вернуть **NULL**. Такая ситуация например может быть при исчерпывании памяти.

Ограничения

Если создавать вышеописанные функции, следуя стандарту языка C, то необходимо будет преодолеть определённые проблемы.

- Пользовательская программа не накладывает никаких ограничений на себя и может требовать столько вызовов функций **malloc** и **free**, сколько посчитает нужным. Таким образом последовательность вызовов этих функций будет произвольна.
- Вызовы **free** получают в качестве аргумента только указатель из функций **malloc**. Отсутствует параметр, отвечающий за размер заранее выделенной памяти.
- Память должна предоставляться незамедлительно, поэтому необходимо выравнивать блоки.
- Если что-то уже было выделено, то эти данные уже не переместить для более рационального использования места.

Производительность. Пропускная способность

Если взять два различных менеджера работы с динамической памятью (две разные реализации), правомерно поставить задачу их сравнения.

Необходимо выделить набор численных характеристик.

1. Один из критериев - достигаемая пропускная способность. То есть, цель - уменьшить количество служебных действий между запросами. Итог - число выполненных запросов за единицу времени. То есть, например, количество вызовов **malloc** и **free** в секунду.
2. Второй показатель - это пиковое использование памяти. То есть, сколько данных, полезных с точки зрения пользователя, менеджер смог разместить в пространстве куча. Рассмотрим порядок запросов вызовов функций **malloc** и **free**. Можно суммарно посчитать, сколько суммарно байт было запрошено функцией **malloc** - это будет **суммарная полезная нагрузка**. Затем суммарная полезная нагрузка делится на размер кучи и получается **пиковое использование памяти** после какого-то числа запросов.

Внутренняя фрагментация. Внешняя фрагментация

Фрагментация бывает двух видов, рассмотрим оба.

1. Внутренняя фрагментация. Данный тип фрагментации возникает из-за того, что помимо пользовательских данных, помимо пользовательских данных, служебные данные тоже нуждаются в хранении. Таким образом, полезная нагрузка будет лишь

частью блока, который занимается. Полезная нагрузка может быть как в начале, так в конце и в середине блока. Кроме того, существуют требования выравнивания. Всё это будет “отъедать” пространство.

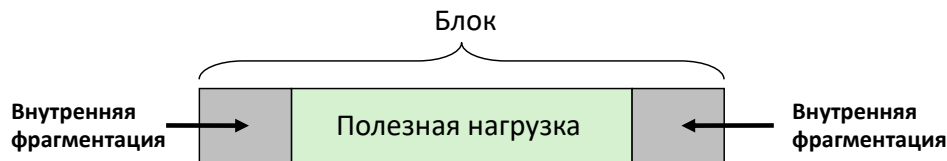


Рис. 14.3: Иллюстрация внутренней фрагментации. Полезная нагрузка занимает лишь часть блока памяти.

2. Внешняя фрагментация. Данное явление трудно подсчитать количественно, так как оно выражается только в тот момент, когда памяти уже перестаёт хватать. Выражается в том, что суммарное количество свободных блоков может быть достаточно большим, но непрерывных блоков нужной ширины не будет. Пример представлен на картинке 14.4.

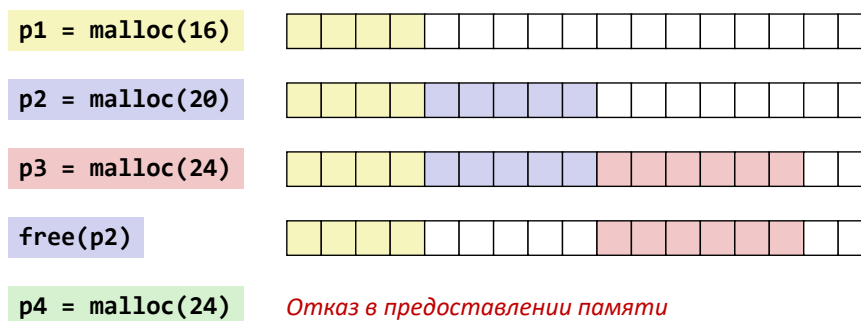


Рис. 14.4: Иллюстрация внешней фрагментации. На последнем шаге не остаётся ни одной свободной непрерывной последовательности блоков памяти нужного размера, хотя суммарно их достаточно.

Проблемы реализации менеджера памяти

В итоге можно выделить ключевые проблемы реализации менеджера, который будет управлять памятью.

1. Как следует запоминать, сколько памяти должно быть освобождено для данного адреса?
2. Как лучше поддерживать информацию о свободных блоках? В какой структуре данных?

3. Если принято решение выделить блок большего размера, чем было запрошено, что делать с лишней памятью? Можно отдать весь выделенный кусок памяти с излишком. Либо можно расщепить блок на два.
4. Какой блок лучше выбрать для выделения?
5. Как лучше распорядиться освобождением блоков? Что будет далее с освобождённым блоком?

Далее ответим на первые два вопроса подробнее.

1. В начале блока выделяемой памяти необходимо сформировать служебный заголовок размером с машинное слово, потому что в нём будет размещена длина блока. По смещению от текущего указателя, который вернулся пользователю от функции, заказывающей память, на -4 будет находиться величина, показывающая размер блока.
2. Вариантов отслеживания свободных блоков может быть несколько.
 - Можно использовать неявный список, в котором хранится в начале каждого блока его размер.
 - Можно использовать явный список свободных блоков с использованием указателей.
 - Раздельные списки. Блоки распределяются по раздельным спискам, в зависимости от размера этих блоков.
 - Можно отсортировать блоки по размеру. Например использовать сбалансированное дерево с указателями в каждом свободном блоке, и с длиной блока в качестве ключа.

Далее отдельно рассмотрим метод неявного, явного и раздельного списка.

Неявный список

Из последовательности размеров будет сформирован однонаправленный список. Каждый раз, зная размер блока, прибавляя его, оказываемся в начале следующего блока. В этом случае возникает проблема хранения состояния. Уже недопустимо использовать ещё одно машинное слово в дополнении к размеру.

Так как память придётся выравнивать в любом случае, то в итоге придём к тому, что размеры блоков будут кратны какому-то числу. Поскольку в компиляторе gcc используется требование по выравниванию в 8 байт, то три младших разряда у длины гарантированно будут нулевыми. Таким образом можно использовать данные разряды

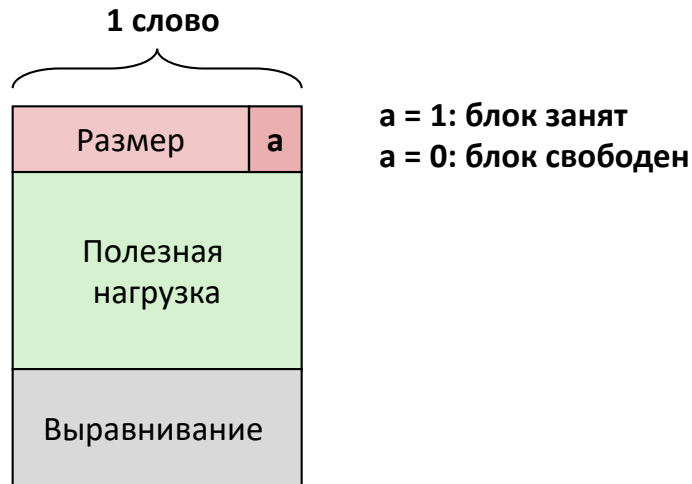


Рис. 14.5: Формат выделенных и свободных блоков.

для хранения дополнительного признака. В самом младшем разряде нуль будет, если он свободен, а единица, если занят. Иллюстрация такой схемы хранения на рисунке 14.5.

В конце неявного списка должен стоять блок-терминатор, имеющий нулевой размер.

- Поиск свободного блока в данном списке - это линейный проход от самого начала. Пример псевдокода для такого поиска:

```
p = start;
while ((p < end) &&
      ((*p & 1) ||
      (*p < len )))
    p = p + (*p & 2);
```

Данный поиск прост, но не очень быстр. Можно продолжить поиск с места, где был найден предыдущий блок - это увеличит скорость поиска.

Можно также просматривать каждый раз весь список в поиске наилучшего свободного места, чтобы уменьшить дальнейшую фрагментацию. В данном случае фрагментация не такая сильная, но скорость работы снижается.

- Выделение свободного блока. После нахождения свободного блока, в случае, если он превосходит требуемый размер, необходимо его расщепить и добавить отщеплённую часть в список.
- Освобождение блока при данном способе работы с памятью будет представлять собой просто запись в младшей разряд размера нулей. Однако, в данном случае возможна ложная фрагментация (рисунок 14.6). Необходимо сливать блоки воедино.

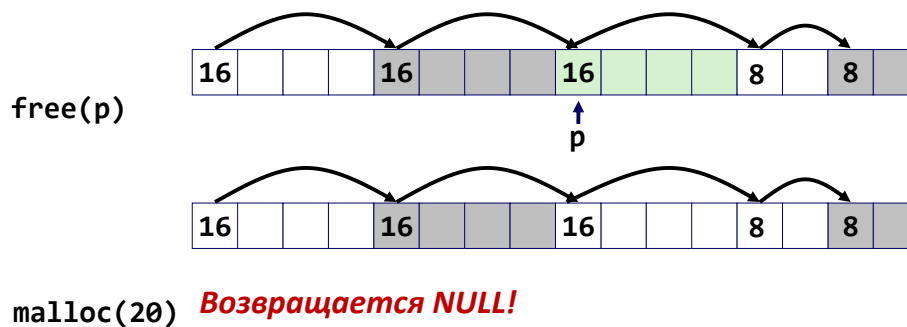


Рис. 14.6: Иллюстрация ложной фрагментации на примере. В данном случае две свободные ячейки разной длины будут стоять подряд, их суммарная длина вмещает запрашиваемый объём, но по отдельности не вмещает. Итого получаем фрагментацию.

- Слияние в прямом направлении произвести просто. Суммарной блок будет просто иметь сумму длин, заголовка второго блока больше не будет существовать с точки зрения менеджера памяти. Если освободился блок памяти, который находится ПЕРЕД уже свободным блоком, то тогда нужен другой способ слияния.
- Двухнаправленное слияние. Ещё Кнут ввёл такое понятие, как граничные теги. Помимо заголовка (header), появляется ещё и нижний заголовок (footer). То есть, в списке блоков неявный двухнаправленный список, по которому можно ходить в обоих направлениях. Тогда слияние с предыдущим блоком производится аналогично слиянию со следующим. Однако, увеличивается внутренняя фрагментация, так как теперь заголовок находится с двух сторон (рисунок 14.7).

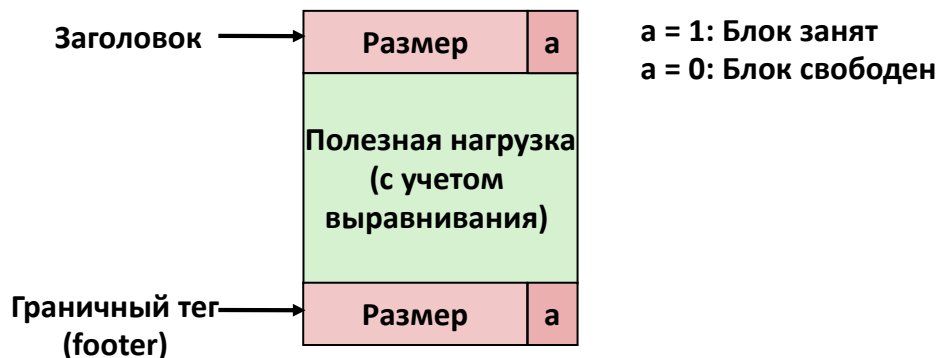


Рис. 14.7: Формат выделения свободных блоков с двумя заголовками.

Слияние во всех четырёх случаях (оба соседа освобождаемого блока заняты, занят только верхний, занят только нижний, оба свободны) происходит за линейное время.

- Оптимизация граничных тегов. Возможно частично избежать внутренней фрагментации. Граничный тег необходим только в том случае, если блок освобождается, и потом с ним будет производиться слияние. Тогда, в заголовок

текущего блока можно добавить ещё индикатор того, свободен ли сосед слева или справа.

Явные списки

В данном случае становится гораздо больше служебных данных. Необходимо будет использовать два дополнительных машинных слова в свободных блоках, которые будут содержать ссылки на следующий и предыдущий блок. Однако, соседние они будут не по адресам, а соседние в списке свободных блоков. Занятые блоки будут выглядеть точно также - с заголовками для слияний. Минимальный размер свободного блока в данном случае вырастает с 8 до 16 байт. На рисунке 14.8 представлены схематично свободный и занятый блоки.

Занятый блок (как и ранее)

Размер	a
Полезная нагрузка и выравнивание	
Размер	a

Свободный

Размер	a
Следующий	
Предыдущий	
Размер	a

Рис. 14.8: Свободный и занятый блоки в явном списке.

Понятно, что блоки могут теперь идти в любом порядке. Работа будет производиться с логической организацией.

- Выделение памяти. При выделении памяти будет осуществляться проход только по свободным блокам. Проход может быть как с начала, так и с какого-то промежуточного шага. При выделении может происходить расщепление свободного блока. Отщепляется занимаемое пространство, оставшийся свободный кусочек вставляется в список.
- Освобождение памяти. Если блок памяти освободился, то вставлять его можно как угодно. Существуют два принципиальных метода:
 1. Возможна вставка в начало списка (**LIFO**), но тогда в начале будет формироваться высокая фрагментация (согласно различным исследованиям).
 2. Возможна вставка в порядке следования адресов. То есть, поместить в список освобождённый блок так, что список всегда поддерживает упорядоченность по адресам. Минус такого подхода в том, что место вставки необходимо искать.

Раздельные списки

Для каждого класса блоков разного размера формируется отдельный список. Эти размеры сначала могут нарастать с каким-то шагом линейно, а затем, как правило, идут по степеням двойки. Тогда процесс выделения блока размером n байт выглядит следующим образом:

- В соответствующем списке ищем блок размера $m > n$.
- Если подходящий блок найден, то расщепляем его и помещаем оставшийся фрагмент в соответствующий список.
- Если блок не найден, то ищем в списке следующего класса. Повторяем процедуру до успеха.
- Если после полного просмотра всех списков блок не найден, то запрашиваем у ОС дополнительную память для кучи, используя **sbrk()**. Затем в новой памяти создаём блок размера n , а оставшуюся память одним блоком помещаем в список класса наибольшего по размеру блоков.

Если после освобождения блока было произведено слияние, то слитый блок помещается в подходящий список.

Итоговые преимущества состоят в более высокой пропускной способности и улучшенное использование памяти.

Лекция 15. Работа с числами с плавающей точкой. Часть 1

Дробные двоичные числа

Число можно представить, сдвинув “двоичную точку” таким образом, что биты слева от неё представляют собой положительные степени двойки, а справа - отрицательные. Таким образом, рациональное число можно представить в виде суммы: $\sum_{k=-j}^i b_k \times 2^k$.

Некоторые процессоры специализируются на вычислениях с дробными числами, поэтому имеют их аппаратную поддержку.

Примеры дробных двоичных чисел:

$$\begin{array}{ll} 5\frac{3}{4} & 101.11_2 \\ 2\frac{7}{8} & 10.111_2 \\ \frac{63}{64} & 0.111111_2 \end{array}$$

Работая с таким представлением чисел, деление и умножение на 2 можно выполнять сдвигом соответственно вправо и влево.

Представимые рациональные числа

Иногда числа приходится округлять, когда невозможно представить число точно. Например, числа $\frac{1}{3}$, $\frac{1}{5}$, $\frac{1}{10}$ будут представлены двоичным числом с периодической частью.

То есть, точно можно представить только рациональные числа вида $\frac{x}{2^k}$

Представление чисел с плавающей точкой

В данном случае пытаемся рассмотреть число, как произведение трёх величин. Первый множитель определяет знак числа, второй - мантисса, а третий - порядок. То есть, число можно представить в виде произведения:

$$(-1)^s \times M \times 2^E \quad (15.1)$$

В данных обозначения s будет знаковым битом. Мантисса M - дробное число в полуинтервале $[1.0, 2.0)$. Порядок E определяет степень двойки.

Определим, как эти биты попадают в общий битовый вектор. В начало вектора попадает знаковый бит, в конец - мантисса. Середина же - степень двойки, определяется не напрямую (рисунок 15.1).

Для того, чтобы закодировать порядок, введём смещение $bias$.

$$bias = 2^{k-1} - 1 \quad (15.2)$$



Рис. 15.1: Представление числа из трёх множителей, как битовой последовательности. Знаковый бит - *s*, мантисса - *frac*, *exp* - закодированный порядок.

И тогда будет кодировать поле *exp*, как сумму:

$$exp = E + bias \quad (15.3)$$

Из-за сдвига, можем кодировать битовой последовательностью как положительные числа, так и отрицательные (положительные и отрицательные степени двойки *E*).

Размеры чисел могут варьироваться. В стандарте языка *C* присутствуют два типа: **float** и **double**. Первое на 32 разряда, а второе на 64. В каждом будет по одному биту на знак, Порядок **float** будет занимать 8 битов, а у **double** - 11 битов. Мантисса занимает 23 бита у **float** и 52 бита у **double**. То есть, мантисса увеличивает значительно.

Нормализованные числа

Определение: нормализованное значение (число) - такое число, у которого порядок не принимает крайние значения (все нули или все единицы). Можно рассмотреть пример представления числа в таком виде. Рассмотрим число 15213.0. Тогда последовательность будет представлять собой: 0 в знаковом бите, 10001100 в *exp* и 11011011011010000000000. Итого получается: 01000110011011011011010000000000.

Ненормализованные числа

В какой-то момент, приближаясь к нулю, мы упрёмся в то, что порядок представимых чисел становится всё меньше и меньше, и на очередном шаге уже некуда смещаться. Это называется область **денормализованных** чисел. То есть, при движении дальше в сторону уменьшения, получим округлением нуль. В данном случае значение порядка не просто нули, а определяется таким образом:

$$E = -bias + 1 \quad (15.4)$$

Помимо того, мантисса, в свою очередь, оказывается на интервале от 0 до 1. Таким образом, у неё появляется лидирующий нуль. Нуль попадает в категорию ненормализованных чисел, причём для него есть два представления: +0 и -0 (просто разная кодировка).

Особые числа

Это случай, когда в поле `exp` одни единицы. Такая комбинация введена для обозначения особых величин - $\pm\infty$, если часть `frac` представляет собой все нули. Если же она - не все нули, тогда закодированы специальные значения NaN - not a number. Они используются в ситуациях, когда значение операции не определено. Причём эти величины бывают разных видов - сигнальный или тихий, который будет поглощаться в вычислениях.

Диапазоны значений

Таким образом, на числовой оси получается симметричная картинка значений чисел (рисунок 15.2).

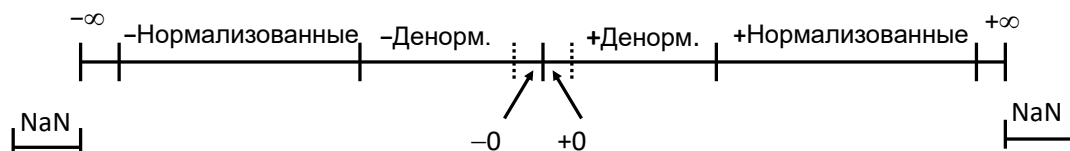


Рис. 15.2: Диапазоны значений чисел.

Ноль имеет два представления - влево и вправо. Затем идёт диапазон с денормализованными числами, потом с нормализованными. Когда не хватает разрядов для хранения чисел, то отображается бесконечность.

Пример чисел, подходящих под все правила - это 8-разрядные числа с плавающей точкой, которые имеют следующую структуру:

- Знаковый бит - старший бит
- Следующие 4 бита - порядок, смещение - 7
- Последние три бита - дробная часть (мантисса)

Таким образом, оказываются выполненными все требования стандарта IEEE 754 к формату числа. Диапазон значений простирается от $\frac{1}{512}$ до 240. Характер покрытия такой, что ближе к нулю числа располагаются сильно гуще, чем в отдалении. Первый диапазон нормализованных чисел идёт с той же частотой, что и денормализованные числа.

Округление

Существуют различные способы округления: к нулю, к наименьшему, к наибольшему, к ближайшему. То есть, числа могут быть отрицательными и положительными. У способов округления к нулю, к наименьшему и к наибольшему недостаток заключается

в том, что при длительных вычислениях погрешность накапливается. Тогда суммарная ошибка может сильно повлиять на вычисления. Поэтому используют округление к ближайшему. В таком случае будет происходить некая компенсация погрешностей округления. В каких-то случаях что-то теряется от точного результата, в каких-то прибавляется к нему. Однако и в таком случае возможна систематическая ошибка.

Рассмотрим два случая:

1. В случае десятичных чисел необходимо производить округление к ближайшему чётному при нахождении числа ровно в середине. Пример (в левом столбце пишется то, что округляем, а в правом то, что получили):

1.2349999	1.23
1.2350001	1.24
1.2350000	1.24
1.2450000	1.24

Как видно, последнее число округлили в сторону чётного, как и предпоследнее.

2. В случае двоичных чисел сначала необходимо определить несколько понятий. **Середина** - это случай, когда отбрасывается последовательность из одной единицы и нулей ($1000\dots_2$). То есть - это ситуация, когда биты справа от позиции к которой происходит округление равняются $1000\dots_2$. Пример (в левом столбце пишется то, что округляем, а в правом то, что получили):

10.00011_2	10.00_2
10.00110_2	10.01_2
10.11100_2	11.00_2
10.10100_2	10.10_2

В примере сравниваются последние три цифры с “серединой”.

Арифметические операции

Основная идея операция над такими числами с плавающей точкой состоит из нескольких шагов:

1. Вычислить точный результат
2. Поместить результат в требуемый диапазон точности. Возможно, придётся произвести округление поля `frac`. Так же возможен случай переполнения, если порядок результирующего числа оказался слишком большой.

Умножение

Возьмём две величины в битовом представлении. Каждому будут соответствовать знаковые биты s_1 и s_2 , мантиссы M_1 и M_2 , порядки E_1 и E_2 . Тогда операции будут производиться следующим образом с каждой составной частью чисел:

$$s = s_1 \hat{s}_2 \quad (15.5)$$

$$M = M_1 \times M_2 \quad (15.6)$$

$$E = E_1 + E_2 \quad (15.7)$$

Затем необходимо произвести исправления в полученных значениях:

- Если мантисса $M \geq 2$, то сдвигаем M вправо (то есть, делим на 2), увеличивая порядок E .
- Если E выходит за пределы, то случилось переполнение, оказываемся в одной из бесконечностей (зависит от знакового бита) .
- Округляем M до соответствующего размера поля `frac`.

Сложение

Со сложением всё сложнее, так как перед выполнением непосредственно сложения, необходимо разместить мантиссы относительно друг друга с учётом того, что порядки чисел могут сильно различаться. Аналогично рассмотрим два числа в битовом представлении с соответствующими s, M, E .

Пусть $E_1 > E_2$. Тогда, при сложении, мантисса первого числа окажется левее, остальное заполним нулями (мысленно), и сложим мантиссы (рисунок 15.3). В результирующем числе берём порядок большего числа. То есть, в данном примере результирующее $E = E_1$.

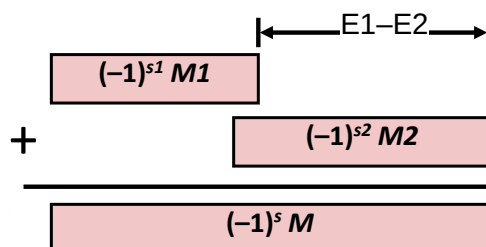


Рис. 15.3: Сложение двух чисел со сдвигом мантисс.

В конце необходимо произвести несколько исправлений:

1. Если $M \geq 2$, сдвигаем M вправо, увеличивая E
2. Если $M < 1$, сдвигаем M влево на k позиций, уменьшая E на k .
3. Переполнение, если E выходит за пределы. Тогда записываем ту или иную бесконечность.
4. Округляем M до соответствующего размера поля $frac$. Значительная часть разрядов после округления может быть отброшена, если слагаемые сильно отличаются.

Математические свойства сложения и умножения

Отметим математические свойства сложения. В процессе сложения образуется группа. Рассмотрим следующие свойства Абелевых групп:

- Замкнутость - выполняется, с учётом того, что присутствуют бесконечности и NaN. Причём неважно, происходит сложение $a + b$ или $b + a$.
- Коммутативность - выполняется.
- Ассоциативность - не выполняется. Так как порядок, в котором будем складывать важен, так как одно число (очень большое) может поглотить другое число (очень малое).
- Нейтральный элемент - ноль, действительно.
- Каждый элемент имеет обратный - почти всегда, за исключением бесконечности и NaN.
- Монотонность - почти всегда выполняется.

Теперь рассмотрим математические свойства умножения. Рассмотрим следующие свойства коммутативных колец (выполняются ли они здесь):

- Замкнутость относительно умножения - выполняется.
- Коммутативность - присутствует.
- Ассоциативность - не выполняется, так как тоже могут быть погрешности, связанные с округлением.
- Мультипликативная единица - 1.0, выполняется.
- Дистрибутивность над сложением не выполняется, так как можно вылететь в переполнение, существуют примеры.

Числа с плавающей точкой в языке Си

Под кодирование выделяется определённое количество разрядов. Исходя из этих свойств, можно говорить о приведении типов между собой.

- При переводе `int` в `double` происходит точное приведение, поскольку `long` и `int` имеют длину 32 бита, что меньше, чем 53.
- Приведение же типа `int` к типу `float` может привести с собой округления, согласно принятым соглашениям.
- Приведение `double` или `float` к `int` происходит посредством отбрасывания дробной части (аналогично округлению к нулю).

Упрощённая схема x87

Сопроцессор `x87` разрабатывался инженерами Intel в конце 70-х годов, которые принимали активное участие в разработке стандарта IEEE-754. То есть, всё делалось одними людьми. Изначально сопроцессор представлял собой отдельно работающее устройство. То есть, отдельно на материнскую плату вставлялось устройство - сопроцессор, который позволял производить вычисления с числами с плавающей точкой. Только на этапе 486 процессора появилось решение, в котором на одном кристалле физически были объединены два устройства - 486 и `x87`. Только в Pentium сопроцессор, стал быть полностью интегрирован в кристалл. Даже в современных процессорах все нужные функциональные возможности, все наборы команд, придуманные 40 лет, назад имеют место быть.

Упрощённо сопроцессор `x87` состоит из регистров, хранящих сами величины - числа с плавающей точкой, и из служебных регистров (управляющий регистр, регистр признаков, регистр состояния), схема на рисунке 15.4.

Работа с сопроцессором `x87` начинается с команды **FINIT**, которая выполняет его инициализацию. Все команды, относящиеся к `x87` начинаются на F. Команда **FINIT** сбрасывает все настройки в начальное состояние. Это осуществляется посредством записи в три служебных регистра определённых последовательностей битов. Интересно то, что `x87` реализует в себе стековую машину. Здесь есть подмножество команд, позволяющее выполнять операции никак не специфицирую операнды. Возможность такой работы базируется на том, что в регистре состояния есть некоторое поле, которое задаёт верхушку стека. Восемь регистров, с которыми ведётся работа, в свою очередь составляют стек регистров. В специальном поле TOP (имеет три разряда) хранится указатель на один из регистров (то есть какое-то имя). Тот регистр, на который ссылается поле, будет называться регистр *ST0*. То, что находится выше называется *ST1*, *ST2* и так далее.

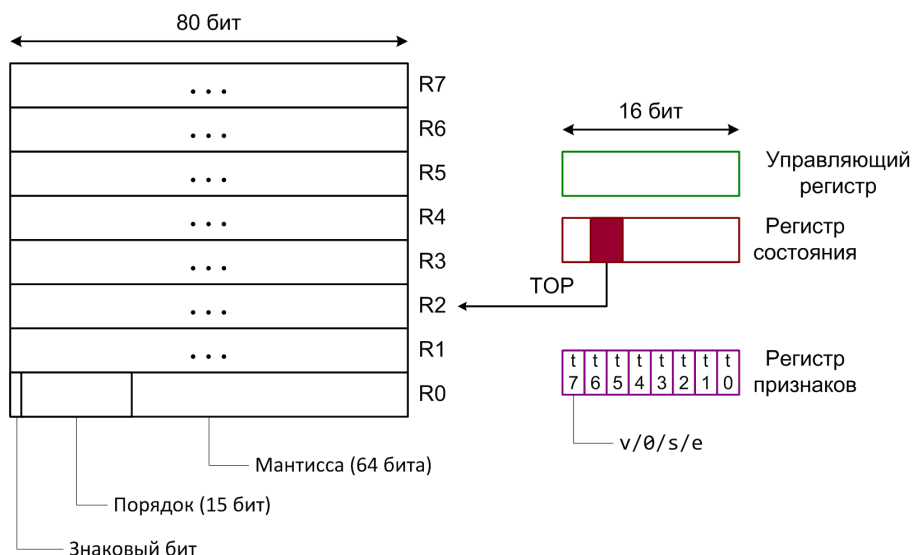


Рис. 15.4: Упрощённая схема x87.

Интересно, что, изменив состояние стека и передвинув поле TOP вниз, изменится вся система наименования. Таким образом, *ST0* окажется уровнем ниже, то, что было *ST0*, станет *ST1*.

Кроме того, регистры содержат в себе 80 разрядов, таким образом, они реализуют расширенную точность. Также, в мантиссе x87 явно кодируется целая часть. Так как исторически x87 и x86 стояли в разных местах на плате, то нет никакой возможности напрямую передавать информацию из регистров общего назначения x86 в эти регистры с плавающей точкой. Обмены могут происходить только с памятью.

Лекция 16. Работа с числами с плавающей точкой. Часть 2. Сопроцессор x87

Слово (регистр) состояния

Ранее было рассмотрено, что сопроцессор x87 имеет 8 регистров для хранения чисел с плавающей точкой, причём эти регистры образуют некий стек. Кроме того, присутствуют служебные регистры.

Первый служебный регистр - регистр состояния. Он содержит в себе 3 разряда (с 11 по 13), которые кодируют в себе текущий номер верхушки стека (из аппаратных регистров). Самый старший разряд (15) показывает, происходят ли в сопроцессоре какие-то вычисления (бит занятости). Разряды 14, 10, 9, 8 - это биты, показывающие специальные условия, связанные со сравнением чисел с плавающей точкой. Далее, от 0 до 7 идут разряды, показывающие наличие тех или иных ситуаций, возникших в ходе вычислений. Интересно то, что теперь у нас появляется возможность реагировать на возникающие прерывания. Схематичное представление на рисунке 16.1.

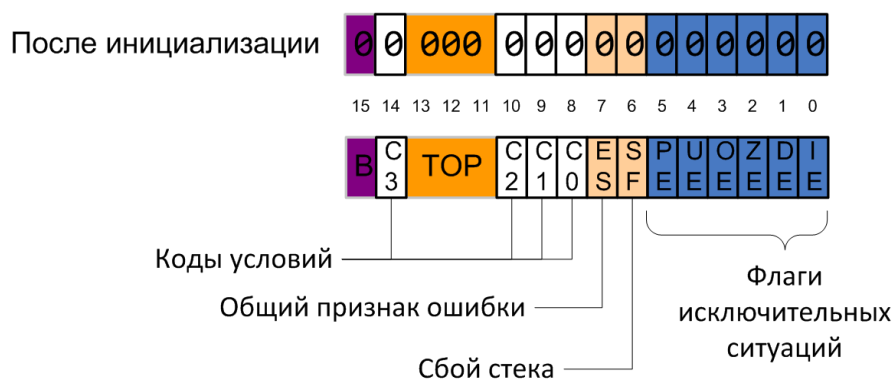


Рис. 16.1: Регистр состояния сопроцессора x87.

Сверху вниз происходит течение времени. Будем считать, что присутствует только одно процессорное ядро, которое выполняет поток команд. Изобразим программу в виде непрерывной линии. В какой-то момент произошло какое-то взаимодействие процессора с внешним миром, например начало работу устройство ввода-вывода. В этот момент происходит **системное прерывание**. Это означает, что аппаратура, получив электрический импульс, способна передать управление на какой-то совсем другой код, который выполняет служебную функцию. Этот служебный код называется обработчик прерывания. Диаграмма изображена на рисунке 16.2

Обработчик вызывается не командами, а просто из ранее заполненных служебных регистров вытаскиваются определённые адреса, начиная с которых находится функции, реагирующие на возникшее событие. Обработчик может отдать управление обратно программе.

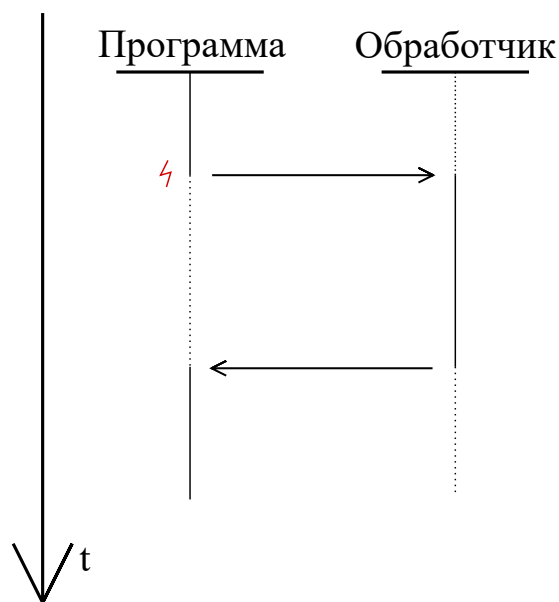


Рис. 16.2: Иллюстрация системного прерывания.

Механизм прерываний используется и как реакция на какой-то сбой. Например, при сбое в программе, управление также передаётся на обработчик. Обработчик решает, что делать с проблемой, он может завершить работу программы или сделать что-то другое.

В регистре состояния фактически может возникнуть 8 состояний, когда что-то пошло не так. На рисунке 16.1 синим отмечены флаги исключительных ситуаций, а персиковым - сбой работы со стеком. Сбой стека - это переполнения в одну или в другую сторону. Сам ход вычислений имеет не такие однозначные сбои.

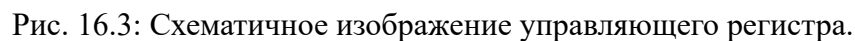
В документации традиционно мнемоники исключений начинаются с решётки, а далее идёт некий мнемонический символ. Запишем некоторые коды исключительных ситуаций и их расшифровки:

#I	inv - invalid operation
#Z	деление на нуль
#P	потеря точности (результат был округлён)
#U	underflow
#O	overflow
#D	операнд денормализован

Как видно, некоторые исключения могут возникать в штатных ситуациях и довольно часто.

Команда **FINIT** выставляет в регистр состояния нули.

Младшие 6 разрядов по сути управляют реакцией на исключения из регистра состояния. То есть, если в маске стоит единичка, то соответствующая исключительная ситуация игнорируется. В самом начале **FINIT** выставляет во все разряды единички. То есть, по умолчанию исключения игнорируются. Также присутствуют поля, управляющие точностью и округлением. По умолчанию **FINIT** выставляет нули в поле округления, что означает округление к ближайшему чётному. В поле точности по умолчанию устанавливаются две единицы, что означает использование расширенной точности при вычислениях. Схематичное изображение регистра на рисунке 16.3.



Регистр признаков показывает, что хранится в регистрах с данными. При инициализации регистр заполняется единицами (16 штук). На каждый регистр с числами приходится по два разряда. Эти два разряда кодируют содержимое регистров с данными следующим образом:

- 0 означает нормализованное число с плавающей точкой
- 1 означает, что в регистре записан нуль
- 2 означает, что в регистре записаны особые числа (такие как бесконечность)
- 3 означает, что в регистр свободен

Таким образом, в начальном состоянии видно, что все регистры свободны.

При разработке программ, NASM предлагает поддержку чисел с плавающей точкой. Можно задавать числа с различной точностью. Точность может быть одинарной, половинной и другой. Все константы могут быть правильно закодированы.

Тип `double` занимает 8 байт, таким образом `dd` превращается в `dq`. Если говорить о расширенной точности, то это обозначается как `dt`. Зачастую двойной и одинарной точности достаточно.

Существуют специальные обозначения для бесконечностей, `NaN` тихого (он игнорируется) и сигнального. Эти константы:

```
__Infinity__  
__NaN__  
__QNaN__  
__SNaN__
```

Пример 1

Разберём простой пример - сложение двух чисел. Первым делом вызывается команда **`finit`**. Далее попробуем сложить два числа, которые определены и размещены в памяти. Загнать эти числа в регистр сопроцессора можно только пересылкой из памяти, так как исторически чип `x87` находился отдельно на плате. Команда **`fld`** позволяет считывать из памяти. Она имеет один операнд, который является памятью различного размера (32, 64, 80).

После считывания второго числа - `fld` отработал два раза, у сопроцессора оказываются заняты два регистра с числами. В регистре `ST0` будет лежать число `y`, которое было считано вторым, поэтому оно будет вершиной стека регистров. В регистре `ST1` будет лежать число `x`, которое было считано первым.

Третья команда **`faddp`**. Она производит сложение верхних двух элементов, результат отправляется в `ST1`, а `TOP` увеличивается на единицу, освобождая то, что раньше находилось на верхушке стека. То есть, после сложения в регистре `ST0` будет лежать элемент `x+y`.

Последний шаг - выгрузка результата в память. Для этого используется команда **`fstp`**, причём буква `p` необязательна, она делает `pop` (снимает элемент со стека). Таким образом, пересылка прошла. Итоговый код процедуры сложения:

CMAIN :

```
finit  
fld dword [x]  
fld dword [y]  
faddp  
fstp dword [z]  
PRINT_HEX 4, z  
NEWLINE  
xor     eax , eax  
ret
```


Теперь ничего не мешает выводить числа с плавающей точкой на печать. Можно вызывать библиотечную функцию `printf`. Для этого, необходимо после выгрузки скопировать результат в пространство аргументов перед вызовом функции `printf`. Стоит обратить внимание на то, что при вызове функции с переменным числом параметров есть одна особенность. Если величина скрыта за многоточием, то она всегда будет приводиться к типу `double` для чисел с плавающей точкой. В данном же примере числа имели формат `float`, поэтому необходимо поменять спецификатор размера (`qword` вместо `dword`). Итоговый фрагмент кода, который выводит число на печать:

```
fst      dword [z]
fstp     qword [esp + 4]
mov      dword [esp], 1c
call     printf
```

Рассмотрим пример вычисления среднего арифметического четырёх чисел. Иногда бывают ситуации, когда 8 регистров не хватает. Для улучшения ситуации сначала приведём запись к польско-инверсной. То есть, было выражение:

$$(w + x + y + z) / 4 \quad (16.1)$$

Стало:

$$wx + y + z + 4 / \quad (16.2)$$

Теперь выражение легко записать в коде:

```
fld      qword [w]
fld      qword [x]
faddp
fld      qword [y]
faddp
fld      qword [z]
faddp
fild     dword [d]
fdivp ; st1 / st0
;...
```

В конце необходимо превратить число в число с плавающей точкой (из 4 сделать 4.0). Можно сохранить отдельно число 4 в памяти, а при загрузке использовать команду **fild**, а не **fld**. Эта команда трактует свой операнд, как 16, 32 или 64 разряда, которые кодируют целое число. В этот момент происходит конвертация и правильное перестроение битового представления числа.

Команда **fdivp** не имеет операндов, записывает в верхушку стека результат деления регистров ST1 на ST0.

Пример 2

Пусть вычисления более сложные. На самом деле есть дерево, которое кодирует вычисление выражения. Пусть корень дерева - это t , имеет двух потомков t_1 и t_2 . Пусть для каждого потомка потребуется n_1 и n_2 регистров на стеке сопроцессора. Будем считать, не ограничивая общности, что $n_1 < n_2$. Это означает, что сначала можно выполнить вычисления, связанные с n_2 , результат сохраним в один регистр, а оставшихся регистров точно хватит для вычисления n_1 . В случае, если $n_1 = n_2$, то потребуется не n_2 регистров, а $n_2 + 1$. Если регистров не будет хватать совсем, то придётся результаты вычислений выгружать периодически в память.

Рассмотренный ранее пример можно представить в виде дерева (рисунок 16.4). Тогда видно, что достаточно будет двух регистров

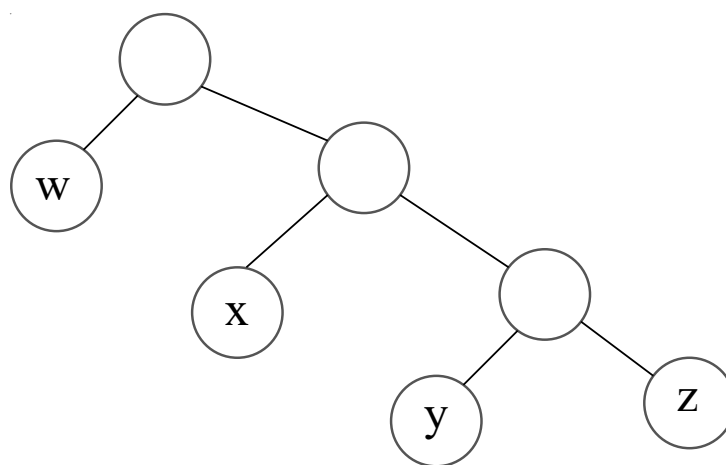


Рис. 16.4: Дерево вычисления среднего арифметического четырёх чисел.

На самом деле, число необходимых регистров - это число Ершова. Рассмотренный порядок вычислений - это упрощённый алгоритм распределения регистров Сети-Ульмана.

Порядок действий

Посмотрим на то, что происходит с самим результатом, когда меняется порядок действий. Можно рассмотреть пример, в котором одно число очень большое, а второе очень маленькое. При одном порядке сложений большее число “проглотит” меньшее. Однако, можно поменять порядок действий, тогда результат получится другой. Пусть есть три числа: x , y , z . Причём:

$$x = 3.14, \quad y = 1e50, \quad z = -1e50 \quad (16.3)$$

В одном случае сначала будем класть на стек y , потом z , а только потом x . Во втором случае сначала положим на стек x , а затем y и z . В первом случае порядок сложения будет

таков:

$$(y + z) + x \quad (16.4)$$

Первые два числа разные по знаку и очень большие. На выходе получим некое приближение числа π .

Во втором случае порядок вычисления будет таков:

$$(x + z) + y \quad (16.5)$$

В данном случае результатом вычислений будет нуль.

Аналогичная ситуация будет происходить и при распределении слагаемых. Рассмотрим пример, когда имеется три очень больших одинаковых числа x , y , z . В первом случае, когда вычисления идут в порядке:

$$(x - y) \cdot z \quad (16.6)$$

Результат будет нулевым.

Во втором случае, когда скобки раскрыты, и вычисления идут в таком порядке:

$$x \cdot z - y \cdot z \quad (16.7)$$

Результатом будет переполнение в двух произведениях. Вычитая одну бесконечность из другой, получается NaN.

Для того, чтобы избежать такого поведения, уберём маскирование обработки таких вычислений. Это (изменить управляющий регистр) можно сделать с помощью двух команд. Первая:

```
fstcw    m16
fldcw    m16
```

Две команды с операндами, которые расположены в памяти 16 разрядов. Таким образом, можно выгрузить текущее значение управляющего регистра, что-то с ним сделать и загрузить обратно. Таким образом, второй способ, в котором происходит переполнение, выведет теперь нас на аварийное завершение программы.

Дополнительные команды

Для удобства заведено несколько команд, которые кладут на верхушку стека регистров некоторые константы. Список этих команд (все без операндов):

- FLD1 +1.0
- FLDL2T $\log_2 10$

- L2E $\log_2 e$
- FLDPI π
- FLDLG2 $\log_{10} 2$
- FLDLN2 $\log_e 2$
- FLDZ +0.0

Сравнение чисел

Особенности сравнения также связаны с историческим аспектом отдельного существования сопроцессора. Результаты сравнения будут запомнены в регистре состояния. Вопрос в том, как результаты сравнения перенести на основной процессор x86. Придётся копировать регистр состояния.

isLe

```
push    ebp
mov     ebp, esp
fld     dword [ebp+16]
fld     qword [ebp+8]
fucompp ; St0 vs. St1
fnstsw  ax
sahf
setbe   al
pop     ebp
ret
```

Команда для сравнения двух чисел - это **fucompp**. Две буквы p означают, что со стека снимаются два элемента. Команда **fnstsw** производит сохранение в регистр, который указан операндом. Далее, уже в процессоре x86 есть специальная команда **sahf**, которая сохраняет старшие 8 разрядов ax внутрь EFLAGS.

Такое поэтапное копирование и сравнение не очень удобно и позже была введена специализированная команда **fucomip**, которая отправляет в EFLAGS напрямую. Однако, она сбрасывает лишь один регистр, поэтому второй надо сбрасывать руками. Сложность в том, что корректное состояние стека зависит от двух признаков, которые находятся в разных местах. Один перезаписывается командой **ffree** регистра ST0, после чего необходимо увеличить STP на единицу (**fincstp**).

Дополнительные возможности работы сопроцессора

Условная пересылка

В поколении Pentium PRO, в котором появилась функция `fcomip`, появились и команды `cmovb`. Это условная пересылка, которую возможно проверить прямо на регистрах процессора, не вовлекая напрямую регистр EFLAGS.

Возвращаемое значение - число с плавающей точкой

В случае с целыми числами возврат идёт через регистр `eax`. В случае с плавающей точкой на входе в функцию все регистры должны быть обязательно пустыми. Когда выходим из функции, все регистры, кроме ST0 должны быть пустые, а в последнем находиться возвращаемое значение.

Лекция 17. Элементы системы программирования

Система программирования

Система программирования - это совокупность каких-то технических средств, программ, документации, которые позволяют программисту написать код, превратить его в работающие программы, а затем дают возможность этим программам работать в среде функционирования.

Среда функционирования - это аппаратура и операционная система, которая позволяет упростить взаимодействие с аппаратурой. Кроме компилируемых бывают интерпретируемые программы. Такие языки не превращаются в команды для процессора, а остаются в некотором высокоуровневом виде, которые потом исполняются другой программой.

Для написания программ требуется комплекс средств, в том числе информационный ресурс. Если брать то, что предлагают крупные производители, то этот инструмент называется SDK (Software Development Kit). Такой комплекс средств включает в себя документацию, библиотеки, программные инструменты.

Помимо инструментальной поддержки важен процесс создания программ - организация и жизненный цикл. Существуют несколько методологий разработки, которые в разном порядке включают в себя проектирование, реализацию, распространение и сопровождение программных продуктов. Возможен такой вот линейный процесс разработки, хотя сейчас более популярна гибкая модель, которая позволяет подстраиваться под изменения. Одна из методологий упрощённо представлена на схеме 17.1.

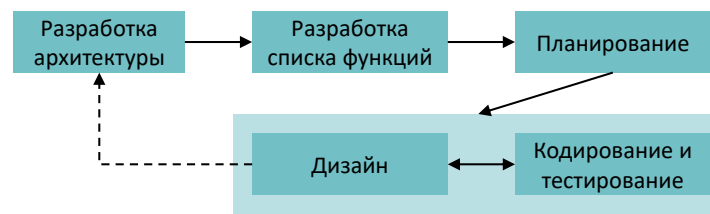


Рис. 17.1: Упрощённая блок-схема одной методологии разработки программ.

Система программирования языка Си

Основное инструментальное средство, организующее автоматизацию превращений программ, - это драйвер компилятора. В данном случае компилятор gcc. Рассмотрим основной “тракт”, по которому проходит Си программа (схематичное изображение всех этапов на рисунке 17.2). На входе имеется Си файл. Язык Си использует независимую компиляцию, то есть каждый отдельный файл проходит по всему

тракту, а в определённом месте происходит слияние **единиц трансляции**. Сначала Си файл раскрывается, производятся все макроподстановки (константы, мнемонические имена, инклюдь). Далее обрабатывает компилятор, который транслирует и компилирует текст в ассемблерный листинг. Далее ассемблер переводит листинг в двоичный код. Компоновщик позволяет собрать несколько файлов с перемещаемым объектным кодом и получить уже то, что почти может выполнять машина. Собирать вместе можно даже написанные на разных языках файлы, главное соблюдать выравнивание и порядок объявления функций.

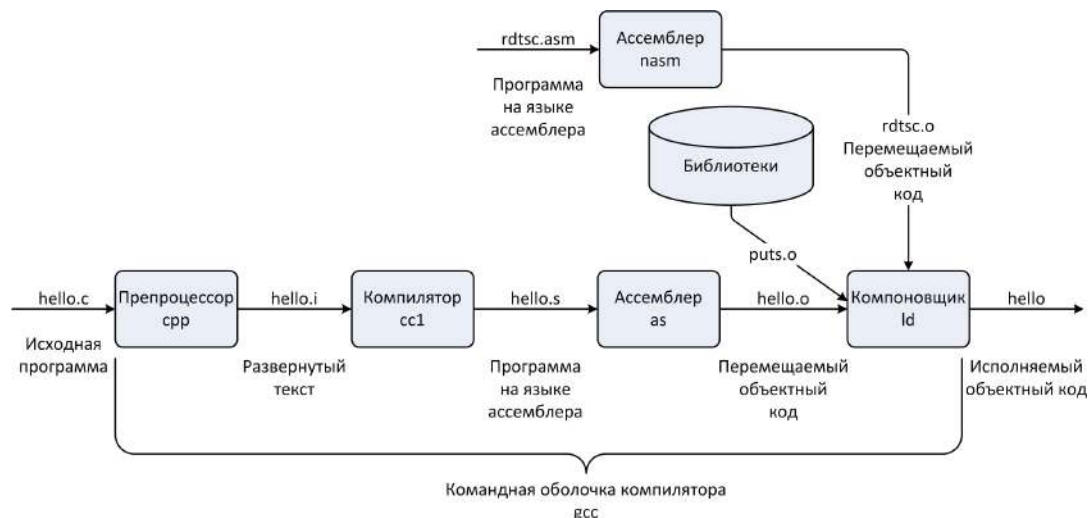


Рис. 17.2: Путь, по которому проходит программа, написанная на языке Си.

После того, как был получен исполняемый файл, его нужно превратить в то, что будет исполнять процессор. За это отвечает загрузчик - специальная программа, которая превращает исполняемый файл, который лежит в файловой системе в то, что находится в оперативной памяти. В этот момент программа уже окончательно докомпоновывается (до этого происходила статическая компоновка кода, а тут происходит динамическая компоновка). Схематично процесс изображён на рисунке 17.3.

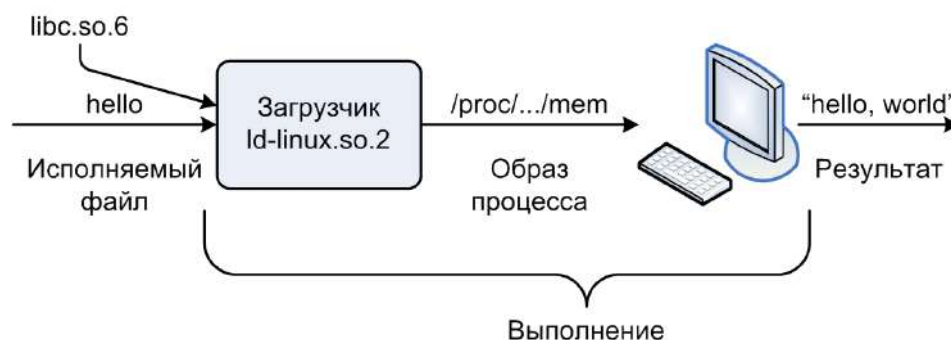


Рис. 17.3: Схема процесса выполнения программы с помощью загрузчика.

Компиляция

Различные компоненты программы компилируются следующим образом:

- Определения и объявления глобальных, статических и локальных статических переменных компилятор на уровне листинга (при построении ассемблерного листинга) превращает всё это в директивы, которые размещены в той или иной секции статических данных. Имена переменных превращаются в метки в той или иной секции.
- Если определяется функция, то на уровне ассемблера по заданным правилам строится код. Начало функции соотносится с некоторой меткой, тело функции можно превратить в ассемблерный код.
- Области видимости переменных в Си и в ассемблере отличаются. В Си по умолчанию всё оказывается в глобальной области видимости для всех единиц трансляции. В ассемблере в силу устоявшихся правил всё наоборот. В ассемблерном файле необходимо дописывать спецификаторы, чтобы на эту метку из другой единицы трансляции возможен был например прыжок.
- При компиляции на Linux имена из Си кода остаются такими же. Однако, в других платформах возможно декорирование имён, например в Windows. Отметим, что в C++ возможна перегрузка функций, то есть они отличаются только параметрами. Таким образом, такие функции будут иметь имя в ассемблерном коде с суффиксом из параметров.

Схема работы ассемблера

После того, как был получен ассемблерный листинг, необходимо перевести текстовую запись в машинную. Казалось бы задача довольно простая, единственная сложность в том, что в ассемблере присутствуют опережающие ссылки (например наверху происходит `jump` на какую-то метку, которая появится сильно внизу). Решение - двойной проход. За первый проход выстраивается частичная трансляция команд, оставив места для заполнения там, где есть метки.

Ассемблер формирует **символ**. Отличие от метки состоит в том, что помимо имени, в символе хранится дополнительная информация (размер, видимость из других модулей данной метки). Символы собираются в служебную структуру данных - **таблицу символов**. Таким образом, на первом проходе строится таблица символов. В других случаях, при обращении к меткам в других единицах трансляции например подготовленную информацию о символах придётся сохранить в о-файле (расширение .o) и дожидаться работы компоновщика, который соберёт это всё вместе.

Таким образом, объектный код включает в себя и содержимое секции и служебную информацию.

Пример Си-программы

Рассмотрим как все механизмы будут работать в Си-программе, состоящей из двух единиц трансляции. Одна называется main.c, код:

```
int buf [2] = {1, 2};
char str [] = "Hello";
char *p = "world";

int swap(char*, char*);

int main( int argc , char* argv [])
    if (argc > 1) {
        swap(str , p);
    }
    return 0;
}
```

Другая называется swap.c, код:

```
extern int buf [];

int *bufp0 = &buf [0];
static int *bufp1;
void swap() {
    int temp;

    bufp1 = &buf [1];
    temp = *bufp0;
    *bufp0 = *bufp1;
    *bufp1 = temp;
}
```

Как видно, в обеих единицах трансляции размещены как функции, так и глобальные данные, которые используются во всех единицах трансляции. Рассмотрим подробно, как будет выглядеть объявление статических данных для файла main.c. Сначала нужно объявить секцию (section data), куда будут помещены инициализированные переменные. Переменная buf превратится в метку, Си строка запишется как есть, только в конце необходимо добавить явным образом ноль. Необходимо добавить директиву global,

так как это всё глобальные переменные. Третья глобальная переменная запишется аналогично. Отметим, что последняя переменная (p) - это указатель, который ссылается на некоторую метку, которая размещена в другой секции. На уровне кода вышеописанное выглядит следующим образом:

```
section .data
    buf      dd 1, 2
    str      db `Hello'. 0
    p        dd .L0
global buf
section .R0data
.L0         db `world', 0
```

На уровне транслированного ассемблером содержимое секции будет представлять собой массив байт. В начале будет находиться buf, потом будет str, затем два неиспользуемых байта для выполнения требований по выравниванию. На правильно выровненной границе появится указатель p, который будет ссылаться на точку L0. То, что находится на этой точке - пока неизвестно, это можно будет доопределить только потом.

При обращении к метке будет зарезервирован один байт, так как это прыжок, и необходимо закодировать его расстояние (считаем, что прыжок небольшой). В местах, где происходит обращение к переменным, вызов функции, резервируется по 4 байта. Эти байты связаны с именами меток. Для того, чтобы построить перемещаемый объектный код, необходимо завести таблицу ссылок и таблицу символов. Эти две служебные структуры данных хранят в себе информацию для вызова функции (откуда) и их имена. Этот перемещаемый объектный код далее переходит на этап статической компоновки.

Статическая компоновка

Компоновщик (в linux он называется ld) получает на вход откомпилированные независимо друг от друга файлы с перемещаемым объектным кодом. В рассмотренном примере компоновщик принимает два файла: main.o, swar.o. На выходе компоновщик выдаёт исполняемый объектный файл.

Зачем нужен компоновщик? Существует несколько причин, и все связаны с эффективностью.

- Разбивая программу на части, сокращается объём кода, который в рамках одного файла (единицы трансляции) находится у человека перед глазами. Есть правило, что увеличение объёма кода в два раза увеличивает сложность работы с ним в 4 раза. Поэтому удобно, когда программа разбивается на файлы относительно небольшого размера.

- Появляется возможность реализовывать библиотеки (повторное использование одного кода). Кроме того, таким образом экономится место на диске.
- Раздельная компиляция уменьшает затрачиваемое время. Например, если изменилось что-то в одном файле, то нет необходимости перекомпилировать всё снова.

Что делает компоновщик

Компоновка состоит из двух больших этапов.

1. Первый шаг - это разрешение символов. На этапе трансляции были подготовлены служебные описания для символов. На устройствах, работающих на linux, каждый символ будет описываться структурой.

```
typedef struct {  
    int name;  
    int value;  
    int size;  
    char type:4,  
    binding:4;  
    char reserved;  
    char section;  
} Elf_Symbol
```

Такого набора полей должно хватать для того, чтобы однозначно соотнести каждое обращение (ссылку) с тем или иным определением, которые должны присутствовать в том наборе единиц трансляции, который подаётся на вход компоновщику.

2. Второй шаг - это перемещение. В каждом модуле есть свой комплект секций, причём секции по именам будут зачастую повторяться. Почти в каждом модуле например будет секция текст. Секции будут собраны, объединены и размещены на некоторых адресах памяти. То есть, каждой собранной секции будет назначен абсолютный адрес, из которого она в памяти работающей программы будет видна. После объединения необходимо поправить обращения ко всем ссылкам, так как абсолютные адреса изменились. Такое переписывание ссылок называется **перебазированием**.

Каждая ссылка уже описывается меньшим количеством полей, сохраняется только тип (type), номер символа в таблице (symbol) и смещение ссылки от начала той или иной секции, где эти ссылки были обнаружены (offset). Вся структура выглядит так:

```
typedef struct {  
    int offset;  
    in symbol:24,  
    type:8;  
} Elf32_Rel;
```

Типы объектных файлов

Существует три типа объектных файлов:

1. Перемещаемые объектные файлы (.o - файлы). Код и данные содержатся в такой форме, что можно проводить компоновку с другими такими файлами.
2. Исполняемые объектные файлы (код и данные в такой форме, что можно запускать выполнение программы)
3. Разделяемые объектные файлы (.so - файлы). Это файлы, которые всплывают на этапе динамической компоновки (в windows - это .dll файлы). Эти объектные файлы вступают в работу уже в тот момент, когда программа запускается. В момент сборки на эти файлы смотрят только для того, чтобы узнать, какие в них содержатся данные и функции.

Executable and Linkable Format (ELF)

Все описанные три типа файлов будут кодироваться и размещаться согласно единому формату Executable and Linkable Format (ELF). Для linux (unix) систем это стандартный бинарный формат объектных файлов. Это довольно старый формат. Файл ELF состоит из “слоёв”. В начале и в конце имеет служебные данные, там содержится информация о заголовках. Между ними находится закодированное двоичное содержимое тех секций, которые были подготовлены во время сборки программы. Каждая секция - это не что иное, как массив. Полная структура представлена на картинке 17.4.

Каждая секция представляет собой различную информацию, опишем некоторые из них:

- Заголовок содержит основную служебную информацию (размер машинного слова, тип файла, порядок байт) для спецификации архитектуры процессора например.
- Нижняя таблица заголовков сегментов необходима при загрузке файла в память. Она не особо нужна для компоновщика.
- Секция .text содержит код.

Заголовок ELF
Таблица заголовков сегментов (необходима в исп. файлах)
секция .text
секция .rodata
секция .data
секция .bss
секция .symtab
секция .rel.txt
секция .rel.data
секция .debug
Таблица заголовков секций

Рис. 17.4: Структура ELF файла.

- Секция `.rodata` содержит данные, доступные только для чтения. Традиционно помещается недалеко от секции `.text`, потому что и одно и другое не должны быть доступны на запись при работе программы.
- Секция `.symtab` - это таблица символов.
- Несколько секций с префиксом `.rel`. Такой префикс означает relocation (то есть ссылки). С помощью суффикса указывается к чему эти ссылки относятся. В секции `.bss` очевидно relocation быть не может.
- Кроме того, может быть ещё что угодно. Например отладочная информация (`.debug`).

Символы в процессе компоновки

В процессе компоновки ведётся работа с символами трёх типов.

1. Глобальные символы. Это символы, соотносящиеся с глобальными функциями или переменными. Они должны быть видны из других модулей. Функция `main` в рассмотренном ранее примере является глобальным символом.
2. Внешние (неопределённые) символы. Это то, к чему происходит обращение в рамках единицы трансляции, но не определено. То есть, символы используются, но

определены где-то в другом месте. Функция `swar` в рассмотренном ранее примере является внешним символом.

3. Локальные символы. Аналогично глобальным символам, только в данном случае они не должны быть видны из других модулей. Важный момент состоит в том, что локальные символы не являются локальными переменными Си-программ.

Лекция 18. Сборка Си-программы

Предварительное определение переменных

Когда что-то определяется, это требует выделения в памяти (в статической памяти), то есть заводится некоторое пространство внутри секции со статическими данными и на это пространство можно потом сослаться через то символьное имя, через ту метку, которое было назначено. Но в языке C есть одна особенность, связанная с тем, как этот язык позволяет определять, объявлять переменные. Выглядит это следующим образом: в отдельной единице трансляции, которую вы разрабатываете, пишете, может быть, скажем так, предварительное определение, которое будет оставаться таким до тех пор, пока компилятор не дойдёт в процессе своей работы до конца файла с исходным кодом. Возможно, это дополнительное определение будет потом дополнено другими объявлениями (определениями), главное, чтобы ему не противоречили. Вы столкнётесь, уже выполняя функции компилятора, с тем, что у Вас будут инициализированные данные. Но, возможно, все как раз и ограничится предварительным определением, которое в результате потребует, чтобы в единице трансляции как бы присутствовала переменная с нулевым значением. Здесь мы рассматриваем несколько таких переменных, для которых введены предварительные определения. Когда мы говорили о том, что помимо предварительного может быть несколько других определений, главное, чтобы при этом не возникало конфликтов по тому, как переменная видна снаружи, а именно по **связыванию имён**.

- Внутренним, оно задаётся в языке C с помощью спецификатора `static`. Таким образом, если всплывает дополнительное определение, то главное потом не противоречить тому, какое связывание Вы изначально ему назначили.
- Если это локальная для модуля переменная, то ничего страшного, но проблемы могут возникать в том случае, если у Вас в разных модулях встречаются предварительные определения одной и той же переменной с внешним связыванием, то есть глобальных переменных и у которых одинаковые имена. Это будет приводить к тому, что Вы столкнётесь с потенциальной проблемой в каждом модуле, в каждой единице трансляции может быть выделена память под эту переменную, и тогда возникает вопрос, а на кого правильно ссылаться.

Сильные и слабые символы. Пример.

Для того, чтобы разобраться с неопределённостью “а на кого правильно ссылаться в данной ситуации?”, на уровне компоновщика есть механизм сильных и слабых символов. Если упрощённо на него посмотреть, то каждый символ в программе может быть либо сильным, либо слабым.

- К **сильным символам** относятся все функции и инициализированные глобальные переменные, если только с помощью специальных возможностей не было специально назначено, что функция или переменная потом должна превратиться в слабый символ.
- **Слабыми символами** будут оказываться все неинициализированные глобальные переменные. Таким образом, в модуле p1.c (рисунок 18.1) переменная foo будет порождать сильный символ, потому что она инициализирована, равно как и сама функция p1. В модуле p2.c это же имя переменной будет порождать слабый символ, потому что переменная не инициализирована (функция по-прежнему сильная).

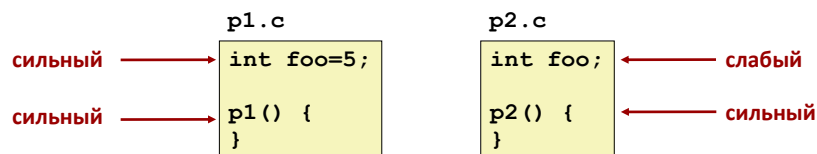


Рис. 18.1: Пример сильных и слабых символов.

Правила работы с символами

1. Одинаковые сильные символы запрещены. Одинаковые символьные символы запрещены, потому что в этом случае мы будем сразу приходить к некоторому конфликту. Например, в двух разных единицах трансляции объявлены функции с одинаковыми именами, возникает конфликт. И если встретятся сильные символы с одинаковыми именами, будет ошибка компоновки.
2. Один сильный символ и несколько слабых – выбираем сильный символ. Если в процессе компоновки есть один сильный символ и несколько слабых, то слабые символы не используются (про них фактически забывают), и все ссылки будут соотноситься с этим единственным сильным символом, и пересчёты ссылок как раз должны будут приводить на сильный символ.
3. Если несколько слабых символов, выбираем произвольный. Если встретилось несколько слабых символов и ни одного сильного, тогда будет выбран некоторых произвольный символ (по каким правилам – неизвестно, особенности реализации). Компоновщик сам выбирает этот символ и все ссылки будут заведены именно на него. В тех опциях, которые использует компилятор по умолчанию, такое поведение может быть изменено дополнительным ключом, который позволяет завести т.н. общие символы. Эту опцию можно включать, либо выключать (`gcc -fno-common`).

Задача. Ошибка компоновки

Возможна ситуация, в которой произойдёт коллизия между слабыми символами или даже сильными. Рассмотрим случаи:

- Ошибка компоновки: два сильных символа. Возникает, если есть две одинаковых функции с одинаковыми именами.
- Различные функции с разными именами, но два слабых одинаковых символа. Например `int x`; в двух файлах, ссылки будут ссылаться на один и тот же неинициализированный `int`, но непонятно на какой. Программа останется работоспособной, но произойдёт перерасход памяти на 4 байта.
- Типы переменных с одинаковыми именами различаются. То есть в одном файле будет `int x`; и `int y`; а в другом `double x`. Могут возникать неприятные ошибки. Если будет производиться запись в переменную `double`, то возможно изменение памяти (переменной `y`), которая находится далее за переменной `int x` из-за того, что размер типов разный.
- Если в предыдущем случае `int x=7`; например, то есть `x` - сильный символ, то описанная перезапись произойдёт гарантированно, так как слабый символ (`double x`) будет гарантированно отброшен. В этом случае ошибка будет стабильно проявляться.
- Все ссылки на `x` будут ссылаться на один инициализированный `int x`, если в одном файле он инициализирован, а в другом - нет.

Наихудшим сценарием может быть такая ситуация, когда компилируются две одинаково слабые структуры, но с разными правилами выравнивания (разными компиляторами откомпилированные). В результате получится ситуация, в которой обращения к одним полям будут портить значения других и так далее.

Чтобы избежать таких ошибок необходимо следовать некоторым правилам.

- Избегать необоснованного использования глобальных переменных.
- В противном случае рекомендуется использовать ключевое слово `static`, чтобы ограничивать область видимости, и не позволять из разных модулей (файлов) обращаться к одному и тому же месту в памяти разными способами.
- Инициализировать глобальную переменную, если всё же приходится её объявлять.

Пример (Правила перебазирования)

Пример состоит из двух модулей: `hello1.c` и `hello2.c`. Код `hello1.c`:

```
extern void func ();
char *buf = "Hello, world! \n";
int main() {
    int ret_code = 0;
    func();
    return ret_code;
}
```

Код hello2.c:

```
#include <stdio.h>

extern char* buf;

void func () {
    printf ("%s", buf);
}
```

Сама программа собирается из двух отдельно скомпилированных .o - файлов (схема на рисунке 18.2).

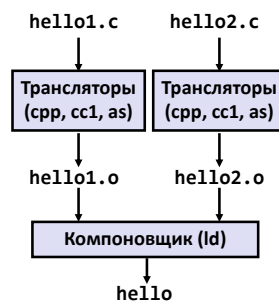


Рис. 18.2: Схема компиляции и компоновки программы из двух .c - файлов.

После того, как получили перемещаемый код, можно убедиться, что в нём присутствуют следующие символы:

- Будет объявлена переменная - указатель с именем buf на строчку Hello world!, которая размещена в секции `rodata`.
- Функция `func`, которая является внешней. Это означает, что, когда мы столкнёмся в ассемблерном коде с командой `call func`, то это приведёт к построению описания неопределённого символа.
- Символ, связанный с кодом (с секцией текста) - это `main`.

На самом деле в .o - файле присутствует больше символом, чем показывает утилита `nm`. Чтобы посмотреть, что там размещено в полном объёме, следует использовать утилиту

readelf. Помимо перечисленных символов, присутствует набор служебных символов. Их тип не связан с кодом или данным, он связан с началом секции или с именем файла.

Кроме того, можно посмотреть какие ссылки внутри этого перемещаемого кода присутствуют.

Посмотрим, что происходит внутри функции main в этом простом примере. Для этого используем утилиту objdump. Будет видно, что ссылки, которые подвергаются перебазируванию полностью соответствуют информации, полученной через утилиту readelf. По смещению 12 обнаруживается ссылка. В секции .data можно обнаружить место, которое зарезервировано под переменную buf.

Для второго модуля аналогично имеется три символа, которые видны с помощью утилиты nm.

- Символ buf в данном случае будет иметь тип undefined.
- Будет определена функция func (секция текст).
- Функция printf библиотечная и тоже окажется undefined.

С ссылками в этом модуле будет немного другая картина. Первая ссылка по смещению +8 будет обращаться к внешней переменной buf (потому что buf здесь выступает в качестве одного из аргументов). Далее происходит обращение к строке, которая находится в .rodata. Третья ссылка - это обращение к библиотечной функции printf. Итого три ссылки, которые подлежат перебазируванию.

Теперь можно произвести связывание символов. Символы buf и func оказались связанными. В первом модуле buf определён, а во втором используется. Символ func, наоборот, определён во втором модуле, а используется в первом. Символ printf из стандартной библиотеки, он находится в отдельном .o - файле, который будет участвовать в компоновке.

Кроме того, необходимо знать, по каким адресам размещены все эти данные и функции. Узнать это можно передав специальный ключ компоновщику (-XLinker).

Вся информация о базовых адресах и ссылках позволяет сразу произвести перерасчёт значений ссылок. Существует два типа преобразования:

- Тип перебазирувания R_{386_32} . При обращении к статическим данным в секции .rodata новое значение ссылки - это сумма абсолютного адреса (S) и добавки (A - addend), которая размещена непосредственно в байтах ссылки.
- Тип перебазирувания R_{386_PC32} . Здесь считается расстояние между местом, куда ссылается от текущего места размещения ссылки. Тогда новое значение ссылки - то $S - P + A$, где P - абсолютный адрес ссылки.

Работа с общими функциями

Существует вопрос о размещении общих функций. Как следует располагать те, что часто используются разными программами? Примеры таких функций - это математика, I/O, управление памятью, работа со строками и так далее. Имея рассмотренный порядок компоновки, можно поступить разными способами:

1. Поместить все функции в один файл (например все функции, связанные с математикой). С одной стороны, это просто, с другой стороны, неэффективно, потому что это означает затягивание всех функций, а не только тех, которые нужны.
2. Поместить каждую функцию в отдельный файл. Этот вариант более эффективен, но это неудобно, потому что придётся отслеживать, что используется, какие функции нужны. Для того, чтобы с этим бороться, придумали статические библиотеки.

Статические библиотеки

Статические библиотеки - это фактически архив, куда помещены близкие по смыслу и функциональности файлы, реализующие различные функции. В момент сборки программы указывается набор архивов, чтобы компоновщик из них вытащил нехватящие объектные файлы и полностью дополнил код, участвующий в статической компоновке. Если в архиве найдётся файл, позволяющий определить символ, то его автоматически включают в компоновку.

Библиотека создаётся отдельной программой (библиотекарем), которая собирает транслированные в расширение .o - файлы в один большой .a - файл. Примером является стандартная библиотека Си `libc.a`, она содержит более 1000 объектов. Схематически процесс представлен на рисунке 18.3.

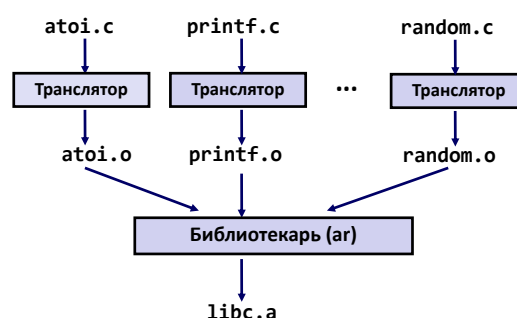


Рис. 18.3: Схема создания статической библиотеки.

Использование статических библиотек

Для разрешения внешних ссылок в итоге компоновщик имеет следующий алгоритм:

- Просматривает командную строку, с которой запущен, смотрит ключи порядке их следования.
- В порядке просмотра формирует список неразрешённых символов.
- Как только появляется новый объектный файл, компоновщик пытается сопоставить символы с тем, что находит в объектном файле. Если попадается ключ с именем библиотеки, то компоновщик пытается вытащить из неё все необходимые .o - файлы.
- Если остался хоть один неразрешённый символ по окончании просмотра, то появляется ошибка линковки.

Получается, что порядок следования опций в командной строке важен. Следует указывать библиотеки последними.

Загрузка исполняемого объектного файла

После того, как прошла статическая компоновка, необходимо выполнить загрузку. Загрузчик выполняет эту задачу.

- Сначала он подготавливает адресное пространство для запускаемых программ.
- Затем происходит копирование соответствующих секций в диапазоны адресов памяти.
- В конце передаётся управление на точку входа программы. Адрес, считающийся точкой входа записан в заголовке .l - файла. Традиционно - это символ `_start`.

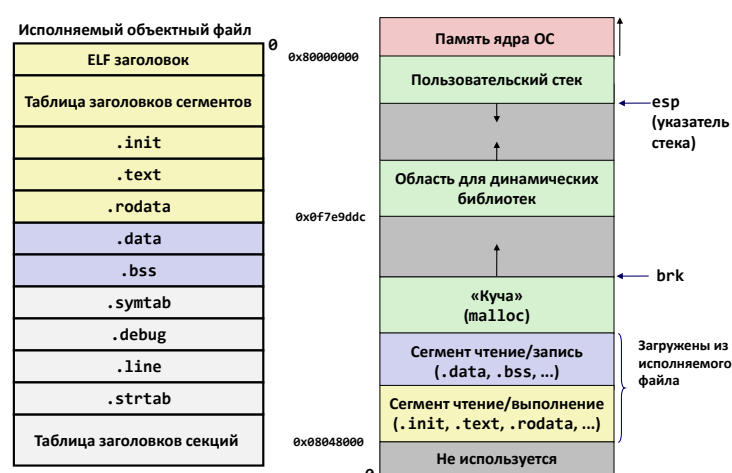


Рис. 18.4: Фрейм при загрузке исполняемого объектного файла.

Для каждого сегмента внутри диапазона адресов заводится описатель, который регламентирует, как пространство памяти должно подготавливаться. В составе описателя есть указания, начиная с какого смещения из файла нужно копировать в память, в каком базовом адресе этот диапазон начинается, количество байт, которые нужно в этот диапазон адресов отправить, величина выравнивания, атрибуты доступа сегмента адресов.

Схема того, как будет выглядеть память, представлена на рисунке 18.4.

Лекция 19. Динамические библиотеки. Динамическое связывание

Динамические библиотеки

Вопросы, которые решают статические библиотеки не решаются в полной мере. Остаются такие проблемы, как:

- Многократное копирование одного и того же кода при построении совсем разных исполняемых файлов. Например стандартная библиотека языка Си нужна всем программам (на Си).
- Копии кода в исполняемых файлах
- Самая острая проблема состоит в том, что в системных библиотеках существуют ошибки. Если таковая находится, то получается, что все разработчики должны не только перекомпилировать программы, но и обеспечить распространение своих обновлений после полной пересборки. Хотелось бы, чтобы исправления в библиотеках “подцеплялось” само в момент запуска программы.

Отметим, что размер кода вырастает на порядок при компиляции исключительно на статических библиотеках (не всегда этот эффект сильный, заметно на маленьких программах преимущественно).

Динамические библиотеки позволяют собирать окончательно программы во время запуска или уже во время работы. В linux динамические библиотеки имеют расширения .so (сокращение от shared object), в windows .dll (dynamic library).

Динамическая компоновка происходит с помощью ещё одной сущности - **динамического компоновщика**. В linux динамический компоновщик - это ld-linux.so. Это специальная программа, которая будет отрабатывать до того, как ваша программа начинает работать. То есть, в момент, когда управление ещё не передано на точку входа, но программа уже загружена в память. Кроме того, динамическая компоновка может происходить даже тогда, когда программа уже работает. Так как динамический загрузчик остаётся на всё время в адресном пространстве работающего приложения, то ничего не мешает обращаться к нему во время работы программы через специальные библиотечные функции.

Таким образом, подключается заголовочный файл **dlfcn.h** для работы с динамическими библиотеками, и используется функция dlopen для открытия файла динамической библиотеки. Фрагмент кода:

```
#include <dlfcn.h>
void * dlopen (const char *filename, int flag);
```

Проблемы при использовании динамических библиотек:

- Динамическая компоновка не позволяет заранее распределить команды и данные динамических библиотек на какие-то заранее определённые адреса. Таким образом, придётся строить позиционно-независимый код.
- Появляются вопросы передачи управления, так как для этого необходимо знать расстояние до точки, куда необходимо передать управление. То есть, до момента компоновки неизвестно куда ведут ссылки.
- Аналогичная проблема может возникнуть с кодом самой динамической библиотеки. Она может работать не только с собственными функциями, из неё могут происходить какие-то переходы в код самой программы или в код других динамических библиотек.

Пример

В качестве иллюстрации рассмотрим пример Hello world. Состоит программа из двух модулей Первый модуль (единица трансляции) - это hello.c, второй - hello2.c.

hello.h:

```
extern void f();
```

```
extern char* buf;
```

hello.c:

```
#include "hello.h"
char * buf = "Hello, n";
int main() {
    int ret_code = 0;
    f();
    return ret_code;
}
```

hello2.c:

```
#include <stdio.h>
#include "hello.h"

void f() {
    printf("%s", buf);
}
```

Первый файл (hello.c) со временем превратиться в программу hello.dlib, а второй (hello2.c) превратится в динамическую библиотеку libhello.so. Видно, что у динамических библиотек появляется префикс и суффикс (расширение).

Сборка несколько усложняется. Строится 32-х разрядный код, для компактности кода можно отключить его защиту. Из файла hello2.c строится ассемблерный листинг, для того, чтобы получился код, не зависящий от позиции, применяется принудительно ключ **-fPIC**. Далее ассемблерный код превращается в перемещаемый объектный код. Затем происходит статическая компоновка с получением динамической библиотеки. Важный момент заключается в том, что, когда происходит статическая компоновка .o - файла первого модуля (hello.o), необходимо указать, помимо .o - файла ещё libhello.so - shared object. Это делается для того, чтобы в процессе статической компоновки можно было оставить ссылки на следующий этап, подсмотрев, что экспортирует динамическая библиотека.

Полный процесс динамической компоновки времени загрузки схематично изображён на рисунке 19.1.

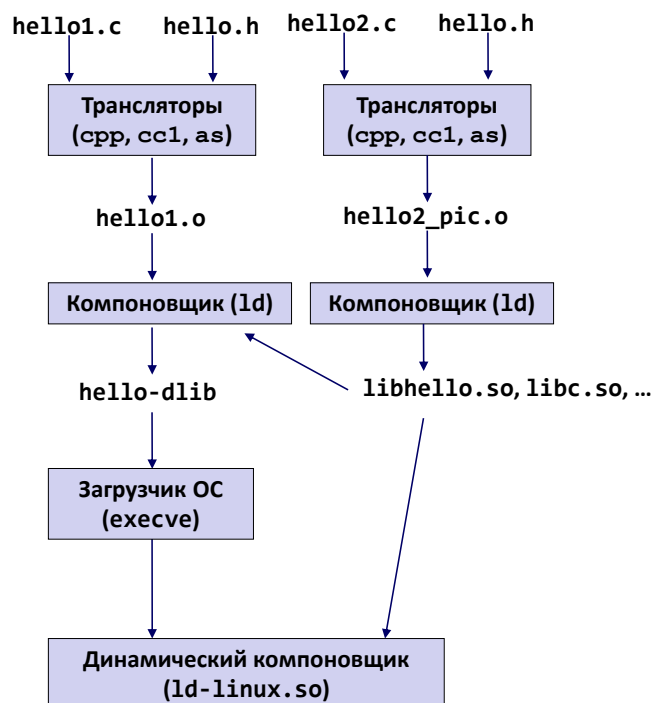


Рис. 19.1: Процесс динамической компоновки времени загрузки.

Позиционно независимый код в IA-32

При построении позиционно независимого кода возникает проблема. В этой архитектуре на уровне набора команд недоступно сразу адресовать относительно текущего положения счётчика команд. Приходится пользоваться тем, что при построении этого модуля (исполняемый файл или динамическая библиотека) вспомогательную таблицу с указателями поместят на известном смещении. Это смещение откроет доступ к

таблице указателей, а за правильное выставление значений в таблице GOT (Global Offset Table) отвечает уже динамический компоновщик.

С помощью GOT можно уже адресоваться на внешние функции и переменные.

Рассмотрим этапы построения GOT.

- На начальном этапе имеется код на Си, который компилятор превращает в ассемблерный код. В ассемблерном листинге появляются особые конструкции - ключевые слова (`@_GLOBAL_OFFSET_TABLE_`, `@GOT`, `@GOTOFF`, `@PLT`). Ассемблер будет строить ссылки определённых типов, отличных от ранее рассмотренных типов ссылок. Это необходимо для построения перемещаемого объектного кода.
- Рассматривается функция `f` из файла `hello2.c`. Из этой функции вызывалась функция `printf`, которая печатала строку, расположенную где-то за пределами модуля. В начале необходимо разобраться, на каких адресах располагается глобальная таблица смещений. После надо будет обратиться к переменной (в которой содержится строка), определив её адрес, который находится где-то в другом модуле. Затем необходимо будет обратиться к своей собственной строке, которая размещена в этом же модуле, где и выполняется текущий код. Необходимо заложить механизм пересчёта в абсолютные адреса для дальнейшего определения адреса строки, которая будет печататься. В конце необходимо будет обратиться к библиотечной функции `printf`, которая также находится в другом месте.
- Ассемблер сформировал ссылки. Необходимо теперь обратиться к данным, но текущий абсолютный адрес изначально неизвестен. Для того, чтобы решить данную проблему, необходимо вспомнить, что в архитектуре IA-32 при вызове какой-либо функции, на стек кладётся адрес следующей команды. Таким образом, можно вызвать функцию, которая снимет то, что лежит в верхушке стека (адрес возврата) и поместит его в регистр `ebx`:

```
_x86.get_pc_thunk.bx:  
    mov     ebx, dword [esp]  
    ret
```

Этот регистр в случае позиционно-независимого кода играет важную роль. В нём хранится якорный адрес, связанный с глобальной таблицей смещения.

Рассмотрим для примера кусок памяти. Где-то в памяти находится команда, с помощью которой будет определяться расстояние до глобальной таблицы смещений. Первый байт - это `81`, затем `C3`, затем идёт значение ссылки, которое необходимо будет пересчитать в процессе компоновки. Вспомогательная функция поместит в `ebx` адрес возврата, таким образом, `ebx` будет указывать на начало и содержать абсолютный адрес.

Далее, расстояние до GOT от `ebx` известно (пусть равно некоему Δ). Кроме того, известна позиция ссылки, которую необходимо пересчитать. Таким образом, если сместить текущее значение `ebx` на Δ , то получается абсолютный адрес глобальной таблицы смещений.

$$\Delta = 2 + (\chi_{GOT} - \chi_{link}) \quad (19.1)$$

Двойка появляется, так как в начале стоят два байта. Адреса таблицы GOT и ссылки - это соответственно χ_{GOT} и χ_{link} . Схематичное изображение фрагмента памяти представлено на рисунке 19.2.

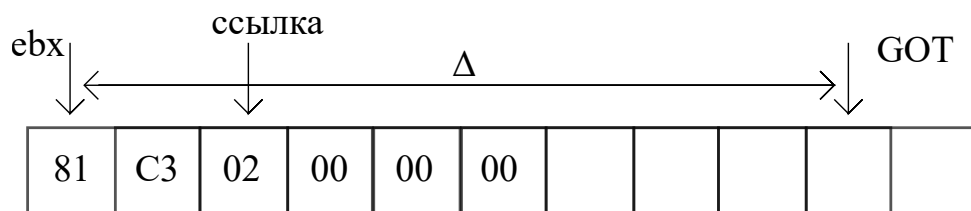


Рис. 19.2: Фрагмент памяти для расчёта расстояния до GOT.

Вообще адрес `GOT(ebx)` не обязательно указывает на конкретно начало таблицы GOT в памяти.

Таким образом, в итоге получается, что:

- Компоновщик, строя код динамической библиотеки, создаст таблицу смещений (GOT)
- Затем все ссылки в коде обновятся
- Создастся дополнительная служебная секция `.dynsym`, в которой компоновщик распишет все те символы, которые должны использоваться в динамической компоновке. То есть, `symtable` появляется такая таблица.
- Создаётся отдельная секция для перебазирования ссылок (`.rel.dyn`), чтобы динамический компоновщик потом знал, где и какие ссылки перебазировать (ссылки, размещённые в GOT).

В тот момент, когда динамическая библиотека будет загружена в память, её секция GOT динамическим компоновщиком будет определённым образом пересчитана.

Можно убедиться, что в динамической библиотеке есть два загружаемых сегмента. Первый идёт прямо от начала файла, он должен быть помещён на какой-то виртуальный адрес. набор секций, покрывающих первый сегмент, будет скопирован и помещён в диапазон адресов, который традиционно называется страницей. Эта страница содержит атрибуты, позволяющие её читать и выполнять, но не перезаписывать.

Второй загружаемый сегмент начинается немного левее позиции, на которую ссылается GOT (вся схема на рисунке 19.3). Таким образом будут захвачены дополнительные две страницы по 4 килобайта, обе страницы получают атрибуты, открывающие чтение, запись, но не доступны для выполнения. Это делается таким образом, так как содержимое GOT необходимо переписать.



Рис. 19.3: Схема двух загружаемых сегмента динамической библиотеки.

Код при вызове функций

В данном случае появляется возможность для оптимизации. При обращении к данным определить, что читается какая-то память или в неё что-то записывается конечно можно, но привязать к каждому такому действию какое-то пользовательское действие не представляется возможным.

При вызове же функций происходит передача управления. К передаче управления можно привязывать дополнительные действия. Это позволяет отложить пересчёт значений GOT до самого последнего момента, до того, как та или иная функция из других модулей будет вызываться. Это делается для того, чтобы загрузка динамических библиотек в память шла гораздо быстрее. Механизм, выполняющий всё это называется **ленивое связывание**.

Плата за такое ускорение состоит в том, что в код придётся добавить много вспомогательных функций-заглушек, которые будут обрабатывать все вышеописанные действия. Тогда ленивое связывание (lazy binding) вместо вызова реальной функции будет обращаться к этой заглушке. Функция заглушки будет знать адрес реальной функции. После определения адреса она сможет переписать ячейку GOT уже нужным значением. Далее все следующие вызовы будут уже перекидывать не на заглушку, а на реальный адрес функции, который появляется в GOT. В заглушке разместятся также дополнительные секции, которые называются .plt (procedure klinking table). Таким образом, появляется ещё одна служебная таблица связывания процедур.

Теперь можем рассмотреть вызов функции printf, которая относится к стандартной библиотеке. Пересчёт ссылки во время компоновки построения динамической библиотеки происходит похожим образом.

Для начала определим контекст, в котором происходит работа. Считается, что в адресном пространстве программы (уже пользовательской) размещён динамический компоновщик и ждёт нужного момента времени, чтобы связать с искомой функцией (`printf` в данном случае). Для того, чтобы динамический компоновщик отработал, ему требуется дополнительная информация о том, где что находится (в частности описания символов в секции `dynamic`). Кроме того, необходимо некоторое дополнительное описание (`libhello.so`), которое компоновщик подготавливает самостоятельно. Вся эта информация доступна по предопределённым элементам в глобальной таблице смещений (по нулевому смещению находится адрес секции `dynamic`, по `+4` находится описание модуля `libhello`, а по смещению `+8` располагается функция самого динамического компоновщика, реализующая линейное связывание). В момент, когда из кода исполнение “вытягивается” на заглушку, происходит `jmp` по абсолютному адресу, который размещён в памяти на месте `ebx+12`. Далее происходит процедура ленивого связывания. После того, как процедура отработала, она определит адрес `printf`, который будет обнаружен в `libc`.

В подготовленной программе, которая была подготовлена с использованием динамической компоновки, появляется дополнительный заголовок **INTERP**. То есть, появляется интерпретатор, который будет обрабатывать после момента начала загрузки до момента передачи управления на входную точку модуля. Программа интерпретатор в рассмотренном случае - это `ld-linux.so`. В начале динамический компоновщик обязательно должен быть загружен, так как он обрабатывает в начале.

Загрузка динамически скомпонованного исполняемого файла

Для реальных программ, которые выполняются в актуальных версиях любой `unix` системы, всё выглядит следующим образом.

- Для начала, чтобы собранная программа отработала, необходимо указать директории, где находятся используемые динамические библиотеки. Сделать это можно с помощью системной переменной `LD_LIBRARY_PATH`. Утилита `ldd` расписывает зависимости между динамическими библиотеками с файлами, которые будут использоваться при запуске. Пример использования утилиты `ldd`:

```
student@pc:~/asm/linking$ LD_LIBRARY=~pwd` ldd hello-dlib
linux-gate.so.1 (0xf7fd5000)
libhello.so => /home/student/asm/linking/libhello.so
(0xf7fca000)
libc.so.6 => /lib32/libc.so.6 (0xf7ddd000)
/lib/ld-linux.so.2 (0xf7fd6000)
```

Утилита показывает, что для работы файла `hello-dlib` необходимо загрузить четыре модуля в память.

- Бесконечный цикла поиска зависимости отсутствует, так как сам компоновщик компонуется полностью статически. Таким образом, после загрузки содержимого исполняемого файла, достаточно догрузить файл с загрузчиком `ld-linux.so`.
- Далее уже загрузчик способен отслеживать все необходимые зависимости и досылать содержимое динамических библиотек в память. Он это делает, просматривая содержимое секции `.dynamic`, где описаны зависимости.
- Далее содержимое GOT переписывается, опираясь на служебные таблицы (обновляются те указатели, с помощью которых будет происходить обращение к данным).
- После того, как динамический компоновщик отработывает, он передаёт управление на точку входа самой программы. Переменная `LD_LIBRARY_PATH` ещё раз доопределяется.

Лекция 20. Аппаратное обеспечение

Логические вентили

В какой-то момент появилась технологическая возможность воплотить элементы счётной машины в жизнь. Идея появилась раньше, чем техническая возможность. Первые механические вычислительные машины вызывали вопросы надёжности, масштабирования и скорости работы. Переломным моментом стало изобретение полупроводников в первой половине 20-го века. Появились различные транзисторы, являющиеся основой для вычислительной техники в наше время. Простое устройство, которое по сути размыкает цепь позволяет быстро производить инвертирование сигнала.

Рассмотрим зависимость напряжения некоторого сигнала, который идёт по линии, от времени. Есть некоторые пороговые значения напряжений, при переходе которых транзистор ведёт себя совершенно другим образом. Если подавать напряжение на затвор транзистора, то в какой-то момент он начинает себя вести как проводник. С одной стороны подаётся питающее напряжение, с другой стороны земля, тогда выходной сигнал V_{out} резко падает из-за эффекта проводимости (стекает на землю). Если в другую сторону снижается входное напряжение (V_{in}), то происходит размыкание проводника и выходное напряжение увеличивается. Зависимость представлена графически на рисунке 20.1.

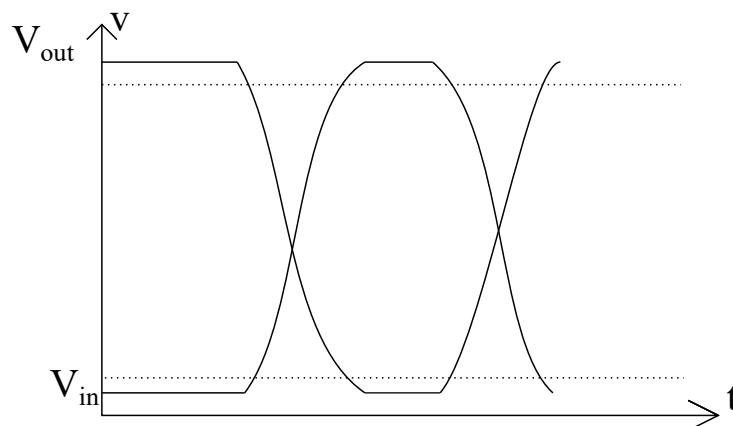


Рис. 20.1: Зависимость напряжения некоторого сигнала, который идёт по линии, от времени.

Если посмотреть на схему с биполярным транзистором, то подача напряжения на базу приведёт к тому, что коллектор и эмиттер будут замыкаться. Таким образом проведена инверсия сигнала. Такая конструкция работает очень быстро, переключение состояний происходит за наносекунды или даже меньше (зависит от вещества транзистора). Современные технологии позволяют размещать транзисторы очень компактно. В современной электронике используются полевые транзисторы, а не биполярные. Последовательные и параллельные соединения транзисторов позволяют строить функции булевой алгебры.

Рассмотрим подробно параллельно подключённые транзисторы. Питающее напряжение будет питать один или другой сток. Схема двух параллельных транзисторов на рисунке 20.2.

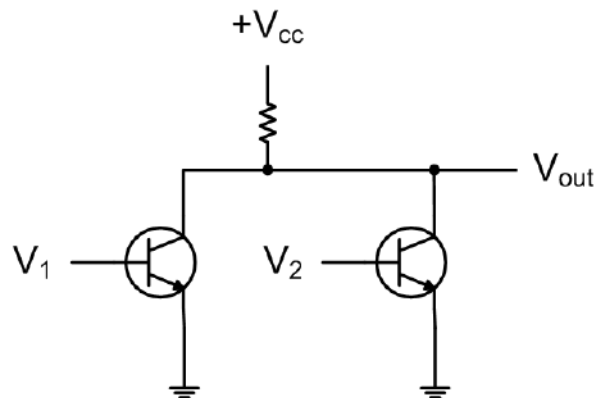


Рис. 20.2: Схема двух параллельных транзисторов.

Случаи, когда питающее напряжение открывает один или другой сток (1 означает, что ток подан, 0 - что не подан). Таким образом, получили логический элемент NOR (НЕ-ИЛИ).

V_{out}	V_1	V_2
1	0	0
0	0	1
0	1	0
0	1	1

Аналогичным образом можно рассмотреть последовательное соединение транзисторов. Это логический элемент NAND (НЕ-И). Схема подключения приведена на рисунке 20.3.

Можно составить аналогичную табличку.

V_{out}	V_1	V_2
1	0	0
1	0	1
1	1	0
0	1	1

Каждый из рассмотренных случаев - это схема с одним выходом и двумя входами. Можно совмещать модули и получать более сложные функции. Например, подключив выход NAND к инвертору получается обычный AND. Аналогично могут быть получены другие более сложные схемы. Например, можно реализовать сравнение одиночных битов. Можно также сравнивать целые слова с помощью каскадного AND.

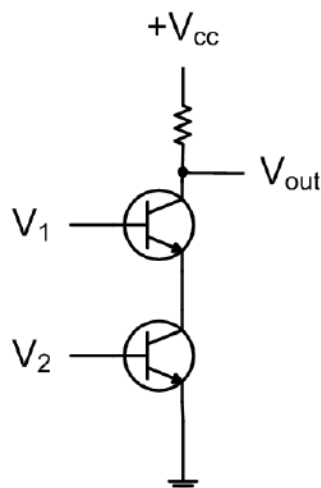


Рис. 20.3: Схема двух последовательных транзисторов.

Сумматор. Полусумматор

Чуть более сложная операция - это сложение. Эта операция имеет два выхода, так как необходимо учесть бит переноса. Схема представлена на рисунке 20.4. Бит переноса - это C, результат - бит S.

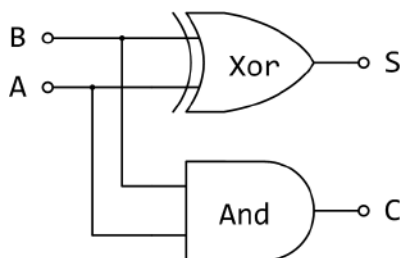


Рис. 20.4: Схема полусумматора.

Сумматор (полусумматор) можно усложнить, учитывая бит, который может заноситься из ранее суммировавшихся разрядов. Схема представлена на рисунке 20.5.

С помощью нескольких сумматоров можно суммировать слова.

Мультиплексор

Каждая рассмотренная цепь выполняет фиксированную операцию над входными сигналами. Можно построить мультиплексор, который будет выбирать в зависимости от селектора значение либо по линии A, либо по линии B. Простейшая схема мультиплексора с одним управляющим битом S представлена на рисунке 20.6.

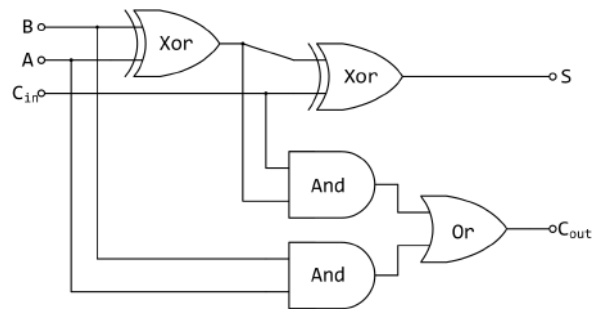


Рис. 20.5: Схема полного двоичного сумматора.

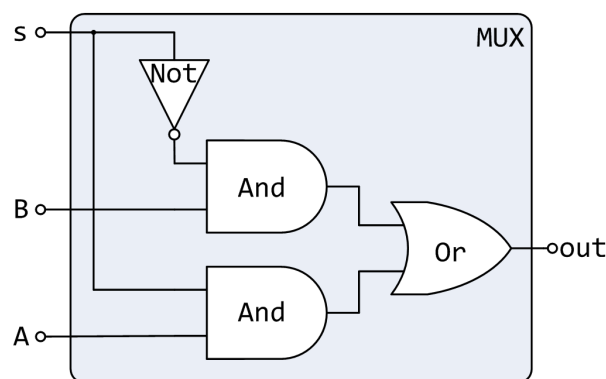


Рис. 20.6: Схема простейшего мультиплексора с одним управляющим битом S.

Арифметико-логическое устройство

Арифметико-логическое устройство может выглядеть с функциональной точки зрения как устройство с мультиплексором, которое выполняет то или иное конкретное действие. То есть, есть возможность, что за операция будет проводиться, а общий вид входных и выходных данных у них у всех одинаковый.

Регистр: сохранение 1 бита

Ранее рассмотренные схемы не имели возможность запомнить в себе то или иное состояние. Самое простое - сохранение одного бита.

В процессоре есть часы. Под часами подразумевается генератор сигналов определённого вида. Традиционно стробирующий сигнал (периодичные квадратные зубцы). Если на каком-то такте произошла запись значения, то на последующий тактах это значение должно выдаваться из ячейки памяти. Необходимо реализовать эту ячейку.

Статическая память

Отличительная черта цепей, реализующих статическую память состоит в том, что выходы схем соединены на входы. Это образует реактивный эффект, который будет проявляться в том, что, имея два разных входа (пусть S и R), которые имеют выходы, соединённые на входы друг друга (то есть, выход R соединён со входом S и наоборот), выходы будут противоположными (например Q и $\bar{Q}$).

За счёт чего это происходит? Каждый элемент (в каждой ветке от входа соответственно) реализует функцию отрицания NOR. Тогда можно составить таблицу значений. Получится, что, если оба входа - нули, то на двух выходах могут находиться только противоположные значения, причём возможны два случая (0 и 1 или 1 и 0). Если подать на какой-то из входов сигнал (только на один), то ситуация будет однозначна.

Q	S	R
?	0	0
1	1	0
0	0	1
?	1	1

Такая схема может запомнить, что было отправлено в неё на другом такте. Такая схема называется **защёлкой**. Если стоят обе единицы, то состояние неоднозначно тоже. Добавив перед каждым входом модуль AND, можно добиться дискретности возникновения событий. Кроме того, можно один сигнал запускать по двум линиям. Итого - необходимо 6 транзисторов для одной такой ячейки памяти.

Отличие статической памяти от динамической памяти

В динамической памяти принципиально другая элементная база. В ней один транзистор, чтобы создавать отсечки. Кроме того, в данной схеме присутствует конденсатор. Если в конденсаторе есть заряд, то можно считать, что в ячейке хранится 1, если нет, то 0. Типичный срок жизни заряда, который хранится в конденсаторе на процессоре составляет порядка 100 мс. Таким образом, время от времени требуются циклы регенерации (чтение ячейки в качестве регенерации).

Разработка интегральных схем

Комбинируя модули можно построить системы любого уровня сложности, но это может занимать очень много времени. Для ускорения разработки электрических цепей применяются языки описания аппаратуры (например VHDL - язык “на основе” ADA или Verilog - язык “на основе” Си). Языки позволяют описать те или иные действия с поступающими на вход модуля данными. Данные поступают таким же образом - это электрические сигналы.

Этапы разработки и изготовления компонент ЭВМ

Всё начинается во внутреннем центре дизайна. Туда “затаскиваются” некоторые сторонние компоненты (как библиотеки высокого уровня). Эти библиотеки компонуется с описанием, которое подготавливается собственными разработчиками. Командная интеграция сводит всё воедино и получают большую программу в текстовом виде например на Verilog. Эта программа по сути является описанием устройства аппаратуры. Далее эта программа передаётся специальному компилятору, который текстовую запись преобразует в схему уровня регистровых передач. Получается схема из элементарных компонент. Далее схема должна превратиться в картинку, которая является последовательностью слоёв, укладываемых на изолятор. Каждый слой будет комбинировать в себе материалы с разной проводимостью. Для создания такой интегральной схемы на подложке используются фотошаблоны. С помощью травления по шаблону получают готовые интегральные схемы, которые собираются в большие платы и работающие устройства. Схема этапов разработки представлена на рисунке 20.7.

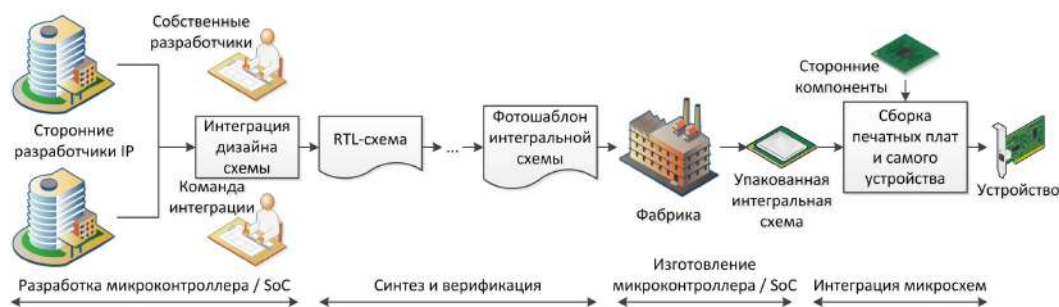


Рис. 20.7: Схема этапов разработки и изготовления компонент ЭВМ.

Закон Мура

Характер роста развития микроэлектроники экспоненциальный. Замечен был ещё в 60-х годах инженером Гордоном Муром. Это эмпирический закон, который всё ещё выполняется. Изначально закон звучал так, что число транзисторов на кристалле каждые два года будет удваиваться. Со временем экспонента немного сгладилась и период удвоения количества транзисторов возрос до трёх лет. В какой-то момент нельзя будет дальше уменьшать транзисторов из-за физических фундаментальных ограничений. Также, нельзя сильно увеличивать площадь кристалла, так как время, за которое сигнал будет проходить от края до края кристалла тогда увеличится (быстрее нельзя из-за того, что сигналы распространяются со скоростью света). Техника стремительно устаревает, поэтому простаивание процессоров - это потеря времени.

Закон Гроша

Примерно в то же время, в которое был сформулирован закон Мура, был сформулирован закон Гроша инженером из компании IBM. Звучал он так: производительность компьютеров будет увеличиваться пропорционально квадрату денежных вложений в его конструирование. В течение 10-15 лет этот закон выполнялся. В то время компьютеры были огромного размера. Как следствие, была выдвинута гипотеза, что достаточно около 5 компьютеров для удовлетворения потребностей всего человечества, а деньги для их создания будут потрачены максимально эффективно. Однако, стало понятно, что бесконечно увеличивать производительность за счёт денег невозможно. К 1997 году закон был полностью опровергнут, тем самым была показана ограниченная применимость к широкому классу компьютеров. Интересно, что закон пережил реинкарнацию в некотором роде. На рынке поисковых систем он имеет место быть, так как по сути люди вложили очень много средств, и теперь небольшое количество поисковых систем покрывает нужды человечества.

Закон Белла

Закон был сформулирован знаменитым инженером Гордоном Беллом в 1972 году и отсылает к понятию класса компьютеров. Закон звучит так: примерно каждое десятилетие появляется новый, более дешёвый класс компьютеров, который использует новую программную, аппаратную платформу. В результате появляются новые отрасли индустрии и области применения компьютеров. Этот закон выполняется до сих пор. Например, 2010-е годы стали эпохой интернета вещей. Нулевые годы были эпохой появления смартфонов, которых ранее не существовало. В 90-е годы появились ноутбуки, а в 80-е персональные компьютеры.

Различные классы компьютеров

Каждый класс компьютеров определяется определёнными характеристиками. Эти характеристики связаны с техническими особенностями, что влияет на цену всей системы. Процессоры бывают разные по сложности, по мощности. Ничего удивительного в сильном различии стоимости нет, так как для каждого класса компьютеров формулируются свои требования, в зависимости от того, где он применяется. Например для мобильных устройств отличительная особенность - это требование к скорости реакции (процессор может быть очень мощным, но из-за каких-то технологических особенностей может быть задержка отклика на действия пользователя). Для сервера же задержка в несколько секунд будет совсем не критична, для него важнее суммарная пропускная способность (суммарное количество платежей за сутки например).

Тем проще устройство, тем большее количество нужно их выпустить. По мировым

продажам доминируют не процессоры с архитектурой x86, а с архитектурами встраиваемых устройств (ARM, ARC, MIPS). Точные числа привести сложно из-за коммерческой тайны, но характер таков, что число процессоров для настольных компьютеров на порядки меньше, чем число процессоров для различных встраиваемых небольших устройств.

Оперативная память

Вне зависимости от класса, все компьютеры имеют оперативную память. Основные характеристики устройства памяти разделяют её на статическую и динамическую (SRAM и DRAM соответственно).

Статическая память обладает следующими свойствами:

- В данном типе памяти ячейки реализуются с помощью транзисторов.
- При наличии питания информация, сохранённая в ячейке может оставаться там неограниченно долго.
- Относительно устойчива к радиации и воздействию электромагнитных полей.
- Быстрее и дороже DRAM.

Динамическая (DRAM) характеризуется следующим:

- Состоит из конденсатора и транзистора
- Сохраняемое значение необходимо обновлять каждые 10-100 мс.
- Более чувствительна к внешнему воздействию.
- Медленнее и дешевле SRAM.

Основные характеристики представлены в таблице:

Тип памяти	Транз. на 1 бит	Относ. время доступа	Устойчивая	Контроль	Относ. стоимость	Применение
SRAM	4 или 6	1 ×	да	нет	100 ×	Кэш
SRAM	1	10 ×	нет	да	1 ×	Основная оперативная память

Если свести ключевые характеристики вместе, то получается картинка исключённого третьего. При формировании памяти можно выбрать лишь две положительные

характеристики. Они будут достигаться за счёт того, что всё плохо будет с третьей. Характеристики: скорость работы, цена, объём памяти.

Динамическая память имеет определённую организацию. Неэффективно обращаться к отдельному биту, поэтому они группируются в более крупные компоненты - суперячейки. Таким образом, оперативная память - это комплект суперячеек размером сколько-то бит (то есть, каждая суперячейка может быть 8, 16 или другое количество бит). Каждая суперячейка рассматривается, как элемент двумерной матрицы, так как пары транзистор-конденсатор размещаются на подложке. Схематичное строение оперативной памяти, как комплекта суперячеек представлено на рисунке 20.8.

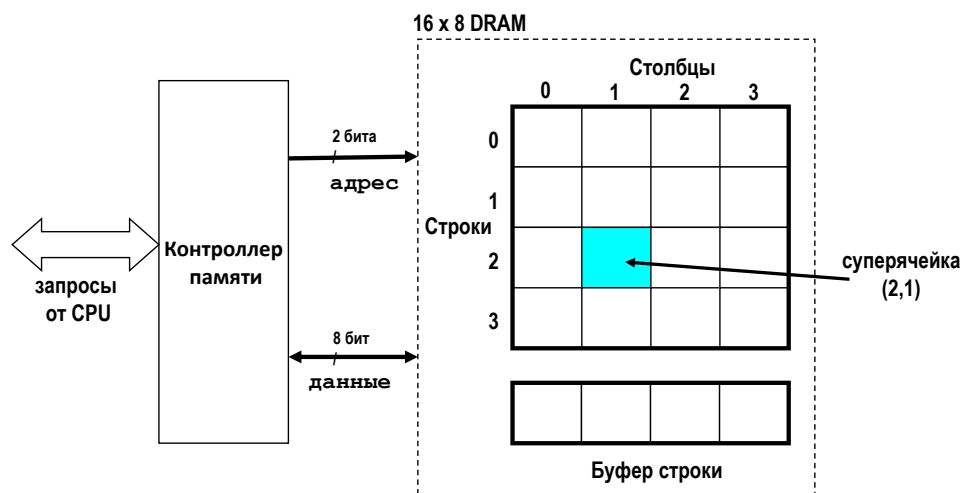


Рис. 20.8: Схема оперативной памяти как двумерного массива суперячеек.

Чтобы считать суперячейку из динамической памяти, необходимо выполнить несколько шагов:

1. Необходимо указать в какой строке матрицы находится интересующая ячейка. Тогда из контроллера памяти по шине приходит строб адреса строки (одно число, которое приходит от процессора за один такт).
2. Содержимое строки отправляется в буфер, где запоминается.
3. Аналогично строке указывается строб адреса столбца.
4. Выбранная суперячейка отправляется из буфера в процессор через контроллер памяти.

Типовой приём ускорения работы с памятью - расслоение. Пусть модуль памяти состоит из 8 отдельных микросхем. Раскидываются элементы большого машинного слова таким образом, чтобы каждая часть оказывалась в своём банке памяти (на отдельной микросхеме). Тогда за одну операцию можно считать не одну часть, а все 8 (ускорение в 8 раз).

Развитие DRAM

Организация ячейки принципиально не менялась уже около 50 лет. В какой-то момент было замечено, что можно повторно использовать адрес строки (то есть, если строку скопировали в буфер, можно выдать последовательность из одного адреса строки и потом из нескольких адресов столбцов, которые идут подряд).

Кроме того, стали производить синхронную память DRAM с удвоенной частотой (так как активация может происходить как на спаде электрического сигнала, так и на подъёме), эта технология называется DDR.

Энергонезависимая память

Энергонезависимая память не теряет информацию при отключении электропитания. Такая память например используется при запуске BIOS. Существует целый набор способов, как построить такой тип памяти.

- Самый простой способ - взять матрицу из полупроводников и пережечь её на производстве. Однако изменить состояние такой матрицы никак нельзя.
- Первая перезаписываемая память могла стереть информацию посредством ультрафиолета.
- Затем придумали электрическую перезаписываемую память (EEPROM). Для стирания информации используется повышенное напряжение. Развитие эта технология получила во флеш-памяти. Технология работы основана на квантовом эффекте туннелирования.

Лекция 21. Организация шин

Шины и адресные пространства.

В этой лекции будут рассмотрены вопросы, связанные саппаратной составляющей. Будут рассмотрены разные аспекты устройства шин и решения прикладных задач с точки зрения разработчика и пользователя.

Система шин – дорожная сеть, связывающая центральный процессор (ЦП), оперативную память, устройства вывода. Через шины данные, которые компьютер получает извне, передаются ЦП и оперативной памяти.

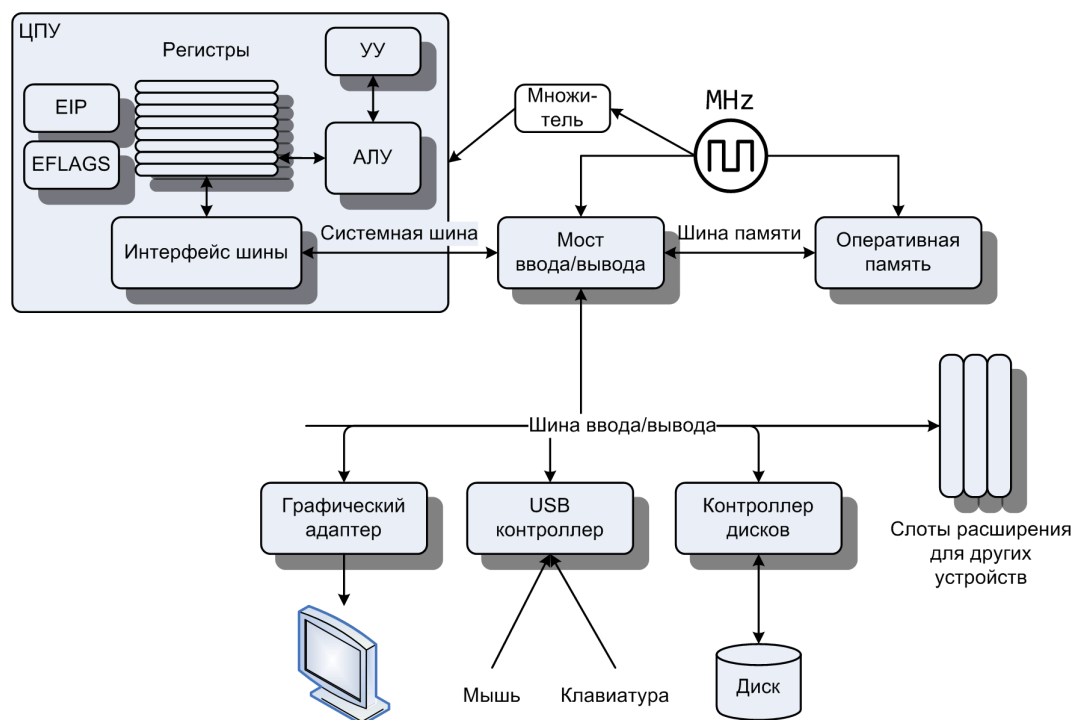


Рис. 21.1: Схема связи устройств

Важнейшая характеристика шин:

- *пропускная способность* – то, какой объем информации может быть передан через шину за единицу времени. Должны быть хорошо сбалансированы связи между шинами, чтобы не получить ”пробки”.

Выделяют следующие типы шин:

- *Системная шина* связывает процессор с тем, что находится вне его.
- *Шина памяти* связывает оперативную память с остальными компонентами.

- *Шина ввода/вывода* разбивается на много шин, связанных между собой. Она связывает устройства долговременного хранения информации, устройства взаимодействия с пользователем и устройства вывода (например, графический адаптер).

Шины работают на более низких частотах чем ЦП, поэтому необходим *тактовый генератор* (см. рис. 21.1), частоты с которого через множитель попадают на процессор. Этот сигнал будет каждый раз запускать очередную порцию обработки информации внутри процессора. На те устройства контроллера шин, которые обеспечивают связь устройства с внешним миром, частота может подаваться без умножений, поэтому на каждую группу выполненных в процессоре команд, приходится одна операция на шине.

Шина – некоторое вспомогательное устройство, которое обеспечивает передачу данных.

Рассмотрим взаимодействие шины с оперативной памятью. Процессор видит некоторый интерфейс шины, за которым может скрываться как прямая линия передачи, так и некая совокупность, объединенная мостами.

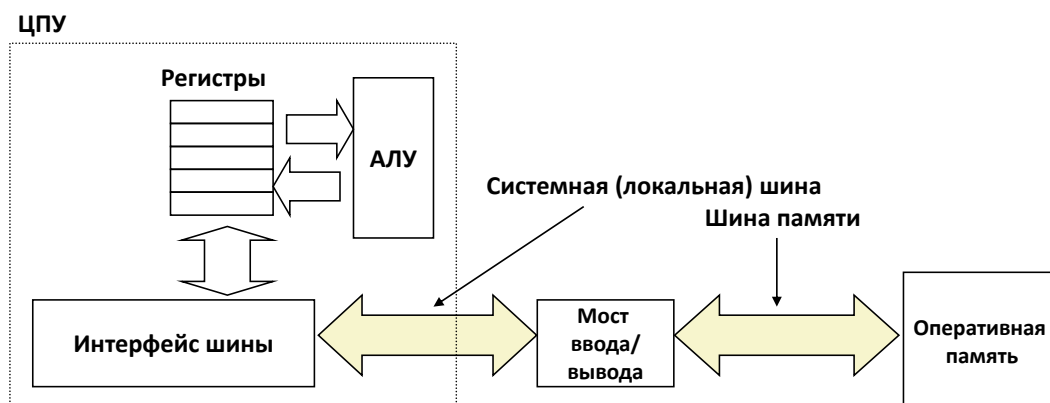


Рис. 21.2: Типовое соединение ЦПУ и оперативной памяти

Определение. *Шина* – набор проводников, используемых для передачи электрических сигналов, кодирующих данные, адреса, управляющие сигналы. Управляющие сигналы говорят, какое действие надо совершить с помощью шины.

Для работы с оперативной памятью всего два действия: чтение и запись.

Шина может использоваться группой устройств. В этом случае шина – некоторая среда передачи сигналов, через которую эти сигналы слышны всем абонентам, которые могут обратиться к шине. То есть любой сигнал, который передается через шину слышим всем подключенным абонентам. Из-за этой особенности нужно синхронизировать работу

подключенных устройств, чтобы они не мешали друг другу.

Обращение к памяти за данными. При обращении к памяти за данными на интерфейс шины попадает обращение, которое выражается внутри команды адресным кодом dword [A]. Из некоторого адреса извлекаются данные, которые затем будут переданы процессору.

Рассмотрим обращение к памяти за данными. **Чтение данных из памяти:**

1. на интерфейс шины попадает адрес A того места памяти, к которому хотим обратиться ;
2. этот адрес проходит по шине, доходит через цепочку контроллеров до оперативной памяти;

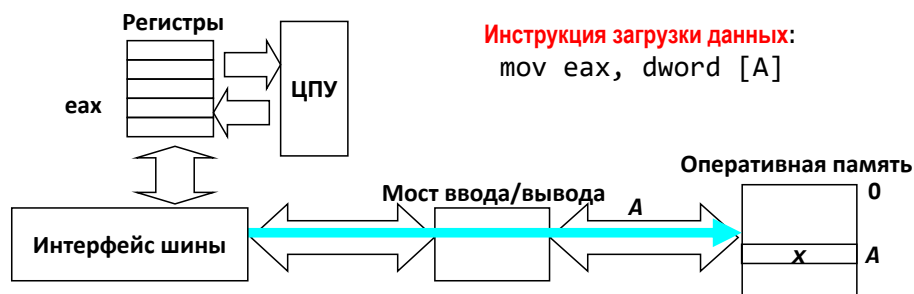


Рис. 21.3: Чтение данных 1

3. оперативная память получает запрос на выборку данных по адресу A из шины, осуществляет выборку значения x, и отправляет его назад в шину;
4. по шине обратно поступают данные поступают на интерфейс, через который эти данные попадают на соответствующие линии процессора;
5. ЦПУ считывает значение x из шины, которое было в оперативной памяти, и посылает его в регистр, в данном случае в регистре eax.

При записи в память работа шины несколько меняется. **Запись данных в память:**

1. ЦПУ передает исполнительный адрес A интерфейсу шины ;
2. оперативная память считывает адрес и ждет посылки соответствующего значения;
3. вслед за адресом отправляется то значение, которое будет помещаться в устройство, подключенное к шине;

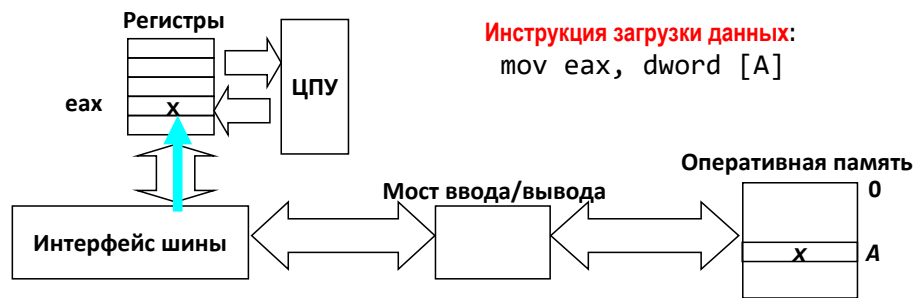


Рис. 21.4: Чтение данных 2

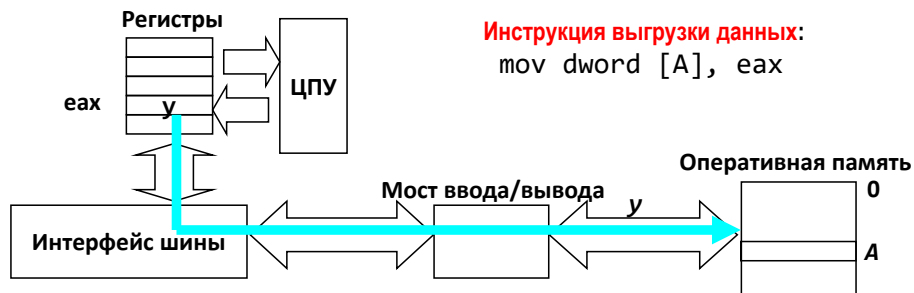


Рис. 21.5: Запись данных

- оперативная память получает двойное слово y из шины и сохраняет его по адресу A .

Информация, проходящая по шине в случае чтения и записи, различается. Кодировается, какая была совершена операция, указывается адрес, указываются значение, которые передаются.

На физическом уровне такого рода пересылки могут происходить параллельно по нескольким линиям в параллельной шине, либо последовательно по одному проводнику — последовательная шина.

Организация ввода/вывода через пространство портов и через память.

Рассмотрим основные принципы подключения устройств к системной шине. Процессор в первую очередь видит основную, системную шину, которая к нему подключена. Он работает с ней в некотором адресном пространстве. Возникает вопрос, как воздействовать на остальные периферийные устройства, не являющиеся памятью, обращени к которым идет не по номерам ячеек.

- Port IO. Введение некоторого дополнительного пространства, к которому можно адресоваться. Характерно для архитектуры X86.

Помимо памяти, пространства с одномерной адресацией, вводятся порты ввода/вывода, которые представляют некоторый одномерный массив некоторых ячеек, к которым можно обратиться (16-ти разрядное пространство). Вводится дополнительная категория двухадресных команд(две команды):IN, OUT. В них задается номер порта и значение, которое записывается в порт, либо считывается в заданный операнд.

Для команды IN в первый операнд считываем информацию из порта, указанного во втором операнде, а для команды OUT в порт, указанный в первом операнде записываем информацию, записанную во втором операнде.

Порты заведены на определенные регистры периферийных устройств. Обращение к какому-либо порту приводит к тому, что запись или чтение оправится в управляющий регистр или статусный регистр. В управляющий регистр можно записать некоторую величину, поменяв порядок работы того или иного устройства. При считывании состояния статусного регистра, можно узнать, в каком состоянии находится устройство. Например, можно следить за ходом приема/оправки информации.

Плюс: память не перекрывается, что удобно в случае, когда мало оперативной памяти.

2. Memory Mapped IO. Отображенный в память ввод/вывод. Все управляющие регистры устройств отображаются на определенные адреса оперативной памяти.
 - Требуется определенная настройка контроллера памяти, чтобы ему было известно, что поределенные адреса связаны не с ячейками оперативной памяти, а с регистрами того или иного устройства.
 - При попытке обращения в оперативную память мы попадаем в те же регистры ввода/выводы.
 - Перекрытая память не используется.
 - Существенно более высокая производительность.

Шина. Точка зрения системного программиста. Системный программист видит шину как некое адресное пространство, которое он может сконфигурировать.

Для системной шины, через которую идет обращение к оперативной памяти, важна настройка, как в адресах потенциально адресуемых ячеек будет размещено содержимой дополнительных устройств.

Шина. Точка зрения системного программиста



Рис. 21.6: Адресное пространство

С некоторого базового адреса 40000000 располагается оперативная память, часть младших адресов соотнесена с микросхемой флэш-памяти, внутри которой находилось некоторое количество определенных разделов:

- загрузчик
- дополнительное программное обеспечение
- блок данных, содержащих специализированную для Flash файловую систему
- защищенная зона

Информация считывается и записывается как в память, но на самом деле все обращения идут в дополнительную микросхему, содержащую флэш память.

Таким образом на шину может быть подсоединена не только флэш-память, но и USB-контроллеры, контроллеры сетевых адапторов и т.д.

В данном случае указана 32-х разрядная шина, которая позволяет адресоваться к чему-то а интервале от 0 до 4 Гб.

Пример. Рассмотрим кусок кода из реального драйвера сетевой карты в операционной системе LINUX.

Есть три параметра: некоторая структура device, дальше передаются два параметра с указанием некоторого буфера данных, с указанием его размера. Задача – передать эти данные через сетевую карту.

```
int device_driver_transmit_data(device *dev,
                                phys_addr_t *buffer,
                                size_t buf_len)
{
    offset_t offset = device->offset;
    uint32_t status;
    time_t time = 0;
    outl(offset + BUFFER_REGISTER, buffer);
    outl(offset + BUFLen_REGISTER, buf_len);
    outl(offset + COMMAND_REGISTER, COMMAND_TRANSMIT);
    do {
        wait(WAIT_INTERVAL);
        time += WAIT_INTERVAL;
        inl(&status, STATUS_REGISTER);
        if ((status & ERROR_MASK) || (time >= TIMEOUT))
            goto error;
    } while (!(status & COMPLETE_MASK));
    return 0;
error:
    return -1;
}
```

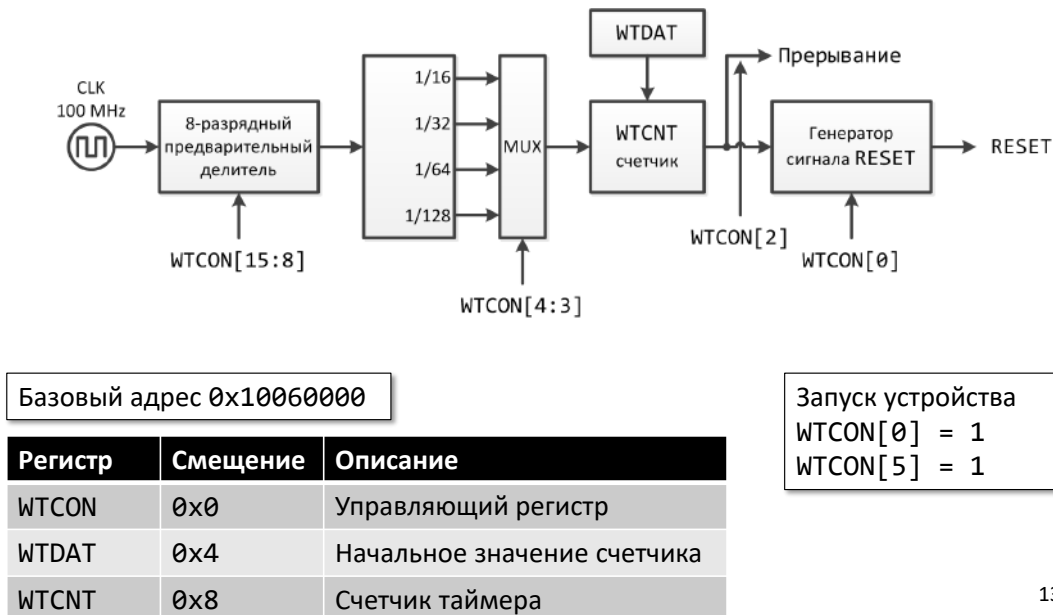
```
void outl(int* m_reg, int val) {
    *m_reg = val;
}
```

Рис. 21.7: Пример кода

- Вытаскивается базовый адрес, начиная с которого в памяти размещены отображенные регистры сетевого адаптера.
- Присутствуют регистры в которых будет задаваться адрес того буфера, который хотим переслать; еще 32-х разрядный регистр, в котором будет помещаться длина того буфера, который будет использоваться; регистр команды и статусный регистр, с помощью которого будем следить за ходом пересылки.
- Обращение к сетевому адаптеру: к базовому адресу добавили некоторое смещение, относящееся к регистру буфера, другое смещение, относящееся к регистру длины. Что-то по этому адресу записали. Потом передали команду.
- Начался цикл ожидания, в котором мы некоторое время ждем, а затем опрашиваем статусный регистр и смотрим, не случилась ли ошибка при передаче, не превышено ли время ожидания, которое было зарезервировано на передачу данных. Это продолжается до тех пор, пока где-то в статусном регистре появится взведенный флаг, показывающий, что передача прошла.

Пример устройства на шине: WatchDog.

Рассмотрим пример простейшего периферийного устройства, так называемой сторожевой собаки.



13

Рис. 21.8: WatchDog @ SoC Exynos4210

В данном случае это не отдельный процессор, а целая система на чипе. Помимо процессора на общей подложке размещено много всего (так дешевле). Для мобильных телефонов характерно присутствие сторожевых собак. Это специальные устройства, которые позволяют отправить компьютер в перезагрузку, если что-то пошло не так, например, ошибка в операционной системе.

Устройство:

- Регистр, который на каждом такте работы уменьшается на 1. Таким образом можно записать начальное состояние счетчика, который живет в собаке. Когда счетчик достигнет 0, устройство запустит сигнал на перезагрузку. Если система не хочет принудительно перезагружаться, она время от времени обращается в собаку и устанавливает значение счетчика через регистр WTDAT.
- Через управляющий регистр WTCN можно выбрать, насколько быстро счетчик будет сбрасываться до 0.
- Через 3 и 4 биты управляющего регистра можно управлять мультиплексором, который позволит сократить скорость на уменьшение счетчика

- Через 1 и 2 бит управляющего регистра можно установить возможность, чтобы при достижении 0 счетчиком выдавалось прерывание.

После базового адреса расположены три регистра, записывая и считывая из которых, можно следить за работой периферийного устройств.

Физические принципы устройства шин. Основные характеристики шин.

Рассмотрим шину с точки зрения разработчика аппаратуры. Шина – совокупность проводов, по которым должны правильно проходить сигналы. Т.е. в определенные моменты времени должны осуществляться скачки с низкого уровня напряжения на высокий, должен быть задан порядок кодирования информации изменениями напряжения, должно быть обеспечено корректное распространение сигнала по длинным проводникам (порядка десятка сантиметров).

В шине могут быть выделены

- управляющие линии,
- линии адреса,
- линии данных.

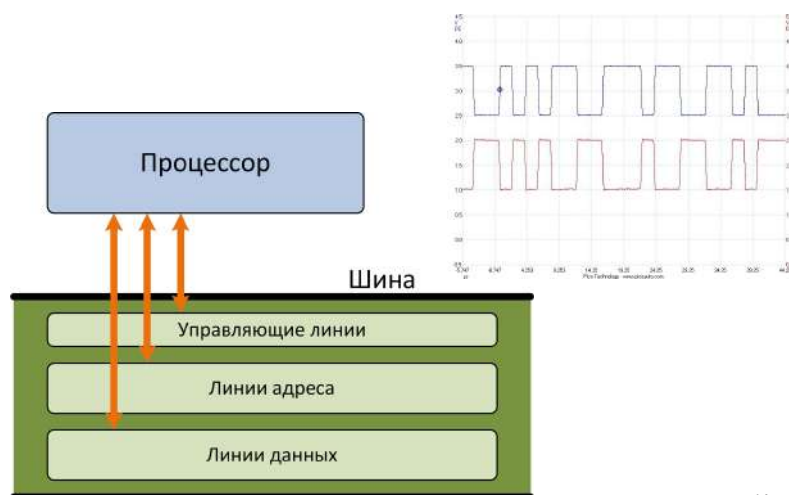


Рис. 21.9: Шина

Характеристики шин:

- Ширина. Количество линий, линии адреса, линии данных, мультиплексирование адресов и данных

- Частота – сугубо физический аспект, связанный с тем, какие стробирующие сигналы на самом низком уровне проходят.

⇒ первые две характеристики влияют на то, какая пиковая пропускная способность шины будет обеспечиваться.

- Пиковая пропускная способность вычисляется как произведение ширины (байты или биты) и частоты (1/сек). Эта частота почти не достигается. При мультиплексировании адресов и данных часть времени будет затрачена на передачу служебной информации.
- Арбитраж – это тот, как принимается решение, кто будет пользоваться шиной. Электрические сигналы идут с конечной скоростью, поэтому одно устройство может обратиться к шине, а другое устройство, еще не зная, что первое к ней обратилось, тоже может к ней обратиться. одновременное обращение нескольких устройств может привести к коллизии доступа. При разработке шины в зависимости от топологии связей внутри шины, решается, как понять, какое устройство может захватить шину и как остальные устройства будут узнавать, что шина занята/свободна.
 - Централизованный. Шины могут иметь топологию ”звезда когда несколько абонентов подключены к некоторому центральному устройству, контроллеру. Т.е. физически присутствуют несколько выделенных линий для отдельных подключенных устройств. Центральный контроллер распределяет, как пойдут данные между этими абонентами.
 - Децентрализованный. Шина представляет собой полно связанный граф. Чем больше элементов связывает такая шина, тем больше физических связей в таком графе присутствуют.
- Возможность горячей замены устройства
- Физическая организация – Выделенные линии данных, адресов, команд – Мультиплексированные линии – Топология связей – Синхронная/асинхронная – Ограничения по длине линии

Развитие системы связанных шин персональных компьютеров

На момент возникновения 32-х разрядной архитектуры , архитектура состояла из двух шин:

1. локальная шина – 32-х разрядная (обращение максимум к 4 мб) шина на линии процессора, через которую доступна оперативная память. Этой шиной управляла

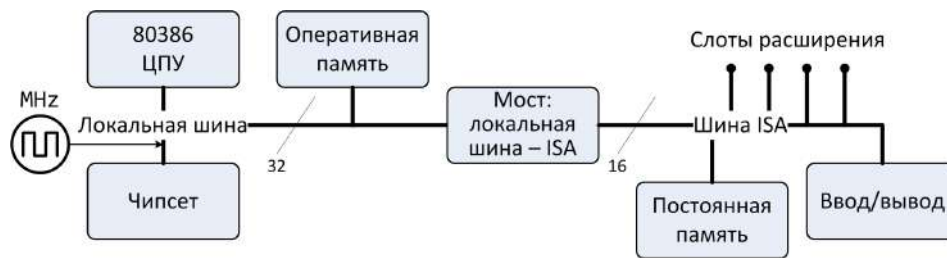


Рис. 21.10: пример 32-х разрядной архитектуры с двумя шинами

группа микросхем (чипсет), тактовая частота определялась некоторым внешним тактовым генератором. На локальной шине сидел мост, который позволял сопрячь локальную шину с более медленной шиной ISA.

- шина ISA (Industry Standard Architecture) – это 16-ти разрядная шина, к которой подключались устройства ввода/вывода, микросхемы, содержащие в себе загрузчик, жесткие диски и т.д. Шина ISA – некоторая стандартизированная шина, спецификация которой была доступна сторонним разработчикам, что позволяло им самостоятельно выпускать устройства, который можно было устанавливать в слоты расширения, подключать. Это вызвало возникновение конкурентной среды, которая способствовала развитию архитектуры персональных компьютеров.

В 1990-х требования увеличения скорости и памяти и уменьшения размера устройства привели к архитектуре, в которой несколько процессоров связывались с внешним миром через *фронтальную шину* (см. рис. 21.11).

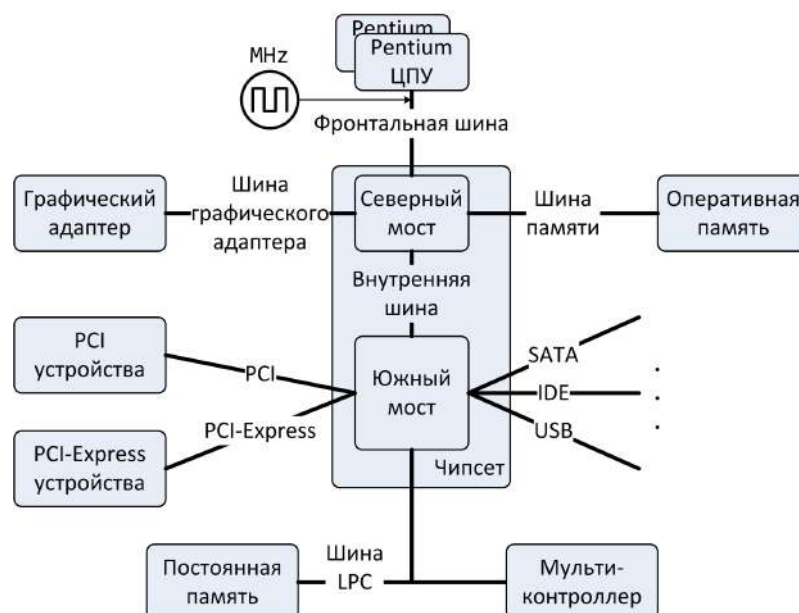


Рис. 21.11: архитектура 1990-х

- Фронтальная шина идеологически не отличается от локальной шины. Отличие заключаются в том, что стробирующий сигнал гарантированно шел с определенным множителем. Она могла соединять между собой пару процессоров и обеспечивала связь с чипсетом.
- Чипсет уже имеет два моста:
 - северный мост отвечает за ввод/вывод. Он связан с оперативной памятью и графическим адаптером (нужна максимальная частота)
 - южный мост обеспечивал взаимодействие с более медленными устройствами: шина PCI, PCI-Express, жесткие диски, USB-устройства.

За самые медленные устройства (клавиатура, мышка) отвечал мультиконтроллер, который подключался вместе с постоянной памятью к шине LPC

Затем на основном кристалле была сформирована некоторая система. Современный 64-х разрядный процессор – это система на чипе, которая состоит из нескольких вычислительных ядер, контроллера памяти, контроллера графического адаптера и т.д. части, требующие высокую пропускную способность и передачу большого объема

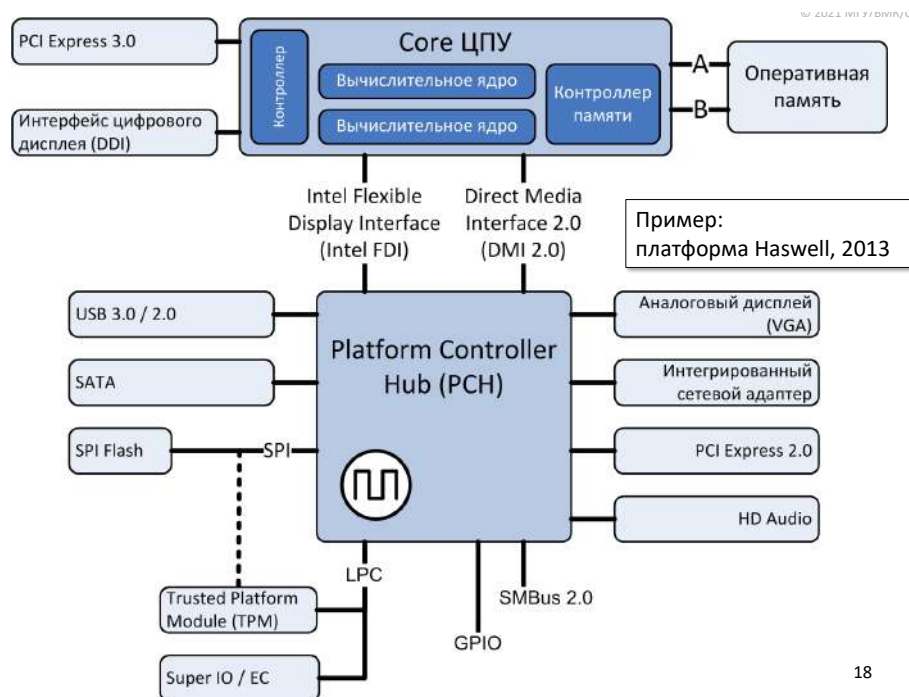


Рис. 21.12: платформа Haswell, 2013

данных размещены на кристалле процессора, остальное обслуживается через Platform Controller Hub.

Примеры шин: фронтальная шина

Фронтальная шина:

- HyperTransport (HT), апрель 2001, AMD. Открытый стандарт - HyperTransport Technology • 2 – 32 разряда, двунаправленная • 200 – 2600 MHz, DDR • В 2017 году на смену HT пришла шина следующего поколения – Infinity Fabric
- QuickPath Interconnect (QPI), ноябрь 2008, Intel • 20 линий, двунаправленная, 4 такта = 64 бита • 2.4, 2.93, 3.2 GHz, DDR • В 2017 году на смену QPI пришла шина следующего поколения – Intel Ultra Path Interconnect (UPI) в процессорах Skylake EX/EP Xeon.

Фронтальная шина обеспечивает соединение точка-точка. Так как связываются ядра процессора, требуются высокие частоты и выделенные линии связи, которые не будут блокировать взаимодействие ядер.

⇒ не требуется арбитраж, так как между каждой парой ядер присутствует физический проводник. Чем больше ядер, тем больше связей в полносвязном графе.

Разрядность таких шин может варьироваться. Частоты растут.

Такие шины двунаправленные, т.е. каждое подключенное процессорное ядро может как отправлять, так и получать данные.

Синхронизация обращений к памяти

Синхронизация обращений к памяти – важное свойство шины, связывающей ядро с памятью.

В силу того, что в процессоре присутствуют много ядер, возникает необходимость организации одновременной параллельной работы программ, выполняющихся одновременно на нескольких ядрах. Может возникнуть ситуация обращения нескольких ядер к общим данным, находящимся в оперативной памяти. Для этого необходимы *механизмы синхронизации*.

Рассмотрим конструкцию с замком, который имеет два состояния:

0 – доступ открыт

1 – доступ открыт

В зависимости от значения замка выполнение на ядре либо находится в бесконечном цикле, либо может продолжаться. С помощью таких замков формируются критические секции в коде. С помощью запрашивания замка кусок кода делается недоступным для обращения других ядер.

Рассмотрим кусок кода, в котором запрашивается замок. В данном цикле выполняется одна команда (см. рис. 21.13)

Разберем команды BTS(Bit Test and Set) и BTR (Bit Test and Reset). В обоих случаях два операнда: регистр/память , регистр/непосредственно закодированная величина.

Во втором операнде задается некоторый бит, который будет отправлен во флаг CF.

- В случае команды set этот бит будет заведен (бит $\leftarrow 1$).
- В случае команды reset этот бит будет сброшен до 0 (бит $\leftarrow 0$).

Попадаем в цикл. Если никто еще не захватил однобайтовую переменную, в первой команде значение нулевого бита из переменной замка отправляется в CF в предположении, что замок был нулевой. Затем проверяется, заведен ли флаг CF.

- Если замок был нулевой, это условие не выполняется, мы выходим из функции.
- Если замок уже захвачен, команда BTS запишет туда единицу и поместит ранее записанную единицу в флаг CF, который нас отправит на следующую итерацию цикла.

Команда BTR отвечает за освобождение замка. В нулевой разряд записывается ноль. Таким образом решается проблема синхронизации.

Гонка. Может быть ситуация, когда два ядра пытаются захватить замок одновременно. Это может произойти следующим образом:

- ядро1 обратилось в память, запросило текущее состояние замка;
- ядро1 посмотрело полученное значение и переслало полученное значение в CF ;
- ядро1 фиксирует свое выполнение, отправляя в память новую величину, где нулевой бит уже взведен
- в ядре2 в это же время происходит чтение с небольшим наложением. В момент, когда результат выполнения команды BTS с ядра1 еще не зафиксирован, вторая уже запрашивает содержимое замка. $\Rightarrow$ оба ядра получают начальное значение замка с нулевым битом . Получаем состояние гонки.



Рис. 21.13: Состояние гонки

Захват шины данных. Чтобы избежать состояние гонки, нужно ввести захват шины данных. Для этого вводится дополнительный префикс lock, который гарантирует, что в

момент времени, когда выполняется одна команда, другие команды не могут использовать шину данных на чтение/запись. Выполнение команд на ядре2 будет дожидаться момента, когда на ядре1 BTS закончится и зафиксирует свои результаты, поместив в память.

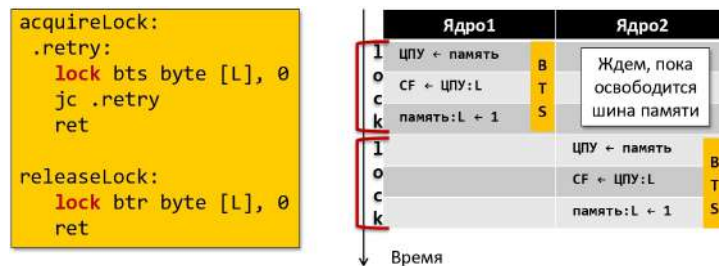


Рис. 21.14: Захват шины данных

Вывод: для корректного функционирования шины в многоядерном устройстве необходима аппаратная возможность блокировать шину.

Пример функции захвата замка. Преобразуем ранее написанный цикл, вытаскивая первую итерацию в виде отдельной процедры. Проверяется, что в замке находится нулевой бит. Если нет, оказывается на команде ret. Если нужно ждать, в тело цикла добавляется команда pause, которая подсказывает процессору, что мы находимся в цикле активного ожидания. Это помогает экономить энергию. В этом состоянии находимся пока не получим нулевой бит в замок.

В современных процессорах команда pause используется как подсказка, что выполнение находится в цикле активного ожидания (busy wait)

```
acquireLock:
    lock bts byte [L], 0
    jc .retry
    ret
.retry:
    pause
    lock bts byte [L], 0
    jc .retry
    ret
```

- Явно указываемый префикс lock. Применим только к некоторым командам ADD, AND, BTC, BTR, BTS, CMPXCHG, ...
Первый операнд команды – память
- Все время выполнения команды процессор удерживает шину памяти, посылая на нее сигнал #LOCK
- Постоянный захват шины негативно сказывается на производительности

ОС Linux использует для реализации активного ожидания не bts/btr, а гораздо более быструю команду cmpxchg.

Примеры шин: PCI.

Появилась в 1990-х годах. Общая шина ввода/вывода для подключения периферийных устройств Peripheral component interconnect (PCI), 1992, Intel, открытый стандарт.

- PCI 1.0 / 2.0
 - Шина содержала 32 линии
 - Передача данных шла транзакциями или пакетами, возможна была приостановка передачи
 - частота 33 MHz
- Расширения
 - PCI 64, PCI 66, PCI 64/66, PCI-X (266 и 533 МГц)
- PCI Express (PCI-E), июль 2002, Intel
 - Топология – звезда, подключение устройств через двунаправленные соединения различной ширины. Ширина обозначается символом x. Каждая линия(x1) – 4 проводника, потому что передача осуществляется в двух направлениях. Передача является полнодуплексной. Дополнительный множитель возникает из-за того, что передача сигнала в направлении реализуется через дифференциальную пару. Пара проводников позволяет снижать зашумленность линии: по одной линии идет одычный сигнал, а по другой – инвертированный. Проводники бывают различной ширины: x1, x2, x4, x8, x12, x16, x32
 - Скорость работы соединения варьируется – 2.5, 5.0, 8.0 и 16GT/s (32GT/s в PCI 5.0), T/s - передач по шине в секунду
 - Избыточное кодирование данных, 128b/130b в PCI 3.0 и более новых, 8b/10b – в старых версиях шины. Т.е. число избыточных данных уменьшилось.

Все спецификации заведены на величину GT/s (gigatransfers per second). Пересылки транзакций в секунду. Транзакция – пересылка одного разряда.

Идентификация (адресация) PCI-устройств. Используется 16 разрядов, которые делятся следующим образом:

- первые 13 разрядов задают физическое устройство, которое может быть включено в компьютер
 - первые 8 разрядов описывают номер шины
 - следующие 5 разрядов описывают номер устройства

- последние 3 разряда кодируют номер функции

Преимущества PCI:

- стандартизация статусных и управляющих регистров, которые составляют суммарно 256 байт
 - первый 16-ти разрядный регистр описывает идентификатор производителя
 - следующие 16 описывают идентификатор устройства
 - регистр для команд
 - регистр для статуса
 - ...
 - регистры, которые позволяют задавать базовые адреса в памяти
- Регистры первых 64 байт стандартизированы
- Чтение и запись в регистры
 - В начальный момент времени доступен механизм работы с PCI-устройствами через порты ввода/вывода. На порт номер 0xCF8 завернут в PCI-контроллере регистр, задающий адрес, а на порт номер 0xCFC завернут регистр, позволяющие обращаться за данными или записывать данные. командами IN и OUT
 - Через отображение регистров PCI-устройства в 4КБ адресного пространства памяти (зависит от реализации...) В адресном пространстве «теряется» 256 МБ памяти.

Можно опросить все возможные функции возможных устройств последовательно опросить и создать картину того, что подключено к системе. Затем распределять диапазоны адресов памяти и т.д.

После этой первичной настройки можно отобразить весь комплект регистров в оперативную память. Т.к. задание дополнительных атрибутов для памяти связано не с реальной физической оперативной памятью, а с гранулярностью в 4 КБ, которые называются страницей, будут потеряны 256 МБ.

После такого отображения работа со всеми PCI будет происходить на максимальной скорости.

Лекция 22. Использование шин в современных компьютерах

Примеры шин: AGP, USB, SATA

- **AGP, 1996.** Непосредственный доступ к оперативной памяти со стороны видеоадаптера, GART (graphics address remapping table). Была вытеснена затем шиной PCI.

Была разработана для эффективного подключения мощных видеоадаптеров для обеспечения высокой скорости работы. Представляла различные усовершенствования, которые используются в современных компьютерах. Например, идея трансляции адресов памяти, которая видна со стороны устройств. В самом процессоре этот механизм называется виртуальной памятью.

Технология GART (graphics address remapping table). Разберем случай видеоадаптера. Если надо выполнять ввод/вывод в обход процессора, напрямую через шину отправляя данные в память, одна операция предполагает некоторый протяженный непрерывный буфер. Когда этот буфер невозможно целиком разместить в физической памяти, в шине AGP можно соотносить куски памяти, видимые для видеокарты, с распределенными по разным местам кусками физической памяти.

- **USB, 1996.**
 - Топология – звезда, к этой шине может подключаться до 127 устройств, разветвители, оконечные точки (15/15 + 1/1 управляющие).
 - Для этой шины допустимо *горячее подключение*, когда устройство подключается к работающему компьютеру без скачков напряжения и сбоев в работе электрических цепей.
 - Поддерживает 4 типа передач: управляющие, поточные, прерывания, изохронные. За счет того, что шина рассчитана на периферию, которая бывает разноплановая (микрофоны, накопители, видео...).
 - Сам контроллер, который управляет работой шины USB является PCI-устройством. $\Rightarrow$ Чтобы работать с шиной USB, надо через шину PCI обращаться к той или иной версии контроллера (например, Open Host Controller Interface), который будет обеспечивать передачу данных по тем линиям, которые подключены к нему.
- **Serial ATA, 2000.** Использует всего 2 линии для приема/передачи. 7 физических проводников:

- 3 линии – земля
- 4 линии образуют 2 дифференциальные пары (прием и передача), чтобы сигнал не искажался во время передачи из-за внешних электромагнитных воздействий.

Пропускная способность в разных версиях: 1.5, 3, 6 GBit/s

Возможность горячей замены

К типовой шине Serial ATA подключается основной носитель информации – магнитный накопитель на жестких дисках.

Жесткий диск

Жесткий диск состоит из набора пластин (со специальным напылением, сохраняющим состояние намагниченности), насаженных на общий шпиндель; считывающей головки, закрепленной на коромысле. Коромысла соединены в гребенку. Данные, полученные при считывании намагниченности пластин, проходят преобразования из аналогового в цифровой сигнал, затем обрабатываются в контроллере жесткого диска.

Геометрия диска.

- На каждой поверхности зафиксированная считывающая головка будет высекать концентрическую дорожку, с которой можно будет считывать информацию. Дорожка делится на сектора, разделенные промежутками.
- Набор равноудаленных от шпинделя дорожек на разных пластинах образуют цилиндр. То есть если спозиционировать гребенку с карамыслами, то комплектом магнитных головок можно последовательно считывать информацию, расположенную в цилиндре, не совершая механических движений кроме вращения пластин вокруг шпинделя.

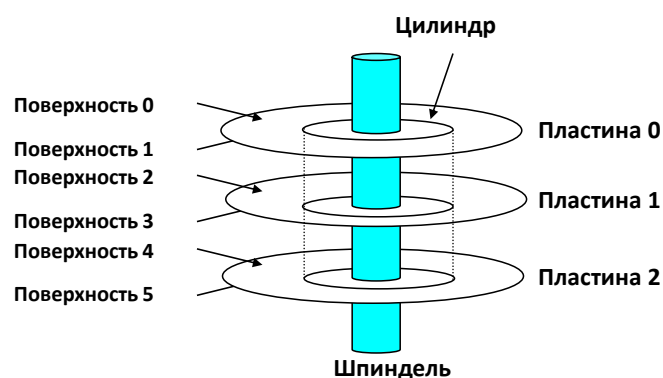


Рис. 22.1: Устройство диска

Емкость диска.

- Разработчики выражают емкость в десятичных гигабайтах, а не обычных (1ГБ=10⁹байт). Различают десятичные (СИ) и двоичные приставки (МЭК) Разница растет: КБ и КиБ ≈ 2.4%, МБ и МиБ ≈ 4.9%, ТБ и ТиБ ≈ 9.95%
- Емкость определяется различными факторами. Чем меньше удастся сделать точку, которая стабильно хранит состояние намагниченности, тем больше плотность записи.
 - Плотность записи – сколько битов может быть размещено на единице длины дорожки(дюйме).
 - Трековая плотность (треки/дюйм – TPI): сколько треков может быть размещено на одном дюйме радиуса.
 - Поверхностная плотность (биты/дюйм²): произведение линейной плотности на трековую плотность
- Современные диски группируют дорожки в несколько зон записи.
 - Каждая дорожка в зоне состоит из одного и того же количества секторов, определяемого длиной самой короткой дорожки.
 - У каждой зоны различное количество дорожек/секторов.

Итоговая емкость:

(#байт/сектор) x (среднее # сектор/дорожка) x (# дорожка/поверхность) x (# поверхность/пластина) x (# пластина/диск)

Работа с диском(одна пластина)

Шпиндель непрерывно вращается на определенной скорости. Считывающая головка, закрепленная на конце коромысла, парит над поверхностью диска на тонкой воздушной подушке. Она может позиционироваться над любой дорожкой. Внутренность жесткого диска заполняется специальной газовой смесью.

Доступ к диску разбивается на несколько этапов. Пусть гребенка спозиционирована на нужную дорожку. На дорожке расположен нужный сектор. Этот сектор считывается, сигнал оцифровывается, компьютер преобразует этот объем информации и т.д. Чтобы прочесть следующий сектор, надо пройти три этапа:

1. *Поиск.* После считывания сектора, необходимо перепозиционировать карамысло. Ищется дорожка, на которой расположен нужный новый сектор.
2. *Латентность вращения.* Находясь на нужной дорожке, ждем, когда диск докрутится до нужного сектора.
3. *Передача данных.* Сектор считывается, сигнал оцифровывается и т.д.

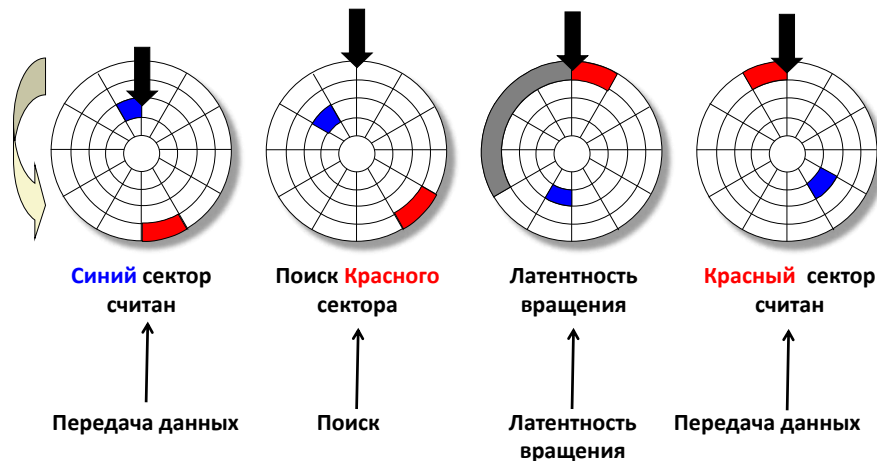


Рис. 22.2: Алгоритм чтения информации с диска

Время доступа к диску.

- $T_{\text{доступа}} = T_{\text{ср. поиск}} + T_{\text{ср. вращения}} + T_{\text{ср. передача}}$
- Время поиска ($T_{\text{ср. поиск}}$)
 - Время, требуемое для перемещения считывающей головки в цилиндр, содержащий требуемый сектор.
 - Как правило $T_{\text{ср. поиск}}$ занимает 3—9 мс.
- Латентность вращения ($T_{\text{ср. вращения}}$)
 - Время ожидания момента, когда первый бит запрашиваемого сектора достигнет считывающей головки.
 - $T_{\text{ср. вращения}} = \frac{1}{2} \times (60 \text{ с.} / \text{RPM})$
 - Типичная скорость вращения – 7200 RPM. $T_{\text{ср. вращения}} \approx 4 \text{ мс.}$
- Время передачи ($T_{\text{ср. передача}}$)
 - Время чтения содержимого сектора.
 - $T_{\text{ср. передач}} = (1/\text{ср. \# секторов на дорожке}) \times (60 \text{ с.} / \text{RPM})$

Пример оценки времени доступа

- Исходные характеристики:
 - Скорость вращения = 7200 RPM
 - Среднее время поиска = 9 мс
 - Среднее # секторов на дорожке = 400
- Оцениваем слагаемые и общую сумму: – $T_{\text{ср. вращения}} = 1/2 \times (60 \text{ с.}/7200 \text{ RPM}) \times 1000 \text{ мс} \approx 4.17 \text{ мс}$ – $T_{\text{ср. передача}} = 60/7200 \text{ RPM} \times 1/400 \text{ с./дорожка} \times 1000 \text{ мс} \approx 0.02 \text{ мс}$ – $T_{\text{доступа}} = 9 \text{ мс} + 4.17 \text{ мс} + 0.02 \text{ мс}$

- Выводы: – Время передачи существенно меньше остальных слагаемых. – Считать первый бит из сектора – «дорогая» операция, считывание остальных битов – «дешево». – Время доступа SRAM ≈ 4 нс для двойного слова, DRAM ≈ 60 нс $\Rightarrow$ Диск медленнее SRAM (статической памяти) в 40,000 раз, и в 2,500 раз, чем DRAM (динамическая память).

Заметим, что время считывания самого сектора – самое маленькое. Самая затратная операция – считать первый бит сектора, который был запрошен.

Производители устанавливают на диск DRAM память и записывают туда сектора, которые считываются с диска. При повторном обращении они уже не ищутся на диске. Это своего рода кэширование.

Также на дорожках определенным образом располагают сектора, чтобы прибегать к позиционированию как можно реже.

Логические блоки. Нумерация секторов зависит от геометрии диска.

- Первоначальный способ задания сектора: $\langle C, H, S \rangle$, где C – номер цилиндра, H – номер считывающей головки в цилиндре, S – номер сектора.
 - В разных зонах – разное число секторов на дорожке
 - Для обращения к диску необходимо знать его геометрию, поэтому при загрузке компьютера во время инициализации устройств нужно было опросить жесткий диск, чтобы узнать его геометрию.
- Более простой метод обращения к данным: геометрия диска скрыта в контроллере, который управляет работой диска. Таким образом все содержимое диска рассматривается как линейная последовательность логических блоков.

Так работает ввод/вывод SATA-устройств

Ввод/вывод (блочного) SATA-устройства.

1. Процессор запускает операцию чтения сектора. Этот запрос через фронтальные шины проходит в контроллер памяти. Те адреса памяти, по которым отправляется содержимое нужных команд, заворачиваются на управляющие регистры SATA-контроллера, которое является PCI-устройством.
2. Команда с номером сектора и указанием операции записывается в регистры, а затем контроллер отправляет эту команду в соответствующий жесткий диск, где все преобразуется в тройку, отвечающую геометрии диска.

Если запрашиваемый сектор уже считывался некоторое время назад, его содержимое может храниться в микросхеме DRAM-памяти диска. В таком случае данные можно считать из нее, а не с пластины.

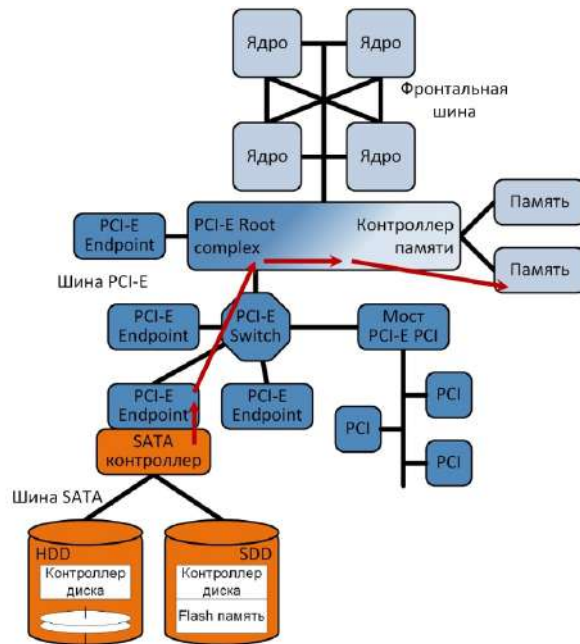


Рис. 22.3: SATA-устройство

3. По шине SATA до контроллера доходит содержимое сектора. SATA-контроллер через шину PCI напрямую в виде серии PCI-транзакций бии за битом в обход процессора запишет содержимое сектора в оперативную память.

4. После завершения записи данных в память SATA-контроллер должен как-то сообщить процессору, что все данные уже помещены в память. Для этого используется механизм прерываний.

5. SATA-контроллер через шину PCI пишет по некоторому адресу памяти (адрес связан с локальным контроллером прерываний вычислительного ядра) сообщение – команду вызвать аппаратное прерывание в ядре (механизм Message Signaled Interrupts). Когда эта информация дошла до контроллера прерываний, он фиксирует, что данные находятся в памяти и ими можно пользоваться. Управление передается коду, который будет работать с новыми данными.

Твердотельные диски (SSD)

Твердотельные диски более надежные и устойчивые к встряскам.

- Данные хранятся блоками – страницами. Размер стреницы : 512 - 4096 байт. Страницы сгруппированы в блоки по 32-128 страниц.
- Данные читаются/пишутся целыми страницами.
- Если нужно подготовить страницу для записи, ее нужно очистить. Данные стираются не по одной странице, а целыми блоками.

- Блок в некоторый момент вырабатывается после некоторого количества перезаписей. На настоящий момент от 1000 до 10000 перезаписей. Изначально было от 100000 перезаписей.
- Если на обычном диске чтение занимало порядка мс, то для SSD диска это время порядка мкс.

Последовательное чтение	250 МБ/с	Последовательная запись	170 МБ/с
Произвольное чтение	140 МБ/с	Произвольная запись	14 МБ/с
Время доступа	30 мкс	Время старта записи	300 мкс

Рис. 22.4: Характеристики

SSD диски и обычные диски. Можно распределять обычный и SSD диски по различным функциям. На SSD дисках размещаются оперативная память и приложения, требующие высокой скорости работы, а для хранения больших объемов данных применяются накопители на магнитных дисках.

Современные контроллеры позволяют реализовывать технологию NCQ (Native Command Queuing). Идет последовательность запросов. Диск эту последовательность переупорядочивает так, чтобы минимизировать время ожидания вращений (см. рис. 22.5).

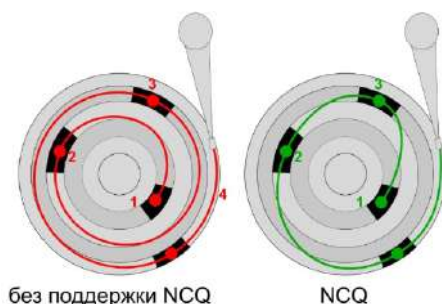


Рис. 22.5: Native Command Queuing

Еще одна оптимизация для случая, когда с SSD-диска данные считываются с большей скоростью, чем шина SATA позволяет передавать. В этом случае рассматриваем твердотельный накопитель как PCI-устройство. Через PCI-Express при достижении определенной электрической совместимости по контактам, подключенный на разъем NVMe диск будет закладывать данные в память ощутимо быстрее.

Можно увидеть, что некоторые характеристики стабильно улучшаются со временем, тогда как для других характеристик уже достигнуты определенные принципиальные пределы.

Рассмотрим хронологию изменения тактовых частот процессора. С ростом частот увеличивалось и энерговыделение. Мощность можно выразить следующим образом:

$$P \approx f \cdot c \cdot V^2,$$

где f – частота, c – емкость, V – напряжение.

SRAM

Метрика	1980	1985	1990	1995	2000	2005	2010	2010:1980
\$/МБ	19,200	2,900	320	256	100	75	60	320
t доступа (нс)	300	150	35	15	3	2	1.5	200

DRAM

Метрика	1980	1985	1990	1995	2000	2005	2010	2010:1980
\$/МБ	8,000	880	100	30	1	0.1	0.06	130,000
t доступа (нс)	375	200	100	70	60	50	40	9
Размер (МБ)	0.064	0.256	4	16	64	2,000	8,000	125,000

HDD

Метрика	1980	1985	1990	1995	2000	2005	2010	2010:1980
\$/МБ	500	100	8	0.30	0.01	0.005	0.0003	1,600,000
t доступа (мс)	87	75	28	10	8	4	3	29
Размер (МБ)	1	10	160	1,000	20,000	160,000	1,500,000	1,500,000

Рис. 22.6: Тенденции в развитии запоминающих устройств

Напряжение уменьшилось на порядок. Емкость связана с площадью кристалла. К 2003-му году тепло выделялось мощностью порядка 100 Вт с площади 1 см², что является большой величиной. С таким тепловым потоком перестали справляться обычные системы охлаждения. Внутри цепей возникали точки перегрева. Поэтому было принято решение увеличивать не тактовую частоту, а количество ядер. Большая производительность достигалась за счет того, что несколько ядер одновременно занимаются вычислениями

Разработчики аппаратуры столкнулись с "Power Wall"

	1980	1990	1995	2000	2003	2005	2010	2015	2015 + 1980
ЦПУ	8086	80386	Pentium	P-III	P-4	Core 2	Core i7 Nehalem	Core i7 Haswell	
Частота (МГц)	1	20	150	600	3300	2000	2500	3000	3000
Длительность такта (нс.)	1000	50	6	1.6	0.3	0.5	0.4	0.25	3000
Количество ядер	1	1	1	1	1	2	4	8	8
Эффективная длительность такта (нс)	1000	50	6	1.6	0.3	0.25	0.1	0.04125	~24250

Рис. 22.7: Частота ЦПУ

над разными данными. С количеством ядер связано время такта. Эффективное время такта продолжает сокращаться.

Помимо энергетической стены разработчики столкнулись со стеной, связанной с быстродействием памяти. Еще одним существенным разрывом является скорость работы обычного жесткого диска и динамической памяти, в которой хранится вся информация.

Был предложен еще один вид энергонезависимой памяти – фазовая память. Информация сохраняется благодаря тому, что при определенном воздействии на

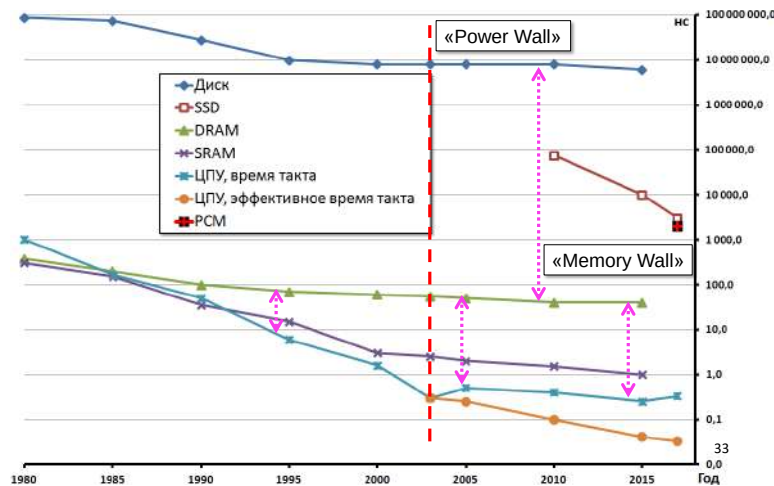


Рис. 22.8: Разрыв между ЦПУ и памятью

материал, являющийся резистором, можно менять его сопротивление. Таким образом можно разградуировать сопротивление на несколько пороговых значений и сопоставить эту величину какому-то числу. Скорость чтения была немного выше скорости SSD.

Локальность в программах

Хорошо написанная программа имеет такое свойство как локальность.

Основной принцип локальности: программа стремится использовать данные и инструкции с адресами близкими (либо точно такими же) к тем, которые использовались ранее. Она обращается к этим данным некоторый промежуток времени.

- Временная локальность: повторные обращения
- Пространственная локальность: в некоторый малый промежуток времени используются ячейки памяти с близкими адресами

Пример 1. В цикле суммируются элементы массива. Видно, что при *выборке данных* выполняется

- *пространственная локальность* за счет того, что осуществляется последовательное обращение к элементам массива, которые идут непрерывно друг за другом;
- *временная локальность* так как переменная `sum` используется на каждой итерации цикла, поэтому идет обращение к одному и тому же месту памяти.

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```

Рис. 22.9: Пример кода

Выборка инструкций обладает *пространственной локальностью*, т.к. осуществляется последовательная выборка инструкций и *временной локальностью* т.к. осуществляется повторное выполнение инструкций.

Пример 2.1 Достигается ли в функции `sum_array_rows` локальность обращений к массиву `a`?

- Да. На вложенном цикле происходит итерация по переменной `j`, которая является вторым индексом. В силу развертки многомерных массивов по строкам, будем идти по памяти последовательно.

```
int sum_array_rows(int a[M][N]) {
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Рис. 22.10: Пример кода

Пример 2.2 Достигается ли в функции `sum_array_cols` локальность обращений к массиву `a`?

Во вложенном цикле происходит итерация по переменной `i`, которая является первым индексом. В силу того, что `i` – первый индекс, будем перемещаться по столбцам, каждый раз перескакивая на большое расстояние в памяти. Таким образом, при выборке данных локальность нарушается.

```
int sum_array_cols(int a[M][N]) {
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Рис. 22.11: Пример кода 2

Пример 3.

Чтобы достичь пространственной локальности, надо, чтобы итерация происходило по наиболее близким адресам. Для этого надо, чтобы второй цикл стал самым внутренним, а цикл, в котором идем по первому индексу, стал самым внешним. $\Rightarrow$ В силу правильного порядка прохода по пространству итерации можно улучшить скорость, которую этот кусок кода будет достигать. Это происходит из-за того, что внутри компьютера сформирована иерархия памяти.

```
int sum_array_3d(int a[M][N][N]) {  
    int i, j, k, sum = 0;  
  
    for (i = 0; i < M; i++)  
        for (j = 0; j < N; j++)  
            for (k = 0; k < N; k++)  
                sum += a[k][i][j];  
    return sum;  
}
```

Рис. 22.12: Пример кода 2

Иерархическая организация памяти, кэширование.

Ряд фундаментальных свойств аппаратуры и ПО:

- Более быстрые устройства хранения стоят дороже, имеют меньший объем и потребляют больше энергии.
- Разрыв в скорости работы между ЦПУ и оперативной памятью увеличивается.
- В хорошо написанных программах демонстрируется хорошая локальность.

Данные свойства выводят на идею *иерархической организации памяти*. Рассмотрим пирамиду из разных уровней запоминающих устройств. Каждый слой, лежащий вверху выступает в качестве кэша для того, что находится ниже. Таким образом получается совмещать минимальную стоимость и максимальный объем с максимальной производительностью.

Основная идея иерархии памяти.

- Для каждого k , более быстрое, меньшее устройство на уровне k выступает как кэш для большего, но более медленного устройства на уровне $k+1$.
- Из-за локальности программа обращается к данным на уровне k гораздо чаще чем к данным на уровне $k+1$.
- Устройство уровня $k+1$ может быть более медленным $\rightarrow$ большего размера, более дешевым.

$\Rightarrow$ Результат: Иерархическая память – большой объем данных, стоит сопоставимо с самой дешевой компонентой, работает с максимальной скоростью.

Основная идея кэша.



Рис. 22.13: Иерархия памяти

Кэш в компьютере работает с блоками, в которые сгруппированы данные. Кэш – небольшая, более быстрая и дорогая память, которая сохраняет подмножество блоков. Допустим в кэш есть 4 позиции. Если нужен какой-то блок, совершается следующая операция: номер блока $\text{mod } 4 = \text{остаток}$. Исходя из этого числа определяется, где искать этот блок в памяти.

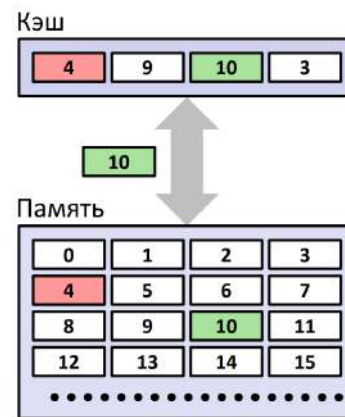


Рис. 22.14: Native Command Queuing

Если блока в кэш нет(промах), блок извлекается из памяти и затем размещается в кэше. Правила размещения определяют, где будет находиться этот блок.

Если одно место оказывается затребовано для размещения нескольких блоков, обращаемся к правилам замещения. **Типы промахов.**

- *Холодные (вынужденные) промахи.* Причина – пустой кэш
- *Промахи из-за конфликтов.*

– Количество мест размещения ограничено (может быть единственным)

Пример: блок i уровня $k+1$ размещается в блоке $(i \bmod 4)$ на уровне k . Те блоки, у которых одинаковый остаток, у будут конфликтовать за одно место, поочередно друг друга выкидывая.

- Прوماхи из-за конфликтов возникают когда несколько блоков размещаются на одном и том же месте.

Пример: запрос блоков 0, 8, 0, 8, 0, 8, ... Будет постоянно вызывать промахи.

- *Прوماхи из-за нехватки емкости.* Причина – используемых блоков (рабочее множество) больше, чем кэш может в себе вместить.

Кэш-память: способы организации.

Кэш памяти представляет собой статическую память, в ячейках этой статической памяти хранятся наиболее часто используемые данные и команды, которые задействованы в ходе выполнения.

Процессор сперва ищет требуемые данные и код в кэше первого уровня(L1), если там не находит, то в L2 и L3, и только потом в оперативной памяти.

Организация кэша (S, E, B). Он разбит на несколько наборов. В каждом наборе есть некоторое количество строк, каждая строка содержит блок. Блок – данные, которые нужны. В строке помимо блока присутствуют служебные данные, которые позволяют определить, находится ли в кэше нужный блок, можно ли пользоваться данными. Совокупность полезной нагрузки и служебных данных формируют строку.

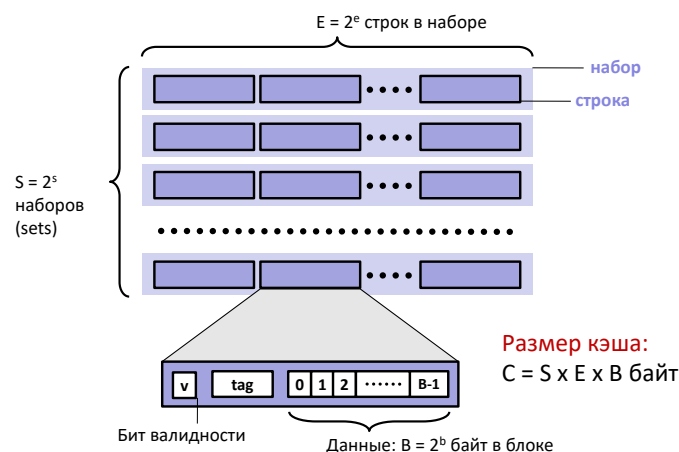


Рис. 22.15: Структура кэша

Чтение данных. При обращении в память вырабатывается исполнительный адрес, по которому происходит выборка данных. Этот адрес бьется на три части:

- самые младшие разряды показывают смещение внутри блока данных;
- средняя часть отвечает на номер набора;
- Старшие разряды – тег.

Определили номер набора, обращаемся к набору, ищем строку в наборе строку, тэг которой совпадет с нашим тэгом. Нашли нужный блок.

Кэш прямого отображения.

Есть несколько видов кэша. Кэш прямого отображения является самым простым. В каждом наборе ровно по одной строке. Каждый блок внутри кэша имеет только одно место, где может храниться.

- Если тэг совпал и данные валидны, то считываем смещение в блоке.
- Если тэг не совпал, эта строка вытесняется из кэша, нужный блок из памяти загружается, поле "тэг" запишется в соответствующее поле внутри строки.

Моделируем кэш прямого отображения. Пусть есть $M=16$ адресуемых байтов, $B=2$ байта в блоке, $S=4$ наборов, $E=1$ блок в наборе

Последовательное чтение	250 МБ/с	Последовательная запись	170 МБ/с
Произвольное чтение	140 МБ/с	Произвольная запись	14 МБ/с
Время доступа	30 нс	Время старта записи	300 нс

Рис. 22.16: Тупики

- Если запросить данные с нулевого адреса, адрес распадется на 3 поля и произойдет "холодный промах". Блок копируется, тэг обновляется.
- Затем происходит попадание

0	[0000] ₂ ,	промах
1	[0001] ₂ ,	попадание
7	[0111] ₂ ,	промах
8	[1000] ₂ ,	промах
0	[0000] ₂	промах

	v	Тег	Блок
Набор 0	1	0	M[0-1]
Набор 1	0	?	?
Набор 2	0	?	?
Набор 3	1	0	M[6-7]

Рис. 22.17: Моделирование кэша прямого отображения

Лекция 23. Работа кэша. Количественные характеристики работы компьютера

Запись в кэш

Многоуровневость памяти несколько усложняет процедуру записи в данных в память.

- Выбор, как будут храниться копии данных на разных уровнях кэша. Несколько копий данных: – L1, L2, L3, оперативная память, диск – В многоуровневом кэше приходится решать, хранить ли копии данных на разных уровнях. В современных компьютерах кэш может быть
 - эксклюзивным, когда на каждом уровне данные хранятся без пересечения. Т.е. на одном из уровней размещена единственная копия данных.
 - инклюзивным, когда считанные из памяти данные размещаются на каждом уровне. Т.е. на каждом уровне имеется своя копия.
- В случае попадания в какой-то уровень кэша, записывать данные можно по-разному:
 - Сквозная запись (пишем в память незамедлительно);
 - Отложенная запись (откладываем до момента вытеснения строки). Более быстрое обращение на запись, но появляется некоторая рассогласованность данных, т.е. в кэше уже есть, а в память еще не записали. В строке кэша нужен бит, говорящий о том, что данные уже различаются
- Промаш:
 - Запись с размещением в кэше. Эффективно когда выполняется несколько записей в последовательные адреса
 - Запись в память напрямую без размещения в кэше
- Типичные комбинации политик управления кэшем
 - Сквозная запись + Запись без размещения
 - Отложенная запись + Запись с размещением

Эти стратегии друг друга дополняют и позволяют сохранить эффективность работы иерархической памяти не только на чтении, но и на записи.

Рассмотрим память современного процессора. На кристалле присутствуют 4 ядра. Ядро – фактически самостоятельный процессор, который состоит из регистров, арифметико-логического устройства, управляющего устройства. Помимо

регистров(самой быстрой памяти) присутствуют еще несколько уровней кэша, которые связаны с конкретным ядром.

L1 и L2 принадлежат отдельному ядру. Заметим, что нарушается один из принципов фон Неймана. На уровне L1 идет явное разделение памяти: память, которая хранит данные (d-кэш) и память, которая хранит инструкции (i-кэш). В L2 код и данные хранятся вместе. L3 – общий для всех ядер. Если в одном ядре произошло изменение, информация должна быть доставлена в кэш других ядер. Время от времени данные из кэша сбрасываются в память.

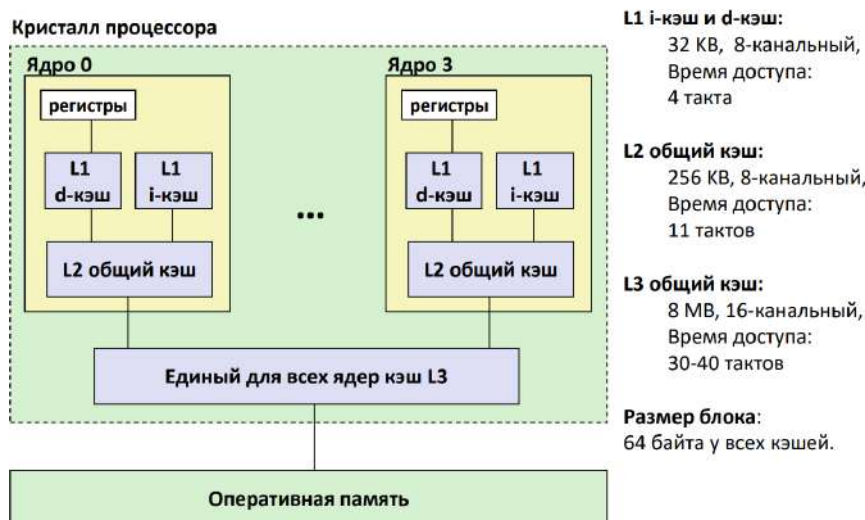


Рис. 23.1: Иерархия кэшей в Intel Core i7

Время обращения при спуске по уровням нарастает. Размер блока на всех уровня кэша одинаковый.

Метрики производительности кэша

- Основная метрика, которая показывает качество работы кэша, завязанная на аппаратную часть – коэффициент промахов. Она показывает долю промахов от общего числа обращений. Показывает выигрыш от реализации комбинаций политик работы кэша.

характерные показатели:

- 3-10% для L1
 - <1% для L2. Чем больше кэш, тем меньше коэффициент.
- Время попадания.

- Длительность извлечения данных из кэша. При обращении в кэш необходимы вспомогательные действия для выяснения наличия и местоположения данных.
- Характерные показатели: 1-2 тактов для L1 и 5-20 тактов для L
- Накладные расходы при промахе. Дополнительное время из-за промаха
 - Обычно 50-200 тактов для обращения к памяти

Из за разницы в два порядка для времени промахов и попаданий небольшое изменение(с 99% до 97%) показателя попаданий уменьшит эффективность работы программы в два раза. Примем, что время попадания – 1 такт, а накладные расходы при промахе 100 тактов. Среднее время доступа к элементу кэша:

- 97% попаданий: $1 \text{ такт} + 0.03 * 100 \text{ тактов} = 4 \text{ такта}$
- 99% попаданий: $1 \text{ такт} + 0.01 * 100 \text{ тактов} = 2 \text{ такта}$

Коэффициент неудач – показатель, связанный с промахами. Более наглядно показывает связь увеличения времени работы программы с количеством промахов.

Дружественный к кэшу код. Программа во многом определяется тем, насколько дружественный кэшу код удастся построить.

- тела вложенных циклов, часто вызывающиеся функции значительно влияют на общее время работы программы.
- минимизировав промахи при обращении к кэшу в теле вложенного цикла, добившись в них локальности временной и пространственной локальности, можно улучшить время программы. – Повторяющиеся обращения к одним и тем же переменным (временная локальность) – Проход по массиву с шагом 1 (пространственная локальность)

Способы оценки производительности компьютеров и программ

Рассмотрим различные аспекты оценки скорости работы и производительности. Это оценка

- Всего компьютера в целом
- Отдельных компонент
- Генерируемого компилятором кода

Измерения производительности зависят не только от свойств аппаратуры, но и от того, какие программы будут выполняться. Для оценки производительности есть специальные программы – *бенчмарки*.

Бенчмарк – задача, служащая эталонным тестом производительности вычислительной системы.

Классификация

– Синтетические / основанные на реальных приложениях(для оценки компьютера в целом, дают более общую картину)

– Микробенчмарк (оценивает характеристики отдельных компонент)

Индустриальные бенчмарки

- SPEC: Standard Performance Evaluation Corporation. Стандартный набор тестов для тех, кто пишет компиляторы.
- Некоммерческая организация, занимающаяся разработкой стандартизированных бенчмарков для оценки производительности и энергоэффективности вычислительных систем. Бенчмарк для ЦПУ - SPEC CPU 2017.
- ScaLAPACK (Scalable Linear Algebra PACKage) – библиотека с открытым исходным кодом, используется для оценки производительности компьютеров с массивно-параллельной архитектурой (например, суперкомпьютеры).
- Решение систем линейных уравнений, обращение матриц, ортогональные преобразования, поиск собственных значений и др.

Аппаратные средства измерения времени.

В процессоре Pentium появился регистр 64-разрядный регистр TSC, подсчитывающий количество выполнившихся тактов с момента включения компьютера. Зная заявленную тактовую частоту, можно пересчитать это число тактов во время выполнения. Опросить

```
section .rodata
format db '0x%08X 0x%08X', 10, 0

section .text
global CMAIN
CMAIN:
...
rdtsc
mov     dword [esp + 8], eax
mov     dword [esp + 4], edx
mov     dword [esp], format
call    printf
...
```

Рис. 23.2: Измерение времени

регистр можно командой rdtsc. Инструкция rdtsc считывает значение регистра TSC и заносит его в пару регистров EDX:EAX

Выполнение команды rdtsc может быть недоступна пользователям на некоторых системах.

Оценка производительности памяти: синтетический бенчмарк (контрольная задача)

Хотим измерить длительность обращения в память, чтобы можно было оценить пропускную способность системы памяти. Выполняется оценочная функция, внутри

```
/* Оценочная функция */
void test(int elems, int stride) {
    int i, result = 0;
    volatile int sink;

    for (i = 0; i < elems; i += stride)
        result += data[i];
    sink = result;
}

/*
Запуск test(elems, stride) и вычисление пропускной способности
при чтении (МБ/с)
*/
double run(int size, int stride, double Mhz) {
    double cycles;
    int elems = size / sizeof(int);

    test(elems, stride);           /* разогрев кэша */
    cycles = fcy2(test, elems, stride, 0); /* вызываем test(elems, stride) */
    return (size / stride) / (cycles / Mhz); /* переводим кол-во циклов в МБ/с
*/
}
```

Рис. 23.3: Измерение длительности обращения в память

которой присутствует цикл `for`. Внутри этого цикла мы с некоторым шагом проходим по массиву `data`. Шаг и размер массива могут варьироваться.

Вызов оценочной функции:

- Для того, чтобы разогреть кэш, выполняется функция `test`.
- Далее из функции-болочки будем вызывать функцию `test`, указывая ей количество элементов, которые нужно перебрать и шаг, с которым будет проходить.
- Пересчитывается количество выполнившихся циклов в то время, которое потребовалось для извлечения данных из памяти. Будут учитываться накладные расходы, связанные с вызовом функции, накладные расходы на выполнение цикла.
- После выполнения бенчмарка, варьируя размер и шаг, с которым мы ходим по массиву, получаем следующую картинку.

При нарастании данных пропускная способность растет. При достаточном количестве данных они будут храниться в кэше L1, и пропускная способность вырастет. Она будет находится на это уровне, пока размера кэша будет достаточно. Когда начнутся промахи в кэш, пропускная способность будет падать на порядки.

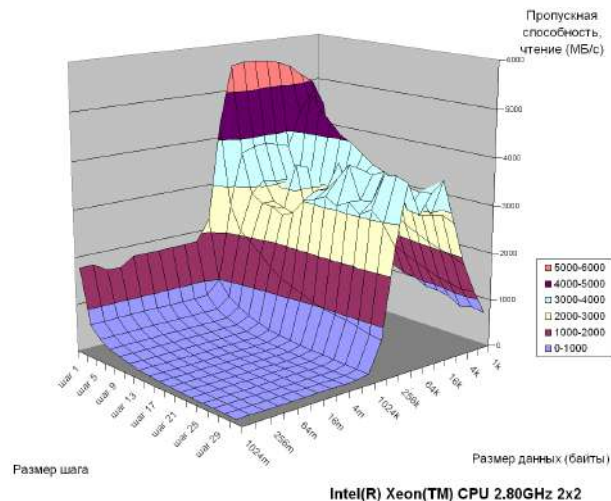


Рис. 23.4: Зависимость пропускной способности от размера шага и размера данных.

При увеличении шага ухудшается локальность, и пропускная способность начинает деградировать.

Рассмотрим картинку, сделанную на более современном процессоре. Простой, ранее

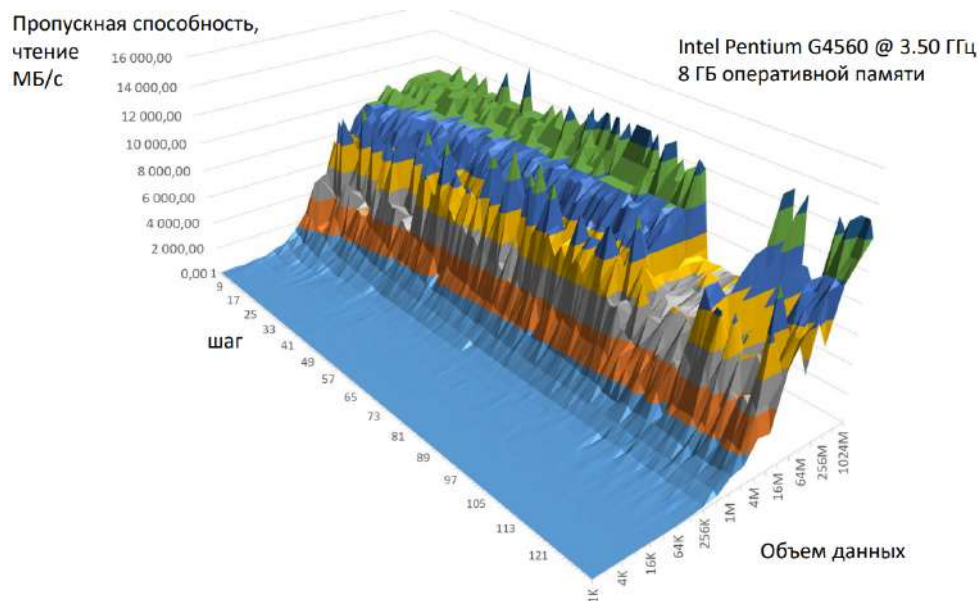


Рис. 23.5: Зависимость пропускной способности от размера шага и размера данных.

рассмотренный бенчмарк на настольных компьютерах и серверах последних поколений показывает «парадоксальные» графики пропускной способности памяти : пропускная способность изначально крайне мала и начинает расти с увеличением объема данных. Такой эффект появился из-за того, что некоторые факторы были не учтены.

- В современных компьютерах появился механизм проска тактов. Пропуск тактов

процессора (дросселирование тактов, CPU throttling) – механизм снижения тактовой частоты процессора.

- Энергопотребление процессора:

$$P \approx C \cdot V^2 \cdot f,$$

где C – электр. емкость, V – питающее напряжение, f – частота

- Снижение частоты снижает энергопотребление, а следовательно, и энерговыделение.

⇒ Защита от перегрева. Чтобы предотвратить физическое сгорание, встроенные механизмы сбрасывают тактовую частоту.

⇒ Экономия энергии, когда компьютер не занят выполнением программ, требующих всех вычислительных возможностей.

- Современные процессоры способны динамически менять свою тактовую частоту, но и по умолчанию дросселируют такты.
 - Снижение и восстановление частоты происходит не мгновенно, оно занимает сотни миллисекунд
 - Быстро отработавший бенчмарк не успевает дождаться восстановления штатной частоты процессора

Когда мы начинаем выполнять тест, разогрели кэш, но дросселирование тактов еще не вывело нас на нужную тактовую частоту. Чтобы с этим справиться, можно наращивать время разогрева, либо можно отключить дросселирование (см. рис. 23.6).

Пропуск тактов отключен. На графике исчезли «зубцы» и виден правильный искомый спад при наращивании шага. Но «правильный» перепад в пропускной способности памяти так и не появился. Это может объяснить тем, что остались и другие неучтенные факторы. Эти факторы определяют микроархитектуру процессора.

Микроархитектура процессора, ее связь с другими архитектурными уровнями.

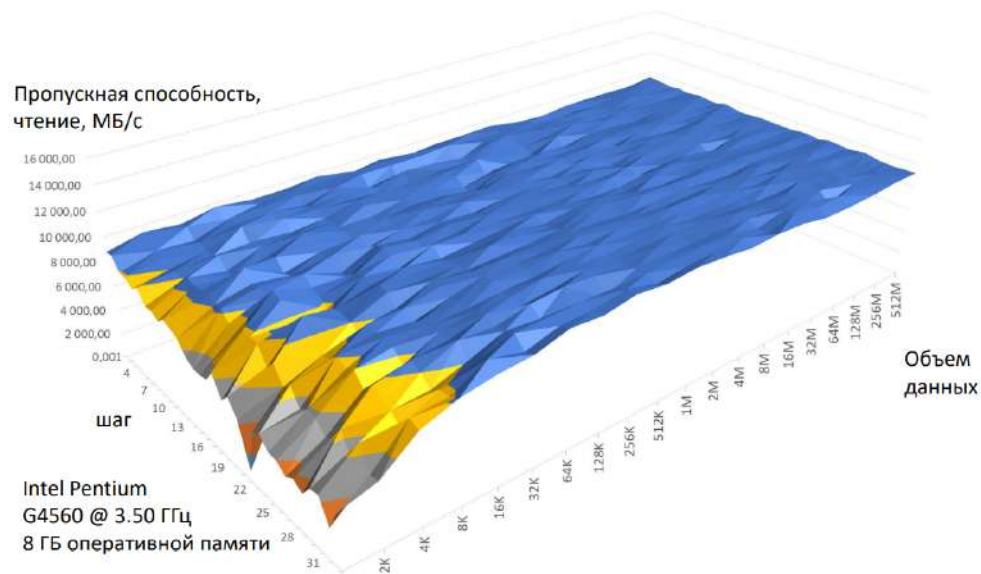


Рис. 23.6: Зависимость пропускной способности от размера шага и размера данных в случае отключенного дросселирования.

Архитектура набора команд – интерфейс, который определяет, что могут делать программы

Микроархитектура определяет, как это будет выполняться.

Разработчики аппаратуры стремятся сохранить совместимость ISA и развить возможности микроархитектуры:

- (Обратная) совместимость – возможность выполнять на новых моделях процессора ранее написанные программы без каких-либо изменений



Рис. 23.7: Микроархитектура процессоров

- ISA можно дополнить новыми командами
- Развитие микроархитектуры улучшает важные для пользователя свойства процессора
 - Производительность
 - Энергопотребление
 - Безопасность

– ...

Изменения микроархитектуры не влияют на архитектуру набора команд. При этом обеспечивается совместимость, т.е. на разных процессорах с разной микроархитектурой можно запускать одни и те же программы.

Упреждающая выборка данных

Используемый бенчмарк обращается за данными строго последовательно, хоть и с различным шагом. Таким образом, для процессора становится возможным предсказывать, за какими данными будут обращения в будущем (упреждающая выборка).

Упреждающая выборка – микроархитектурный механизм для повышения производительности за счет извлечения инструкций и/или данных из памяти в кэш до того, как они фактически потребуются. В идеальном случае сможем получать пиковую производительность всех архитектурных решений на уровне шины, организации динамической памяти и т.д. В той модели процессора, на которой запускался бенчмарк, аппаратно реализовано 4 различных алгоритма упреждающей выборки.

⇒ При таком регулярном обращении, когда мы просто проходим по массиву, померять пропускную способность очень тяжело.

Пропускная способность vs. Латентность

Попробуем измерить другую характеристику системы памяти, которую труднее «спрятать» микроархитектурными решениями.

Пропускная способность памяти – объем данных, который может быть передан из памяти за единицу времени. Если построить аналогию с трубопроводом – это ширина трубы.

Микроархитектурные решения стремятся приблизить эту характеристику к пиковой величине

- Совокупность решений: расслоение памяти, многоканальность, многоуровневое кэширование, упреждающая выборка, ...
- Хорошо работают при «предсказуемых» обращениях в память, таких как последовательный проход .
- Ограничениями становятся пропускные способности шин и модулей памяти.

Латентность (задержка) – время, которое затрачивается на чтение из памяти минимальной порции данных (одного слова). В примере с трубой – это длина трубопровода

Для оценки требуется обращаться в память по случайным адресам

⇒ Заметим, что пропускная способность и латентность – относительно независимые друг от друга характеристики

Оценка латентности памяти на синтетическом тесте.

Нужно поменять синтетический микробенчмарк, который будет использоваться. Вместо массива берется однонаправленный список, в котором будет находиться поле payload, к которому мы будем обращаться, и указатель, который ведет на следующий элемент списка.

Измеряется время, затрачиваемое на проход по однонаправленному списку

1. Элементы списка размещаются в памяти последовательно
2. Список «перемешивается», каждый элемент ссылается на другой в произвольном порядке. Это будет гарантированно приводить к промахам в кэш и мешать алгоритмам упреждающей выборки.
3. Для снижения погрешностей измерения времени по небольшим спискам проходят несколько раз

Попытаемся измерить время доступа для списков разных размеров

1. Проход по «перемешанному» списку
2. Последовательный проход по списку
3. Последовательный проход с отключенной упреждающей выборкой

```
typedef struct Node Node;  
  
struct Node {  
    uint64_t payload;  
    Node* next;  
};
```

а)

```
for (Int it=0; it<iters; ++it) {  
    Node* node = start_node;  
    while (node) {  
        node = node->next;  
    }  
}
```

б)

Рис. 23.8: а) Время прохода по однонаправленному списку. б) Время доступа для списков разных размеров

Задача – сделать измерения для случая включенной упреждающей выборки и для случая выключенной упреждающей выборки.

Отключение упреждающей выборки.

Каждая модель процессора может иметь свои микроархитектурные особенности. Управлять ими через специализированные команды – крайне неудобно для разработчиков процессора, придется постоянно дорабатывать ISA. Проще организовать управление через специальный набор управляющих и статусных регистров (Model specific register, MSR)

Моделезависимые регистры (MSR) – это еще одно логическое пространство, с которым можно работать. Если основное пространство памяти – физическая память, другое пространство – порты ввода/вывода, третье пространство – Моделезависимые регистры.

- Обращение к регистру по его номеру
- Для работы с регистрами есть команды чтения/записи – команды RDMSR и WRMSR. Они могут выполняться только операционной системой
- Разработчики процессоров имеют право свободно менять состав и поведение регистров от модели к модели
- На некоторые MSR доступна документация
- В Linux обратиться к MSR можно через специальные утилиты, названия которых повторяют имена соответствующих машинных команд

У используемой автором слайдов модели процессора упреждающая выборка управляется MSR 0x1a4.

Если выполнить все подготовительные действия и програть новый бенчмарк, увеличивая размер списка, получим похожие на теорию три графика (см. рис. 23.9).

Для последовательного прохода по списку видно, что только когда данные перестают попадать в кэш, появляется небольшой прирост. Дальше упреждающая выборка позволяет сохранять неплохую латентность.

Если обращаться в память в случайном порядке, видно, как происходит рывок и время доступа увеличивается. Время стабилизируется пока данные находятся внутри кэша 3 уровня. Затем время увеличивается до 100 нс.

Последовательный проход с отключенной предвыборкой оказывается посередине.

Выводы.

- Измерение даже простых характеристик компьютера требует учета многих факторов, без гарантий, что про все эти факторы найдется информация в открытых источниках (зеленая линия на графике). Эти микроархитектурные решения могут исказить или выворачивать наизнанку некоторую теоретическую картину.

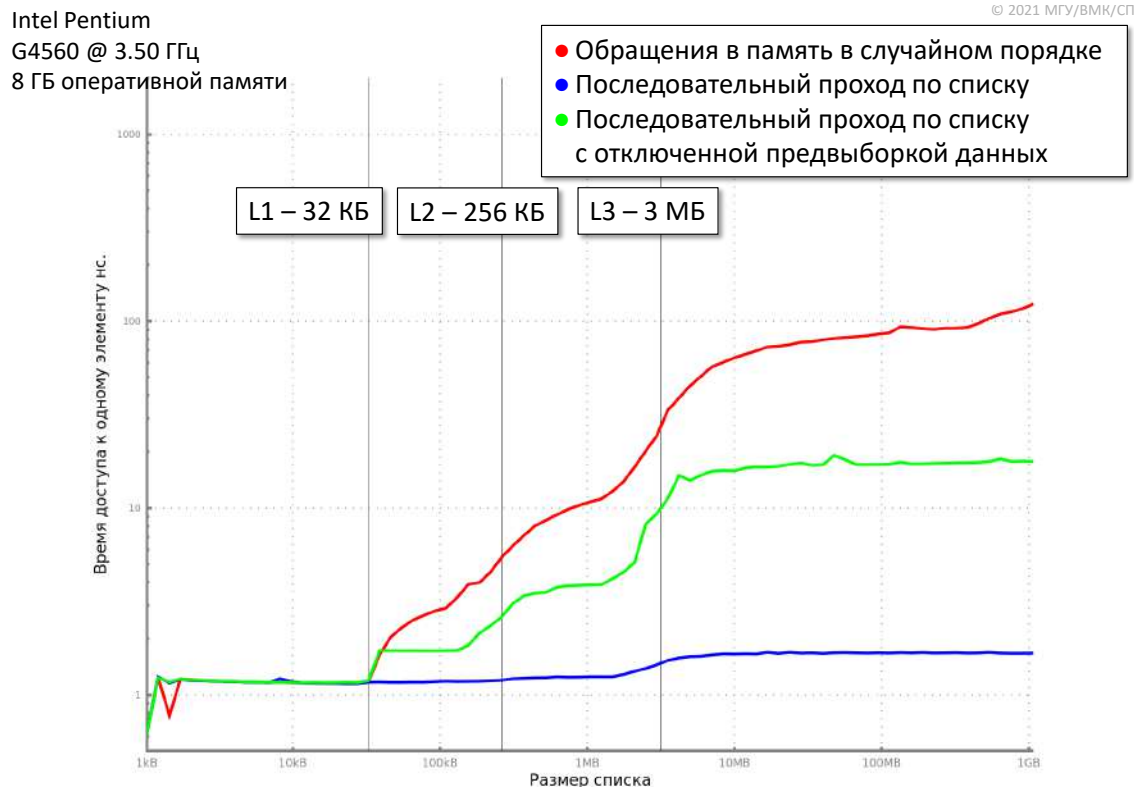


Рис. 23.9: Зависимость времени доступа от размера списка для случая, когда прпуск тактов отключен

- Отключены только пропуск тактов и упреждающая выборка. Последовательные обращения памяти замедлились на порядок, но все еще на порядок быстрее, чем обращения к произвольным адресам.
- Если в разрабатываемой программе несколько теряется требуемая характеристика локальности, современный компьютер способен этот огрех подчистить и все равно выдавать хорошую скорость работы.
- Регулярные, последовательные, обращения к памяти могут сохранять высокую пропускную способность и латентность даже при потере локальности, благодаря комплексу дополняющих друг друга микроархитектурных решений (синяя линия на графике)
- Латентность памяти скрыть гораздо тяжелей по ряду причин
 - В частности, потому что память физически удалена от процессора
 - Пример: 1ГГц, длительность 1 такта
 - 1 нс. За это время световой сигнал в вакууме успеет распространиться примерно на 30 см.

- Скорость распространения электрического сигнала в меди – 60-90% скорости света
- Необходимо передать в память команду на считывание, строб строки, строб столбца, получить обратно данные, ...
- Если данные не умещаются в кэш, то длительность произвольных обращений к ним микроар

Микроархитектурные решения

- Конвейер команд работающих конвейеров
 - Суперскалярная архитектура
 - Несколько параллельно
- Многоуровневое кэширование
 - Отдельные кэши для кода и данных
- Многоядерность
- Предсказание передачи управления (переходов)
- Упреждающая выборка кода и данных
- Микрокод
- Внеочередное выполнение команд

Конвейерная обработка команд, проблемы конвейерной обработки

Конвейер – совмещение разных действий в один момент времени. Аналогия с эскалатором: можно перевозить одного человека за раз, либо можно сделать много ступенек, и на каждой ступеньке будет стоять один человек. Вдремя доставки из одной точки в другую нельзя, но можно каждый временной интервал (меньше времени доставки) получать выходящего человека. Чем меньше ступени(больше людей), тем больше пропускная способность.

- Общая для различных предметных областей методика
- Длительность обработки неизменна или несколько увеличивается
- Увеличение пропускной способности

Упрощенная схема работы конвейера x86 состоит из пяти этапов(их может быть больше в реальных процессорах):

1. (Fet) Извлечение инструкции из памяти
2. (Dec) Декодирование, обновление EIP
3. (D-F) Извлечение данных, подготовка операндов к выполнению операции
4. (Exe) Непосредственное выполнение операции
5. (Wrt) Запись результата

При реализации идеи конвейерных вычислений, можно столкнуться с определенными проблемами и ограничениями физического уровня.

Организация конвейерных вычислений В отсутствии конвейера все вычисления проходят некоторую логическую схему, на что требуется некоторое количество времени, после чего сохраняются внутри некоторого регистра на некоторое время. Задержка работы для такой системы 320 пс. Полусим пропускную способность 3.12 GIPS (Giga Instructions Per Second).



Рис. 23.10: Обычная схема для вычислений

Раздробим логическую схему на равные куски. При делении на 3 части каждый такой кусочек будет обрабатывать за 100 пс, но потребуются промежуточные регистровые станции (еще 20 пс). За счет дробления появляется необходимость в промежуточном сохранении результатов. Общая задержка возрастает до 360 пс. Но каждые 120 пс на выходе получаем очередную завершившуюся команду. пропускная способность выросла до 8.33 GIPS.

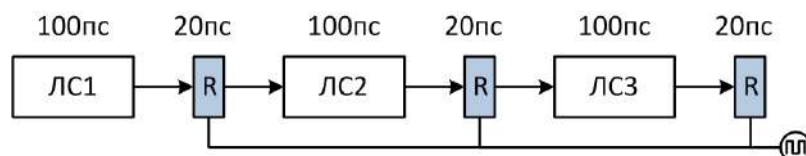


Рис. 23.11: Схема для конвейерных вычислений

Неоднородность ступеней конвейера. Всегда удастся разбить конвейер на равные по времени работы части. Выравнивать время работы придется по самой длинной

ступени(см. рис. 23.12). В силу того, что на каждом такте теперь требуется по 170 секунд, задержка будет 510 пс, а пропускная способность уменьшится до 5.88 GIPS.

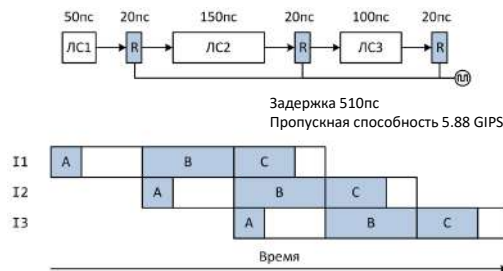


Рис. 23.12: Конвейер с неоднородными ступенями

Проблемы конвейерной организации.

- Опустошение конвейера: сброс промежуточных результатов уже выполняющихся в конвейере команд.
 - Современные процессоры могут содержать конвейеры длины 15 и более (P4 Prescott – 31 стадия конвейера)
 - Изменение EIP сбрасывает промежуточные результаты выполнения следующих инструкций
 - Возникновение исключительной ситуации при обращении к памяти или при выполнении операции над данными
- Зависимости по данным между «близко» расположенными командами
- Остановки из-за обращения к памяти

Опустошение конвейера при передаче управления. При последовательной выборке команд выбрали, декодировали и дальше выполняем сравнение, затем условную передачу управления. Только когда понятно, куда перепрыгиваем или не перепрыгиваем, в конвейере уже будут находиться извлеченные, декодированные и выполняющиеся следующие четыре команды. Когда становится понятно, что нужно перепрыгнуть вперед, все эти четыре команды нужно выбросить. Стараемся предсказать, куда будет переход. Своего рода техника кэширования.

Приостановка конвейера. Некоторые команды могут обращаться к памяти. Если происходит промах, надо долго ждать, когда данные будут загружены. Пока они не поступили в процесс, конвейер останавливается.

Бывает, что в одной команде нужно обновить регистр, а в другой команде мы должны им пользоваться. Внутри аппаратной реализации процессора сделать bypass заготовки, в которые записываются текущее значение до записи. Оно одновременно уже идет внутрь

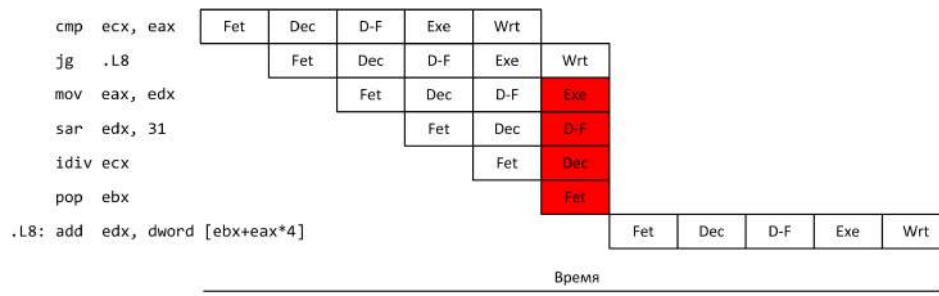


Рис. 23.13: Опустошение конвейера при передаче управления

схем, которые начинают выполнять следующую команду. Это приводит к усложнению внутренней логики процессора.

Двойной конвейер. В середине 90-х годов было предложено сделать несколько экземпляров конвейеров. Если в очереди выборки оказывалась последовательность из команд, часть которых могла пойти по U-конвейеру, а другая по V-конвейеру, то после этапа выборки можно было их параллельно пустить на выполнение. Быстро работающая выборка могла одновременно эти два конвейера заполнять потоком команд, если потомки позволяли это делать.

Двойной конвейер с общим этапом выборки команд

- (Fet) Извлечение инструкции из памяти
- (Dec) Декодирование, обновление EIP
- (D-F) Извлечение данных
- (Exe) Непосредственное выполнение операции
- (Wrt) Запись результата

Возможности конвейеров неравноценны.

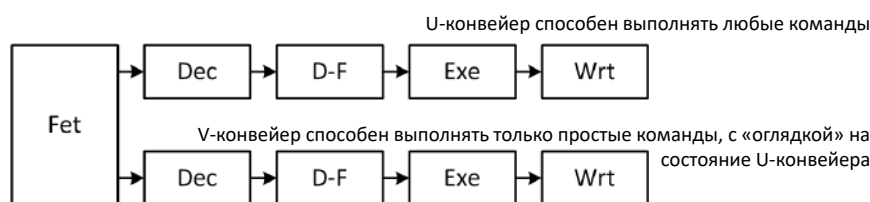


Рис. 23.14: Двойной конвейер

Суперскалярная и VLIW архитектуры. Можно увеличить число функциональных устройств, тех компонентов логических цепей, которые выполняют нужные действия на этапе выполнения. Таким образом получим суперскалярную архитектуру, когда доходим до этапа получения операндов, после чего поток команд расходится на несколько рукавов.

- Многие операции над данными не могут быть реализованы за один такт.
- В суперскалярной архитектуре выполняющиеся команды на стадии операции расходятся по нескольким функциональным устройствам (ФУ).

- За распределение (планирование) потока команд по ФУ отвечает аппаратура процессора.

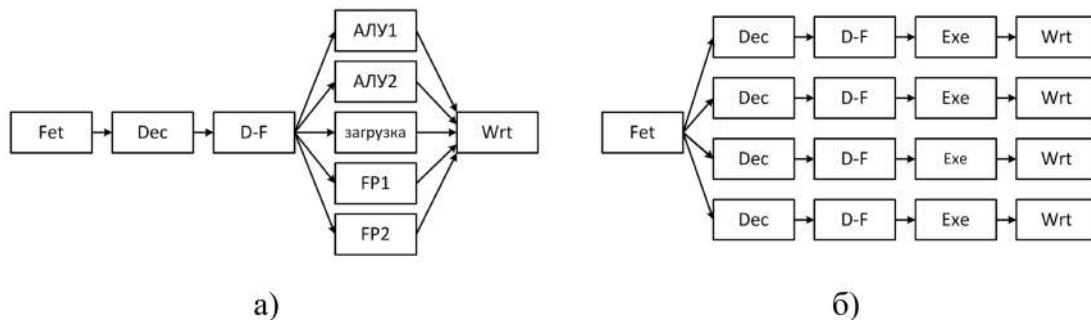


Рис. 23.15: а) Суперскалярная архитектура; б) VLIW архитектура

Крайняя форма такого распределения команд по разным линиям конвейера – VLIW (Very Long Instruction Word) архитектура. Во VLIW-архитектуре каждая команда состоит из нескольких микрокоманд, независимо друг от друга, выполняющих операции над данными. Конвейер почти полностью дублируется, двоичный код команды бьется на куски, которые являются самостоятельными командами. Эти куски проходят по линиям из логических цепей.

За то, что бы команды действительно не конфликтовали между собой и могли выполняться параллельно, отвечает не процессор, а компилятор.

Суперскалярная архитектура и прерывания Помимо сложностей с планированием, есть еще сложности. Время прохождения функциональных устройств может различаться в зависимости от операций.

Поэтому может получиться так, что следующее в потоке команд действие с целыми числами уже завершилось, а команда с плавающей точкой, которая шла раньше в очереди, еще не закончилась. Получим рассогласованное состояние программы.

- Усложнение внутреннего устройства процессора может повлиять на ход выполнения
- Выполнение команды может вызвать исключительную ситуацию (сбой), что приведет к выбросу прерывания (например, с целью восстановления состояния программы)
- Целые числа и числа с плавающей точкой обрабатываются разными ФУ, за различное время, например $t_1 < t_3$
- Прерывание, обусловленное сбоем в x87 команде может быть доставлено для обработки уже после того, как следующая команда с целым числом записала свой результат
- Необходима синхронизация доставки прерывания
- Большая часть команд x87 синхронизирует обработку исключений от ранее выполнявшихся команд x87

Пример. В первой команде мы грузим из памяти целое число. Оно конвертируется внутри функционального устройства представления числа с плавающей точкой, укладывается на стек регистров. Счетчик увеличивается на 1, а затем вычисляется квадратный корень того, что было положено на стек регистров.



Рис. 23.16: Варианты написания кода

Может произойти сбой при конвертации в число с плавающей точкой. Или может произойти ситуация, когда не хватает регистров (уже увеличили счетчик на 1). Тогда происходит доставка прерывания. Задействуются аппаратные механизмы процессора, которые стопарят текущее выполнение и перекидывают управление на специальные предопределенные функции.

Такое прерывание не пытается воостановить работу программы из-за того, что неизвестно, зафиксирован ли результат переменной `cnt`.

Для того, чтобы исправить это, поменяем порядок команд так, чтобы `cnt` увеличивалось только после работы команды `FSQRT`, потому что последовательная выполнение команд будет синхронизировать доставку прерывания.

Либо можно выполнить специальную команду `FWAIT`, которая гарантированно зафиксирует доставку всех исключений и требований прервать работу, обусловленных какими-то событиями в процессора `x87`.

Производительность: особенности современной архитектуры Intel64 и не только

Рассмотрим основные приемы и механизмы, направленные на улучшение производительности.

- Многоядерность
- Многоуровневый кэш, отдельный кэш кода и данных
- Суперскалярная архитектура – такая архитектура, когда несколько функциональных устройств, одновременно обслуживающих этап выполнения
- Векторные команды. Содержимое какого-нибудь регистра трактуется как некоторая совокупность однородных данных. Она реализована в таких расширениях набора команд как MMX, 3DNow!, SSE, ...
- Внеочередное выполнение команд

- Предсказание переходов
- Внутреннее RISC-ядро. Современный процессор, не смотря на то, что он сохраняет всю системы команд, имеет внутри еще один процессор, который отвечает за выполнение того, что туда подается.
- ...

CISC и RISC архитектуры.

RISC (Reduced Instruction Set Computer) – сокращенный набор команд

CISC (Complex Instruction Set Computer) – сложный набор команд.

- Исторически CISC предшествовал RISC
 - PDP-11 → VAX / CISC, 1977 г.
 - MIPS, SPARC / RISC, конец 80-х

Нарастивать тактовую частоту становилось все сложнее. Поэтому было принято решение попытаться получить максимальную простоту команды. Электрические цепи, реализующие ее, смогут работать на больших частотах. Таким образом простота внутреннего устройства позволит улучшить показатели на выходе .

Принципы:

- Простые операции
 - Ограниченный набор простых команд (например, нет деления)
 - Команда выполняется за один такт
 - Фиксированная длина команды (простота декодирования)
- Конвейер. Если сформировать конвейер исполнения, он будет просто устроен.
 - Каждая операция разбивается на однотипные простые этапы, которые выполняются параллельно
 - Каждый этап занимает 1 такт, в т.ч. декодирование
- Регистры
 - Много однотипных взаимозаменяемых регистров (могут использоваться и для данных, и для адресации). Их можно использовать для хранения чисел, с которыми работаем, и для произвольных арифметических операций.
- Модель работы с памятью. В действиях с какими-то числами сокращается число типов операндов. Если хотим работать с памятью, доступны всего две команды.
 - Отдельные команды для загрузки/сохранения в память Альтернативное название RISC-архитектур
 - Load/Store-машина
 - Команды обработки данных работают только с регистрами. Становится гораздо проще оценить длительность команды.

- Результат
 - Устройство ядра процессора становится проще
 - * Проще наращивать частоту процессора
 - * Меньше энергопотребление
 - Несмотря на то, что число команд, требуемых для реализации какой-то функциональности становится больше, суммарно они все равно будут выполняться гораздо быстрее.
 - По сравнению с CISC объем кода (число команд) при реализации того же алгоритма на RISC-машине будет больше
- Сложность оптимизаций перенесена из процессора в компилятор
 - Производительность сильно зависит от компилятора

RISC-V – архитектура RISC-микропроцессоров с открытым исходным кодом.

Существует две версии процессора и, соответственно, два набора команд : 32 и 64 разряда.

Характеристики RV 32 ISA

- Машинное слово – 32 разряда. Такой размер имеют: адрес, регистры и размер команды.
- 32 регистра общего назначения x0 – x31. Счетчик команд pc. Один из регистров x0 «запаян» всегда хранить 0, он не реагирует на записи.
- Базовый набор команд над целыми числами RV32I – 47 команд. Этот набор может расширяться. Например, команды умножения и деления являются дополнительным набором. Только простые операции: сложение, вычитание, логика, сдвиги, сравнение, передача управления.
- Большинство команд трехадресные. Для компилятора такая форма команд является наиболее удобной.
- Умножение, деление и взятие остатка вынесены в расширение набора команд RV32M, который необязателен для реализации, как и другие расширения.
- Флагов состояний нет, аппаратной поддержки стека нет, как и многих других излишеств, например, команды MOV. Нет команды пересылки из регистра в другой регистр.

Отличия RISC-V от x86.

- **x86.** Система кодирования достаточно сложная. Есть код операции (1-3 байт), который может предвостаться префиксами (+4 байта). Затем кодируются операнды (2 байта). Если один операнд представляет из себя что-то лежащее в памяти, в адресном коде может потребоваться закодировать сещение (+4 байта). Если еще один операнд – что-то лежащее в теле команды, нужно еще 4 байта.

⇒ Таким образом длина команды от 1 до 15 байт.

- **RISC-V.** – всегда 32 разряда.

Из памяти извлекается 4 байта, после чего увеличивается счетчик команд. Затем переход на выборку следующей команды.

Присутствуют всего 4 простых формата : R-type, I-type, S-type, U-type.

Нумерация разрядом идет справа на лево. Всего 7 разрядов.

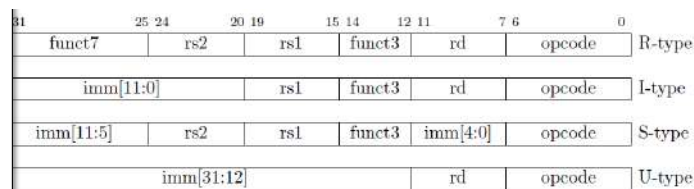


Рис. 23.17: RISC-V

Декодирование максимально простое.

Проблемы:

- сложение регистра с 32-х разрядной константой
- обращение к произвольному месту памяти

Длинные константы, которые не умещаются внутри команды, приходится бить на 2 куска: отдельно загружаются старшие 20 разрядов, а затем используется непосредственно закодированная величина immediate operand, в котором указывается 12 разрядов(см. рис 23.18).

```
LUI x5, 0x12345 ; загружаем старшие 20 разрядов x5
ADDI x5, x5, 0x678 ; заполняем младшие 12 разрядов x5

LW x5, 0xc(x6) ; загружаем в x5 слово из памяти
                ; по адресу x6+0xc
                ; константа кодируется 12 бит
                ; как получить произвольный базовый
                ; адрес – см. пример выше
```

Рис. 23.18: Сложение регистра с 32-х разрядной константой

- Вызова функций в привычном виде не существует.
- Вместо стека для адреса возврата используется явно заданный регистр x1.
- Регистры и флаги можно не сохранять а использовать в рамках одной команды.

- Безусловный переход

```
JAL x1, 0x12345 ; jump and link [register]
                ; x1 ← pc + 4
                ; pc ← pc + ОП2 (20 разрядов)
JALR x1, x28, 0x20 ; x1 ← pc+4
                ; pc ← ОП2 + ОП3 (12 разрядов)
```

- Сравнения

```
SLT[U] x28, x29, x30 ; if (ОП2 <[u] ОП3) {
                    ; ОП1 ← 1 } else {
                    ; ОП1 ← 0
                    ; }
```

- Условные переходы

```
BEQ x28, x29, 0x10 ; if (ОП1 == ОП2) {
                  ; pc ← pc + (2 * ОП3)
                  ; } // ОП3 – 12 разрядов
```

Рис. 23.19: Сравнения и передача управления

Конвейер RISC-V

В каждой команде присутствует операция над данными из регистров. Даже при обращении в память происходит сложение. Таким образом выполнение команды загрузки идеологически мало чем отличается от сложения.

- – Если происходит обращение в память, исполнительный адрес получается сложением базового регистра и непосредственно закодированного значения

Этапы конвейера похожи на то, что мы видели в x86. Принципиально они даже проще.

Этапы конвейера RISC-V:

1. Извлечение команды из памяти, увеличение pc на 4
2. Декодирование и извлечение значений операндов – регистров и непосредственно закодированной константы
3. Выполнение операции
4. Обращение к памяти, если только команда с памятью работает
5. Запись результата в регистр.

В силу такой простоты и открытости исходного кода, архитектура RISC-V получила распространение.

История развития x86 • 4004 – ноябрь 1971. 4-битный микропроцессор. Первый в мире коммерчески доступный однокристалльный микропроцессор.

- 8008 – апрель 1972. 8080 – апрель 1974. 8-битные процессоры.
- 8086 – 1978. Размер слова – 16 бит, ширина адресной шины – 20 бит. Адреса вычисляются с использованием сегментных регистров.
- 80186 – 1982. Добавлено несколько новых инструкций.
- 80286 – 1982. Ширина адресной шины – 24 бита, добавлено устройство контроля памяти – MMU. Процессор мог переключаться между двумя режимами – реальным и защищенным.
- 80386 – октябрь 1985. Снят с производства в 2007. Размер слова – 32-разряда. Процессор мог переключаться между тремя режимами – реальным, защищенным, виртуальным. Адресуемая память – 4 ГБ.
- ...
- Intel Xeon E7-8870 – 2011, 10 ядер, до 8 ЦПУ в системе. 2.4 (2.8) ГГц; 2.2×10^9 транзисторов; системная шина QPI: 6.4 ГТ/с; обращение к памяти одного процессора: 32 ГБ/с (4×8 ГБ/с), адресуемая память – 4 ТБ; Кэш L1: 640КБ L2: 2.5МБ, L3: 30МБ.
- ...
- Intel Xeon 8180M – 2017, 28 ядер, до 8 ЦПУ в системе. 2.5 (3.8) ГГц; транзисторов в аналоге от AMD 19×10^9 ; системная шина UPI: 10 ГТ/с; обращение к памяти одного процессора: 120 ГБ/с (6×20 ГБ/с); адресуемая память – 1.5ТБ. Кэш L1: 896+896КБ, L2: 28МБ, L3: 38.5 МБ.
- ...

Развитие системы команд в архитектуре x86

Параллельно с развитием процессоров развивались технологии, которые позволяют их строить. В силу этого сохранялся экспоненциальный рост числа транзисторов и линейно росла их сложность.

Последние 23 расширения команд в период 2011-2016 преимущественно представляют собой системные команды, которые отвечают не столько за производительность, сколько за вопросы безопасности, изоляцию программ и данных друг от друга.

Помимо этого появилась возможность запускать несколько различных операционных систем параллельно.

В последние годы не происходило сильных качественных изменений во внутреннем устройстве процессора. Происходил количественный рост: увеличивалось число ядер, размер кэша.



Рис. 23.20: Развитие системы команд

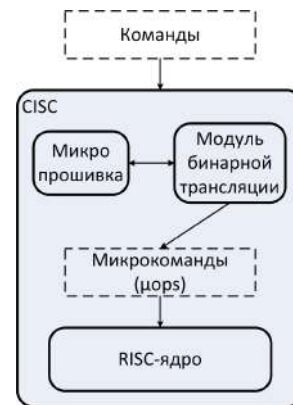


Рис. 23.21: Путь команды

Микрокод. По мнению исследователя Andrew Baumann на протяжении нескольких лет аппаратное устройство процессоров Intel принципиально не менялось, для поддержки новых команд обновлялась его «прошивка» – микрокод.

На входе получаем команды привычной нам архитектуры с новыми дополнениями. Они попадают в модуль бинарной трансляции, который их переводит в поток более мелких и простых команд внутреннего RISC-процессора. Управляется это все служебной программой. Процессор рекурсивно реализовал идею полноценного компьютера.

Именно принципы RISC-архитектуры позволяют развивать и улучшать характеристики, сохраняя простоту его внутреннего устройства.

Микрокод — программа, реализующая набор инструкций процессора

Современные процессоры x86 динамически, на лету, транслируют команды x86 в микрокоманды (μops), которые выполняются внутренним RISC-ядром

- Подход применяется не только в x84 / Intel
- Удобно исправлять ошибки в поведении сложных команд и распространять эти исправления, т.е. новую версию микрокода (пакет linux-firmware)
- Удобно расширять набор команд

Идея микрокода была предложена еще на заре эпохи компьютеров

Лекция 24. Системное управление работой современного компьютера

Режимы работы современного процессора архитектуры Intel64/AMD64, загрузка.

Современный 64-х разрядный процессор представляет собой целую совокупность различных процессоров. С расширением набора команд появлялись различные режимы работы, а старые режимы сохраняются.

Изменения хода выполнения называются *режимами работы* (см. рис 24.1). Начальная

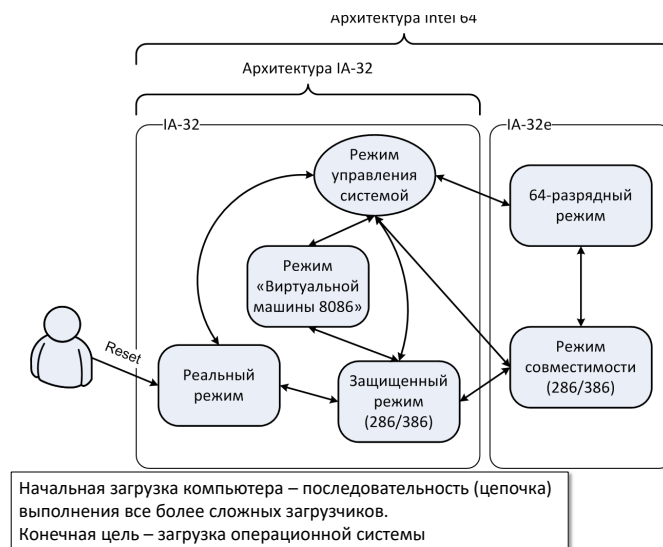


Рис. 24.1: Архитектура 64-х разрядного процессора

загрузка компьютера – последовательность (цепочка) выполнения все более сложных загрузчиков. Конечная цель – загрузка операционной системы.

- Когда современный компьютер включается, его работа начинается в реальном режиме, когда он представляет собой 16-ти разрядный компьютер начала 1980-х годов.
- Затем можно перейти в защищенный режим, который соответствует тому, что изучалось в данном курсе (32-х разрядный режим) .
 - Если остановиться в 32-х разрядном защищенном режиме, для некоторых запускаемых программ можно демонстрировать возможности аппаратуры так, будто они работают на 16-ти разрядной машине – это режим "виртуальной машины 8086"
- Из защищенного режима можно перейти в полноценный 64-х разрядный режим через промежуточный режим совместимости.

- В режи управления системой процессор может переключаться в зависисмости от событий. Например, при подсоединении флэшки происходит замыкание цепей и процессор переходит в этот режим. Он считывает некоторый встроенный во флешку код, который потом позволит с ней работать, его устанавливает, затем возвращается в обычный режим работы.

Прохождение по разным режимам – задача загрузчика (для x86 он называется BIOS).

Переход из одного такого состояния в другое – это некоторая согласованная последовательность действий, которая выражается в разипи определенный значений в некоторые управляющие регистры. Только нужно последовательность действий переведет работающий процессор из одного согласованного состояния в другое.

Встраиваемое ПО, обеспечивающее начальную загрузку компьютера: BIOS, ACPI, UEFI

Схема загрузки IA-32. При включении питания некоторую последовательность действий выполняет сам процессор. В первую очередь выполняется диагностика сохранности электрических цепец, проверка на отсутствие коротких замыканий и обрыва линий.

- Выполнение команд начинается с адреса 0xFFFFFFF0
 - Содержимое памяти ROM отображается на адресное пространство таким образом, что содержащийся в ней код становится доступен, начиная с этого адреса.
 - Микросхема ROM содержит BIOS – набор микропрограмм, предоставленных производителем аппаратуры, разработчиком материнской платы. BIOS проводит дополнительное тестирование, проверяет доступность памяти. Работа микропрограмм направлена на то, чтобы дать возможность провести конфигурацию аппаратуры и программно взаимодействовать с ней.
- В тот момент, когда началась загрузка, код BIOS расположен в микросхеме. BIOS выполнит следующие действия
 - Копирование своего кода из ROM (микросхемы) в RAM (оперативную память компьютера)
 - Самопроверка кода
 - Начальная конфигурация аппаратуры
 - Размещение в оперативной памяти ACPI таблиц, которые описывают периферийные устройства.

После того, как служебная информация помещена, передается управление следующему загрузчику, который находится в цепочке загрузки.

- MBR – устройство, которое хранит информации в блочном виде. Последовательно пытаемся копировать MBR (сектор №0, 512 байт) каждого загрузочного устройства в память по адресу 0x7c00. Если сектор скопировать удалось, BIOS передает управление на адрес 0x7c00 и заканчивает свою работу.
- MBR-загрузчик копирует в память загрузчик операционной системы и передает ему управление.
- Загрузчик операционной системы копирует в память ядро операционной системы и передает ему управление. Уже этот полноценный код копирует модуль самой операционной системы.

Реальный режим.

- В каждый момент времени работает только одна программа
 - Эта программа управляет всеми ресурсами (ЦПУ, память, ввод/вывод)
- Машинное слово 16 разрядов, адрес памяти 20 разрядов
 - Сегментные регистры работают так, как работали 40 лет назад.
 - Исполнительный адрес = (seg_reg) « 4 + смещение. Таким образом 16 разрядов сместили до 20. Так можем адресоваться внутри адресного пространства размером 1 Мбайт.
- На вычислительном ядре так стартует только начальное загрузочное ядро. На нем выполняется одна единственная программа, не имеющая ограничений, которой доступны все настройки. Доступна вся адресуемая память
- Периферийные устройства управляются через порты ввода/вывода (команды in и out).

UEFI. Есть определенная сложность в использовании тех служебных функций, которые размещают BIOS в памяти для того, чтобы можно было работать с периферийными устройствами. Сам BIOS – 16-ти разрядная программа.

После того, как BIOS все настроил мы либо остаемся в 16-ти разрядном мире, либо загрузчик операционной системы переключит в 32 разряда, т.е. все функции BIOS становятся бесполезными. Для того, чтобы справиться с этими проблемами, возник определенный стандарт того, как загрузчик должен быть устроен. Этот стандарт называется UEFI – (Unified) Extensible Firmware Interface.

Выполнение задачи перевода системы в 32-х разрядный режим и более сложная настройка аппаратуры – обязанности загрузчика. Вспомогательные функции, которые он будет размещать, становятся применимыми со стороны операционной системы. Этот интерфейс, который позволяет реализовать возможности аппаратуры называется интерфейсом EFI(Extensible Firmware Interface).

Спецификация состояния аппаратуры перед загрузкой ОС с помощью ACPI таблиц.

Стандарт ACPI (Advanced Configuration and Power Interface) был разработан для того, чтобы можно было эффективно управлять энергопотреблением различных периферийных устройств.

Проблема: управление энергопотреблением периферийных устройств и набора микросхем. Поэтому ОС должна содержать драйвер для каждой модели материнской платы.

Решение: организовать драйвер в виде двух отдельных компонент: описания аппаратуры и интерпретатора этого описания. Таким образом часть операционной системы, которая отвечает за управление конкретным устройством, разбилась на писательную и стандартизированную части. Таким образом он размещает в памяти два вида таблиц.

BIOS размещает в оперативной памяти две группы ACPI таблиц:

- Перечисление имеющейся в компьютере аппаратуры и некоторых ее свойств
- Формируется таблица, набор микропрограмм, описывающих работу с устройствами на архитектурно независимом языке AML

В состав ОС входит интерпретатор языка AML.

⇒ Таким образом, если раньше операционная система включала в себя значительное количество драйверов, теперь количество драйверов и сложность их устройства можно уменьшить. Главное – наличие AML интерпретатора, который может запросить ACPI таблицу из оперативной памяти.

⇒ Это устройство взаимодействия унифицировалось и упростилось.

Многозадачная работа компьютера: требования к аппаратуре.

При запуске нескольких пользовательских программ одновременно возникает ситуация, когда они мешают друг другу. Необходимо урегулировать, какое приложение использует какой диапазон физической памяти, порядок их обращения к устройствам ввода/вывода и т.д.

Для решения этой проблемы нужно

- изолировать программы
- для работы с уникальными ресурсами прикладные программы должны запрашивать разрешение у операционной системы.

1. Привилегированный режим

- Разделение машинных команд на две категории: пользовательские и привилегированные.

Внутри процессора должен быть служебный регистр, который будет определять, в каком состоянии мы находимся: работает операционная система или работает приложение.

Необходимые аппаратно реализованные механизмы:

2. Механизм защиты памяти

- Изоляция кода/данных различных программ (и операционной системы) друг от друга

3. Таймер – устройство, которое время от времени генерирует сигнал, по которому все останавливается, а управление возвращается внутрь операционной системы на одну из ее функций(механизм прерываний).

- Принудительное вытеснение программы с процессора
- Многозадачность: невытесняющая и вытесняющая

4. Механизм прерываний

- Возможность процессора отреагировать на возникновение некоторого события – передать управление на определенный код и изменить состояние регистров и памяти
 - (не)штатный ход выполнения очередной команды
 - сигналы от других устройств, в том числе – от таймера

Модели памяти.

Модели памяти в IA-32

Есть несколько режимов работы. Главные отличия – между реальным и защищенным режимами.

- Реальный режим. Модель памяти с реальной адресацией линейный адрес = сегментный регистр « 4 + смещение; Адрес физической памяти = линейный адрес
- Режимы защищенной памяти. Появляется дополнительный уровень косвенности. Тот же сегментный регистр, который используется при вычислении линейного адреса трактуется другим образом: это некоторый номер описателя сегмента внутри таблицы, размещенной в памяти. Из этого описателя можно извлечь базовый адрес, который потом будем складывать со смещением.

В рамках такой дополнительной защиты на уровне прав доступа можно регламентировать, с каким куском адресного пространства что мы может делать (читать/писать), для какого кода он доступен.

Помимо сегментного механизма еще дополнительно включается страничная трансляция адресов.

- Базовая плоская модель. Защищенная плоская модель
 - Мультисегментная модель
- Независимо от используемой модели памяти может быть включена страничная трансляция адресов

Плоская модель. Все те же 6 сегментных регистров отсылают к описателю, который лежит где-то в памяти. Для него есть флаги доступа, есть базовый адрес. Считаем, что у всех этих сегментов нулевой базовый адрес, длина 4Гб. Каждый такой сегмент будет целиком покрывать всю потенциально адресуемую память, внутри которой будет размещен код, статические данные и т.д.

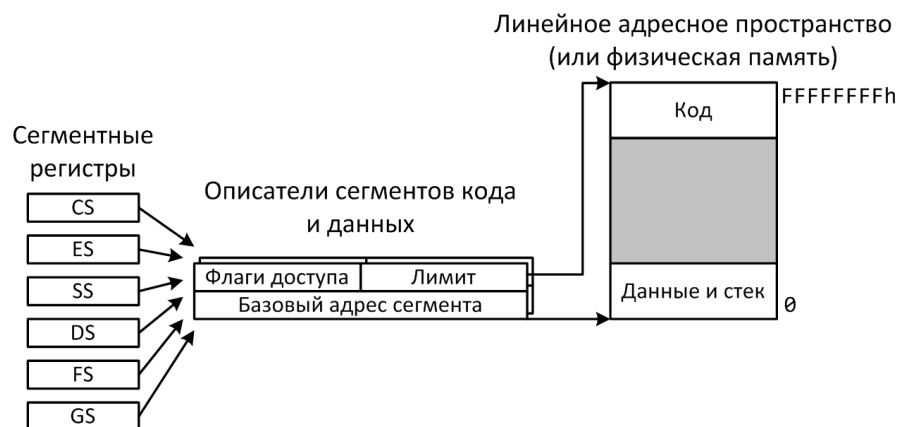


Рис. 24.2: Плоская модель

В тот момент, когда идет выборка команды, используется сегментный регистр CS, который неявно добавляется к исполнительному адресу, с которого происходит выборка. Когда идет обращение к данным, неявно добавляется регистр DS, а когда идет работа со стеком, используется регистр SS.

В такой плоской модели каждая программа видит адресуемую память как пространство от 0 до FFFF, внутри которого размещено все, что ей нужно.

Мультисегментная модель. Каждый сегментный регистр отправляет к уже к разным описателям, внутри этих описателей используются базовые адреса, который могут ссылаться на разные точки внутри памяти. Будут появляться куски, для которых

определены права доступа, определенные размеры. Попытка выйти, прибавляя смещение, за пределы этого лимита, приведет к сбою, который ОС будет фиксировать как прерывание.

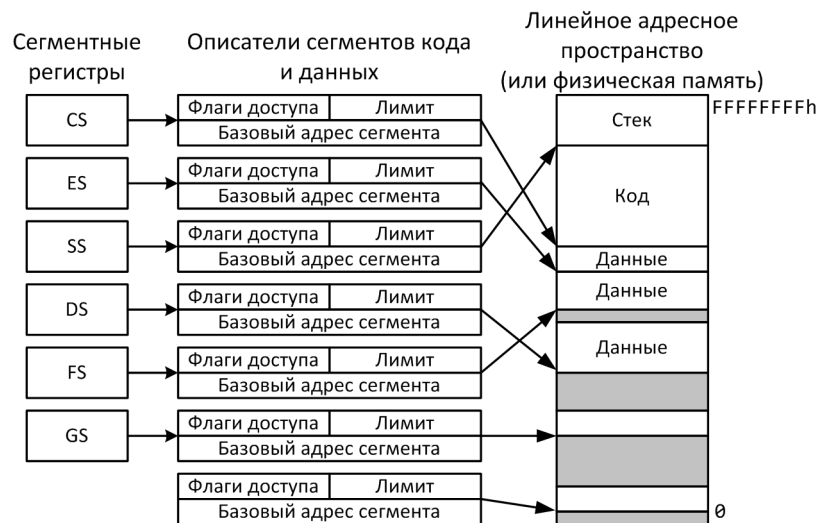


Рис. 24.3: Мультисегментная модель

Описателей в таблице может быть достаточно большое количество. Сегментные регистры внутри себя могут содержать разные индексы, переставляясь с одного описателя на другой.

В реальности происходит некоторое совмещение. Плоская модель связана с 4 сегментными регистрами, которые работают с кодом, с данными, со стеком. Все они ссылаются фактически на почти одинаковые описатели, у которых база – 0, а лимит – 4 Гб. Ограничения на уровне сегментной трансляции адресов не используются.

Есть еще два сегментных регистра:

- FS (используется в Windows системах);
- GS (используется в Linux системах),

которые отсылают к описателю базовый адрес которого фактически является начальным адресом некоторой структуры в памяти, которая описывает текущее состояние потоков выполнения.

На служебные данные ссылаемся явно, а для всего остального используется простая плоская модель.

Аппарат защиты памяти.

- Переключившись в защищенный режим, мы меняем настройки процессора так, что машинное слово и адрес становятся 32-х разрядными; становятся доступны два

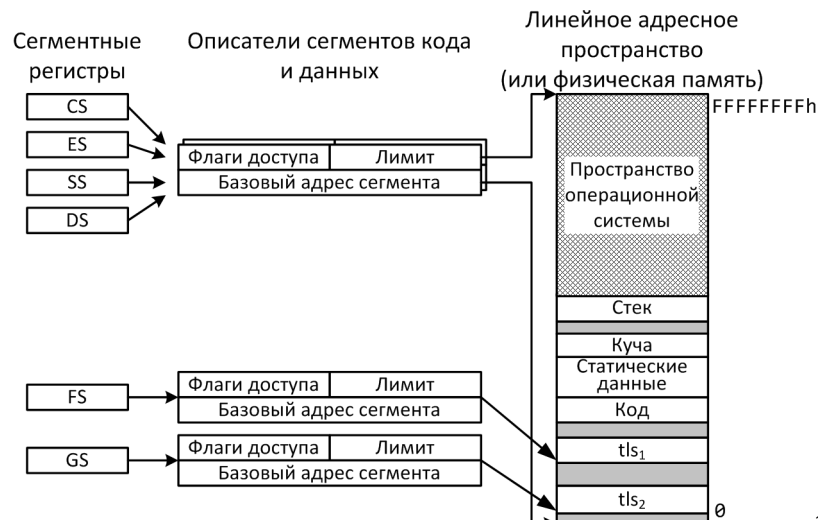


Рис. 24.4: Реальная модель

дополнительных сегментных регистра gs и fs. Архитектура набора команд будет позволять выполнять команды, которые с ними работают.

- Доступны для перехода в этот режим четыре регистра управления CRn. Запись каких-то predetermined значений будет приводить к тому, что будет меняться настройка работы.

Например, CR3 используется для управления доступом к памяти.

- Появляется аппаратная защита памяти.
 - Для работы которой потребуются базовые адреса, по которым будут расположены таблицы с регистрами. Это таблицы GDTR, LDTR, IDTR
 - Становится доступка мультисегментная модель (фактически не используется)
 - Появляется многоуровневая трансляция адресов памяти.
- На уровне аппаратуры есть поддержка аппаратного переключения задач. Т.е. сама аппаратура позволяет разграничивать контекст выполнения той или иной программы, и потом эти контексты выполнения менять (не используется).

Сегментные селектор и дескриптор. Рассмотрим сегментный регистр в защищенном режиме.

Регистр называем селектором. В данном случае 16 разрядов, относящиеся к регистрам cs, ds, ss, позволяют выбирать по заданному индексу из одной из таблиц дескриптор.

Дескриптор состоит из двух 32-х разрядных слов. Их может быть 2^{13} .

- Отдельным разрядом указывается тип таблицы: глобальная (общая таблица с описателями сегментов для всех программ, которые работают на компьютере) или локальная (сегменты, относящиеся только к одной текущей программе) таблица.



Рис. 24.5: Селектор

В некотором виде такое разделение оказывается избыточным.

- Еще два разряда описывают уровень привелегий, который соотнесен с этим сегментом памяти. Этим уровнем может быть 4.

Сегментный дескриптор содержит в себе 32 разряда, описывающие базу.

Длина кодируется в 20 разрядах. Отдельный бит указывает на гранулярность этой длины. Она может быть байтовая, либо порциями по 4 Кбайта ($4K=2^{12}$), таким образом захватывается все потенциально адресуемое пространство памяти. Главное, что здесь



Рис. 24.6: Дескриптор

присутствует – база, длина и некоторый права доступа.

Сегментная адресация в защищенном режиме. В зависимости от выборки (кода/данных) берется сегментный селектор, из которого берется соответствующий индекс, берется регистр GDTR, в котором содержится базовый адрес этой таблицы описателей. По этому базовому адресу вытаскиваем базовый адрес сегмента, который затем складывается со смещением $[eax+4*ecx]$

После сложения идет обращение в память за данными. На выходе получаем линейный адрес.

Страничная виртуальная память.

Рассмотрим еще один уровень преобразования адреса – уровень страничной организации.

Основные проблемы:

- Нехватка физической памяти

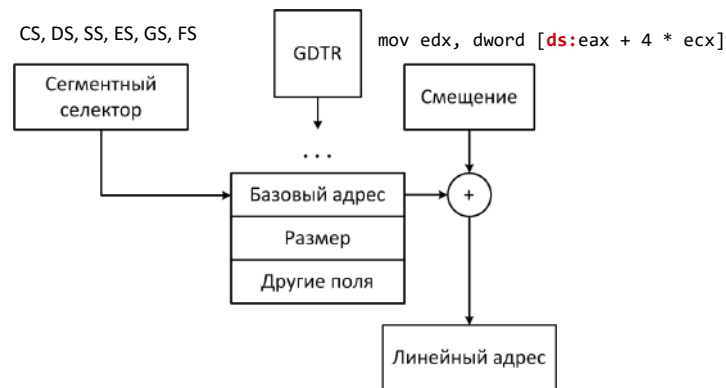


Рис. 24.7: Адресация в защищенном режиме

- Необходимость изоляции одновременно работающих программ.

Вся память бьется на куски равных размеров. Старшие разряды адреса будут преобразовываться так, что номер виртуальной страницы будет меняться на номер физической страницы. За счет этого можно будет формировать видимое адресное пространство нужного размера из этих кусков. Можно некоторые куски памяти одновременно использовать в разных адресных пространствах различных работающих программ и защищать это все друг от друга, закрывая доступ к физической памяти нужной одной программы от других программ.

Преобразованием адресов занимается отдельная компонента процессора MMU, которая и должна часть адреса определенным образом пересчитывать.

Пример. Пусть есть физическая память из 8 страниц, каждая из которых занимает 4 байта. Эти страницы чем-то заполнены. Два программы работают. Каждая из программ хочет по 6 страниц. Можно сформировать виртуальную память на стороне каждой работающей программы, описав устройство виртуальной памяти в таблице страниц, привязанной к конкретной программе.

- В программе А определено следующее отображение:
 - 0 страницу виртуальной памяти отображаем на 0 страницу физической памяти.
 - Первая страница виртуальной памяти связана со второй страницей физической памяти.
 - Вторую страницу ассоциирует с первой страницей физической памяти.
 - Следующие несколько страниц недоступны, а пятая страница виртуальной памяти связана с седьмой страницей физической памяти.
- В программе Б присутствует своя таблица страниц, в которой только одна страница недоступна.

- Нулевой адрес связан с тем, что программа А не видит(3 страница физической памяти).
- ...
- седьмая страница физической памяти одновременно отображена в адресное пространство и программы А, и программы Б. Через этот общий кусок физической памяти программы могут обзаться, если будут придерживаться определенного протокола.



Рис. 24.8: Страничная организация памяти

Любое обращение в память будет выражаться в том, что мы должны поменять номер виртуальной страницы на номер физической страницы. Затем внутри страницы определяется нужное смещение. **Запись в таблице страниц (на примере 4 КБ страниц).** Адрес состоит из 32-х разрядов. Так как размеры порции занимают 4КБ, то для адреса физической страниц достаточно 20 разрядов. Первые 12 разрядов говорят о том, присутствует ли эта реально или нет, можно ли ее читать/писать. Следующий бит указывает на то, может ли пользовательский код читать/писать в эту страницу. Еще бит указывает, будет ли кэшироваться содержимое страницы. Дальше бит показывает, обращались ли к этой странице по чтению/записи. Последний бит содержит информацию о том, что в страницу зачитывали данные.

Разберем, что будет происходить при попытках в команде `mov edx что-то считать по адресу eax+4*ecx`.

1. Вырабатывается логический адрес, который состоит из сегментного селектора и посчитанного смещения. Это определение всей нужной информации для последующей работы.
2. По сегментному селектору, исходя из значения базового адреса глобальной таблицы дескрипторов, берется описание сегментного дескриптора. В нем узнается линейный адрес, к которому затем добавляется смещение. → Получили линейный адрес в линейном адресном пространстве программы.

На этом завершилась работа сегментной организации.

3. Системный регистр CR3 указывает на базовый адрес каталога страниц.
4. Линейный адрес бьется на 3 куска.
 - (a) младшие 12 разрядов – смещение внутри страницы
 - (b) старшие 20 разрядов тоже разбиваются на 2 части:
 - старшие 10 разрядов указывают на запись внутри каталога страниц. Эта запись содержит некоторый базовый адрес, начиная с которого идет таблица страниц.
 - младшие 10 из этих 20 разрядов определяют индекс внутри таблицы страниц. Из нее вытаскиваем запись, из которой определили базовый адрес физической страницы, к которой будем обращаться.

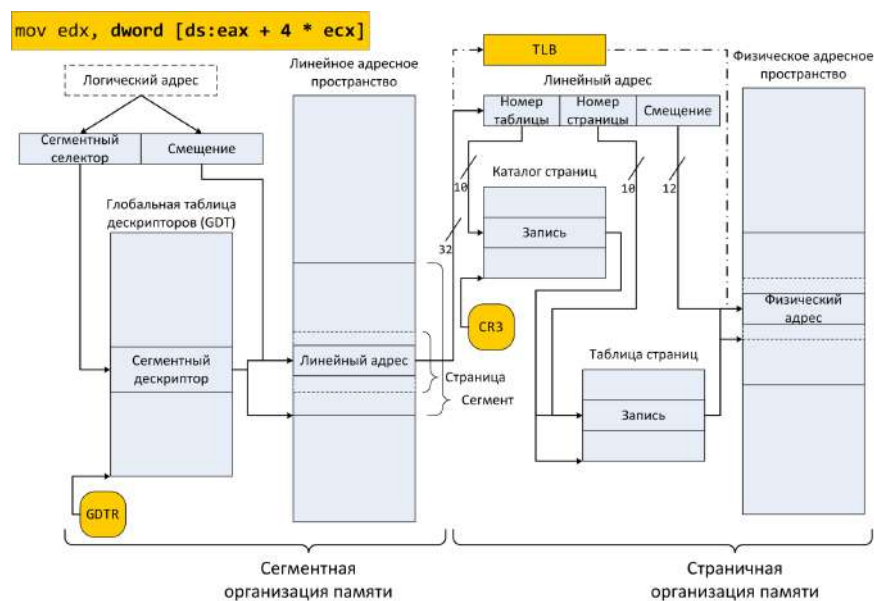


Рис. 24.9: Память

- Сегментный селектор. Его значение меняется крайне редко, меняет только операционная система. Когда она записывает в него индекс, в этот момент внутри процессора кэшируется содержимое соответствующего сегментного дескриптора, который туда перегоняется. Таким образом прямо внутри процессора можно посчитать линейный адрес.
- На уровне страничной организации проходит похожий процесс. Результаты транслирования номеров виртуальных страниц в номера физических страниц сохраняется в служебном кэше, который называется TLB. Если программа имеет в себе локальность, раз за разом она будет обращаться примерно к одним диапазонам. Раз за разом будут транслироваться одни и те же адреса. Таким образом вся схема не будет обращаться в память за содержимым всех таблиц и каталогов.

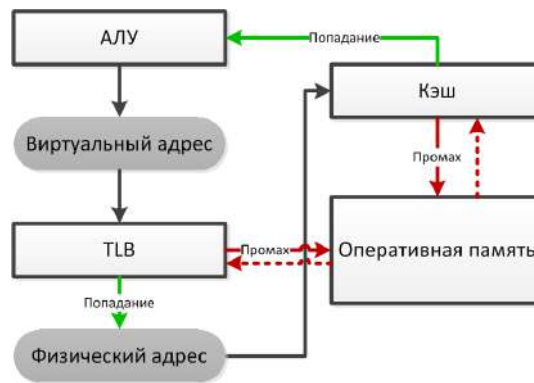


Рис. 24.10: Упрощенная схема извлечения данных из виртуальной памяти

- **TLB – буфер ассоциативной трансляции.** TLB играет ключевую роль, потому что от него зависит, насколько успешно будут происходить трансляции.
 - TLB кэширует записи таблицы страниц, т.е. является служебным (специализированным) кэшем – В отличие от кэша данных хранятся не блоки, а номера страниц. Обращение к ним идет по той же схеме, как и в случае кэша данных.

Упрощенная схема извлечения данных из виртуальной памяти

1. Логический адрес пересчитали в линейный, используя сегменты.
2. Нужно пересчитать линейный адрес в физический.
 - (a) Если для соответствующего номера в TLB его готовый адрес, берем его, переставляем старшие разряды адреса и получаем то, что нужно
 - (b) Иначе обращаемся к таблицам трансляции, оттуда по нужному индексу берется нужный адрес физической страницы, который затем запоминается в TLB.
3. Обращение в физическую память будет предвостановляться проверкой кэша.

Примечание. Процент промахов в TLB: 0.1 – 1.0 %

Уровни (кольца) защиты

В описателях есть определенные биты, которые разделяют привелегии между операционной системой и прикладными программами.

На уровне сегментов размер этого поля состоял из двух разрядов. Таким образом получается 4 уровня, которые называются кольцами защиты.

- В 0 зоне работает операционная система.
- В 1 кольце с уже меньшим уровнем привелегий работают программы, у которых привилегия равна 1.
- Уровни защиты 1 и 2, как правило, не используются
- Уровень 3 – зона работы пользовательской программы.

Смена уровня привилегий – затратное по времени действие, потому что нужно поменять много всего в служебных регистрах процессора. Это происходит только при наступлении определенных событий. Переход с одного кольца на другой – долгое событие (порядка 100 тактов).

Переход между уровнями контролируется полями PL в дескрипторах сегментов. Возможность выполнения определяется текущим уровнем привилегий. Его можно примерно соотнести с полем PL в дескрипторе.

Итог: есть уровень привилегий, который поддерживается аппаратно. У операционной системы и у пользовательских программ эти уровни разные. Обращаться к тому коду и данным, которые соотнесены с уровнем ноль, прикладная программа обращаться не может. Если она обратится, произойдет прерывание.

Прерывания.

Прерывание – это некоторое событие, на которое аппаратура должна отреагировать определенным образом.

Два типа прерываний – один механизм обработки

- Асинхронные прерывания – события, происходящие в периферийных устройствах (Ввод/Вывод). Не связаны с тем, что происходит на центральном процессоре
- Синхронные прерывания (исключения) – возникновение определенных ситуаций при выполнении команд процессором

- Ловушки возникают при целенаправленном выполнении команды, зная, что она будет перехвачена и управление передастся в операционную систему.
- Сбои возникают в случае ожидания верного исполнения команды, которая по какой-то причине отработать не может.
- Аварийная остановка возникает в случае неустранимой ошибки. Например, перегрев или нарушение физической целостности компьютера.

Вектор (№)	Мнемоника	Описание	Источник
0	#DE	Ошибка при делении	Команды DIV и IDIV
3	#BP	Точка останова	Команда INT 3
13	#GP	Сбой защиты	Любые ситуации, связанные с нарушением защиты памяти
14	#PF	Отсутствие страницы	Обращение к отсутствующей странице
16	#MF	Сбой в x87 FPU	Команды x87
18	#MC	Непоправимый сбой в работе аппаратуры	Механизм самопроверки аппаратуры
32-255	–	Определяется пользователем	Внешние прерывания или команда INT <i>n</i>

Рис. 24.11: Примеры прерываний

Реакция на прерывание. Если произошло прерывание, управление потом может быть передано обратно на то место, где прерывание было зафиксировано (доставка прерывания), либо не возвращено.

В любом случае нужно отдать управление в операционную систему. При этом можно повысить уровень привилегии. Т.е. вызов обработчика будет сопровождаться тем, что поменяется сегмент, связанный с кодом. В этот момент сегмент может получить права операционной системы.

Для того, чтобы понять, какой сегмент взять, ориентируется на номер/вектор прерывания и таблицу описателей прерываний, где определенные адреса обработчиков размещены.

Некоторые прерывания можно маскировать. Т.е. работу с ненормализованным числом можно видеть, а можно и не видеть.

За обработку и доставку прерываний отвечают определенные компоненты, которые называются программируемыми контроллерами прерываний (PIC). Они разделены на две части:

- локальный контроллер прерываний (Local APIC);

- контроллер прерываний ввода/вывода (I/O APIC).

А также контроллер прерываний содержит таймер: через заданное пользователем число тактов будет выбрасываться прерывание от таймера. Он останавливает работу и может выполнявшуюся ранее пользовательскую программу сдвинуть с вычислительного ядра и поместить туда на выполнение другую программу.

Local APIC и I/O APIC в многоядерном компьютере. Разберем схему с двумя контроллерами. Есть некоторое количество процессоров (ядер). Каждое ядро содержит свой локальный программируемый контроллер прерывания. Локальный контроллер завязан на системную(фронтальную) шину. Это фронтальная шина занимается в первую очередь занимается рассылкой уведомлений через локальные контроллеры прерываний о том, что кэши в отдельных ядрах между собой рассинхронизировались. Внешние события поступают на контроллер ввода/вывода. Все происходит на едином

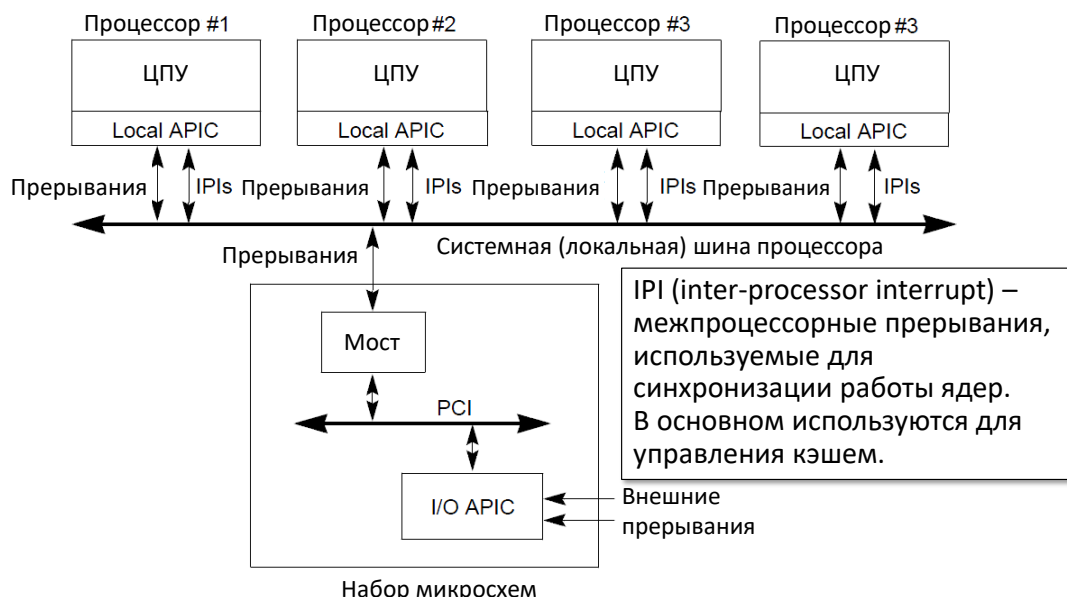


Рис. 24.12: Local APIC и I/O APIC в многоядерном компьютере

механизме доставки сообщений через шину PCI.

Если пришло прерывание в устройство на его APIC, он формирует служебный пакет данных, который через шину PCI, через мост доставляет информацию в локальный контроллер, через который процессор узнает о событии.

Когда процессор узнал о том, что что-то произошло, он смотрит на служебный регистр IDTR(см. рис 24.12), по номеру прерывания находит нужный описатель прерывания. Далее получает некоторое смещение внутри локальной таблицы, берет оттуда сегментный дескриптор, складывает базовый адрес с самим смещением и передает управление на обработчик прерывания, который где-то живет в памяти.

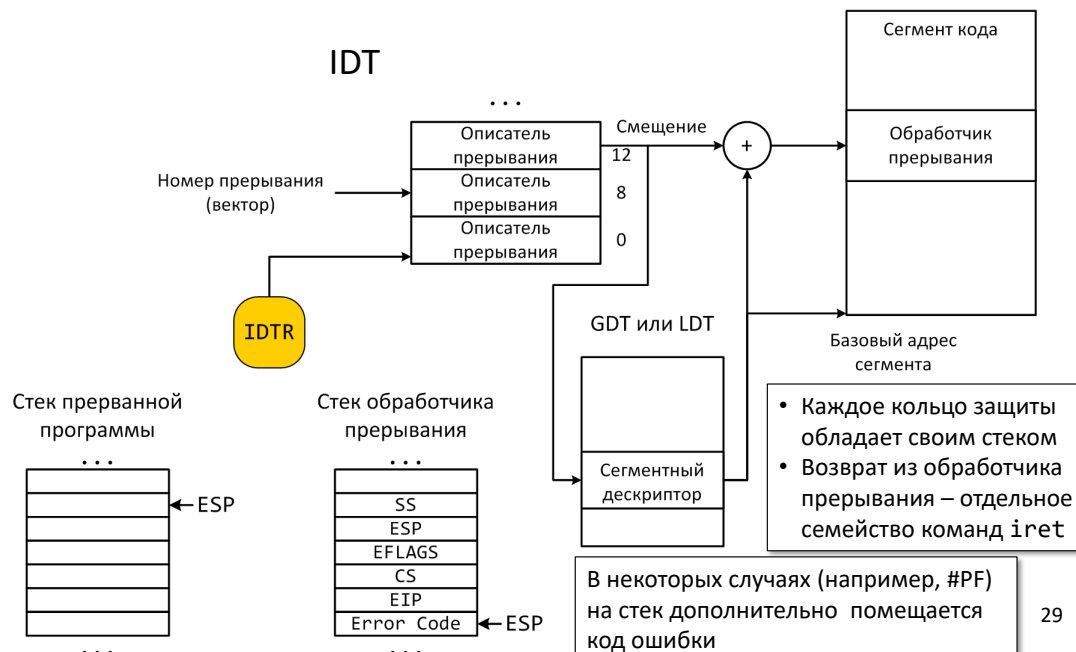


Рис. 24.13: Передача управления в обработчик прерывания в защищенном режиме

В этот момент на СТЭК могут быть помещены некоторые важные регистры, которые описывают, что происходило в этот момент в пользовательской программе и т.д.

В каждом кольце защиты существует свой стек.

Если при прерывании мы перепрыгиваем из пользовательского кода в систему.

Если прерывание произошло при работе операционной системы, изменений уровня привилегий не происходит.

Механизм прерываний – то, с помощью чего можно попросить операционную систему сделать для нас что-то полезное. Например, аппаратура, память операционной системы и других программ не доступны. Управляющие регистры лежат в тех диапазонах, которые не видны пользовательскому коду и т.д. Получается, что единственное, что видит программа – это регистр процессора и некоторый диапазон адресов физической памяти, которые доступны ей после настройки трансляции. Чтобы попросить операционную систему что-то для нас сделать, нужно выполнять определенные команды.

Системные вызовы Системный вызов – основной способ «обратиться» к операционной системе

- `syscall/sysret`
- `sysenter/sysexit`
- `int/iret`

В ОС Linux системный вызов реализован как прерывание

- `int 0x80`

- Передача параметров – через регистры: номер функции – eax параметры – ebx, ecx, edx, esi, edi

EAX	Название	EBX	ECX	EDX
1	sys_exit	int	–	–
3	sys_read	unsigned int	char *	size_t
4	sys_write	unsigned int	const char *	size_t
5	sys_open	const char *	int	int
6	sys_close	unsigned int	–	–
116	sys_sysinfo	struct sysinfo *	–	–

Рис. 24.14: Примеры некоторых системных вызовов ОС Linux

Hello, world!

Рассмотрим прим пример, в котором нужный функционал, а именно вывод строки, будет делать операционная система.

Данный кон неперонисим между другими UNIX системами, потому что там будут свои правила системных вызовов.

- Определяем строчку "Hello, world!" в секции data, определяем ее размер
- После метки start, которая соотносится с точкой входа в программу, на которую передаст управление загрузчик, как только программа будет готова к запуску, укажем, что нужна функция 4.
- Функция 4 – функция записи в файл. Укажем номер файла.
- 1 – стандартный вывод
- В ecx и edx отправляем адрес начала строки и ее длину.
- Вызываем int 80h, что приводит к тому, что операционная система с заданного адреса на консоль напечатает заданную строку.
- Это завершается вызовом системного вызова номер 1, указав нулевой код возврата.
- Финишируем прогамму с нулевым кодом.

© 2021 МГУ/ВМК/СП

```
snoop@earth:~/samples$ nasm -f elf32 -o syscall.o syscall.asm
snoop@earth:~/samples$ nm syscall.o
00000000 T _start
00000000 d msg
0000000e a msg_len
snoop@earth:~/samples$ ld -o syscall syscall.o
snoop@earth:~/samples$ nm syscall
080490b2 A __bss_start
080490b2 A _edata
080490b4 A _end
08048080 T _start
080490a4 d msg
0000000e a msg_len
snoop@earth:~/samples$ ./syscall
Hello, world!
snoop@earth:~/samples$
```

```
#include <unistd.h>
#include <stdlib.h>

void main() {
    write(1, "Hello, world!\n", 14);
    exit(0);
}
```

```
section .data
    msg db `Hello, world!\n`
    msg_len equ $-msg

section .text
global _start
_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, msg
    mov edx, msg_len
    int 80h

    mov eax, 1
    mov ebx, 0
    int 80h
```

Рис. 24.15: Hello, world!



ФАКУЛЬТЕТ
ВЫЧИСЛИТЕЛЬНОЙ
МАТЕМАТИКИ И
КИБЕРНЕТИКИ
МГУ ИМЕНИ
М.В. ЛОМОНОСОВА

teach-in
ЛЕКЦИИ УЧЕНЫХ МГУ